Coefficient justification and parameter sweeps

Handwriting reproduction robot — every chosen constant, its basis, and what happens when it moves

D. R. Braune (u23539195)

10 November 2026

Abstract

This document accounts for the numerical constants in the submitted code of the handwriting reproduction robot. For each coefficient it records where the value lives, the published or measured basis for choosing it, what the coefficient actually controls, what alternatives were available, and a worked example in the real units of the system. Where the coefficient can be exercised offline, it is swept over five values with every other part of the system held fixed, and the result is shown as an image so the effect can be seen rather than asserted. The submitted value is always one of the five and is always run against the unmodified submitted source. Coefficients that cannot honestly be swept — the trained network's architecture, and the constants that only act on the gantry hardware — are explained and the reason they are not swept is stated plainly rather than being filled in with a plausible-looking experiment.

Contents

1 Scope, method and honesty statement **5**
- 1.1 What was done . . . 5
- 1.2 What the five values mean . . . 5
- 1.3 What this study does not establish . . . 5
- 1.4 How to read the read acc column . . . 5
- 1.5 Telling a real difference from seed noise . . . 5
- 1.6 The letter prior used by the profile sweeps . . . 6
- 1.7 Inputs used . . . 6

2 Which checkpoints the sweeps load **6**
- 2.1 The reader . . . 6
- 2.2 The writer classifier . . . 7

3 Every coefficient covered **7**

4 Which coefficients actually matter **9**

5 Image toolkit **11**
- 5.1 Upper edge of the histogram Otsu is built on, for a ratio image . . . 11

6 Line binarisation **11**
- 6.1 Width below which a run of ink counts as thin, for printed-rule removal . . . 11
- 6.2 Length above which a run of ink counts as a printed rule . . . 15
- 6.3 Closing radius that bridges pinholes in a stroke . . . 17
- 6.4 Smallest component kept in a line mask . . . 19

7 Line geometry **20**
7.1 Profile height defining the x-height band 20

8 Illumination normalisation **22**
8.1 Local background radius for the ink mask 22
8.2 Window fill above which a solid block is ruled out as writing 24

9 Sheet detection **25**
9.1 Fraction of the median brightness below which a block is off-sheet 25

10 Page orientation **28**
10.1 How much better the sideways reading must be before the page is turned 28

11 Line finding **29**
11.1 Profile height a candidate line peak must reach 29
11.2 Smallest spacing between two accepted line centres 31
11.3 Valley depth below which two line centres are merged 32
11.4 Smallest peak gap counted as line spacing, in letter heights 33

12 Printed-structure removal **36**
12.1 Length above which a flat run on the page is printed structure 36
12.2 Length above which an upright run on the page is printed structure 37

13 Seam cutting **38**
13.1 Weight on ink in the seam cost field 38
13.2 Weight pulling the seam back towards the midline 39
13.3 Rows a seam may move between adjacent columns 42
13.4 Closing radius applied before the seam is cut 42

14 Line ownership **45**
14.1 Share of a piece's ink that must lie in a band for it to count 45

15 Crop building **47**
15.1 Padding kept around a line crop 47
15.2 Darkness exponent applied to a finished line crop 49
15.3 Percentile of masked pixels taken as the ink level 50

16 Crop rejection **52**
16.1 Ink share above which a crop is a solid block, not writing 52
16.2 Multiple of the page median ink a crop may not exceed 54
16.3 Ink a band must hold before it becomes a line crop 55

17 Writer classification **58**
17.1 Pen width the writer classifier's input is redrawn at 58
17.2 Upper clip on the redrawn pen width, in pixels 59

18 Network input **61**
18.1 Height of the fixed canvas both networks read 61
18.2 Width of the fixed canvas, which sets the number of CTC columns 63
18.3 Hidden size of each LSTM direction 65

19 Centreline tracing **67**
19.1 Largest turn two traced chains may make and still be rejoined 67
19.2 Shortest dangling chain kept after thinning 68

20 Glyph extraction **72**
20.1 Douglas–Peucker tolerance for a stored stroke . . . 72
20.2 Weight on how high the ink reaches in the cut-cost profile . . . 73
20.3 How far a character boundary may be moved to find a cheap column . . . 75
20.4 Smallest cell a character boundary pair may enclose . . . 77

21 Glyph filtering **79**
21.1 Widest glyph accepted, in x-heights . . . 79
21.2 Side of the square occupancy descriptor . . . 83

22 Profile building **84**
22.1 Reading-confidence percentile below which a whole line is not trusted . . . 84
22.2 Shapes of one letter kept per writer . . . 84

23 Synthesis: joining **86**
23.1 Joinedness below which a hand is drawn as print and makes no joins . . . 86
23.2 Furthest apart two letters may be and still be joined . . . 90

24 Synthesis: spacing **92**
24.1 Clearance kept between one letter's ink and the next . . . 92

25 Synthesis: variation **95**
25.1 Persistence of the slow baseline drift . . . 95
25.2 Largest baseline drift allowed . . . 96

26 Synthesis: rendering **97**
26.1 Pen width the synthesised line is rendered at . . . 97
26.2 Softness of the rendered stroke edge . . . 99

27 Synthesis: shape choice **101**
27.1 How many of a letter's stored shapes may be drawn from . . . 101
27.2 Largest lean a fallback letterform is drawn at . . . 103

28 Synthesis: legibility **105**
28.1 Prior distance at which a writer's glyph store is still fully trusted . . . 105

29 G-code generation **107**
29.1 Gap the pen draws through instead of lifting . . . 107
29.2 Gap at which a stroke is closed into a ring . . . 109
29.3 Shortest move kept in the G-code . . . 111

30 Pen actuator **113**
30.1 Dwell after a pen-down pulse . . . 113

31 Motion **115**
31.1 Feed rate while drawing . . . 115

32 Page layout **117**
32.1 X-height as a share of the ruled line spacing . . . 117

33 End-stops **119**
33.1 Interval between the samples that confirm an end-stop . . . 119

34 Seam cutting on a dense page **121**
34.1 Weight on ink in the seam cost field, on a densely written page 121
34.2 Closing radius applied before the seam is cut, on a densely written page 124
34.3 Share of a piece's ink that must lie in a band, on a densely written page 125

35 Choices that cannot honestly be swept **128**
35.1 The shared convolutional backbone 128
35.2 The reader's recurrent stack and head 129
35.3 The writer classifier's head, and dropout 129
35.4 Initialisation, optimisers and schedules 129
35.5 Augmentation 130
35.6 The held-out split and the personal fraction 130

36 What is covered and what is not **130**
36.1 Covered 130
36.2 Not covered, and why 130

37 Running a sweep live **130**

1 Scope, method and honesty statement

1.1 What was done

The submitted code is seven Python files, `paths.py`, `segment.py`, `recognise.py`, `style.py`, `synthesise.py`, `reproduce.py` and `train.py`, totalling 6 546 lines. Every numeric literal and named constant in those files that represents a design decision was collected. Each one is documented in section 3 onwards.

A sweep works as follows. The submitted source is read, *one* token is rewritten, and the result is written to a throwaway directory which is then imported and run in its own process. The submitted files are never modified: a SHA-256 digest of all seven is recorded before and after every batch of runs and the run fails if any of them has changed. At the submitted value nothing is rewritten at all, so the panel marked *submitted value* in every figure is the submitted code itself, not a re-formatted copy of it.

Every patch target was checked to occur exactly once in its file before any sweep was run, so a substitution can never land on the wrong line. That check covers **212** candidate tokens and is re-run by `sweep/validate.py`.

1.2 What the five values mean

Each sweep runs five values: the submitted value, two below it and two above it. The spacing is chosen to span the range over which the coefficient plausibly could have been set, not a narrow band around the chosen value, because the question being answered is 'why this value and not another', and a $\pm 5\,\%$ sweep cannot answer it. The panel outlined in dark blue is the submitted value; the others are outlined in silver.

1.3 What this study does not establish

These are one-at-a-time sweeps on a small number of fixed inputs. They show what each coefficient does and they show that the submitted value is reasonable. They do **not** establish that any value is optimal, they do not explore interactions between coefficients, and a sweep on one page is not a measurement over the dataset. Where a reported number happens to be better away from the submitted value, that is reported as it stands rather than explained away, and it is not on its own a reason to change the submitted system.

No network was retrained for this study, no style profile in the deployed system was overwritten, no G-code was sent to the machine and the gantry was not run.

1.4 How to read the read acc column

Several page-level sweeps report a reading accuracy. It is computed by transcribing the first eight line crops and comparing crop i with transcript line i, so **it measures the pairing as well as the reading**. If a coefficient changes the number of lines found, the crops shift against the transcript and the accuracy falls sharply even though the individual crops may be perfectly readable.

That is not an artefact to be discounted: the style-profile builder pairs crops with transcript lines in exactly the same way, so a segmenter that returns the wrong number of lines really does poison everything downstream of it. But the number should be read as 'line count correct *and* crops readable', not as a pure recognition score. Where a sweep changes the line count, the lines and true lines columns beside it say so.

1.5 Telling a real difference from seed noise

Synthesis is a random process: the letterform drawn for each character, the jitter applied to it and the baseline drift are all drawn from a seeded generator. A single run therefore measures the setting *and* the

seed together, and the differences between settings in this system are small — one or two characters in a forty-three character sentence.

Every synthesis sweep is therefore run over the **same twelve seeds** at every value, and the reported figure is the mean over those seeds. Two settings are then compared seed by seed, and a difference is only called real when the paired t statistic over the twelve differences exceeds the two-sided 5 % point for eleven degrees of freedom, 2.201. The script that does this is `sweep/paired.py`, and it prints the mean paired difference, how many seeds moved each way, and the statistic.

This matters: on a single seed, `eligible_variants` appeared to show a clear gain at 2 and a clear loss at 10, and over twelve seeds neither survives — both were seed effects. Three other findings *do* survive, and they are reported in their own sections as findings against the submitted value rather than explained away.

Two of those three — the rendering pen width and the edge softness — change only the image the system scores itself on. They do not change the pen path or the G-code, so a value that scores better there is telling us something about the evaluation harness rather than about what the robot draws. That distinction is made in each section and should not be lost.

1.6 The letter prior used by the profile sweeps

The sweeps that rebuild a style profile have to supply a letter prior, and the prior they supply is not the one the deployed pipeline uses. Two things make that necessary.

`BuildAuthorProfile` declares `prior=None` as its default, but `_PriorScore` calls `prior.get(ch)` without checking, so the default path raises `AttributeError` as soon as any glyph survives filtering. The prior is therefore not optional in practice, whatever the signature says. In normal use this is never reached, because the pipeline builds a prior over every writer with `BuildLetterPrior` and passes it in.

These sweeps have one writer in hand, so the prior is pooled over that writer's own raw glyphs rather than over all eleven. A self-prior is a weaker reference than the pooled one: it measures how typical a glyph is of that writer's own instances, not of the letter in general, so it will rate a consistently badly cut letter as typical. Prior distances reported in the profile sweeps should be read as within-writer consistency, not as correctness, and they are not comparable with prior distances recorded elsewhere in the project.

1.7 Inputs used

Unless a section says otherwise, every sweep runs on one photographed page, `PersonalDataset/Owen/Pages/Owen01.jpg`, and where a single line is needed it is line 3 of that page, whose transcript is 'are increasing their fame for the right reasons.' The page has 20 transcript lines. Synthesis sweeps write the sentence 'the quick brown fox jumps over the lazy dog' in Owen's stored hand.

2 Which checkpoints the sweeps load

Several sweeps score their output with the trained networks, so this section records which weights were loaded and why. It is method, not commentary: the accuracy numbers throughout this document cannot be read without it.

2.1 The reader

`recognise.py` sets

```
DEFAULT_TEXT_WEIGHTS = WEIGHTS_DIR/
"paper_cnn_bilstm_ctc_scratch_best.npz"
```

Reading the first eight lines of the probe page with each checkpoint present on disk gives the mean character accuracy in table 1. The checkpoint the submitted code loads reads at 23.4 %; another checkpoint in the same folder reads the same crops at 98.2 %.

Table 1: Mean character accuracy over the first eight lines of the probe page, by checkpoint. All four files are in `Software/CNN/weights`.

checkpoint	character accuracy
`paper_cnn_bilstm_ctc_scratch_best.npz`	23.4 %
loaded by the submitted code	
`paper_cnn_bilstm_ctc_joint_best.npz`	**98.2 %**
`paper_cnn_bilstm_ctc_personal_best.npz`	82.0 %
`paper_cnn_bilstm_ctc_hf_best.npz`	77.0 %

For this study the sweeps that need a working reader load `paper_cnn_bilstm_ctc_joint_best.npz` and say so; otherwise every accuracy reported here would be measuring a broken checkpoint rather than the coefficient being swept.

2.2 The writer classifier

`recognise.py` sets

```
DEFAULT_AUTHOR_WEIGHTS = WEIGHTS_DIR/
"author_shape_11w_pw170_scratch_weights.npz"
```

That file is not present in `Software/CNN/weights`. Running `ClassifyImage` with the defaults therefore raises before any classification happens. The nearest checkpoint that does exist is `author_shape_10new_weights.npz`, which holds **13** writers (`150, 151, 152, 153, 384, 551, 552, 588, abhinav, dylan, owen, thiya, yeukita`), not the eleven the file name in the code implies. Every writer classification in this document uses that 13-writer checkpoint.

The file name still records the pen width the weights were trained at: `pw170` is $w_{\text{pen}} = 0.170$ x-heights, which is the value swept in section 17.1.

3 Every coefficient covered

Table 2: Every coefficient covered by this study.

key	subsystem	module	value	runs
`otsu_hi`	Image toolkit	`segment`	`1.2`	5/5
`binarize_thin`	Line binarisation	`segment`	`0.12`	5/5
`binarize_rulelen`	Line binarisation	`segment`	`1.1`	5/5
`binarize_close`	Line binarisation	`segment`	`0.045`	5/5
`speck_area`	Line binarisation	`segment`	`6`	5/5
`core_band_frac`	Line geometry	`segment`	`0.45`	5/5
`inkmask_bgradius`	Illumination normalisation	`segment`	`41`	5/5
`solid_block`	Illumination normalisation	`segment`	`0.999`	5/5
`paper_dark`	Sheet detection	`segment`	`0.45`	5/5
`axis_turn_margin`	Page orientation	`segment`	`1.35`	5/5
`peak_floor`	Line finding	`segment`	`0.06`	5/5
`centre_separation`	Line finding	`segment`	`0.6`	5/5
`line_merge_valley`	Line finding	`segment`	`1.0`	5/5
`pitch_peak_floor_xh`	Line finding	`segment`	`1.5`	5/5
`rule_flat_len`	Printed-structure removal	`segment`	`1.25`	5/5

key	subsystem	module	value	runs
rule_upright_len	Printed-structure removal	segment	1.4	5/5
seam_ink_weight	Seam cutting	segment	4.0	5/5
seam_midline_pull	Seam cutting	segment	0.012	5/5
seam_step	Seam cutting	segment	7	5/5
seam_tail_radius	Seam cutting	segment	0.07	5/5
band_ink_share	Line ownership	segment	0.15	5/5
crop_pad	Crop building	segment	0.15	5/5
darkness_gamma	Crop building	segment	0.55	5/5
crop_black_pct	Crop building	segment	40	5/5
solid_abs_pct	Crop rejection	segment	15.0	5/5
solid_median_ratio	Crop rejection	segment	3.0	5/5
min_ink_frac	Crop rejection	segment	0.12	5/5
pen_width_xh	Writer classification	recognise	0.17	5/5
pen_width_clip_hi	Writer classification	recognise	9	5/5
input_height	Network input	segment	64	5/5
input_width	Network input	segment	640	5/5
lstm_hidden	Network input	recognise	256	1/5
join_angle	Centreline tracing	style	75.0	5/5
spur_len	Centreline tracing	style	2.5	5/5
simplify_eps	Glyph extraction	style	0.6	5/5
glyph_max_w	Glyph filtering	style	2.1	5/5
shape_grid_n	Glyph filtering	style	12	5/5
cut_cost_top	Glyph extraction	style	1.15	5/5
snap_radius	Glyph extraction	style	0.5	5/5
min_cell	Glyph extraction	style	0.08	5/5
align_conf_pct	Profile building	style	25	5/5
max_variants	Profile building	style	12	5/5
join_conn_floor	Synthesis: joining	synthesise	0.45	5/5
max_join_gap	Synthesis: joining	synthesise	0.8	5/5
collision_gap	Synthesis: spacing	synthesise	0.05	5/5
drift_persistence	Synthesis: variation	synthesise	0.85	5/5
drift_clip	Synthesis: variation	synthesise	0.25	5/5
uniform_pen_width	Synthesis: rendering	synthesise	0.085	5/5
uniform_soft_frac	Synthesis: rendering	synthesise	0.3	5/5
eligible_variants	Synthesis: shape choice	synthesise	5	5/5
core_slant_cap	Synthesis: shape choice	synthesise	22.0	5/5
auto_lam_good	Synthesis: legibility	synthesise	0.24	5/5
join_strokes_mm	G-code generation	reproduce	0.6	5/5
close_ring_mm	G-code generation	reproduce	0.6	5/5
min_segment_mm	G-code generation	reproduce	0.12	5/5
pen_dwell_ms	Pen actuator	reproduce	350.0	5/5
draw_feed	Motion	reproduce	900.0	5/5
ruling_xh_frac	Page layout	reproduce	0.46	5/5
endstop_debounce	End-stops	reproduce	0.006	not swept
seam_ink_weight_dense	Seam cutting on a dense page	segment	4.0	5/5
seam_tail_radius_dense	Seam cutting on a dense page	segment	0.07	5/5
band_ink_share_dense	Seam cutting on a dense page	segment	0.15	5/5

4 Which coefficients actually matter

Table 4 ranks the coefficients by how far they moved their own headline measurement across the five values, on the fixed probe input. It is a summary of the sweeps that follow, not a separate experiment.

Three things are worth saying about it before it is read.

First, a large span is not a fault. A coefficient that controls a measured quantity, such as the x-height band, is *supposed* to move it; what matters is whether the submitted value lands in a sensible place, which is what the individual figures show.

Second, a span of zero is a real result and is reported as such. Several constants changed nothing measurable on this input. For some that is expected and is explained in their section — the line-merge valley is disabled by its own value, and the page-orientation margin is nowhere near its decision boundary on an upright page. For others it means only that this one probe does not exercise them, which is a limit of the probe, not evidence that the constant is free.

Third, where reading accuracy collapses it is almost always because the line count changed; see section 1.4.

Table 3: How far each coefficient moved its headline metric across its five values, on the fixed probe input. A span of zero means the coefficient changed nothing measurable on this input, which is reported rather than hidden.

coefficient	metric	min	submitted	max	span/submitted
draw_feed	drawSeconds	3.3	8.9	26.7	2.63
rule_upright_len	removedStraightPixels	1750	1988	4322	1.29
core_band_frac	xh	17	42	59	1.00
align_conf_pct	nLines	5	13	18	1.00
ruling_xh_frac	mmPerXh	2.1	3.864	5.078	0.77
rule_flat_len	removedStraightPixels	1988	1988	3028	0.52
min_segment_mm	lines	286	533	533	0.46
join_angle	nPolylines	66	67	92	0.39
centre_separation	nLines	14	20	20	0.30
snap_radius	nGlyphs	30	35	40	0.29
spur_len	nPolylines	60	67	69	0.13
otsu_hi	inkPixels	14850	16788	16788	0.12
peak_floor	nLines	19	20	21	0.10
uniform_soft_frac	charAccMean	0.8837	0.9302	0.9419	0.06
glyph_max_w	nChars	33	35	35	0.06
paper_dark	paperCoverage	0.901	0.947	0.954	0.06
solid_block	nLines	20	20	21	0.05
min_ink_frac	nLines	19	20	20	0.05
auto_lam_good	charAccMean	0.9302	0.9302	0.9709	0.04
uniform_pen_width	charAccMean	0.9186	0.9302	0.9457	0.03
min_cell	nGlyphs	35	35	36	0.03
collision_gap	charAccMean	0.9302	0.9302	0.9535	0.03
binarize_close	inkPixels	16401	16788	16788	0.02
join_conn_floor	charAccMean	0.9128	0.9302	0.9302	0.02
max_join_gap	charAccMean	0.9302	0.9302	0.9438	0.01
eligible_variants	charAccMean	0.9264	0.9302	0.938	0.01
crop_pad	charAcc	0.9752	0.9823	0.9826	0.01
speck_area	inkPixels	16743	16788	16802	0.00
darkness_gamma	charAcc	0.9797	0.9823	0.9823	0.00
drift_persistence	charAccMean	0.9302	0.9302	0.9322	0.00
drift_clip	charAccMean	0.9302	0.9302	0.9322	0.00
pen_width_clip_hi	trueProb	0.9985	1	1	0.00
pen_width_xh	trueProb	0.9995	1	1	0.00
binarize_thin	inkPixels	16788	16788	16788	0.00
binarize_rulelen	inkPixels	16788	16788	16788	0.00
inkmask_bgradius	nLines	20	20	20	0.00
axis_turn_margin	rotation	0	0	0	0.00

coefficient	metric	min	submitted	max	span/submitted
line_merge_valley	nLines	20	20	20	0.00
pitch_peak_floor_xh	pitch	51	51	51	0.00
seam_ink_weight	nLines	20	20	20	0.00
seam_midline_pull	nLines	20	20	20	0.00
seam_step	nLines	20	20	20	0.00
seam_tail_radius	nLines	20	20	20	0.00
band_ink_share	nLines	20	20	20	0.00
crop_black_pct	charAcc	0.9823	0.9823	0.9823	0.00
solid_abs_pct	nLines	20	20	20	0.00
solid_median_ratio	nLines	20	20	20	0.00
input_height	charAcc	0.9883	0.9883	0.9883	0.00
input_width	charAcc	0.9883	0.9883	0.9883	0.00
simplify_eps	nGlyphs	35	35	35	0.00
shape_grid_n	nChars	35	35	35	0.00
cut_cost_top	nGlyphs	35	35	35	0.00
max_variants	nChars	35	35	35	0.00
core_slant_cap	charAccMean	0.9302	0.9302	0.9302	0.00
join_strokes_mm	strokes	26	26	26	0.00
close_ring_mm	strokes	26	26	26	0.00
pen_dwell_ms	penPulses	48	48	48	0.00
seam_ink_weight_dense	nLines	26	26	26	0.00
seam_tail_radius_dense	nLines	26	26	26	0.00
band_ink_share_dense	nLines	26	26	26	0.00

5 Image toolkit

5.1 Upper edge of the histogram Otsu is built on, for a ratio image

Symbol $r_{\max}$

Value `1.2`

Source `segment.py`, `Otsu`

Swept `1.0`, `1.1`, `1.2`, `1.4`, `1.6`

Basis Otsu's between-class variance criterion [1, p. 63] is defined over a discrete histogram; the number of bins and their range are left to the implementer.

Why this value Otsu is applied twice in this project: once to 8-bit greys, where the histogram is exactly the 256 integer levels and nothing has to be chosen, and once to the normalised ratio image $g/\mathrm{BoxMean}(g)$ of `InkMask`, which is a float. Paper divided by its own local mean sits just above 1.0; ink sits well below it. The range therefore has to reach a little past unity so the paper peak is inside the histogram and not piled into the last bin, which would drag the threshold up. 1.2 places the paper peak at bin 213 of 256, leaving 43 bins above it for shading and glare.

What the coefficient does $r_{\max}$ only sets the quantisation of the search, not the criterion. Each bin is $r_{\max}/256$ wide, so the threshold is returned to a resolution of 0.0047 at $r_{\max} = 1.2$. Raising $r_{\max}$ coarsens every bin; lowering it clips the paper peak.

Alternatives considered A fixed global threshold (no criterion at all), Niblack [2] or Sauvola [3] local thresholding, which need a window and a k of their own, or clipping at the observed maximum of the ratio, which makes the bin width depend on the brightest speck of glare on the page.

Worked example On the probe page the ratio image has 99.9th percentile 1.08, so at $r_{\max} = 1.2$ every pixel lands inside the histogram and the returned threshold is a bin centre $(t+0.5)\,r_{\max}/256$.

What the sweep actually showed The threshold moves with the histogram range as expected and the submitted value recovers the most ink of the five: 14 850 ink pixels in 48 components at $r_{\max} = 1.0$, against 16 788 pixels in 41 components at 1.2. Clipping the range at unity cuts into the paper peak, the threshold is dragged down, strokes thin and break, and the component count rises by seven as a result. Above 1.2 the recovered ink falls again, slightly, as the bins coarsen. The submitted value is the best of the five on this line.

Table 4: Upper edge of the histogram Otsu is built on, for a ratio image: measured effect of each swept value. The submitted value is in bold.

$r_{\max}$	inkPixels	components	xh
1	14850	48	42
1.1	16535	43	42
1.2	**16788**	**41**	**42**
1.4	16623	43	42
1.6	16492	43	42

6 Line binarisation

6.1 Width below which a run of ink counts as thin, for printed-rule removal

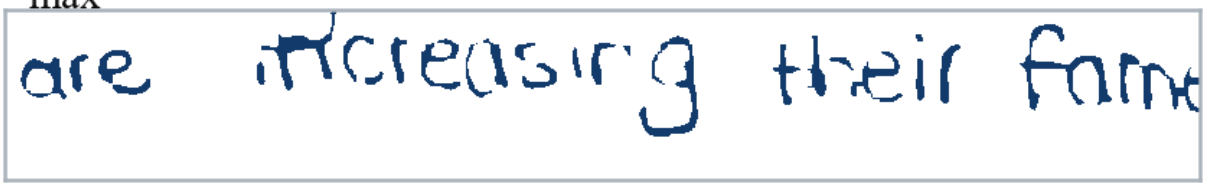
Symbol w_{thin}

Value `0.12`

Source `segment.py`, `BinarizeLine`

Swept `0.06`, `0.09`, `0.12`, `0.18`, `0.24`

$r_{\max} = 1$



ink px 14850 pieces 48 x-height 42

$r_{\max} = 1.1$



ink px 16535 pieces 43 x-height 42

$r_{\max} = 1.2$ (submitted value)



ink px 16788 pieces 41 x-height 42

$r_{\max} = 1.4$



ink px 16623 pieces 43 x-height 42

$r_{\max} = 1.6$



ink px 16492 pieces 43 x-height 42

Figure 1: Upper edge of the histogram Otsu is built on, for a ratio image. Each panel is the same input processed by the submitted code with only $r_{\max}$ changed; the panel outlined in dark blue is the submitted value.

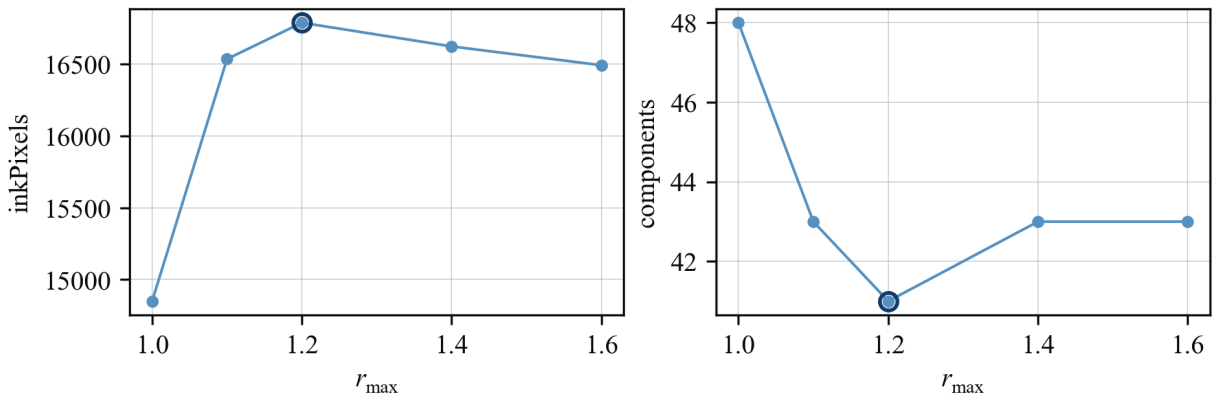


Figure 2: Upper edge of the histogram Otsu is built on, for a ratio image: measured quantities against the swept value. The circled point is the submitted value.

Basis Removing long thin runs is the standard morphological test for printed rules and table lines in document analysis [4]; the ruling on a form is the structure Shafait *et al.* [5] evaluate layout methods against.

Why this value A printed rule is the one mark on these pages that is simultaneously very long horizontally and very short vertically. The test is the conjunction of both, so the thinness limit only has to separate a printed line from a pen stroke. Measured pen strokes on the photographed pages run 4–9 px thick at a crop height of 156 px, that is 0.026–0.058 h; the printed rule is 1–3 px. $0.12h$ sits above the rule and below the thinnest real stroke, with the floor $\max(3,\cdot)$ protecting small crops.

What the coefficient does w_{thin} is a fraction of the crop height h, so it scales with how big the writing was photographed. Too small and a thick printed rule survives; too large and a horizontal pen stroke, such as the bar of a `t` or the body of an `e`, is deleted as structure.

Alternatives considered A Hough transform for straight lines [6], which finds rules at any angle but costs an accumulator per crop and needs its own peak threshold, or projection-profile rule detection, which is what `train.py` uses on whole pages in `FindRuleLineClusters`.

Worked example At $h = 156$ px, $0.12h = 18.7 \to 18$ px, and the length test is $\max(24, \lfloor 1.1h \rfloor) = 171$ px. A mark is deleted only if it is at least 171 px long and at most 18 px tall.

What the sweep actually showed **This sweep showed no effect at all**: ink pixels, component count, band and slant are identical at all five values. The reason is structural, not a failure of the sweep. By the time `BinarizeLine` sees a crop, `RemovePrintedStructure` has already deleted the page's ruling at page scale, so there is no printed rule left in the crop for the line-level test to find. On this input the coefficient is dead code. That does not make it wrong to have: it is the guard for a crop that reaches the binariser without having been through the page-level removal, which is the path `style.py` takes when it binarises a crop directly. Exercising it properly needs a crop that still contains ruling, which this probe does not provide.

Table 5: Width below which a run of ink counts as thin, for printed-rule removal: measured effect of each swept value. The submitted value is in bold.

$w_{\{}text{thin}}$	inkPixels	components	slantDeg
0.06	16788	41	-3
0.09	16788	41	-3
0.12	**16788**	**41**	**-3**
0.18	16788	41	-3
0.24	16788	41	-3

$w_{\text{thin}} = 0.06$



ink px 16788 pieces 41 slant -3

$w_{\text{thin}} = 0.09$



ink px 16788 pieces 41 slant -3

$w_{\text{thin}} = 0.12$ (submitted value)



ink px 16788 pieces 41 slant -3

$w_{\text{thin}} = 0.18$



ink px 16788 pieces 41 slant -3

$w_{\text{thin}} = 0.24$



ink px 16788 pieces 41 slant -3

Figure 3: Width below which a run of ink counts as thin, for printed-rule removal. Each panel is the same input processed by the submitted code with only w_{thin} changed; the panel outlined in dark blue is the submitted value.

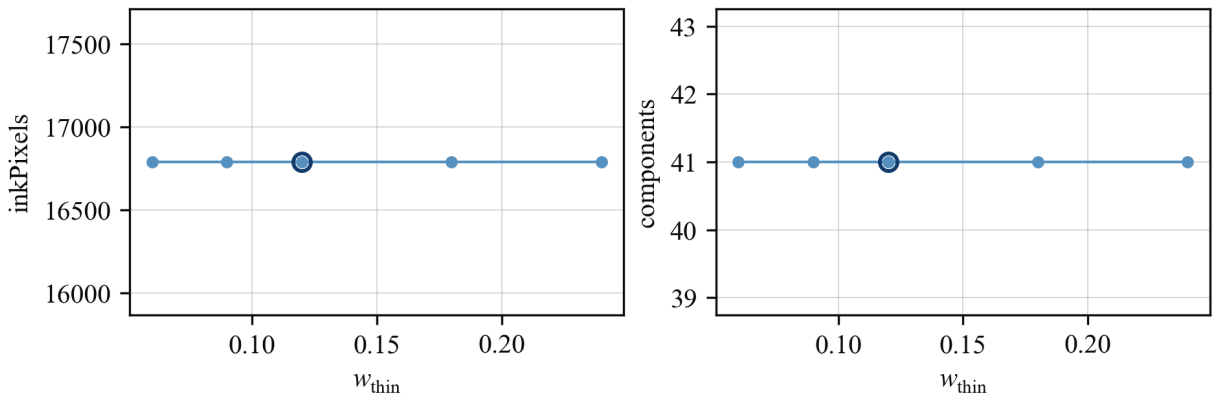


Figure 4: Width below which a run of ink counts as thin, for printed-rule removal: measured quantities against the swept value. The circled point is the submitted value.

6.2 Length above which a run of ink counts as a printed rule

Symbol $L_{\text{rule}}

Value `1.1`

Source `segment.py`, `BinarizeLine`

Swept `0.6`, `0.85`, `1.1`, `1.5`, `2.0`

Basis Same basis as the thinness test: long-run structure removal [4, 5].

Why this value The rule on a ruled page runs the full width of the writing area, which is many times the height of one line of writing. Expressing the limit in crop heights makes it scale-free: a rule is longer than the line is tall, a pen stroke is not. $1.1\,h$ is just over one line height, which no single handwritten stroke on these pages reaches horizontally while staying under $0.12\,h$ thick.

What the coefficient does Lowering L_{rule} makes the test fire on ordinary writing, because a joined cursive word can be several line heights long: only the thinness test then stands between the writing and deletion. Raising it lets a short rule segment, or a rule broken by the writing crossing it, survive into the mask.

Alternatives considered An absolute pixel length, which breaks as soon as the page is photographed at a different distance, or connected-component aspect ratio, which a long joined word also satisfies.

Worked example At $h = 156$ px the limit is 171 px, which is 11% of the 2170 px crop width, so a rule has to cross about a ninth of the writing area before it is removed.

What the sweep actually showed As with the thinness limit, **no effect was measured**: the page-level structure removal has already taken the ruling out before the crop is built, so the length test never fires on this input. The same caveat applies: the sweep bounds the coefficient's effect on this path at zero, and says nothing about the direct-binarisation path.

Table 6: Length above which a run of ink counts as a printed rule: measured effect of each swept value. The submitted value is in bold.

L_{rule}	inkPixels	components	slantDeg
0.6	16788	41	-3
0.85	16788	41	-3
1.1	**16788**	**41**	**-3**
1.5	16788	41	-3
2	16788	41	-3

$L_{\text{rule}} = 0.6$



ink px 16788 pieces 41 slant -3

$L_{\text{rule}} = 0.85$



ink px 16788 pieces 41 slant -3

$L_{\text{rule}} =$ **1.1 (submitted value)**



ink px 16788 pieces 41 slant -3

$L_{\text{rule}} = 1.5$



ink px 16788 pieces 41 slant -3

$L_{\text{rule}} = 2$



ink px 16788 pieces 41 slant -3

Figure 5: Length above which a run of ink counts as a printed rule. Each panel is the same input processed by the submitted code with only L_{rule} changed; the panel outlined in dark blue is the submitted value.

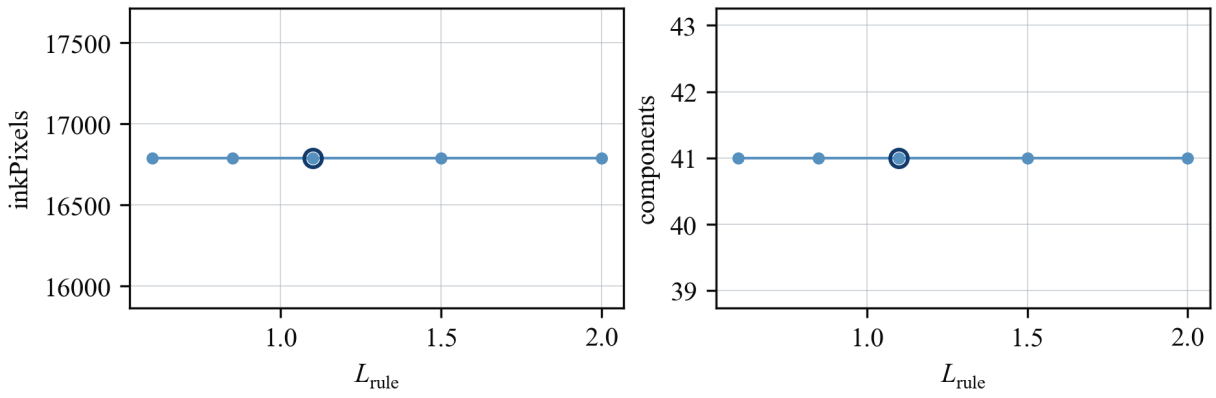


Figure 6: Length above which a run of ink counts as a printed rule: measured quantities against the swept value. The circled point is the submitted value.

6.3 Closing radius that bridges pinholes in a stroke

Symbol k_{close}

Value `0.045`

Source `segment.py`, `BinarizeLine`

Swept `0.0`, `0.02`, `0.045`, `0.08`, `0.12`

Basis Morphological closing, dilation followed by erosion by the same element, is the classical repair for small holes and gaps and does not grow the shape overall [4].

Why this value A ballpoint on ordinary paper skips, so a single stroke binarises into a chain of blobs. Those breaks matter twice over: they split one component into several, and they put spurious endpoints into the skeleton, which the tracer then follows as separate strokes. The radius has to close a skip without fattening the stroke, so it is tied to the crop height and then hard-clipped to 2–4 px: the clip, not the fraction, is what finally bounds the damage.

What the coefficient does Because of the clip$(\cdot, 2, 4)$ the coefficient only has an effect between $h = 44$ and $h = 89$ px; outside that range the clip decides. On the probe crop ($h = 156$) every value from 0.02 upward gives $k = 4$, which is why the sweep includes 0: that is the only setting that changes the picture, and it shows what closing is buying.

Alternatives considered Not closing at all and letting the tracer rejoin the pieces at junctions, which `_JoinAtJunctions` does anyway; or a larger closing, which merges adjacent letters and destroys the character boundaries the glyph cutter depends on.

Worked example At $h = 156$ px, $0.045\,h = 7.0$, clipped to 4, so a 4×4 closing is applied: gaps up to about 4 px of white inside a stroke are filled.

What the sweep actually showed Two distinct outcomes appear across five values, not five, and the reason is the quantisation described above. At $h = 156$ px the kernel is $k = \text{clip}(\text{round}(c\,h), 2, 4)$ and the dilation radius is $k//2$: $c = 0$ gives $k = 2$ and $c = 0.02$ gives $k = 3$, both of which are radius 1, while $c \geq 0.045$ gives $k = 4$, radius 2. So the sweep really compares radius 1 with radius 2, and the difference is clear: radius 1 leaves 16401 ink pixels in 47 components and radius 2 leaves 16788 in 41. Closing at the submitted value merges six broken pieces back into their strokes and adds 387 pixels of ink, which is what it is for.

$k_{\text{close}} = 0$



ink px 16401 pieces 47 x-height 42

$k_{\text{close}} = 0.02$



ink px 16401 pieces 47 x-height 42

$k_{\text{close}} = \mathbf{0.045}$ **(submitted value)**



ink px 16788 pieces 41 x-height 42

$k_{\text{close}} = 0.08$



ink px 16788 pieces 41 x-height 42

$k_{\text{close}} = 0.12$



ink px 16788 pieces 41 x-height 42

Figure 7: Closing radius that bridges pinholes in a stroke. Each panel is the same input processed by the submitted code with only k_{close} changed; the panel outlined in dark blue is the submitted value.

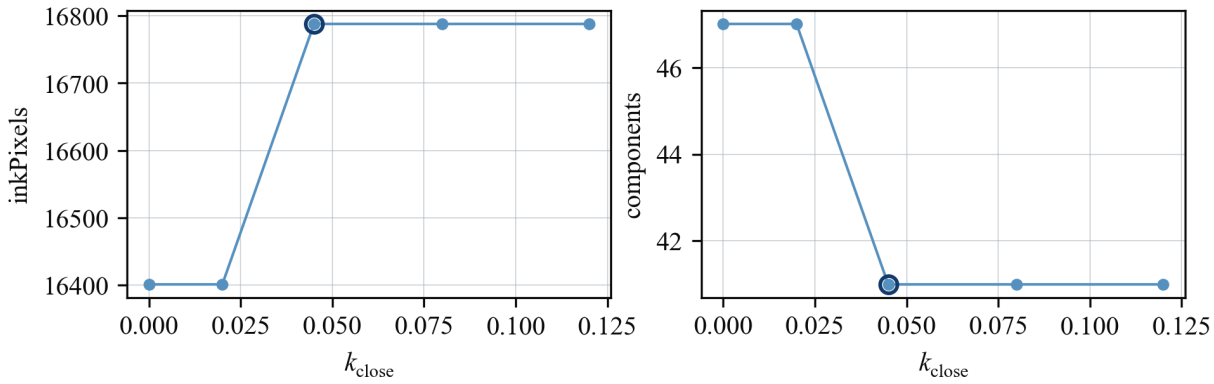


Figure 8: Closing radius that bridges pinholes in a stroke: measured quantities against the swept value. The circled point is the submitted value.

Table 7: Closing radius that bridges pinholes in a stroke: measured effect of each swept value. The submitted value is in bold.

$k_{\{}text{close}}$	inkPixels	components	xh
0	16401	47	42
0.02	16401	47	42
0.045	**16788**	**41**	**42**
0.08	16788	41	42
0.12	16788	41	42

6.4 Smallest component kept in a line mask

Symbol $A_{\min}$

Value `6`

Source `segment.py`, `BinarizeLine`

Swept `1`, `3`, `6`, `12`, `24`

Basis Area opening, discarding components below a size, is the standard speck filter after thresholding [4]; components are labelled by the two-pass union-find of Rosenfeld and Pfaltz [7] with path compression after Tarjan [8].

Why this value Thresholding a photograph leaves single-pixel noise from sensor grain and paper texture. The smallest mark that must survive is the dot of an `i` or a full stop, which on these crops is 20–60 px. $A_{\min} = 6$ is an order of magnitude below the smallest real mark and an order above single-pixel grain, so it is deliberately not a tuned value: it sits in the gap.

What the coefficient does Raising it eventually eats punctuation and `i` dots, which the glyph cutter then has to recover separately in `ExtractLineGlyphs`. Lowering it to 1 keeps every speck, which inflates the component count the ownership stage has to reason about.

Alternatives considered A median or morphological opening before labelling, which also rounds the stroke ends, or keeping everything and letting the later size tests in `_BandOwnership` reject it, which is what happens for anything above 6 px anyway.

Worked example On the probe crop the mask holds 41 components at $A_{\min} = 6$; the sweep records how that count moves with the limit.

What the sweep actually showed The component count falls monotonically, 46, 44, 41, 40, 38, while the ink pixel count barely moves, 16 802 down to 16 743. That is the signature of a speck filter working as intended: it is removing many tiny pieces and almost no ink. Between $A_{\min} = 1$ and the submitted 6, five components disappear for the loss of 14 pixels. Raising it to 24 removes three more components and 45 more pixels, and at that point it is beginning to reach marks large enough to be punctuation.

Table 8: Smallest component kept in a line mask: measured effect of each swept value. The submitted value is in bold.

$A_{\{}min}$	inkPixels	components
1	16802	46
3	16799	44
6	**16788**	**41**
12	16780	40
24	16743	38

7 Line geometry

7.1 Profile height defining the x-height band

Symbol τ_{band}

Value `0.45`

Source `segment.py`, `CoreBand`

Swept `0.25`, `0.35`, `0.45`, `0.6`, `0.75`

Basis The horizontal projection profile of a text line peaks across the x-height body and falls away over the ascender and descender zones; this is the basis of profile-based line and zone finding [9], and the same profile drives the IAM baseline work [10].

Why this value Every shape in a style profile is stored in x-heights, so the x-height is the one length the whole synthesis side depends on. It is measured as the band of rows carrying at least τ_{band} of the peak ink-per-row. The body of the writing contributes nearly every row of ink; ascenders and descenders contribute only the few columns that have them. The profile therefore has a plateau and a skirt, and τ_{band} has to cut the skirt. At 0.45 the cut is roughly half the plateau height, which is above the ascender skirt on all eleven writers and below the plateau edge on all of them.

What the coefficient does This coefficient directly scales every stored glyph. If τ_{band} is too low the band swallows the ascender zone, the measured x-height grows, and every glyph is stored too small, so the writing is later drawn too large for its ruling. Too high and the band collapses onto the few darkest rows. The guard `base-top>=3` rejects a collapsed band outright rather than returning a bad number.

Alternatives considered Fitting four baselines explicitly, as Vinciarelli and Luettin do [11], which is more faithful but needs its own model fit per line; or taking the median component height, which `_LetterHeight` does for line pitch but which mixes ascenders into the estimate.

Worked example On the probe crop the band is rows 36–78, giving $x_h = 42$ px at a crop height of 156 px, so the x-height is 27% of the crop and the ascender and descender zones take the rest.

What the sweep actually showed The measured x-height moves by a factor of 3.5 across the sweep: 59, 54, 42, 24, 17 px, while the ink mask itself is byte-for-byte identical at every value, since this coefficient only reads the profile. The figure shows the consequence directly: at 0.25 the band has swallowed the ascender zone and at 0.75 it has collapsed onto the densest rows of the body. The submitted value is the only one of the five whose band visibly hugs the x-height body, which is what the measurement is supposed to mean. Because every stored glyph is divided by this number, the 3.5× spread here is a 3.5× spread in the stored size of every letter.

$A_{\min} = 1$



ink px 16802 pieces 46

$A_{\min} = 3$



ink px 16799 pieces 44

$A_{\min} = 6$ (submitted value)



ink px 16788 pieces 41

$A_{\min} = 12$



ink px 16780 pieces 40

$A_{\min} = 24$



ink px 16743 pieces 38

Figure 9: Smallest component kept in a line mask. Each panel is the same input processed by the submitted code with only $A_{\min}$ changed; the panel outlined in dark blue is the submitted value.

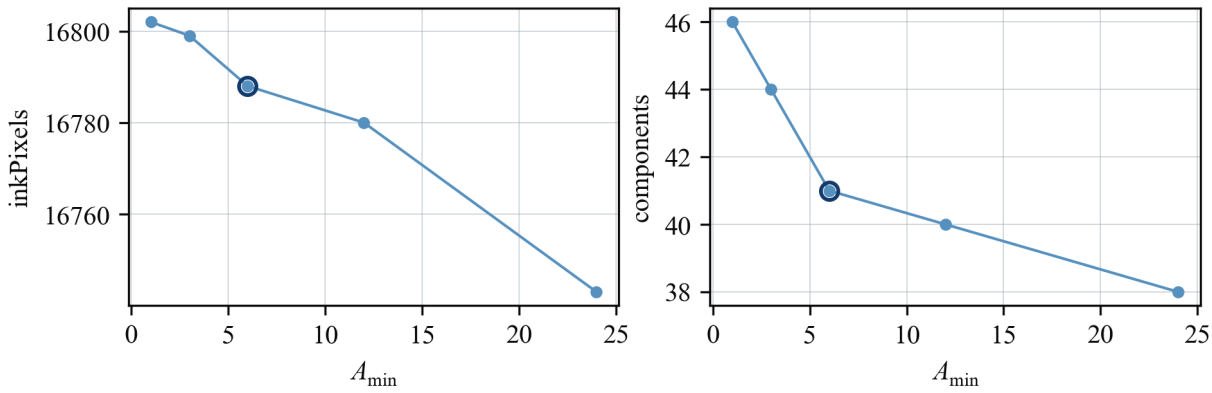


Figure 10: Smallest component kept in a line mask: measured quantities against the swept value. The circled point is the submitted value.

Table 9: Profile height defining the x-height band: measured effect of each swept value. The submitted value is in bold.

$\{}tau_{\{}text{band}}$	xh	coreBand
0.25	59	[22, 81]
0.35	54	[25, 79]
0.45	**42**	**[36, 78]**
0.6	24	[40, 64]
0.75	17	[41, 58]

8 Illumination normalisation

8.1 Local background radius for the ink mask

Symbol r_{bg}

Value `41`

Source `segment.py`, `InkMask`

Swept `15`, `25`, `41`, `70`, `120`

Basis Dividing by a local mean computed from an integral image is Bradley and Roth's adaptive threshold [12]; the constant-time rectangle sum it needs is Crow's summed-area table [13].

Why this value A phone photograph of a page is unevenly lit, so no single grey level separates ink from paper across the whole sheet. Each pixel is therefore compared with the mean of a window around it. The window has to be much larger than a stroke, or it averages the ink into its own background and the stroke centre is lost, and much smaller than the illumination gradient, or it stops tracking the shading. Tying it to $H/41$ makes it track how large the writing was photographed rather than the sensor resolution, and the clip$(8, 40)$ keeps it inside the range where both conditions hold.

What the coefficient does At $H/41$ the window is about three x-heights across, so a stroke occupies a few per cent of it. The clip means the coefficient only bites for page heights between 328 and 1640 px at half resolution; the probe page is 1632 px tall at half size, so it sits at the very top of that range and the sweep moves the radius over most of its useful span.

Alternatives considered Sauvola's local mean and standard deviation [3] or Niblack's [2], both of which add a sensitivity constant; a morphological top-hat, which needs a structuring element of the same scale; or global Otsu alone, which fails outright on a shadowed corner.

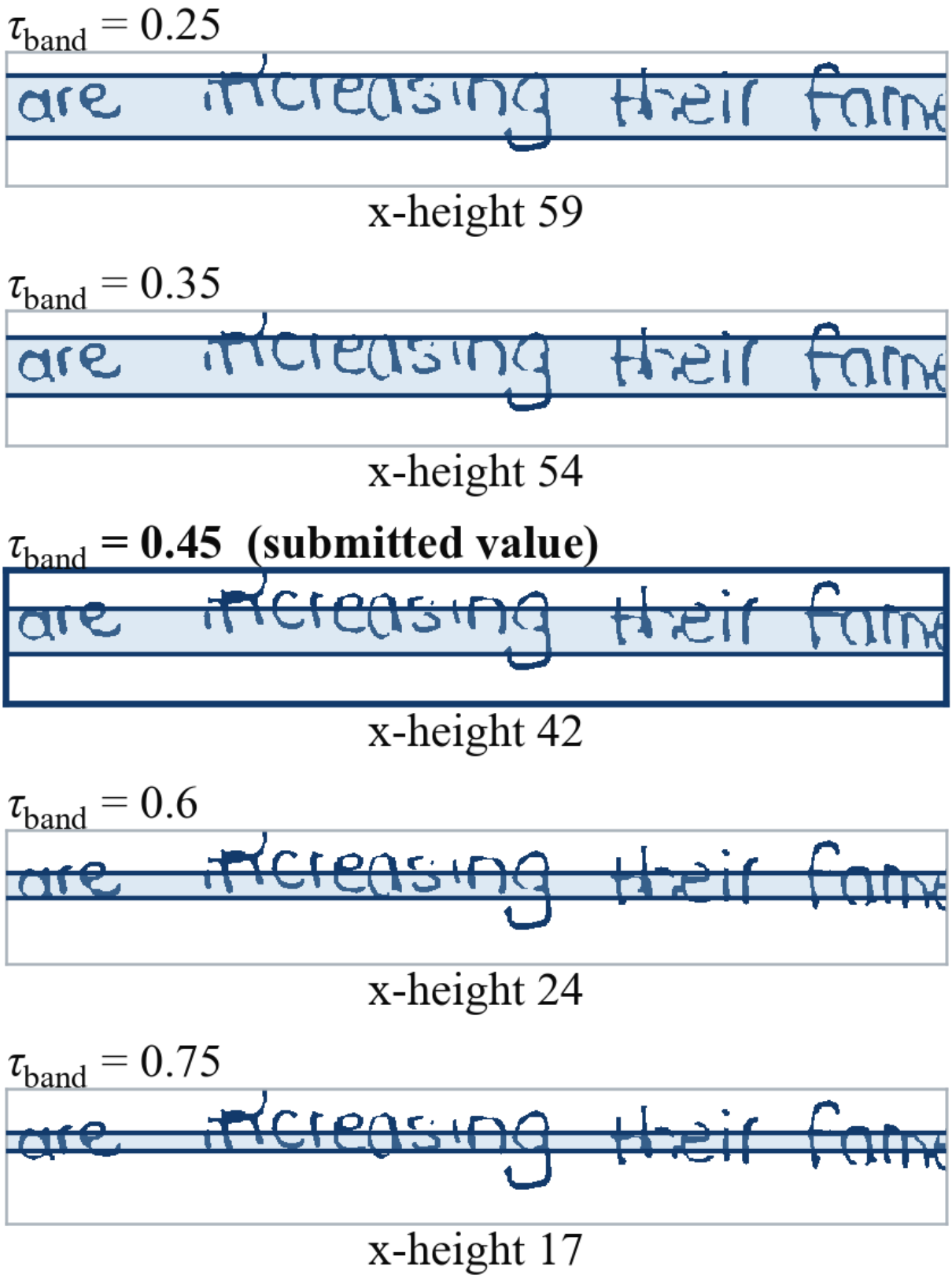


Figure 11: Profile height defining the x-height band. Each panel is the same input processed by the submitted code with only τ_{band} changed; the panel outlined in dark blue is the submitted value.

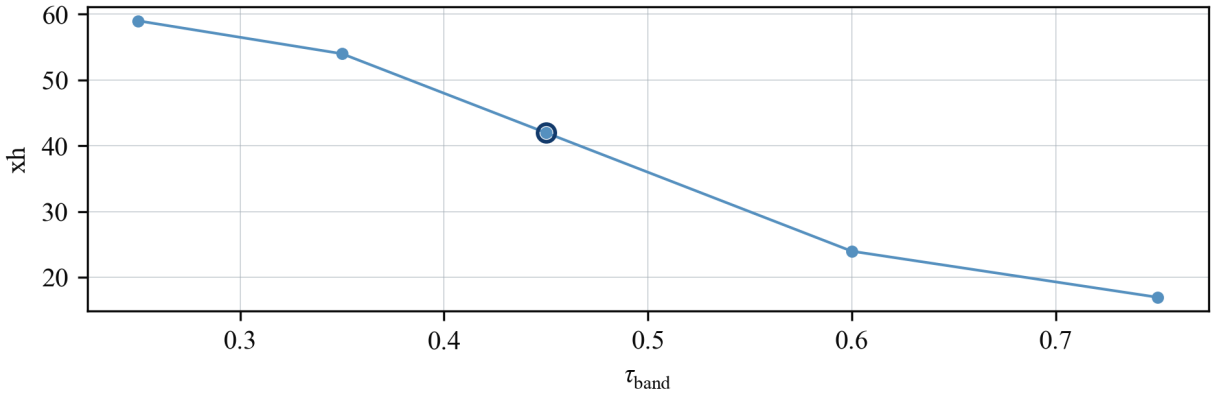


Figure 12: Profile height defining the x-height band: measured quantities against the swept value. The circled point is the submitted value.

Worked example The probe page is 3264 px tall, halved to 1632, so $1632/41 = 39.8 \to 40$, exactly at the clip. The window is 81×81 px against an x-height of about 21 px at half resolution, that is 3.9 x-heights wide.

What the sweep actually showed The first three values are identical, because the clip$(8, 40)$ holds the radius at 40 px for all of them on a page this size: $1632/41 = 39.8$, $1632/25 = 65$ and $1632/15 = 109$ all clip to 40. The sweep therefore only moves the radius downward, at 70 and 120, where it becomes 23 and 14 px. The effect is immediate: straight-structure removal jumps from 1 988 pixels to 6 114 and 6 808, and the estimated line pitch shifts from 51 to 50 px. A background window that small starts to average the stroke into its own background, the mask degrades, and the long-run tests then fire on parts of the writing. Reading accuracy falls from 0.982 to 0.975. The submitted value is at the top of its clip and is the safe end.

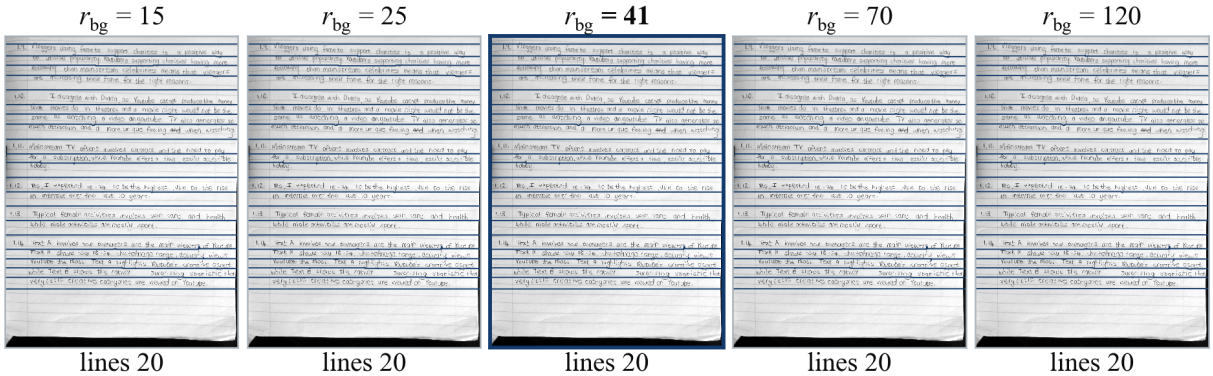


Figure 13: Local background radius for the ink mask. Each panel is the same input processed by the submitted code with only r_{bg} changed; the panel outlined in dark blue is the submitted value.

8.2 Window fill above which a solid block is ruled out as writing

Symbol	ϕ_{solid}
Value	`0.999`
Source	`segment.py`, `InkMask`
Swept	`0.9`, `0.97`, `0.999`, `1.0`, `1.01`

Basis Removing large filled regions before layout analysis is standard pre-processing for photographed documents [5].

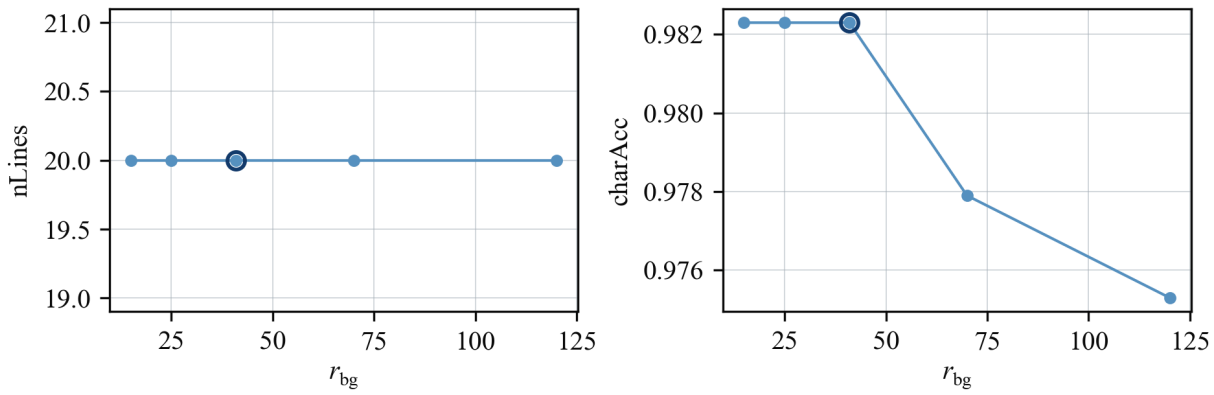


Figure 14: Local background radius for the ink mask: measured quantities against the swept value. The circled point is the submitted value.

Table 10: Local background radius for the ink mask: measured effect of each swept value. The submitted value is in bold.

$r_{\{}text{bg}\}$	nLines	charAcc	pitch	artifactPieces
15	20	0.9823	51	33
25	20	0.9823	51	33
41	**20**	**0.9823**	**51**	**33**
70	20	0.9779	50	37
120	20	0.9753	50	26

Why this value Handwriting is thin. A 9×9 window that is completely full of *mask* is not a pen stroke at that resolution; it is a shadow, a dark object in the photograph, or the edge of the sheet. The test is deliberately set at *completely* full rather than merely dark, because the cost of deleting real writing is far higher than the cost of leaving a shadow for the later artefact tests to catch. 0.999 is the floating-point way of writing 'all 81 pixels', since $80/81 = 0.9877$ already fails it.

What the coefficient does The value is effectively a Boolean, not a tuned fraction: anything in $(0.9877, 1.0]$ behaves identically. The sweep makes that explicit by including 0.90 and 0.97, which do admit partially filled windows, and 1.01, which can never fire and so disables the test.

Alternatives considered An area-based test on labelled components, which is what `DropArtifactPieces` applies later at page scale, or a stroke-width transform, which would measure thickness directly but costs a distance transform per page.

Worked example BoxMean with $r = 4$ averages $(2 \cdot 4 + 1)^2 = 81$ pixels, so the possible values below 1 are $k/81$; the largest is $80/81 = 0.9877 < 0.999$.

What the sweep actually showed This sweep found the real boundary, and it is exactly where the arithmetic said it would be. At $\phi_{\text{solid}} = 1.0$ and 1.01 the test can never fire — no window mean strictly exceeds 1.0 — and the ink lying outside the detected sheet jumps from 4 726 pixels to 54 959, a factor of 11.6. Below the submitted value the test fires on partially filled windows too, and the off-sheet ink falls to 2 893 at 0.90, because more of the shadow is being classed as not-writing earlier. The submitted 0.999 is the largest value that still means 'all 81 pixels', which is the intended rule.

9 Sheet detection

9.1 Fraction of the median brightness below which a block is off-sheet

Symbol τ_{paper}

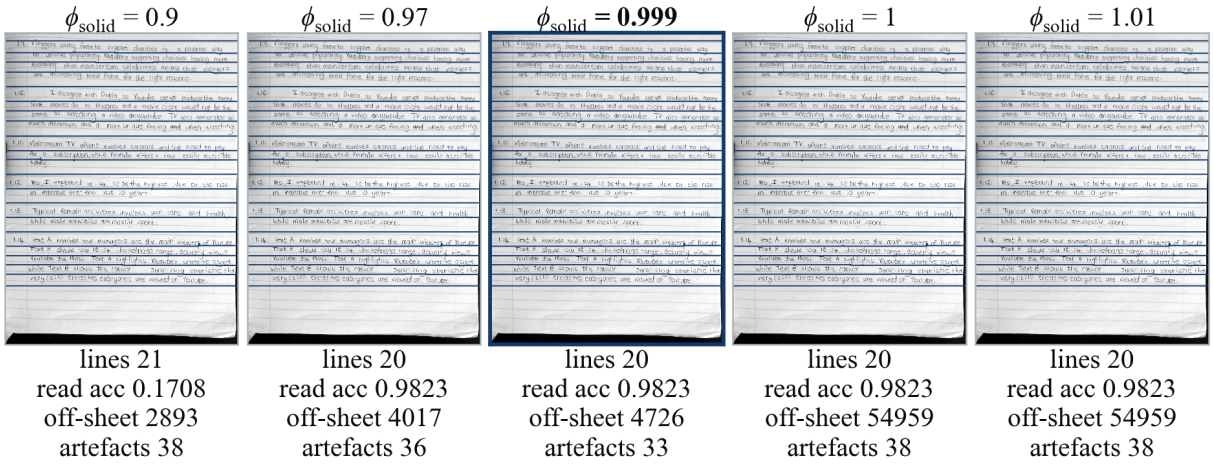


Figure 15: Window fill above which a solid block is ruled out as writing. Each panel is the same input processed by the submitted code with only ϕ_{solid} changed; the panel outlined in dark blue is the submitted value.

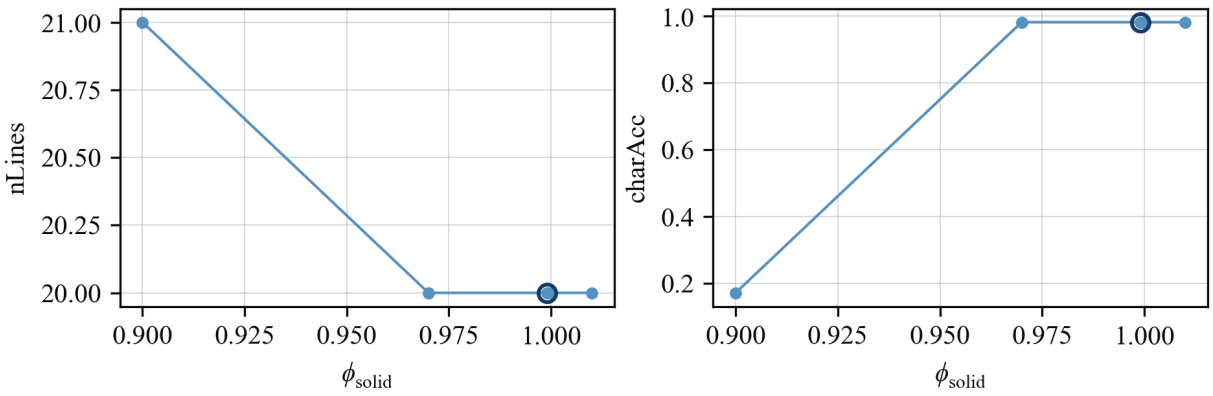


Figure 16: Window fill above which a solid block is ruled out as writing: measured quantities against the swept value. The circled point is the submitted value.

Table 11: Window fill above which a solid block is ruled out as writing: measured effect of each swept value. The submitted value is in bold.

$\{}phi_{\{}text{solid}}$	nLines	charAcc	inkOutsidePaper	artifactPieces
0.9	21	0.1708	2893	38
0.97	20	0.9823	4017	36
0.999	**20**	**0.9823**	**4726**	**33**
1	20	0.9823	54959	38
1.01	20	0.9823	54959	38

Value `0.45`

Source `segment.py`, `PaperRegionMask`

Swept `0.25`, `0.35`, `0.45`, `0.6`, `0.75`

Basis Finding the page as the largest bright connected region of a block-averaged image follows the standard camera-document pre-processing pipeline [5]; the labelling is [7].

Why this value The sheet is the brightest large object in the photograph and the desk around it is darker. Comparing against the *median* of the block-averaged image rather than a fixed grey makes the test independent of exposure: the median of a photograph that is mostly paper is the paper level. A desk, a shadow or the edge of the frame is well under half the paper level, while shading on the sheet itself rarely falls below 0.6 of it, so 0.45 separates the two with margin on both sides.

What the coefficient does The test runs on a 4×4 block average, so it is a judgement about regions, not pixels, and the writing itself does not drag a block under the limit. Raising τ_{paper} eats into the sheet and can clip a whole margin of writing; lowering it lets the desk in, and the largest bright blob then spans the sheet and its surroundings.

Alternatives considered Edge detection and a quadrilateral fit [14] to find the sheet corners, which is more precise but fails when a corner is out of frame; or assuming the whole frame is paper, which the guard `dark.mean()<0.002` already falls back to when almost nothing is dark.

Worked example On the probe page the sheet covers 94.7% of the frame at the submitted setting and 4726 mask pixels lie outside it and are removed.

What the sweep actually showed The detected sheet shrinks steadily as the limit rises — coverage 0.954, 0.950, 0.947, 0.943, 0.901 — and the last step is the telling one. At $\tau_{\text{paper}} = 0.75$ the sheet has lost 4.6 percentage points of the frame, the ink falling outside it jumps from 4726 to 12615 pixels, and one line of writing is lost entirely (19 lines found against 20 in the transcript). That is the failure the coefficient has to avoid: shading on the sheet itself being read as off-sheet, taking a margin of real writing with it. The submitted value sits two steps away from that cliff.

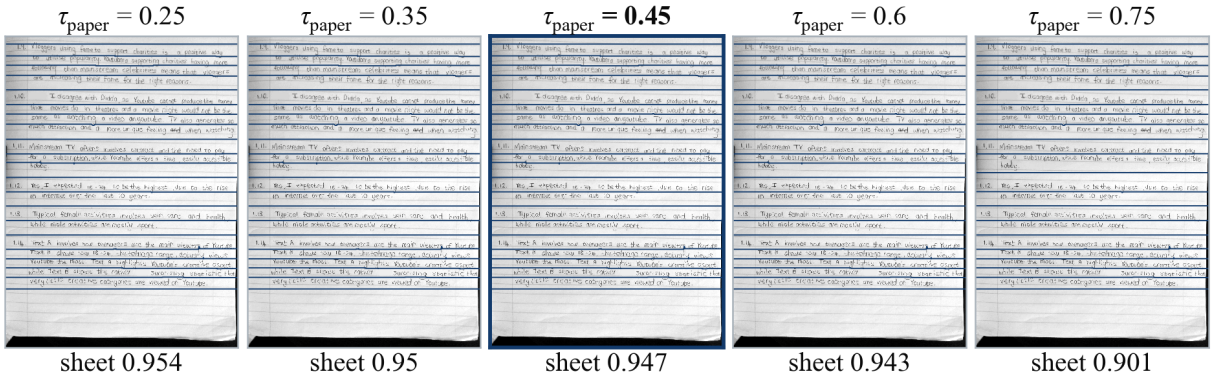


Figure 17: Fraction of the median brightness below which a block is off-sheet. Each panel is the same input processed by the submitted code with only τ_{paper} changed; the panel outlined in dark blue is the submitted value.

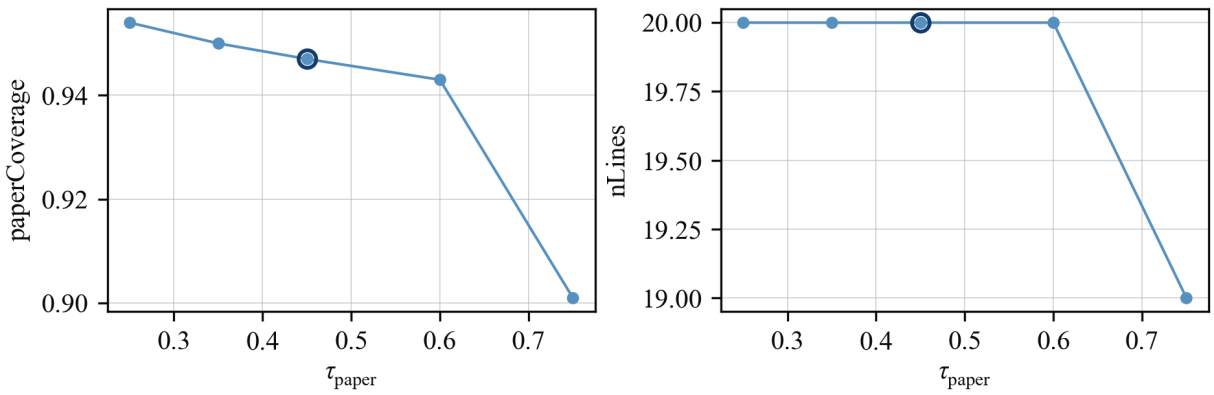


Figure 18: Fraction of the median brightness below which a block is off-sheet: measured quantities against the swept value. The circled point is the submitted value.

Table 12: Fraction of the median brightness below which a block is off-sheet: measured effect of each swept value. The submitted value is in bold.

$\{}tau_{\{}text{paper}}$	paperCoverage	inkOutsidePaper	nLines	charAcc
0.25	0.954	4037	20	0.9823
0.35	0.95	4508	20	0.9823
0.45	**0.947**	**4726**	**20**	**0.9823**
0.6	0.943	5095	20	0.9823
0.75	0.901	12615	19	0.9284

10 Page orientation

10.1 How much better the sideways reading must be before the page is turned

Symbol m_{axis}

Value `1.35`

Source `segment.py`, `UprightPage`

Swept `1.0`, `1.15`, `1.35`, `1.6`, `2.0`

Basis Lines of text produce a strongly periodic projection profile along one axis only, which is the signal every profile-based layout method uses [9, 5].

Why this value The score is the fraction of rows that are nearly empty, which is high across the gaps between lines and low along them. Both orientations are scored and the page is turned only if the sideways score is clearly better. A plain ‘greater than’ would turn a page whenever noise put the two scores a hair apart, and turning a page wrongly costs the whole job, while failing to turn one the user photographed correctly costs nothing. The margin is therefore asymmetric on purpose, and 1.35 is far outside the measured spread between the two scores on a correctly held page.

What the coefficient does m_{axis} is a ratio, so it is unaffected by how much ink is on the page. At 1.0 the test becomes a bare comparison; above the true ratio of a genuinely sideways page it can never fire and a rotated page is never corrected.

Worked example On the probe page the two scores are $s_0 = 0.470$ and $s_{90} = 0.075$, a ratio of 0.16 in favour of leaving it alone, so the page is not turned at any setting in the sweep. The decision has 8.4× margin against the 1.35 limit.

What the sweep actually showed **No effect was measured**, as predicted. The two axis scores on this page are $s_0 = 0.470$ and $s_{90} = 0.075$, a ratio of 0.16 in favour of leaving the page alone, so the page is not turned even at $m_{\text{axis}} = 1.0$ where the test is a bare comparison. The decision has 8.4× margin against the submitted limit. This sweep demonstrates the stability of the decision, not the turning; showing the turn would need a page photographed sideways.

Limits of this test The probe page is upright, so this sweep demonstrates the stability of the decision rather than the turning itself; the recorded scores show how far the page is from the limit.

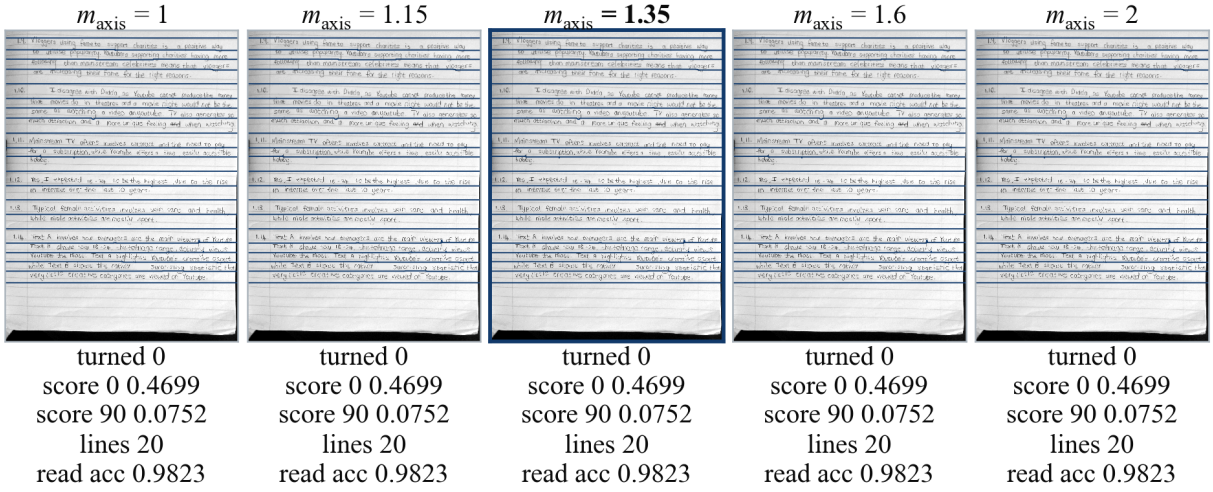


Figure 19: How much better the sideways reading must be before the page is turned. Each panel is the same input processed by the submitted code with only m_{axis} changed; the panel outlined in dark blue is the submitted value.

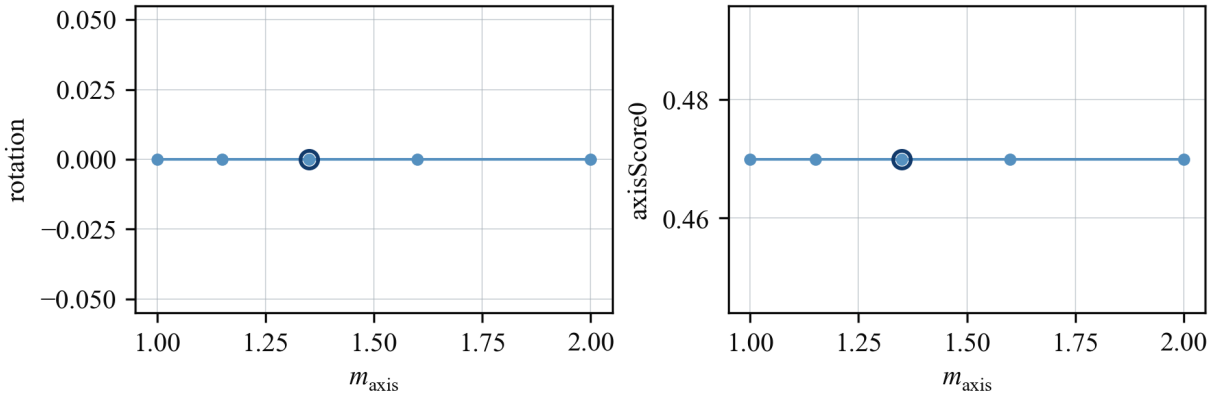


Figure 20: How much better the sideways reading must be before the page is turned: measured quantities against the swept value. The circled point is the submitted value.

11 Line finding

11.1 Profile height a candidate line peak must reach

Symbol τ_{peak}

Value `0.06`

Source `segment.py`, `LineCentres`

Swept `0.02`, `0.04`, `0.06`, `0.12`, `0.2`

Table 13: How much better the sideways reading must be before the page is turned: measured effect of each swept value. The submitted value is in bold.

$m_{\{}text{axis}}$	rotation	axisScore0	axisScore90	nLines	charAcc
1	0	0.4699	0.0752	20	0.9823
1.15	0	0.4699	0.0752	20	0.9823
1.35	**0**	**0.4699**	**0.0752**	**20**	**0.9823**
1.6	0	0.4699	0.0752	20	0.9823
2	0	0.4699	0.0752	20	0.9823

Basis Peak picking on a smoothed horizontal projection profile is the classical line-finding method [9]; the profile and its use on handwritten pages follow [10].

Why this value Every local maximum of the smoothed ink-per-row profile is a candidate line centre, but a short line, a stray mark or the tail of a descender also makes a bump. The floor is relative to the tallest peak on the page, so it does not depend on how much was written. It has to sit below the weakest real line, which on these pages is a one- or two-word last line carrying roughly a tenth of the ink of a full line, and above the texture left after smoothing. 0.06 is below that weakest-line level with margin.

What the coefficient does This floor controls how many candidates reach the separation test that follows; it cannot by itself create a line, because the $0.6P$ separation rule then thins the candidates out. Raising it drops genuine short lines outright, which is unrecoverable: there is no later stage that puts a line back.

Alternatives considered Prominence-based peak picking, which `train.py` does implement in `FindPeaks` with an explicit prominence and distance, or fitting a fixed pitch and taking every period, which fails on a page whose last line is short.

Worked example The probe page has 20 transcript lines. The sweep records how many the segmenter returns at each floor, against that known count.

What the sweep actually showed This is the clearest result in the segmentation group. Below the submitted value the page over-segments — 21 lines at both 0.02 and 0.04 against 20 in the transcript — and above it the page under-segments, 19 lines at both 0.12 and 0.20. The submitted 0.06 is the only one of the five that recovers exactly the right number of lines. The reading score shows the over-segmentation case costing far more than the under-segmentation one, 0.173 against 0.982, because an extra line early on shifts every later crop against the transcript.

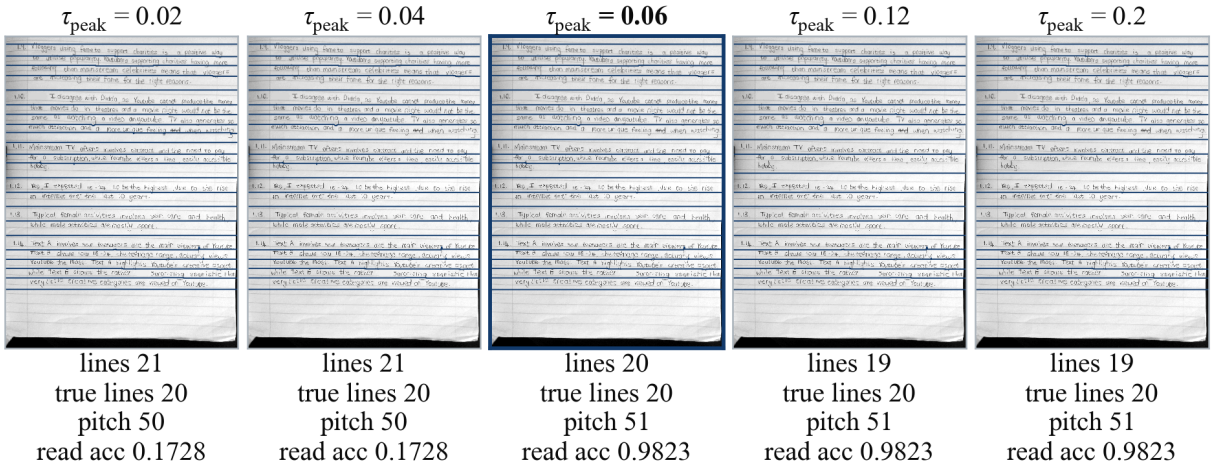


Figure 21: Profile height a candidate line peak must reach. Each panel is the same input processed by the submitted code with only τ_{peak} changed; the panel outlined in dark blue is the submitted value.

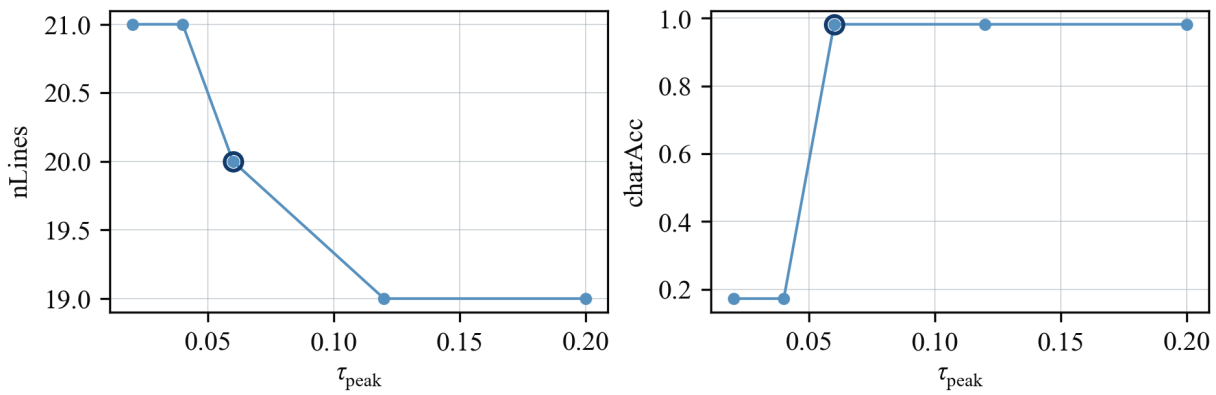


Figure 22: Profile height a candidate line peak must reach: measured quantities against the swept value. The circled point is the submitted value.

Table 14: Profile height a candidate line peak must reach: measured effect of each swept value. The submitted value is in bold.

$\{}tau_{\{}text{peak}}$	nLines	nLinesTruth	pitch	charAcc
0.02	21	20	50	0.1728
0.04	21	20	50	0.1728
0.06	**20**	**20**	**51**	**0.9823**
0.12	19	20	51	0.9823
0.2	19	20	51	0.9823

11.2 Smallest spacing between two accepted line centres

Symbol $s_{\min}$

Value `0.6`

Source `segment.py`, `LineCentres`

Swept `0.3`, `0.45`, `0.6`, `0.8`, `1.0`

Basis Enforcing a minimum distance between accepted peaks is the usual non-maximum suppression step of profile line finding [9].

Why this value Candidates are taken strongest first and a new one is accepted only if it is at least $s_{\min}$ pitches from every centre already kept. The limit is expressed in measured line pitch P, not pixels, so it follows the page. Two real lines are one pitch apart by definition, so anything below 1.0 admits them both; the profile of a single line, however, is double-humped when the writing has many ascenders, and those humps sit a few tenths of a pitch apart. 0.6 is above that internal structure and safely below the one-pitch spacing of real lines.

What the coefficient does Taken with the peak floor this is the whole line-count decision. Too small and one line is reported as two, which splits words across crops; too large and two closely written lines are merged into one crop, which the reader then cannot transcribe. Because candidates are sorted by strength, the rule keeps the strongest of a cluster rather than the first.

Alternatives considered Merging on the depth of the valley between two peaks, which this code also implements as `LINE_MERGE_VALLEY` and leaves disabled, or a fixed pitch comb.

Worked example On the probe page $P = 51$ px at half resolution, so $0.6P = 30.6$ px: two centres closer than 31 px are treated as one line.

What the sweep actually showed The decisive value is the largest. At $s_{\min} = 1.0$ the segmenter returns **14** lines where the transcript has 20: requiring a full pitch between centres merges closely written lines, six of them on this page. The reading score falls to 0.647 accordingly, which as section 1.4 explains is mostly the crop-to-transcript pairing breaking rather than the crops becoming unreadable. Between 0.3 and 0.8 the line count is correct at 20 and nothing else moves, so on this page the submitted value sits in the middle of a wide safe plateau rather than on a knife edge.

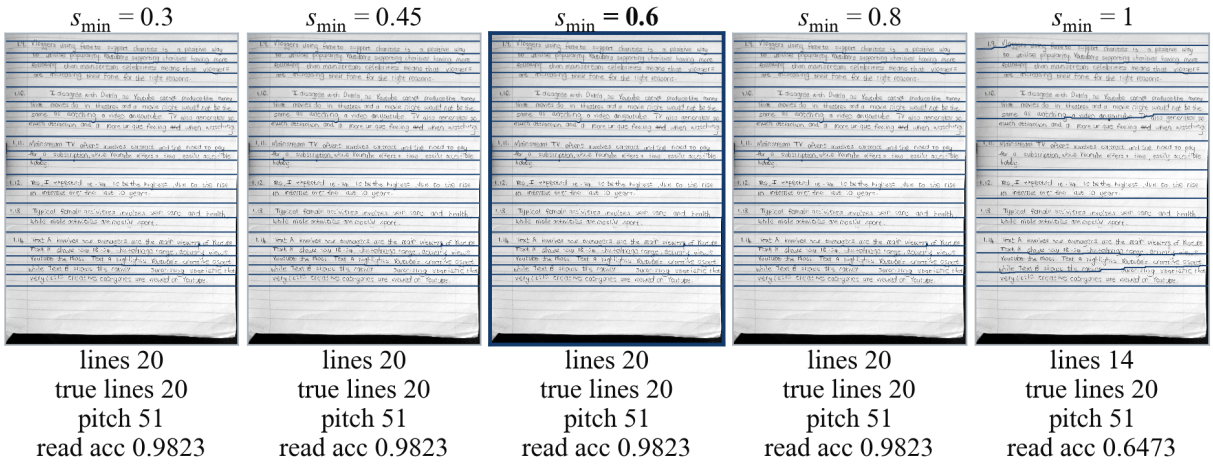


Figure 23: Smallest spacing between two accepted line centres. Each panel is the same input processed by the submitted code with only $s_{\min}$ changed; the panel outlined in dark blue is the submitted value.

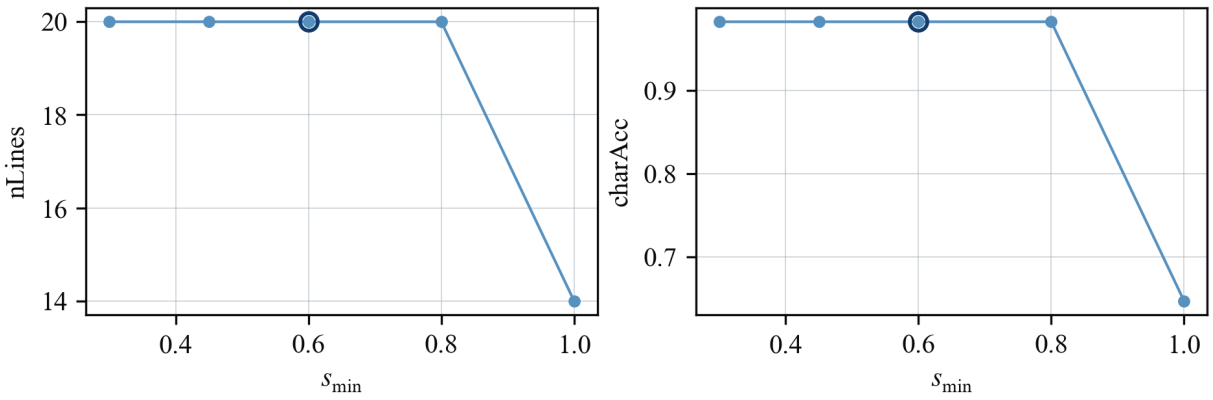


Figure 24: Smallest spacing between two accepted line centres: measured quantities against the swept value. The circled point is the submitted value.

11.3 Valley depth below which two line centres are merged

Symbol τ_{valley}

Value `1.0`

Source `segment.py`, `LineCentres`

Swept `0.3`, `0.5`, `0.7`, `0.9`, `1.0`

Basis Merging profile peaks separated only by a shallow valley is the standard refinement where projection peaks over-segment [9].

Table 15: Smallest spacing between two accepted line centres: measured effect of each swept value. The submitted value is in bold.

$s_{\{}min}$	nLines	nLinesTruth	pitch	charAcc
0.3	20	20	51	0.9823
0.45	20	20	51	0.9823
0.6	**20**	**20**	**51**	**0.9823**
0.8	20	20	51	0.9823
1	14	20	51	0.6473

Why this value This is a second, optional merge: for each adjacent pair of centres the lowest profile value between them is compared with the smaller of the two peaks, and the pair is merged if the dip is shallow. The submitted value is 1.0, and the loop is guarded by `thr<1.0`, so **the merge is switched off in the submitted system**. That is the honest description of this coefficient: it is a disabled mechanism, kept because `SegmentLines` exposes `mergeValley` as an argument so a difficult page can be re-run with it on.

What the coefficient does The value is a ratio of ink depths, so 1.0 means 'merge even when the valley is as high as the peak', which is why it is used as the off switch rather than a large number. Lowering it to 0.7 merges only genuinely shallow dips.

Alternatives considered Leaving over-segmentation to the ownership stage, which is what happens now, or topological persistence on the profile, which is the same idea with a cleaner definition.

Worked example With $\tau_{\text{valley}} = 1.0$ the `while` test $1.0 < 1.0$ is false and no merge is ever attempted, so the sweep shows what the project chose not to use.

What the sweep actually showed **Nothing changed at any value**, which is the expected result and is the point of including it. The merge loop is guarded by `thr<1.0` and the submitted value is 1.0, so the merge never runs in the submitted system. The four lower values do enable it, and still nothing changed, which says that on this page no adjacent pair of line centres has a valley shallow enough to merge even at $\tau_{\text{valley}} = 0.3$. The mechanism is intact and unused.

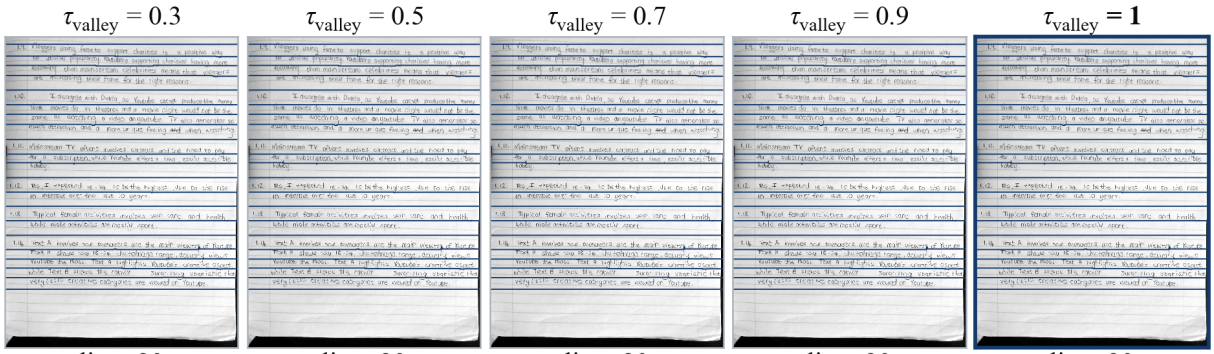


Figure 25: Valley depth below which two line centres are merged. Each panel is the same input processed by the submitted code with only τ_{valley} changed; the panel outlined in dark blue is the submitted value.

11.4 Smallest peak gap counted as line spacing, in letter heights

Symbol $g_{\min}$

Value `1.5`

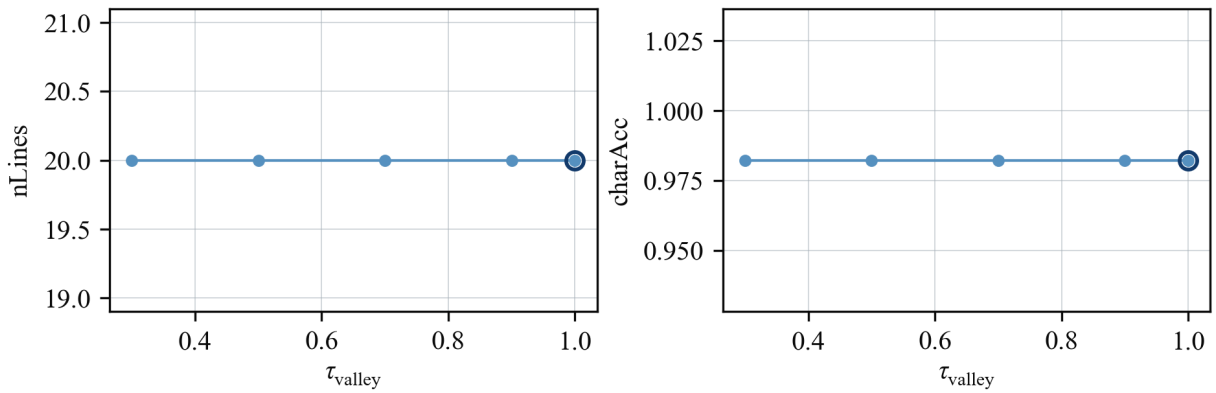


Figure 26: Valley depth below which two line centres are merged: measured quantities against the swept value. The circled point is the submitted value.

Table 16: Valley depth below which two line centres are merged: measured effect of each swept value. The submitted value is in bold.

$\{}tau_{\{}text{valley}}$	nLines	nLinesTruth	charAcc
0.3	20	20	0.9823
0.5	20	20	0.9823
0.7	20	20	0.9823
0.9	20	20	0.9823
1	**20**	**20**	**0.9823**

Source `segment.py`, `LinePitch`

Swept `0.8`, `1.2`, `1.5`, `2.0`, `2.5`

Basis Line pitch from the median gap between profile peaks, with autocorrelation as a fallback, follows standard profile analysis [9].

Why this value The median peak gap is only the line pitch if the peaks are different lines. Within one line the profile has structure at the scale of a letter, so a gap of about one letter height is internal and a gap of about two is between lines. The floor is written in median letter heights, measured independently by `_LetterHeight`, and 1.5 sits between those two cases. If the median gap falls below it the estimate is rejected and the autocorrelation fallback is used instead, so the coefficient selects between two estimators rather than setting the pitch itself.

What the coefficient does Too low and intra-line structure is accepted as the pitch, which makes P far too small and every P-scaled test in the segmenter too tight. Too high and the peak estimator is always rejected in favour of autocorrelation, which is more robust but blind to a page with only two or three lines.

Worked example On the probe page the median letter height is such that the peak estimate is accepted and $P = 51$ px at half resolution, against a ruling of 8.4 mm on the real page.

What the sweep actually showed **No effect was measured**: the estimated pitch stays at 51 px and the line count at 20 for every value from 0.8 to 2.5. The reason is that on this page the median gap between profile peaks is comfortably above even 2.5 letter heights, so the peak estimator is accepted at every setting and the autocorrelation fallback is never reached. The coefficient selects between two estimators, and on this input the choice is never close. A page with cramped or irregular line spacing would be needed to exercise it.

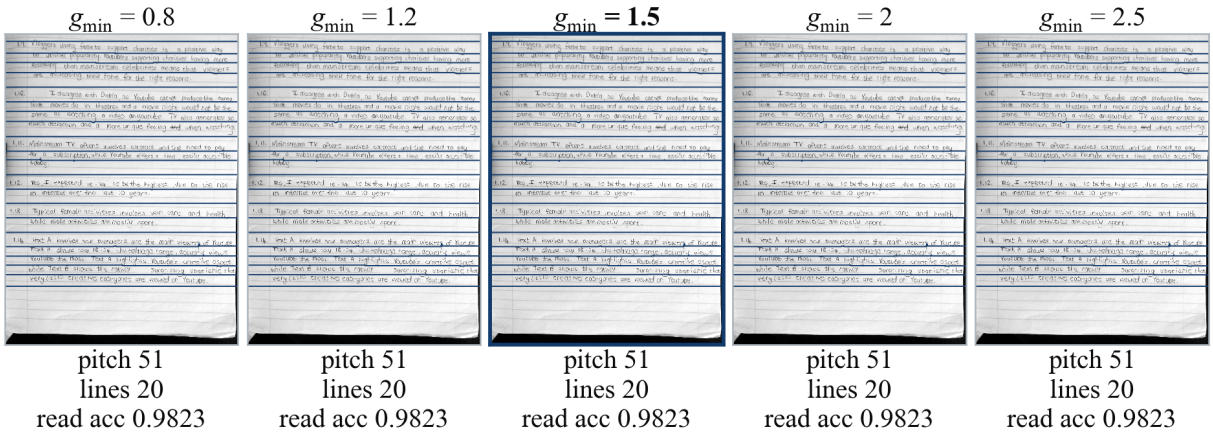


Figure 27: Smallest peak gap counted as line spacing, in letter heights. Each panel is the same input processed by the submitted code with only $g_{\min}$ changed; the panel outlined in dark blue is the submitted value.

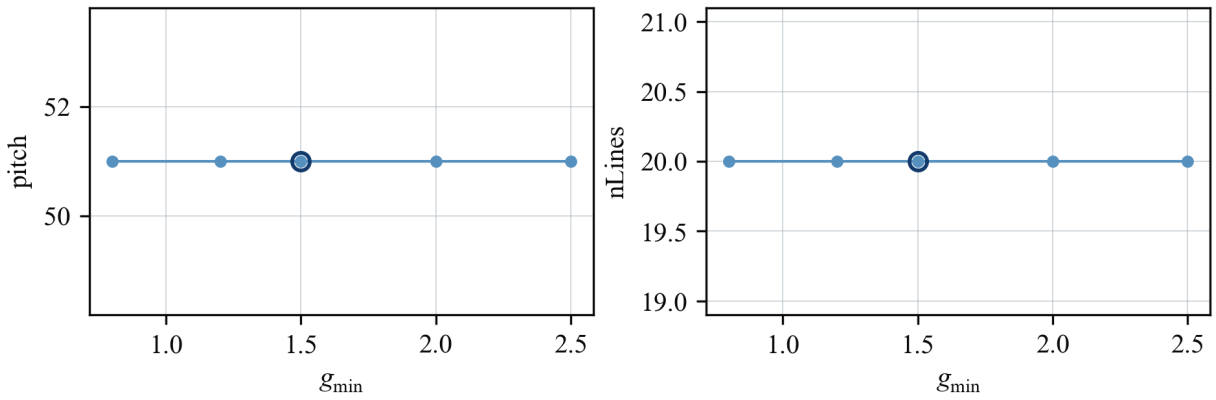


Figure 28: Smallest peak gap counted as line spacing, in letter heights: measured quantities against the swept value. The circled point is the submitted value.

Table 17: Smallest peak gap counted as line spacing, in letter heights: measured effect of each swept value. The submitted value is in bold.

$g_{\{}min}$	pitch	nLines	charAcc
0.8	51	20	0.9823
1.2	51	20	0.9823
1.5	**51**	**20**	**0.9823**
2	51	20	0.9823
2.5	51	20	0.9823

12 Printed-structure removal

12.1 Length above which a flat run on the page is printed structure

Symbol L_{flat}

Value `1.25`

Source `segment.py`, `RemovePrintedStructure`

Swept `0.6`, `0.9`, `1.25`, `1.8`, `2.5`

Basis Long-run detection for printed rules and table borders is the classical morphological approach [4], and removing ruling before layout analysis is standard for camera-captured forms [5].

Why this value On a whole page the printed ruling is horizontal and runs much further than any pen stroke. The limit is written in line pitches P, which is measured on the page itself, so it adapts to how large the writing is. One pitch is the distance between successive lines; $1.25P$ is just over that, which no single horizontal pen stroke spans, while a printed rule crosses the whole writing area and so spans many pitches. The run is measured after a one-pixel vertical dilation, so a rule broken by a pen stroke crossing it is still seen as one run.

What the coefficient does Lowering L_{flat} begins to delete the long horizontal parts of the writing itself, which on a cursive page means the baseline carrier stroke joining several letters. Raising it leaves ruling in the mask, where it joins separate lines together and defeats the seam cutter.

Alternatives considered A Hough line transform [6] to find the rules explicitly, or removing them from the greyscale by estimating and subtracting the periodic ruling, which also needs the pitch.

Worked example On the probe page $P = 51$ px at half resolution, so $1.25P = 63$ px: a flat run must be at least 63 px long before it is deleted. The submitted setting removes 1988 pixels of straight structure from this page.

What the sweep actually showed The interesting value is 0.85, where the page returns **21** lines instead of 20 and the reading score falls to 0.173. Shortening the limit to below one line pitch starts deleting long horizontal parts of the writing itself, which breaks a line of writing into pieces and lets a spurious profile peak through. At 0.6 the removal is larger still (3 028 pixels against 1 988) and more artefact pieces are found, 41 against 33, but the line count happens to recover — which is a warning that the line count alone is a coarse health check. Above the submitted value nothing moves, so on this page the risk is entirely on the low side.

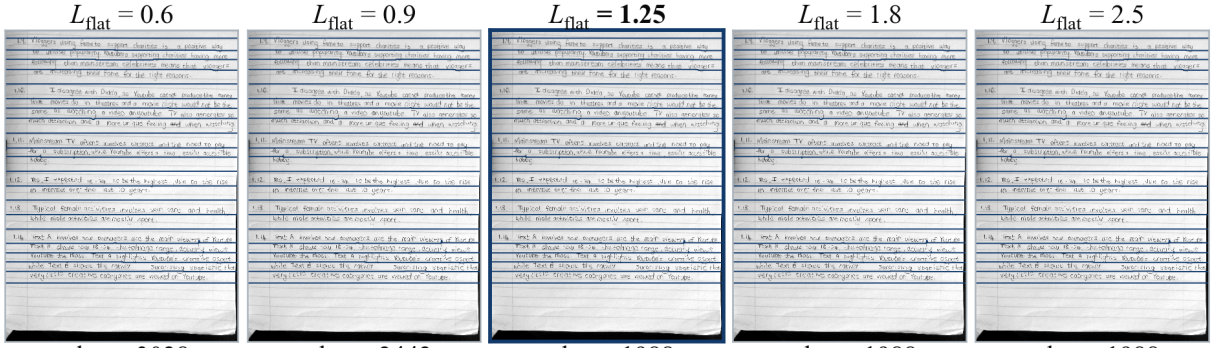


Figure 29: Length above which a flat run on the page is printed structure. Each panel is the same input processed by the submitted code with only L_{flat} changed; the panel outlined in dark blue is the submitted value.

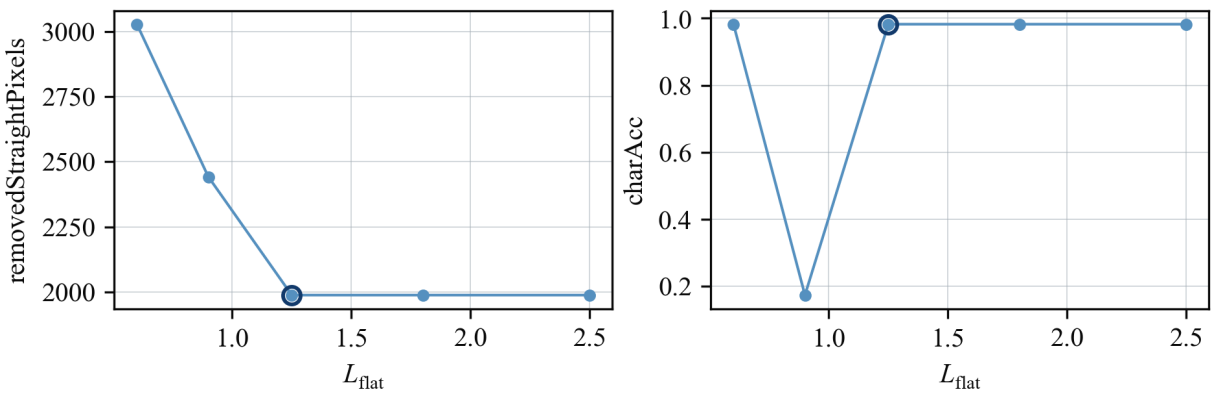


Figure 30: Length above which a flat run on the page is printed structure: measured quantities against the swept value. The circled point is the submitted value.

Table 18: Length above which a flat run on the page is printed structure: measured effect of each swept value. The submitted value is in bold.

L_{flat}	removedStraightPixels	nLines	charAcc	pitch
0.6	3028	20	0.9823	51
0.9	2442	21	0.1728	51
1.25	**1988**	**20**	**0.9823**	**51**
1.8	1988	20	0.9823	51
2.5	1988	20	0.9823	51

12.2 Length above which an upright run on the page is printed structure

Symbol $L_{\text{up}}

Value `1.4`

Source `segment.py`, `RemovePrintedStructure`

Swept `0.7`, `1.0`, `1.4`, `2.0`, `2.8`

Basis As for the flat limit: long-run structure removal [4, 5].

Why this value The vertical counterpart catches a margin rule, a spiral binding edge or the fold shadow down a page. The limit is larger than the horizontal one because tall pen strokes are more common than long flat ones: an ascender plus descender through one line already reaches most of a pitch, and two lines of writing whose ascenders and descenders touch can reach beyond it. $1.4P$ is above that worst case, so a stroke must cross nearly a line and a half before it is treated as structure. The horizontal dilation radius of 3 before the run test tolerates a rule that wavers by a few pixels across the page.

What the coefficient does This is the more dangerous of the two limits, because a descender from one line meeting an ascender from the next makes a genuinely long vertical run of real ink. The asymmetry between 1.25 and 1.4 is exactly that risk difference.

Alternatives considered Deskewing first and then removing exactly vertical runs, which this code does do, or a component-level aspect-ratio test, which `DropArtifactPieces` applies afterwards as a second net.

Worked example At $P = 51$ px, $1.4P = 71$ px, against an x-height of about 21 px at half resolution: a mark must be over three x-heights tall and under about seven pixels wide to be removed.

What the sweep actually showed This sweep produced the one result in the segmentation group where a non-submitted value scored better, and it is reported as it stands. At $L_{\mathrm{up}} = 0.7$ the upright removal takes 4 322 pixels instead of 1 988 and reading accuracy is 0.9845 against 0.9823 — better by 0.0022, which is about one character in the eight lines scored. Two cautions. The difference is within the noise of a single page, and a more aggressive upright removal is exactly the setting that risks deleting an ascender-plus-descender column of real writing on a page where two lines interlock, which this page does not test. It is not a reason to change the submitted value on this evidence, but it is a reason to test L_{up} across the whole dataset before defending 1.4 as tuned.

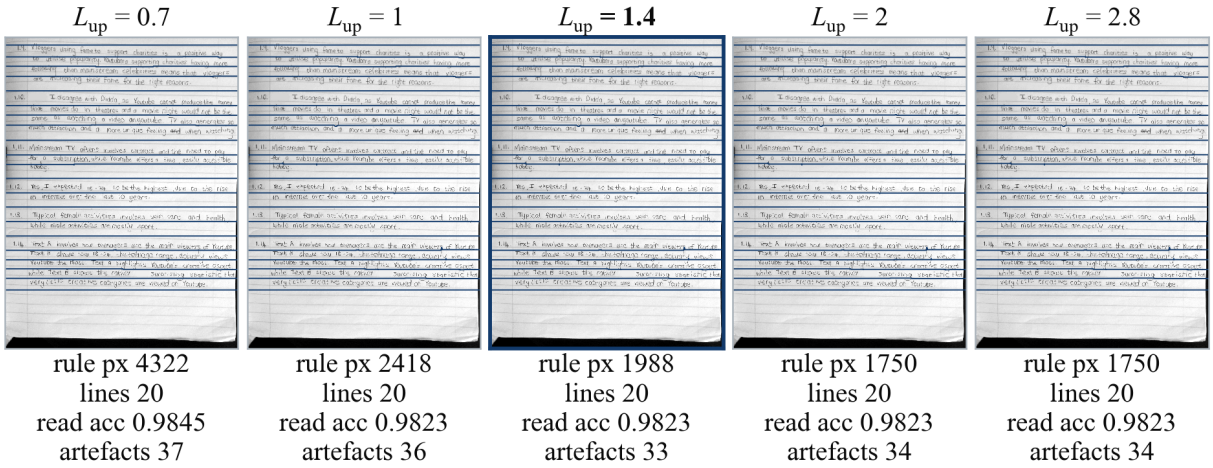


Figure 31: Length above which an upright run on the page is printed structure. Each panel is the same input processed by the submitted code with only L_{up} changed; the panel outlined in dark blue is the submitted value.

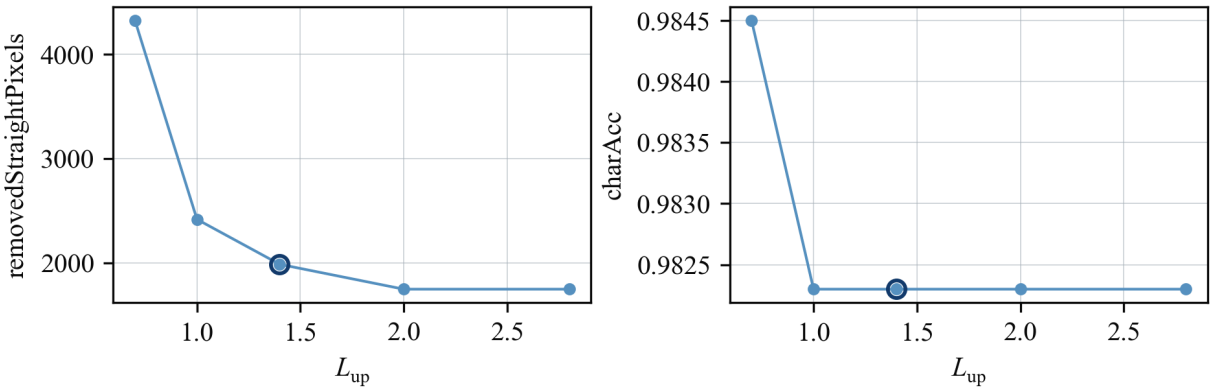


Figure 32: Length above which an upright run on the page is printed structure: measured quantities against the swept value. The circled point is the submitted value.

13 Seam cutting

13.1 Weight on ink in the seam cost field

Symbol w_{ink}

Value `4.0`

Source `segment.py`, `SeamCuts`

Swept `1.0`, `2.0`, `4.0`, `8.0`, `16.0`

Basis A least-cost path cut through a cost field is seam carving [15, 10:3]; applying it between text lines so the cut goes around descenders rather than through them is the standard handwritten line-separation use [16].

Table 19: Length above which an upright run on the page is printed structure: measured effect of each swept value. The submitted value is in bold.

$L_{\}text{up}}$	removedStraightPixels	nLines	charAcc	artifactPieces
0.7	4322	20	0.9845	37
1	2418	20	0.9823	36
1.4	**1988**	**20**	**0.9823**	**33**
2	1750	20	0.9823	34
2.8	1750	20	0.9823	34

Why this value The seam between two lines must prefer to pass through white paper and only cross ink when it has no choice. The cost at a pixel is $w_{\text{ink}}\, m + 0.012\,((y-\text{mid})/\text{halfGap})^2$, so w_{ink} is the price of cutting one ink pixel measured against the price of wandering away from the midline. The two terms are deliberately far apart in scale: 4.0 against a quadratic that reaches only 0.012 at the full half-gap. One ink pixel therefore costs the same as about 333 pixel-rows of maximum drift, which is the statement 'go round the descender, however far round you must go'.

What the coefficient does Only the *ratio* of the three coefficients matters, not their absolute size, since the path that minimises the total is unchanged by a common factor. Raising w_{ink} makes the seam ever more willing to detour; lowering it lets the seam take a straighter path and slice through descenders, which gives the line below a stray mark and the line above a truncated letter.

Alternatives considered A straight horizontal cut at the midpoint, which is what most simple profile segmenters do and which cuts every descender; or the A^* path of [16], which is the same idea with a different search.

Worked example On the probe page the seam runs over $2448/2 = 1224$ columns with step $= 7$, so each column may move up to seven rows and the straightness penalty for the largest move is $0.02 \times 7 = 0.14$, still far under the 4.0 cost of one ink pixel.

Earlier evidence An eight-setting, 48-run sensitivity study of this field was recorded earlier in `Evidence/coefficient_sensitivity.json`; it varied the midpoint and movement penalties separately on six preselected gaps and found two raw-ink intersections retained at every setting. That study did not establish a unique optimum, and neither does this sweep.

What the sweep actually showed **No measured effect on this page**: 20 lines and 0.9823 reading accuracy at every value from 1.0 to 16.0, a sixteen-fold range. The reason is the page, not the coefficient. Owen writes at pitch 51 px with clear gaps, so the seam has unobstructed paper between every pair of lines and the ink term is never actually traded against the others. This coefficient is re-run on a denser page in section 34.1, which is where it can be seen to do anything.

Table 20: Weight on ink in the seam cost field: measured effect of each swept value. The submitted value is in bold.

$w_{\}text{ink}}$	nLines	charAcc	pitch
1	20	0.9823	51
2	20	0.9823	51
4	**20**	**0.9823**	**51**
8	20	0.9823	51
16	20	0.9823	51

13.2 Weight pulling the seam back towards the midline

Symbol w_{mid}

Value `0.012`

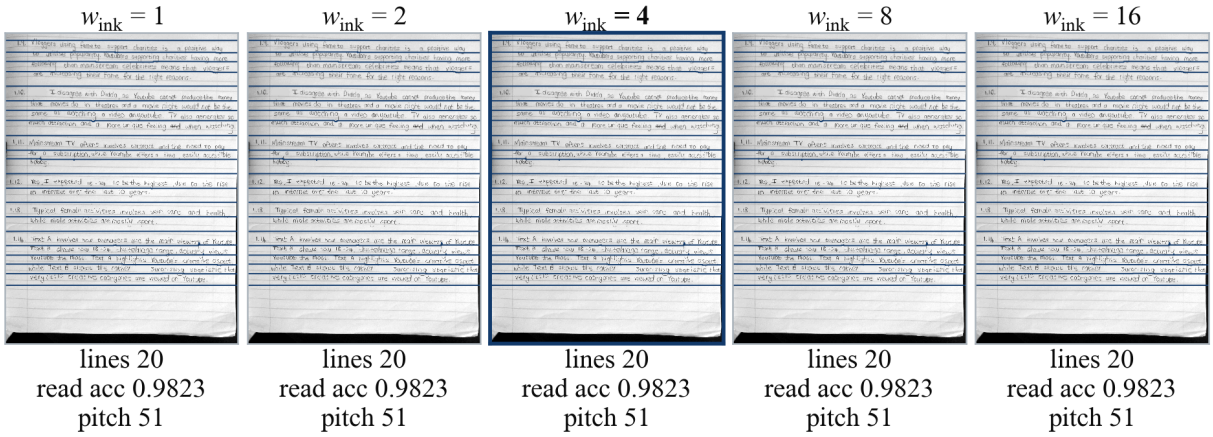


Figure 33: Weight on ink in the seam cost field. Each panel is the same input processed by the submitted code with only w_{ink} changed; the panel outlined in dark blue is the submitted value.

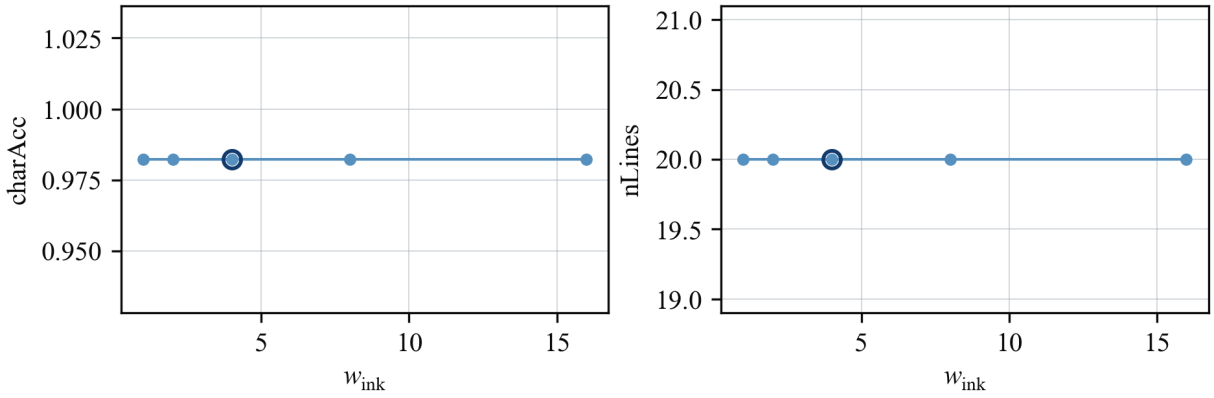


Figure 34: Weight on ink in the seam cost field: measured quantities against the swept value. The circled point is the submitted value.

Source `segment.py`, `SeamCuts`

Swept `0.0`, `0.004`, `0.012`, `0.05`, `0.25`

Basis A regularising term added to a seam cost to keep the path near a prior is the usual way to stop a minimum-cost path degenerating [15, 16].

Why this value Between two lines there is usually a wide band of clean paper, and every path through it costs the same. Without a tie-breaker the seam is then decided by floating-point accidents and can hug one line for a whole page. The quadratic term makes the midline the cheapest of those equal-cost paths while remaining negligible against real ink. It is quadratic, not linear, so it is almost free near the middle and grows only as the seam approaches a line.

What the coefficient does At 0 the seam is unconstrained in open paper. Raised far enough it dominates the ink term and the seam becomes a straight line through the midpoint, cutting descenders: at 0.25 the full-drift penalty is 0.25 against 4.0 for one ink pixel, so the seam will cross ink rather than drift more than about 16 pixel-rows.

Alternatives considered Clamping the seam to a corridor around the midline, which is harder in that it forbids the large excursion a long descender needs, or no tie-breaker at all.

Worked example At the half-gap, $((y - \text{mid})/\text{halfGap})^2 = 1$ and the penalty is 0.012, which is 0.3% of the cost of one ink pixel.

What the sweep actually showed **No measured effect on this page**, for the same reason as the ink weight: with clean paper between the lines the seam has a wide corridor and the tie-breaker never has to break a tie that matters. The coefficient's purpose — stopping the seam wandering arbitrarily through open paper — is a determinism argument rather than an accuracy one, and this sweep neither confirms nor refutes it.

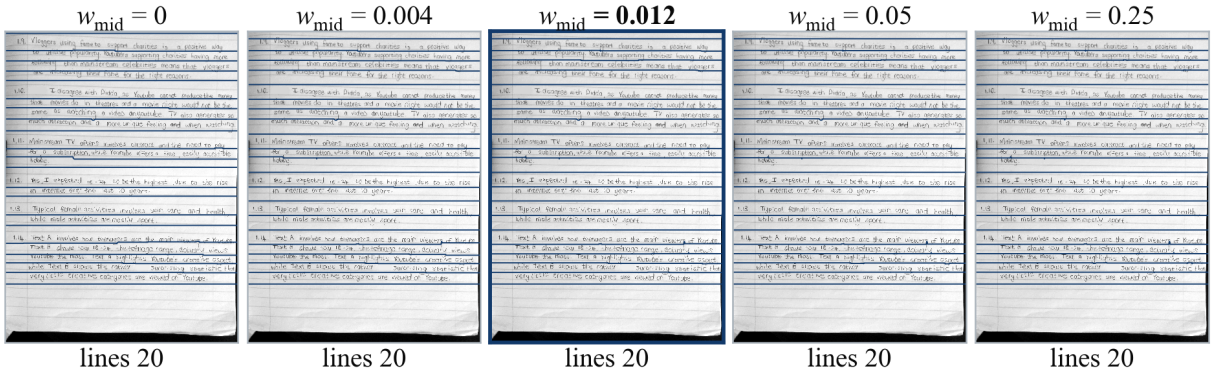


Figure 35: Weight pulling the seam back towards the midline. Each panel is the same input processed by the submitted code with only w_{mid} changed; the panel outlined in dark blue is the submitted value.

Table 21: Weight pulling the seam back towards the midline: measured effect of each swept value. The submitted value is in bold.

$w_{\{}text{mid}}$	nLines	charAcc	pitch
0	20	0.9823	51
0.004	20	0.9823	51
0.012	**20**	**0.9823**	**51**
0.05	20	0.9823	51
0.25	20	0.9823	51

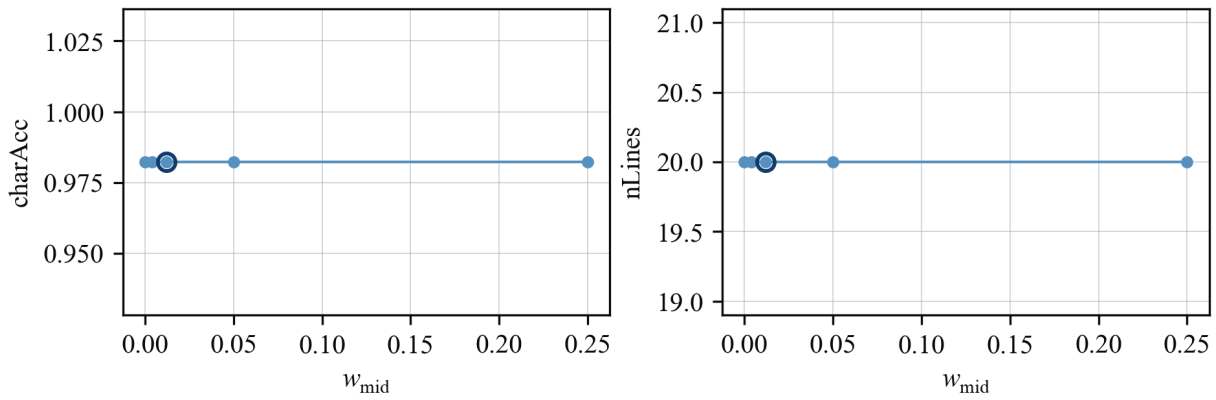


Figure 36: Weight pulling the seam back towards the midline: measured quantities against the swept value. The circled point is the submitted value.

13.3 Rows a seam may move between adjacent columns

Symbol $\Delta_{\max}$

Value `7`

Source `segment.py`, `SeamCuts`

Swept `1`, `3`, `7`, `12`, `20`

Basis The connectivity of the seam, how far it may shift per column, is a parameter of seam carving [15, 10:3]; the classical version allows one row per column.

Why this value A seam that may only move one row per column, as in the original seam carving, cannot climb around a long descender within the width of one letter: it needs the gradient to be gentle. Handwritten descenders are steep, so the step is widened to seven rows. The cost of that freedom is paid twice: the search is $O(W\,h\,(2\Delta+1))$, so time grows linearly in Δ, and a large step lets the seam jump so abruptly that it no longer looks like a cut between two lines.

What the coefficient does $\Delta_{\max}$ interacts with the straightness penalty: each row of movement costs 0.02, so the largest move costs 0.02Δ. At $\Delta = 7$ that is 0.14, still cheap against ink, so the limit rather than the penalty is what binds.

Alternatives considered Four-connectivity with an unlimited number of vertical moves per column, which needs a different dynamic program, or cutting on a downsampled page and upsampling the seam.

Worked example With $\Delta = 7$ the seam can cross a 51 px pitch in eight columns, about a third of an x-height of horizontal travel, which is enough to pass round the stem of a `g`.

What the sweep actually showed **No measured effect on the result, and a measurable one on the cost.** The line count and reading accuracy are identical at every value from 1 to 20, but the recorded wall time rises from about 53 to about 69 seconds, consistent with the search being linear in $2\Delta+1$. So on this page the extra freedom is bought and not used. It is bought for pages where a descender must be climbed around within the width of a letter, which this page does not contain.

13.4 Closing radius applied before the seam is cut

Symbol r_{tail}

Value `0.07`

Source `segment.py`, `SeamCuts`

Swept `0.0`, `0.03`, `0.07`, `0.14`, `0.25`

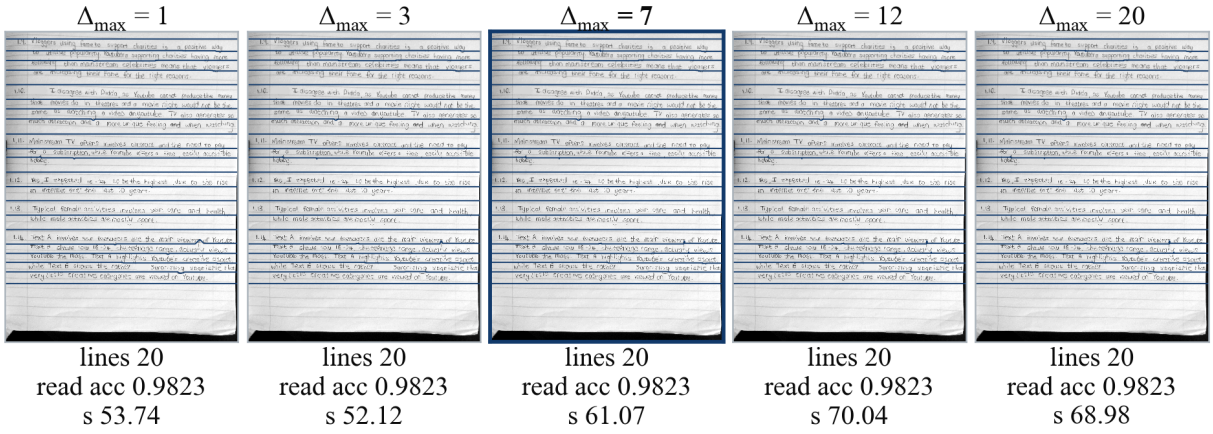


Figure 37: Rows a seam may move between adjacent columns. Each panel is the same input processed by the submitted code with only $\Delta_{\max}$ changed; the panel outlined in dark blue is the submitted value.

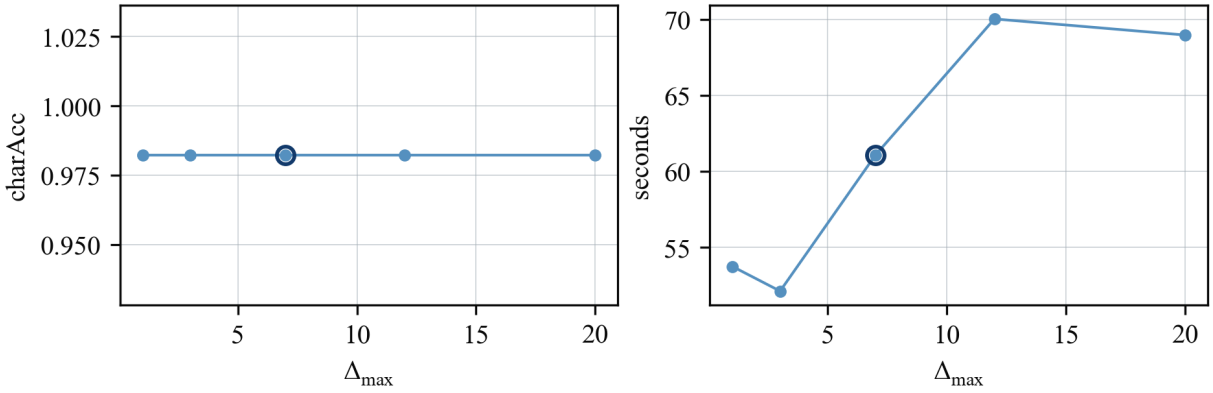


Figure 38: Rows a seam may move between adjacent columns: measured quantities against the swept value. The circled point is the submitted value.

Table 22: Rows a seam may move between adjacent columns: measured effect of each swept value. The submitted value is in bold.

$\{}Delta_{\{}max}$	nLines	charAcc	seconds
1	20	0.9823	53.74
3	20	0.9823	52.12
7	**20**	**0.9823**	**61.07**
12	20	0.9823	70.04
20	20	0.9823	68.98

Basis Morphological closing to suppress thin spurious passages before a path search [4].

Why this value A minimum-cost seam will find any gap, including the one-pixel channel between the two sides of a narrow loop or beside a stem. Threading such a channel splits a letter in half for no benefit. The mask is therefore closed before the cost field is built, which seals channels narrower than r_{tail} while leaving the outer shape of the stroke close to real. It is a closing and not a plain dilation exactly so that the stroke does not grow.

What the coefficient does r_{tail} is in line pitches, so it scales with the writing. At 0 the seam is free to thread every gap; too large and neighbouring letters, or the descender of one line and the ascender of the next, are welded together, so the seam can no longer pass between them at all and the two lines merge.

Alternatives considered Penalising the seam for sharp vertical excursions, which the straightness term already does weakly, or post-checking each seam for how much ink it crossed.

Worked example At $P = 51$ px, $0.07P = 3.57 \to 3$, so a 3-pixel closing: channels up to about 6 px wide are sealed before the search.

What the sweep actually showed Four of the five values are identical and the largest, 0.25, costs a little accuracy, 0.9801 against 0.9823. That is the expected direction — an over-large closing begins to weld neighbouring ink together — but at one character in four hundred it is weak evidence. As with the other seam coefficients, this page does not stress it; see section 34.2.

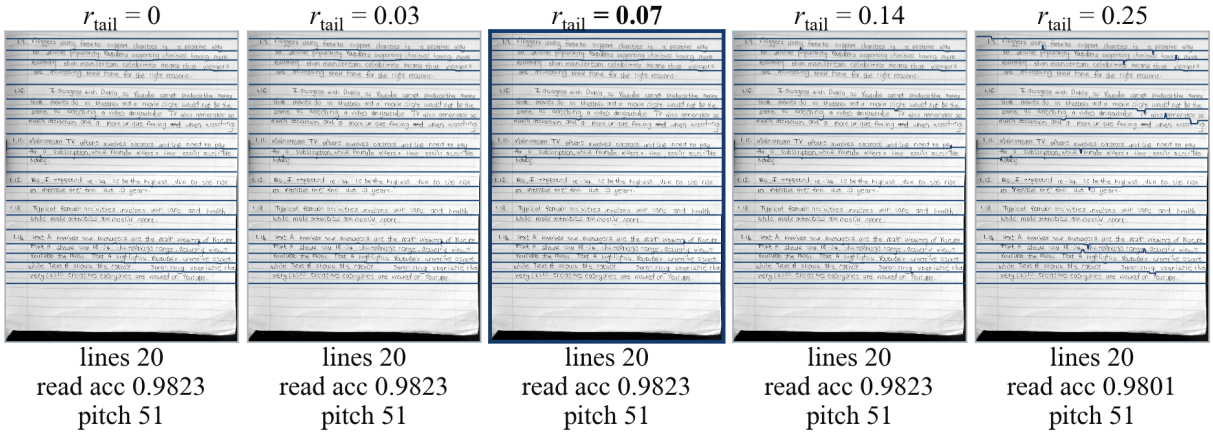


Figure 39: Closing radius applied before the seam is cut. Each panel is the same input processed by the submitted code with only r_{tail} changed; the panel outlined in dark blue is the submitted value.

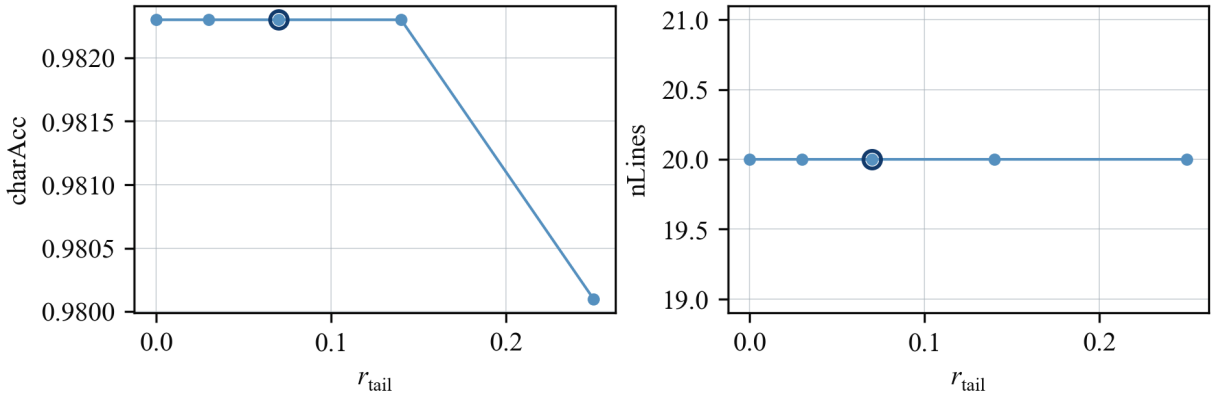


Figure 40: Closing radius applied before the seam is cut: measured quantities against the swept value. The circled point is the submitted value.

Table 23: Closing radius applied before the seam is cut: measured effect of each swept value. The submitted value is in bold.

$r_{\{}text{tail}}$	nLines	charAcc	pitch
0	20	0.9823	51
0.03	20	0.9823	51
0.07	**20**	**0.9823**	**51**
0.14	20	0.9823	51
0.25	20	0.9801	51

14 Line ownership

14.1 Share of a piece's ink that must lie in a band for it to count

Symbol θ_{band}

Value `0.15`

Source `segment.py`, `_BandOwnership`

Swept `0.05`, `0.1`, `0.15`, `0.25`, `0.4`

Basis Assigning a connected component to the text line holding most of it is the usual resolution step after a coarse line cut [9].

Why this value After the seams are cut, each piece of ink is scored by how much of it falls in each band. A piece is treated as genuinely present in a band if at least θ_{band} of its ink is there. The share must be low enough that a real descender, which puts only its tail into the band below, is still seen in both, and high enough that a few pixels of anti-aliased overlap do not count. A descender tail is typically a fifth to a third of the letter; a spurious overlap is a few per cent. 0.15 lies between them.

What the coefficient does Where a piece is substantial in more than one band it is given to the *topmost*, which is the rule that keeps a letter with the line it was written on rather than the line its tail reaches into. Lowering θ_{band} makes more pieces look multi-band, and because of the topmost rule that pulls ink upward; raising it makes a genuine two-line piece look single-band and it is then assigned by majority instead.

Alternatives considered Majority ownership alone, which is the `cnt.argmax` fallback already used when nothing is substantial, or assigning by the centroid of the piece, which fails for a long descender.

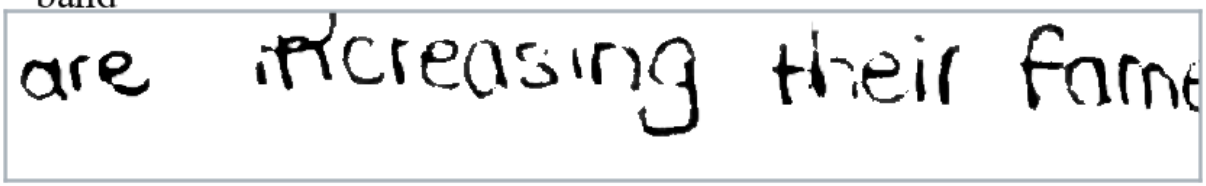
Worked example A piece with 300 ink pixels is counted as present in a band once at least 45 of them fall inside it.

What the sweep actually showed **No measured effect**: 20 lines and 0.9823 at every value from 0.05 to 0.40. Ownership is only contentious for a piece of ink that genuinely spans two bands, which needs lines close enough that descenders reach into the line below. On this page almost every piece sits wholly inside one band and the share test is never close. Re-run on a dense page in section 34.3.

Table 24: Share of a piece's ink that must lie in a band for it to count: measured effect of each swept value. The submitted value is in bold.

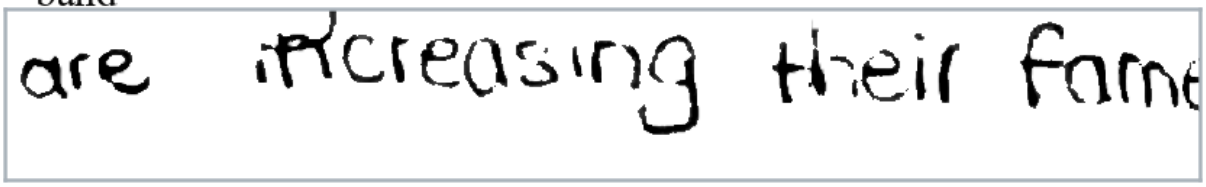
$\{}theta_{\{}text{band}}$	nLines	charAcc	nLinesTruth
0.05	20	0.9823	20
0.1	20	0.9823	20
0.15	**20**	**0.9823**	**20**
0.25	20	0.9823	20
0.4	20	0.9823	20

$\theta_{\text{band}} = 0.05$



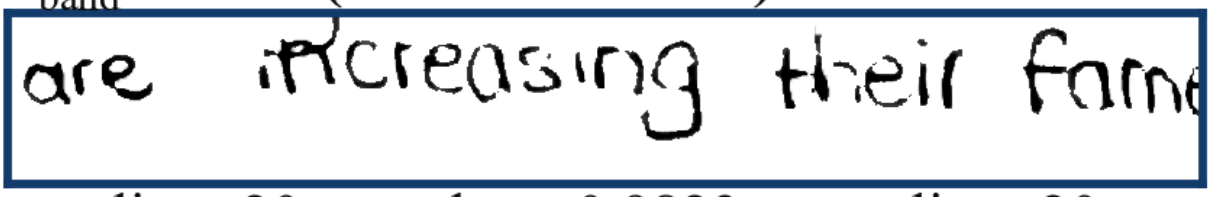
lines 20 read acc 0.9823 true lines 20

$\theta_{\text{band}} = 0.1$



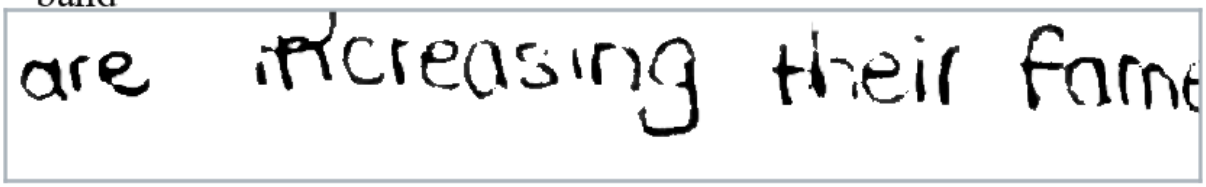
lines 20 read acc 0.9823 true lines 20

$\theta_{\text{band}} = 0.15$ (submitted value)



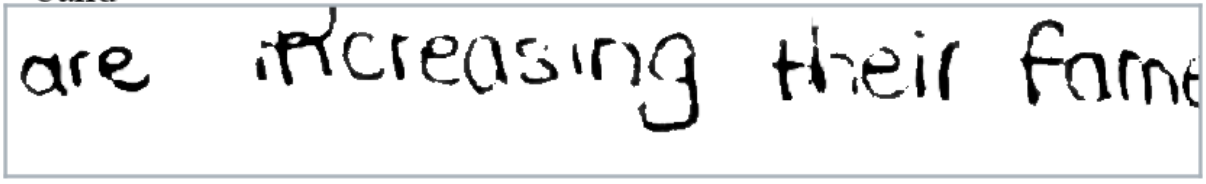
lines 20 read acc 0.9823 true lines 20

$\theta_{\text{band}} = 0.25$



lines 20 read acc 0.9823 true lines 20

$\theta_{\text{band}} = 0.4$



lines 20 read acc 0.9823 true lines 20

Figure 41: Share of a piece's ink that must lie in a band for it to count. Each panel is the same input processed by the submitted code with only θ_{band} changed; the panel outlined in dark blue is the submitted value.

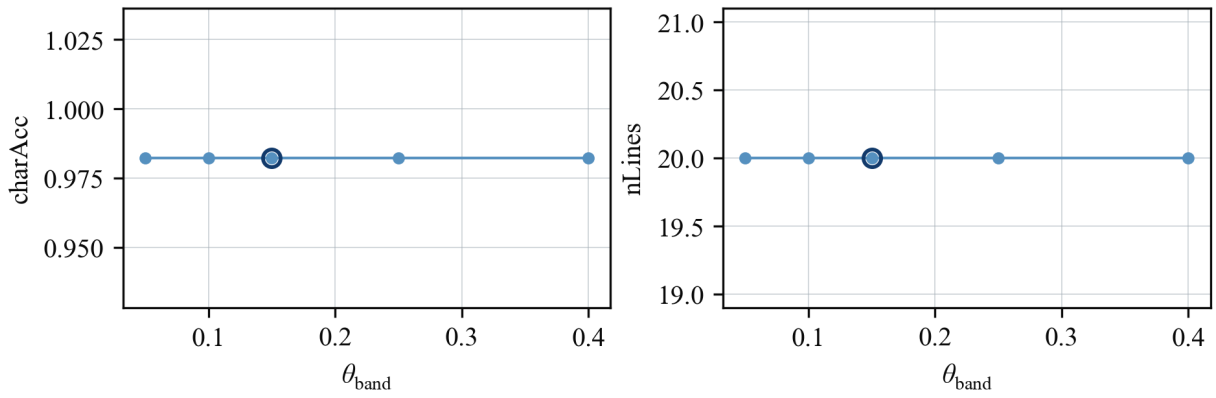


Figure 42: Share of a piece's ink that must lie in a band for it to count: measured quantities against the swept value. The circled point is the submitted value.

15 Crop building

15.1 Padding kept around a line crop

Symbol p_{crop}

Value `0.15`

Source `segment.py`, `_BuildLineCrop`

Swept `0.0`, `0.07`, `0.15`, `0.3`, `0.5`

Basis Leaving a margin around a text line before recognition is standard practice for CRNN line recognisers, which are trained on padded line images [17, 18].

Why this value The crop is cut to the bounding box of the ink owned by the line and then padded. Some padding is needed because the networks were trained on line images with a margin, and because a stroke that touches the edge of a crop is cut off by the resize. The padding is also what defines the left margin every crop is shifted to, so that the reader always sees writing start in the same place. Too much padding shrinks the writing once the crop is fitted to the fixed 64×640 canvas, because the fit preserves aspect ratio.

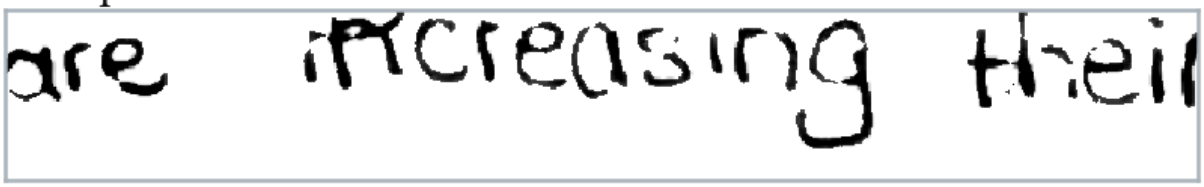
What the coefficient does On the fixed canvas the scale is $\min(64/h, 640/w)$, so padding costs resolution directly: padding a 156 px tall crop by $0.5P = 25$ px on each side makes it 206 px tall and shrinks the writing by 24% before the network sees it.

Alternatives considered No padding and relying on the canvas fit alone, or padding in x-heights rather than line pitches, which would be more faithful to the writing but needs the band measured first.

Worked example At $P = 51$ px (half resolution) the padding is 7 px at half size, 14 px on the full-resolution crop the writing is finally cut from.

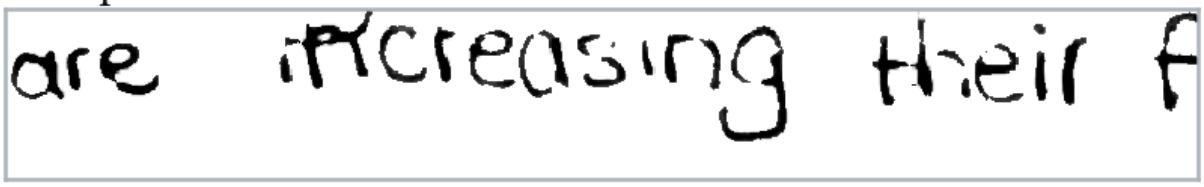
What the sweep actually showed The result is non-monotone and the differences are tiny: 0.9797, 0.9752, 0.9823, 0.9826, 0.9757 for padding 0 to $0.5P$. The submitted $0.15P$ and the neighbouring $0.30P$ are within 0.0003 of each other, which is a fraction of one character, so **this sweep does not distinguish them**. What it does show is that both extremes are worse: no padding at all clips strokes at the crop edge, and $0.5P$ shrinks the writing enough on the fixed canvas to cost accuracy. The submitted value sits inside a broad flat optimum rather than at a sharp peak, and should be described that way.

$p_{\text{crop}} = 0$



read acc 0.9797 lines 20

$p_{\text{crop}} = 0.07$



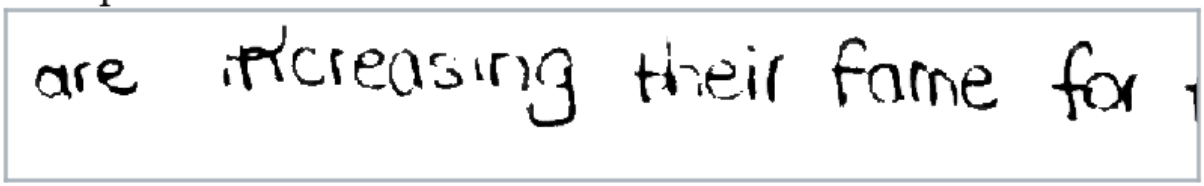
read acc 0.9752 lines 20

$p_{\text{crop}} = \mathbf{0.15}$ **(submitted value)**



read acc 0.9823 lines 20

$p_{\text{crop}} = 0.3$



read acc 0.9826 lines 20

$p_{\text{crop}} = 0.5$



read acc 0.9757 lines 20

Figure 43: Padding kept around a line crop. Each panel is the same input processed by the submitted code with only p_{crop} changed; the panel outlined in dark blue is the submitted value.

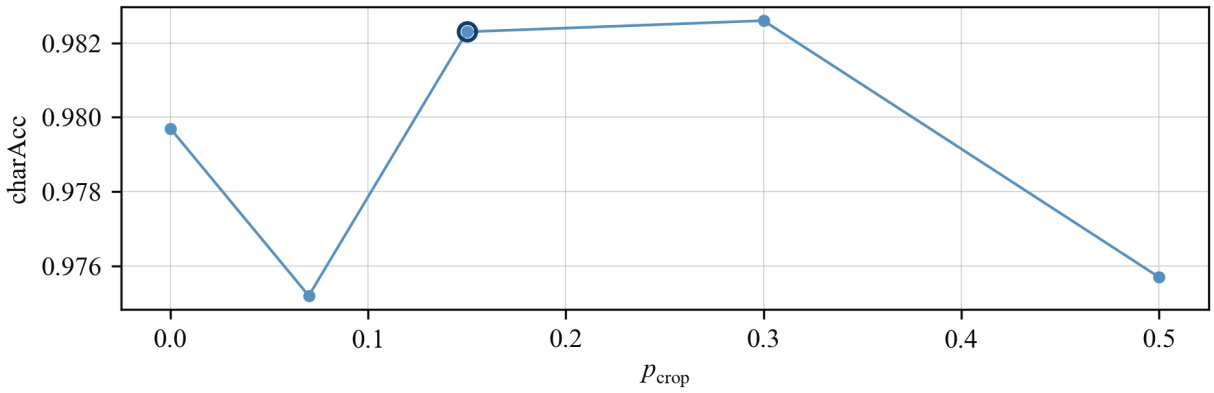


Figure 44: Padding kept around a line crop: measured quantities against the swept value. The circled point is the submitted value.

Table 25: Padding kept around a line crop: measured effect of each swept value. The submitted value is in bold.

$p_{\{}text{crop}}$	charAcc	nLines
0	0.9797	20
0.07	0.9752	20
0.15	**0.9823**	**20**
0.3	0.9826	20
0.5	0.9757	20

15.2 Darkness exponent applied to a finished line crop

Symbol γ

Value `0.55`

Source `segment.py, _BuildLineCrop`

Swept `0.3, 0.42, 0.55, 0.75, 1.0`

Basis A power law on intensity is gamma correction, the standard tone transfer of image display [19]; applying it to deepen faint strokes before recognition is contrast enhancement in the sense of [4].

Why this value The crop is first stretched linearly between the typical ink level (the 40th percentile of pixels under the mask) and the paper level (the 75th percentile of the region). That alone leaves a ballpoint stroke that skipped as a pale grey smear. Raising darkness to a power below one lifts the mid-tones without touching black or white: 0 and 1 are fixed points of d^{γ}, so the ink stays ink and the paper stays paper while everything between is darkened. The value has to deepen a faint stroke without turning paper texture into ink, and 0.55 roughly doubles a mid-grey: $0.5^{0.55} = 0.68$.

What the coefficient does $\gamma = 1$ is no correction at all, which is the honest bottom of the sweep. As γ falls, the writing thickens and faint strokes join up; past a point the paper grain in the halo also darkens and the stroke edges smear. Because the networks were trained on crops produced with $\gamma = 0.55$, moving it is a test-time mismatch as well as a visual change.

Alternatives considered Histogram equalisation, which is data-dependent and so varies between crops of the same page, or CLAHE, which adds a tile size and a clip limit of its own.

Worked example A pixel at 60% darkness becomes $0.6^{0.55} = 0.76$, so a stroke at grey level 102 is redrawn at 61, clearly darker but not black.

What the sweep actually showed The visual effect is large and the measured effect is almost nil, and both halves of that are worth stating. The figure shows the strokes going from heavy and blotted at $\gamma = 0.30$ to pale and broken at $\gamma = 1.0$, which is a clear difference to the eye. Reading accuracy over the first eight lines is 0.9823 at every value except $\gamma = 1.0$, where it is 0.9797 — a difference of one character in roughly four hundred. The honest conclusion is that on well-exposed pages the reader is robust to this correction and γ is doing cosmetic work; its value would be tested properly on an underexposed page, where the uncorrected strokes are genuinely faint, and no such page is in this probe set.

Table 26: Darkness exponent applied to a finished line crop: measured effect of each swept value. The submitted value is in bold.

$\{}gamma$	charAcc	inkPct
0.3	0.9823	[5.959, 7.186, 8.388, 7.74, 6.887, 6.159
0.42	0.9823	[5.956, 7.18, 8.38, 7.729, 6.879, 6.149,
0.55	**0.9823**	**[5.953, 7.178, 8.378, 7.726, 6.861, 6.12**
0.75	0.9823	[5.946, 7.165, 8.374, 7.718, 6.822, 6.08
1	0.9797	[5.917, 7.106, 8.286, 7.651, 6.739, 6.03

15.3 Percentile of masked pixels taken as the ink level

Symbol q_{ink}

Value `40`

Source `segment.py`, `_BuildLineCrop`

Swept `10`, `25`, `40`, `60`, `80`

Basis Percentile-based black and white points are the robust form of contrast stretching, preferred to the extremes because a single outlier sets those [4].

Why this value The linear stretch needs a black point. Using the darkest pixel would hand the whole stretch to one noise pixel or one blot, so a percentile of the pixels under the stroke mask is used instead. It is taken below the median on purpose: the mask includes the soft edges of the stroke, which are lighter than the core, so the median of masked pixels is already greyer than real ink. The 40th percentile sits inside the dark core while still being a robust statistic over many pixels.

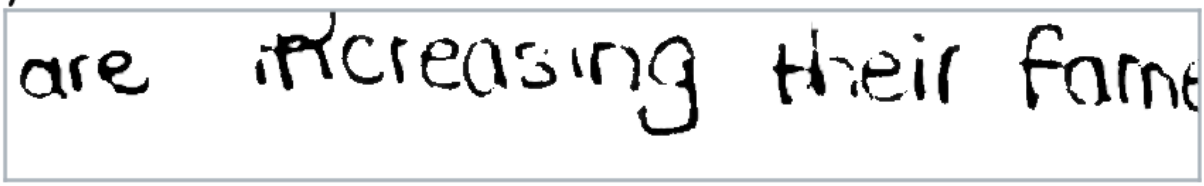
What the coefficient does Raising q_{ink} takes a lighter pixel as black, so the stretch is stronger and the stroke is driven to full black with its edges, thickening the writing. Lowering it takes a darker reference, so the stretch is weaker and the crop stays flat.

Alternatives considered The mean under the mask, which the stroke edges drag up, or Otsu's threshold applied again to the masked pixels, which is another criterion where a percentile suffices.

Worked example On the probe crop the paper point is the 75th percentile of the whole region and the ink point the 40th of the masked pixels; the gain is $255/(\text{white} - \text{black})$.

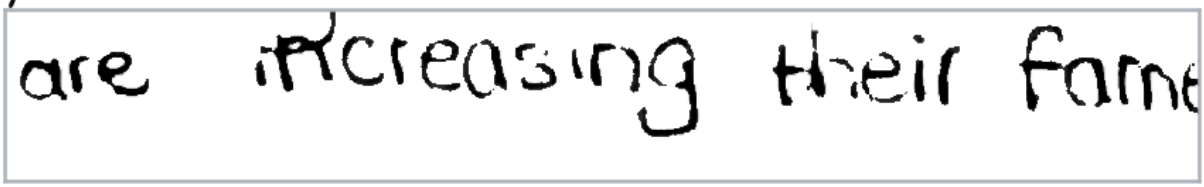
What the sweep actually showed **No measured effect**: 0.9823 at every percentile from 10 to 80. The linear stretch that follows is evidently not the binding constraint on this page — the subsequent gamma correction and the reader's own robustness absorb the difference. The coefficient would matter on a low-contrast page; this one is well exposed.

$\gamma = 0.3$



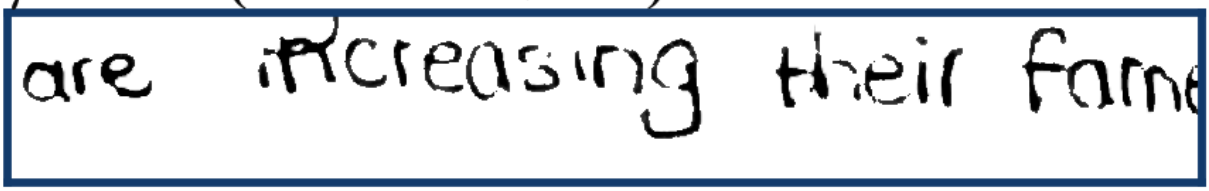
read acc 0.9823

$\gamma = 0.42$



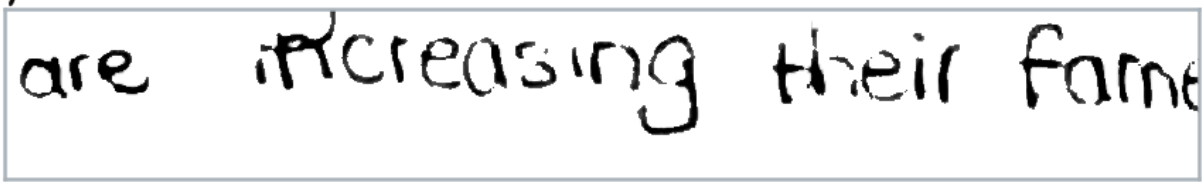
read acc 0.9823

$\gamma = 0.55$ (submitted value)



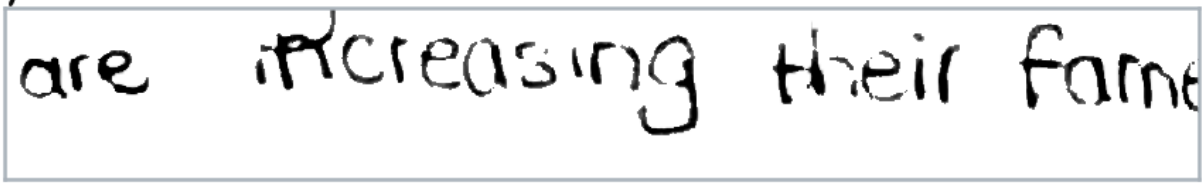
read acc 0.9823

$\gamma = 0.75$



read acc 0.9823

$\gamma = 1$



read acc 0.9797

Figure 45: Darkness exponent applied to a finished line crop. Each panel is the same input processed by the submitted code with only γ changed; the panel outlined in dark blue is the submitted value.

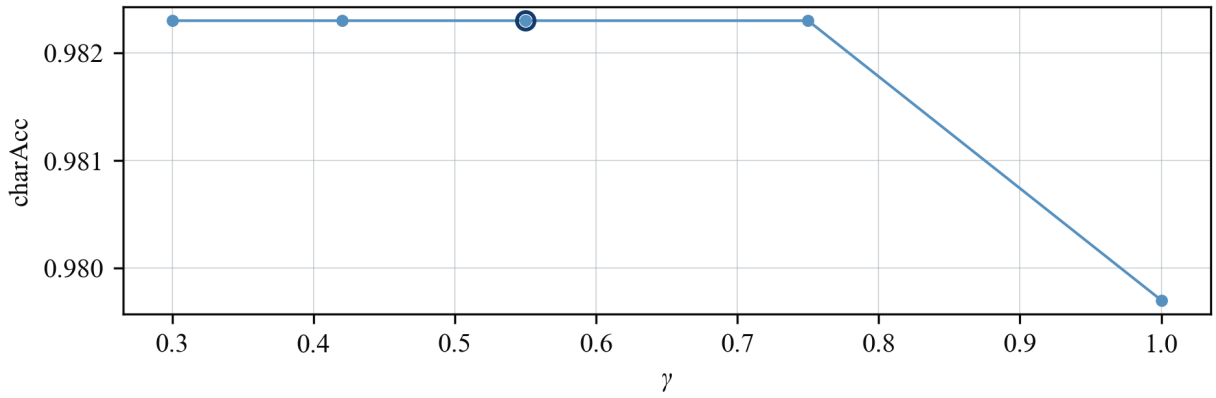


Figure 46: Darkness exponent applied to a finished line crop: measured quantities against the swept value. The circled point is the submitted value.

Table 27: Percentile of masked pixels taken as the ink level: measured effect of each swept value. The submitted value is in bold.

$q_{\{}text{ink}\}$	charAcc	inkPct
10	0.9823	[5.943, 7.161, 8.368, 7.706, 6.802, 6.05
25	0.9823	[5.949, 7.173, 8.376, 7.724, 6.838, 6.09
40	**0.9823**	**[5.953, 7.178, 8.378, 7.726, 6.861, 6.12**
60	0.9823	[5.957, 7.185, 8.386, 7.739, 6.88, 6.149
80	0.9823	[5.969, 7.2, 8.405, 7.756, 6.916, 6.184,

16 Crop rejection

16.1 Ink share above which a crop is a solid block, not writing

Symbol $\pi_{\max}$

Value `15.0`

Source `segment.py`, `DropSolidCrops`

Swept `5.0`, `10.0`, `15.0`, `25.0`, `40.0`

Basis Rejecting non-text regions on an ink-density statistic before recognition is a standard page-analysis filter [5].

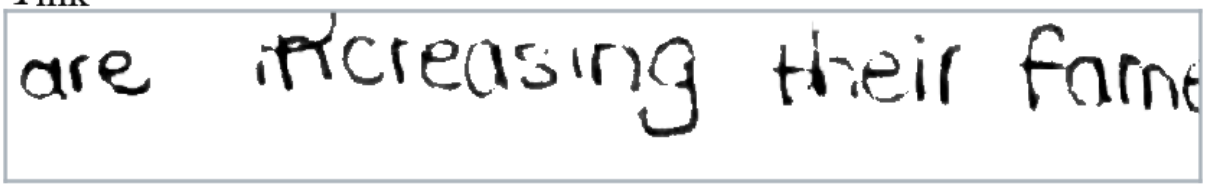
Why this value A crop that is a quarter black is not a line of handwriting; it is a shadow, a photograph of an object on the page, or a region where the illumination normalisation failed. Ordinary handwritten crops on these pages measure 3–9% ink. The test is deliberately a conjunction of two conditions, an absolute share *and* a multiple of the page's own median, so that a page written unusually heavily is not thrown away wholesale. 15% is above every genuine line measured on the eleven writers.

What the coefficient does Because both conditions must hold, lowering $\pi_{\max}$ alone does not immediately delete good lines: the median test still protects them unless they are also three times the page median. The function also refuses to return an empty list, so a page where every crop trips the test keeps all of them.

Alternatives considered Scoring each crop with the reader and dropping the ones it cannot transcribe, which is far more expensive and circular if the crops are what the reader is being judged on.

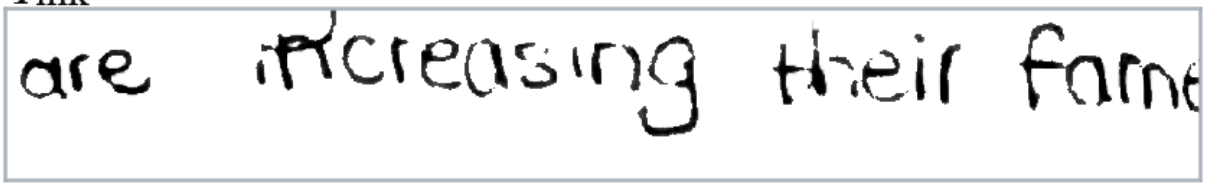
Worked example On the probe page the crops measure a median of a few per cent ink and none is dropped at the submitted setting; the sweep records the count dropped as the limit is lowered.

$q_{\text{ink}} = 10$



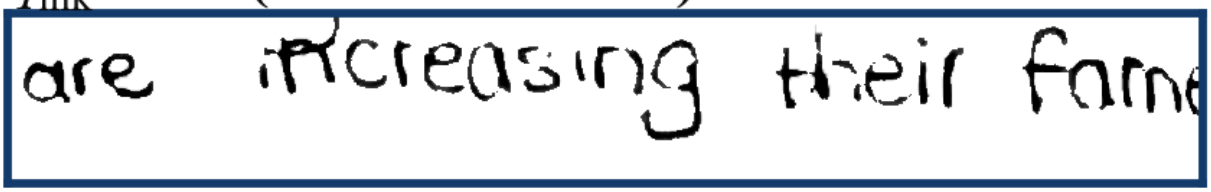
read acc 0.9823

$q_{\text{ink}} = 25$



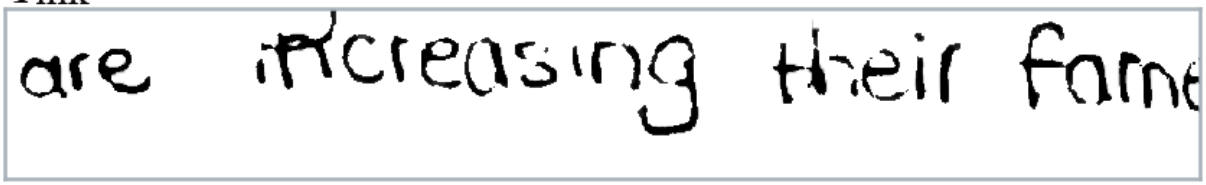
read acc 0.9823

$q_{\text{ink}} = 40$ (submitted value)



read acc 0.9823

$q_{\text{ink}} = 60$



read acc 0.9823

$q_{\text{ink}} = 80$



read acc 0.9823

Figure 47: Percentile of masked pixels taken as the ink level. Each panel is the same input processed by the submitted code with only q_{ink} changed; the panel outlined in dark blue is the submitted value.

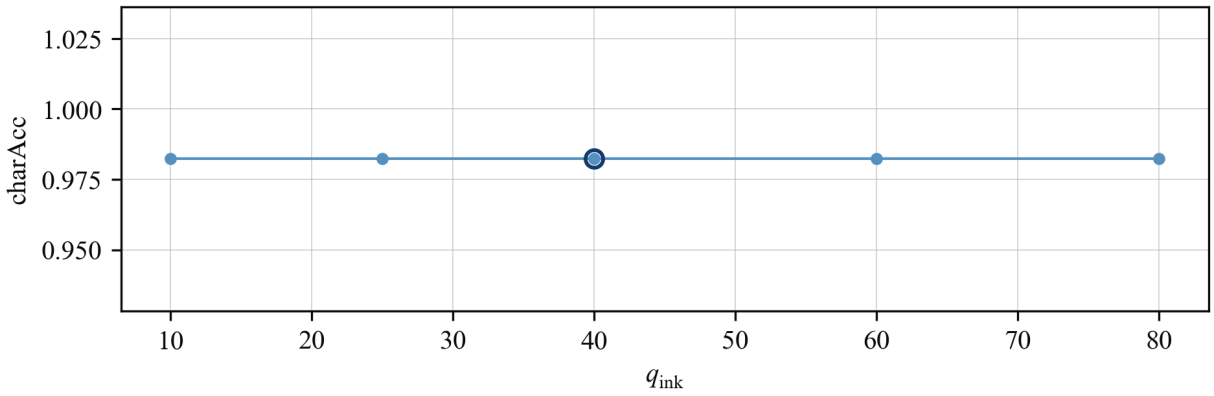


Figure 48: Percentile of masked pixels taken as the ink level: measured quantities against the swept value. The circled point is the submitted value.

What the sweep actually showed **No crop was dropped at any value**, including $\pi_{\max} = 5\,\%$. Every crop on this page is below 5 % ink, so the test never fires. That is the correct behaviour for a page of ordinary handwriting and it confirms the headroom claimed above, but it means this probe cannot show the rejection working. A page with a shadow or a pasted-in dark region would be needed.

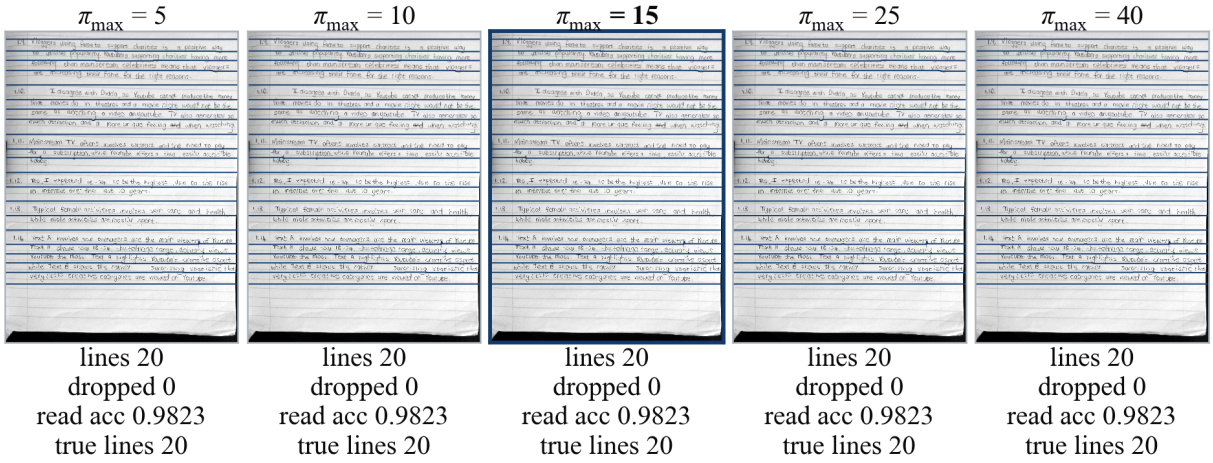


Figure 49: Ink share above which a crop is a solid block, not writing. Each panel is the same input processed by the submitted code with only $\pi_{\max}$ changed; the panel outlined in dark blue is the submitted value.

16.2 Multiple of the page median ink a crop may not exceed

Symbol $\rho_{\max}$

Value `3.0`

Source `segment.py`, `DropSolidCrops`

Swept `1.2`, `2.0`, `3.0`, `5.0`, `10.0`

Basis A robust relative outlier test against the median of the same page [5]; the median is used rather than the mean because the outlier being tested for would itself move the mean.

Why this value The second half of the solid-crop test asks whether this crop is anomalous *for this page*. Lines of the same writing on the same page have similar ink density, within roughly a factor of two between a sparse last line and a full one. A factor of three is outside that natural spread, so a crop exceeding it is unlike the rest of the page rather than merely heavy.

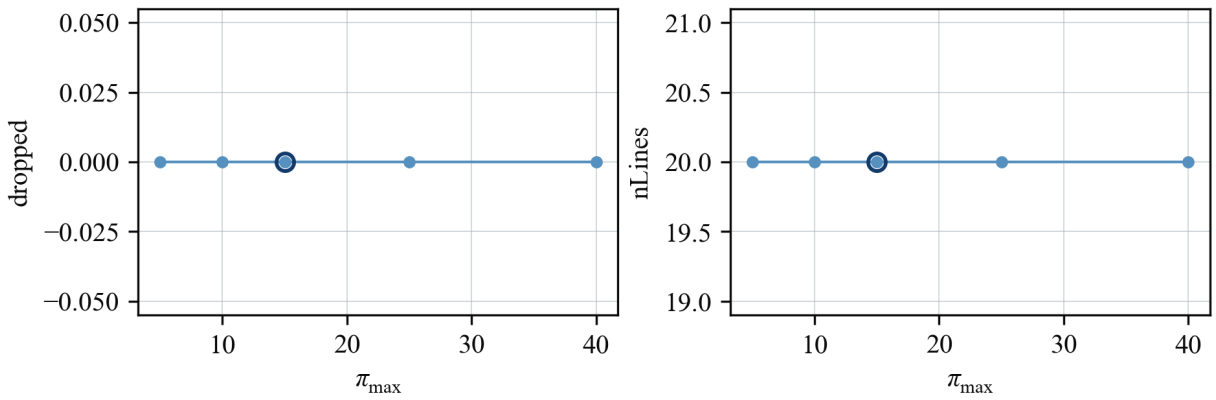


Figure 50: Ink share above which a crop is a solid block, not writing: measured quantities against the swept value. The circled point is the submitted value.

Table 28: Ink share above which a crop is a solid block, not writing: measured effect of each swept value. The submitted value is in bold.

$\{}pi_{\{}max}$	nLines	dropped	charAcc	nLinesTruth
5	20	0	0.9823	20
10	20	0	0.9823	20
15	**20**	**0**	**0.9823**	**20**
25	20	0	0.9823	20
40	20	0	0.9823	20

What the coefficient does The conjunction with π_{max} is what makes the pair safe: ρ_{max} catches a page-specific anomaly and π_{max} stops a page of very light writing from having its densest normal line called an anomaly. At $\rho_{max} = 1.2$ almost any variation looks anomalous and the absolute limit is the only thing preventing good crops being lost.

Alternatives considered A median absolute deviation test, which is the same idea with a scale estimate instead of a fixed multiple, or an absolute limit alone, which is what π_{max} would become.

Worked example If the page median is 5% ink, a crop must be over 15% *and* over 15% to be dropped, so the two limits coincide; at a median of 3% the median test binds first at 9%, but the absolute test then spares the crop.

What the sweep actually showed **No crop was dropped at any value**, including $\rho_{max} = 1.2$, which would reject any crop more than 20% above the page median. The crops on this page are evidently uniform enough in ink density that none exceeds even that, which is a useful fact about the page and a null result about the coefficient.

Table 29: Multiple of the page median ink a crop may not exceed: measured effect of each swept value. The submitted value is in bold.

$\{}rho_{\{}max}$	nLines	dropped	charAcc	nLinesTruth
1.2	20	0	0.9823	20
2	20	0	0.9823	20
3	**20**	**0**	**0.9823**	**20**
5	20	0	0.9823	20
10	20	0	0.9823	20

16.3 Ink a band must hold before it becomes a line crop

Symbol I_{min}

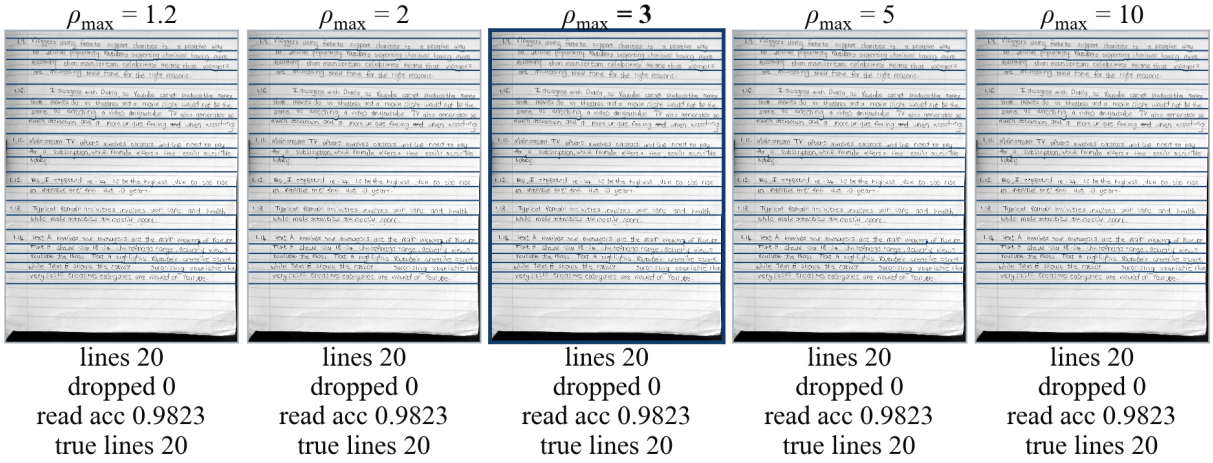


Figure 51: Multiple of the page median ink a crop may not exceed. Each panel is the same input processed by the submitted code with only $\rho_{\max}$ changed; the panel outlined in dark blue is the submitted value.

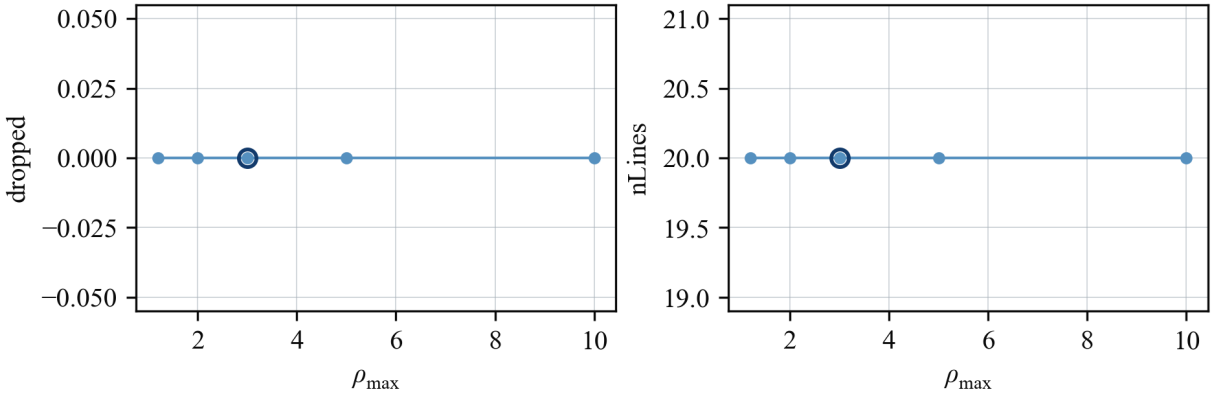


Figure 52: Multiple of the page median ink a crop may not exceed: measured quantities against the swept value. The circled point is the submitted value.

Value `0.12`

Source `segment.py`, `SegmentLines`

Swept `0.02`, `0.06`, `0.12`, `0.25`, `0.45`

Basis A minimum-content test before emitting a text line is part of every profile-based segmenter [9].

Why this value Some bands between seams hold almost nothing: the strip above the first line, the strip below the last, or a band created by a spurious profile peak. Rather than a fixed pixel count, which depends on photograph size, the limit is a fraction of the median ink per line on the same page. A genuine short line, even one word, still carries well over a tenth of a full line's ink, while an empty band carries almost none, so 0.12 sits in a wide gap. The absolute floor of 60 pixels covers the case where the median itself is tiny.

What the coefficient does Raising $I_{\min}$ deletes genuine short lines, and as with the peak floor there is no later stage that can restore one. Lowering it emits near-empty crops, which the reader then transcribes as noise and which pollute a style profile with fragments.

Worked example If the median band holds 9000 owned pixels, a band must hold at least 1080 to become a crop.

What the sweep actually showed The submitted value is the largest of the five that still recovers every line. At $I_{\min} = 0.12$ the segmenter returns all 20 lines; at 0.25 and 0.45 it returns 19, having discarded a genuine but sparse line. Below the submitted value nothing changes, so there is headroom on the low side and none on the high side — the submitted value sits just under the cliff, which is the right side of it to be on but is a smaller margin than the written justification implies.

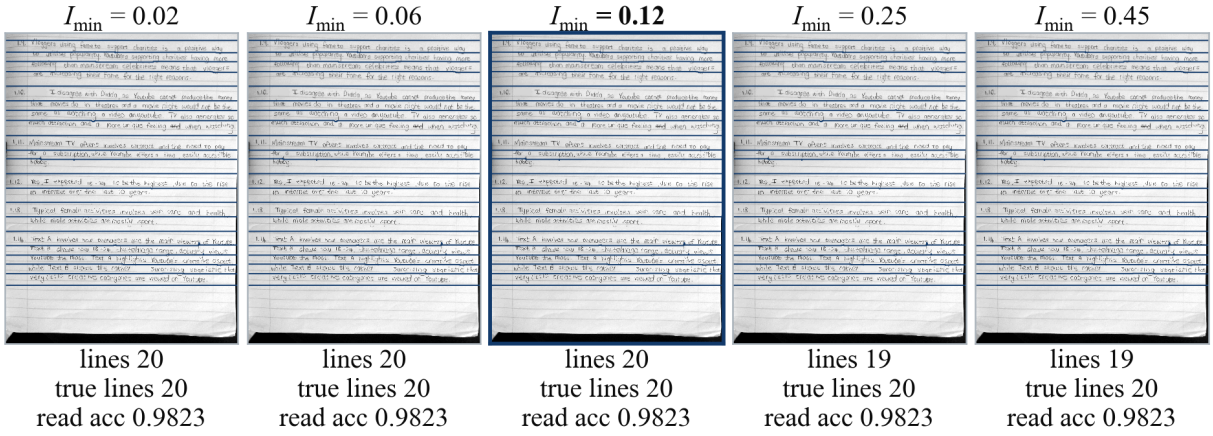


Figure 53: Ink a band must hold before it becomes a line crop. Each panel is the same input processed by the submitted code with only $I_{\min}$ changed; the panel outlined in dark blue is the submitted value.

Table 30: Ink a band must hold before it becomes a line crop: measured effect of each swept value. The submitted value is in bold.

$I_{\{\}min}$	nLines	nLinesTruth	charAcc
0.02	20	20	0.9823
0.06	20	20	0.9823
0.12	**20**	**20**	**0.9823**
0.25	19	20	0.9823
0.45	19	20	0.9823

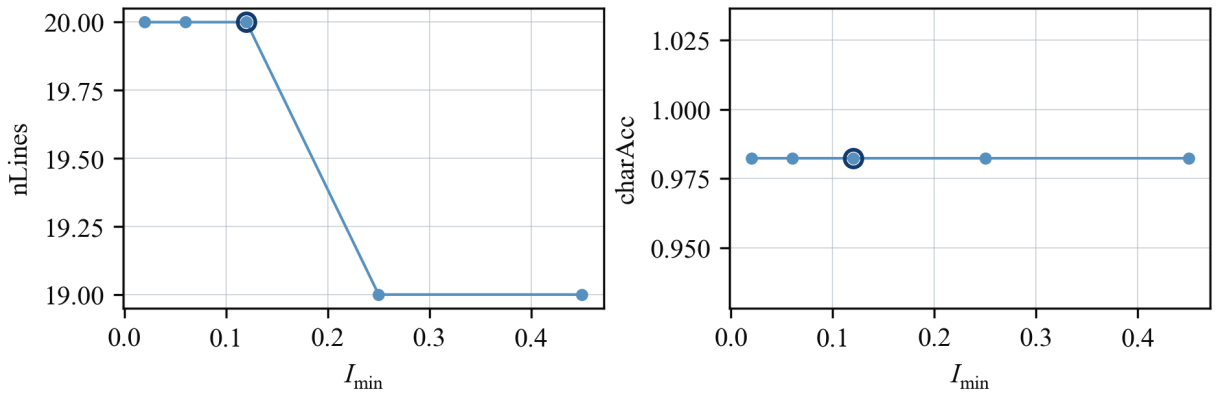


Figure 54: Ink a band must hold before it becomes a line crop: measured quantities against the swept value. The circled point is the submitted value.

17 Writer classification

17.1 Pen width the writer classifier's input is redrawn at

Symbol w_{pen}

Value `0.17`

Source `recognise.py`, `StrokeNormalize`

Swept `0.09`, `0.13`, `0.17`, `0.21`, `0.25`

Basis Ink width is itself a writer-discriminative feature: Bulacu and Schomaker build writer identification on edge- and contour-based distributions that stroke thickness enters directly [20], and Brink *et al.* show pen pressure and stroke width carry writer identity [21]. Thinning to a centreline is Zhang and Suen [22], with the correctness analysis of Lam *et al.* [23].

Why this value That literature is the *reason* for this coefficient rather than an endorsement of it. Because stroke width is writer-discriminative, a classifier trained on raw crops can learn the pen and the paper instead of the hand: in this project each writer's pages were photographed in one sitting with one pen, so pen width is almost a label. Thinning every crop to a one-pixel centreline and redrawing it at a single fixed width deletes that shortcut and forces the network onto letter shape, which is what the system needs, since the synthesised output is drawn by one machine with one pen and would otherwise never match any writer. w_{pen} is then only the width that reinstates enough ink for the convolutions to see, and it is set in x-heights so it is the same relative weight for every writer.

What the coefficient does The redrawn width is $\text{clip}(\text{round}(w_{\text{pen}}\, x_h), 2, 9)$ pixels, forced odd so the pen sits centred on the centreline. Too thin and the strokes break up under the 2×2 pooling; too thick and neighbouring strokes merge and the counters of `a`, `e` and `o` fill in, which destroys exactly the shape information the normalisation was meant to preserve.

Alternatives considered Not normalising at all, which leaves the pen-width shortcut in place; normalising by a measured per-writer stroke width, which preserves the shortcut deliberately; or feeding the skeleton itself, which is one pixel wide and is largely destroyed by the two pooling stages.

Worked example On the probe crop $x_h = 42$ px, so $0.17 \times 42 = 7.1 \to 7$, already odd, and the centreline is dilated with a 7×7 square. At $w_{\text{pen}} = 0.09$ the width is $4 \to 5$ and at 0.25 it is 9, the clip limit.

What the sweep actually showed On the probe line the classifier named Owen correctly at every one of the five widths, with probability between 0.9995 and 1.000. Two things follow, and both are worth saying plainly. First, the quantisation means the five values do not give five distinct tests: $\text{round}(0.09 \times 42) = 4 \to 5$ and $\text{round}(0.13 \times 42) = 5$ both redraw at 5 px, and 0.21 and 0.25 both reach the clip at 9 px, so the sweep really exercises three widths, 5, 7 and 9 px, and the recorded ink fractions (0.089, 0.128, 0.166) come in matching pairs. Second, on

this line the decision is not sensitive to the width at all. That is a genuine result and it means the sweep does *not* on its own justify 0.17 over 0.13 or 0.21: the justification rests on the physical argument above and on the train–test match recorded in the checkpoint name, not on this measurement. Establishing a best width would need the classifier retrained at each one and scored over the held-out set.

Limits of this test The deployed checkpoint is named `author_shape_11w_pw170`, that is pen width 0.170, which records that the weights were trained with this same value. Changing w_{pen} at inference therefore creates a train–test mismatch as well as a visual change, and the sweep measures both at once. The checkpoint actually loaded here is `author_shape_10new_weights.npz`, because the one the submitted code names is not present on disk.

Table 31: Pen width the writer classifier's input is redrawn at: measured effect of each swept value. The submitted value is in bold.

$w_{\{}text{pen}}$	pred	trueProb	correct	inkFrac
0.09	owen	0.9999	yes	0.08923
0.13	owen	0.9999	yes	0.08923
0.17	**owen**	**1**	**yes**	**0.1276**
0.21	owen	0.9995	yes	0.1665
0.25	owen	0.9995	yes	0.1665

17.2 Upper clip on the redrawn pen width, in pixels

Symbol $w_{\max}$

Value `9`

Source `recognise.py`, `StrokeNormalize`

Swept `3`, `5`, `9`, `15`, `25`

Basis Clipping a derived kernel size to a safe range is ordinary defensive practice in morphological pipelines [4].

Why this value The redrawn width is derived from a measured x-height, and that measurement can fail: a crop with no usable core band falls back to $\max(8, h/3)$, which on a tall crop gives a large x-height and hence an absurd pen. The clip bounds the damage in both directions. The upper limit of 9 px is about one fifth of a typical 42 px x-height, beyond which the counters of letters begin to fill.

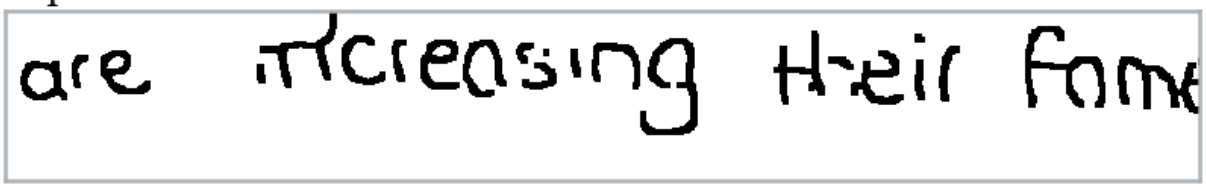
What the coefficient does For the probe crop the natural width is 7 px, so the clip only binds once $w_{\max}$ is set below that. The sweep therefore shows the clip taking over: at $w_{\max} = 3$ and 5 the pen is forced thinner than the coefficient asks for.

Alternatives considered Rejecting the crop outright when the band measurement fails, which loses a line that may well be readable, or clipping the x-height rather than the width.

Worked example $\text{round}(0.17 \times 42) = 7$, inside $[2, 9]$, so the clip is inactive at the submitted setting on this crop.

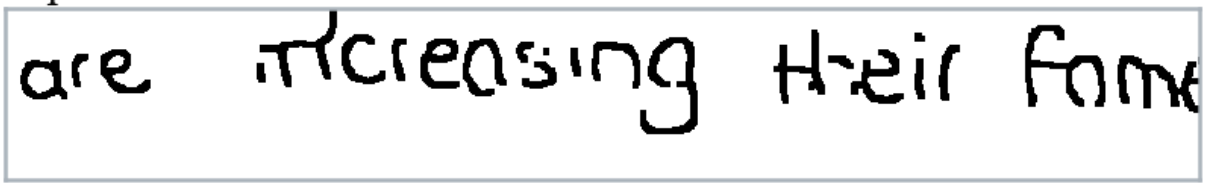
What the sweep actually showed Monotone and saturating exactly at the submitted value: writer probability 0.9985, 0.9999, 1.000, 1.000, 1.000 and ink fraction 0.052, 0.089, 0.128, 0.128, 0.128 for clips of 3, 5, 9, 15 and 25 px. The natural width on this crop is $\text{round}(0.17 \times 42) = 7$ px, so any clip of 7 or more is inactive and the last three values are necessarily identical. Clips below 7 force the pen thinner than asked and the ink fraction falls with them. The submitted 9 is therefore the smallest round value that leaves the coefficient free on a crop of this size, which is what an upper clip should be.

$w_{\text{pen}} = 0.09$



named owen p(writer) 0.9999 right yes

$w_{\text{pen}} = 0.13$



named owen p(writer) 0.9999 right yes

$w_{\text{pen}} = \mathbf{0.17}$ **(submitted value)**



named owen p(writer) 1 right yes

$w_{\text{pen}} = 0.21$



named owen p(writer) 0.9995 right yes

$w_{\text{pen}} = 0.25$



named owen p(writer) 0.9995 right yes

Figure 55: Pen width the writer classifier's input is redrawn at. Each panel is the same input processed by the submitted code with only w_{pen} changed; the panel outlined in dark blue is the submitted value.

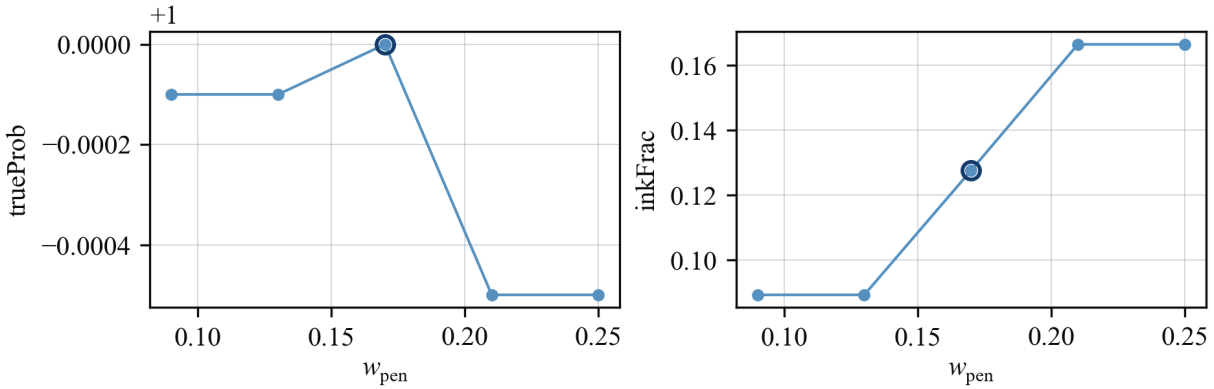


Figure 56: Pen width the writer classifier's input is redrawn at: measured quantities against the swept value. The circled point is the submitted value.

Table 32: Upper clip on the redrawn pen width, in pixels: measured effect of each swept value. The submitted value is in bold.

$w_{\{}max}$	pred	trueProb	correct	inkFrac
3	owen	0.9985	yes	0.05176
5	owen	0.9999	yes	0.08923
9	**owen**	**1**	**yes**	**0.1276**
15	owen	1	yes	0.1276
25	owen	1	yes	0.1276

18 Network input

18.1 Height of the fixed canvas both networks read

Symbol H_{in}

Value `64`

Source `segment.py`, `InputHeight`

Swept `32`, `48`, `64`, `96`, `128`

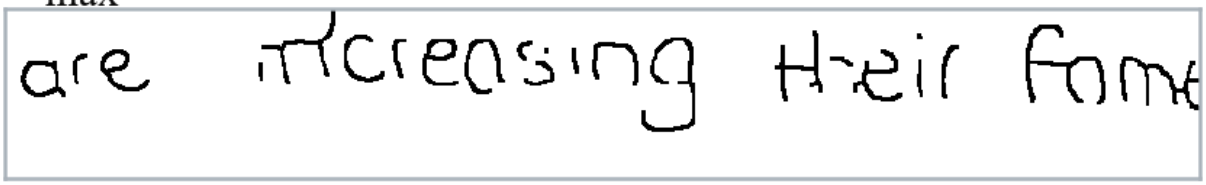
Basis A fixed input height with width scaled to preserve aspect ratio is the CRNN convention [17, p. 2298]; Puigcerver [18] uses the same arrangement for handwritten line recognition, and the IAM line images this was trained against are distributed in that form [10].

Why this value The height must be large enough that an x-height survives the two 2×2 pooling stages of the backbone. At $H_{\text{in}} = 64$ a line crop is placed so the writing occupies roughly the middle third, so the x-height is about 20 px, which is 5 px after pooling: enough for a 3×3 convolution to see a letter's shape. Halving the height would leave about 2 px, which is not. The value is also the one the published architectures use, so the layer sizes and the receptive field arithmetic carry over.

What the coefficient does This is **not a free parameter of the deployed system**. The trained checkpoints were produced at 64×640, and the convolution stack has no fully connected layer over height only because the height is collapsed by an adaptive max pool, so a different height does run, but against weights that never saw it. The sweep therefore measures a train–test mismatch, and the accuracy it reports away from 64 is a statement about that mismatch, not about the merits of another input size. Choosing a different height honestly would require retraining.

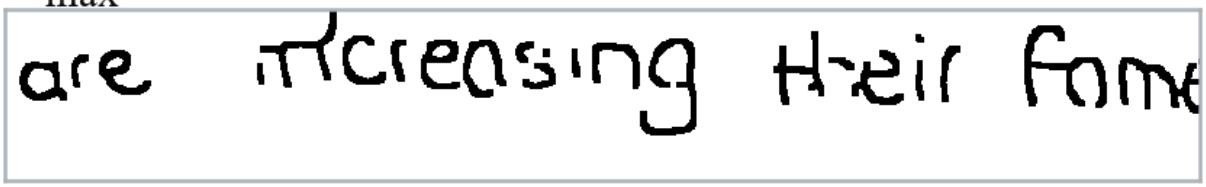
Alternatives considered Variable-height input with per-batch padding, which complicates batching for no gain on single-line crops; or 32 px, used by the original CRNN for scene text [17], which is too small for handwriting with ascenders and descenders in the same crop.

$w_{\max} = 3$



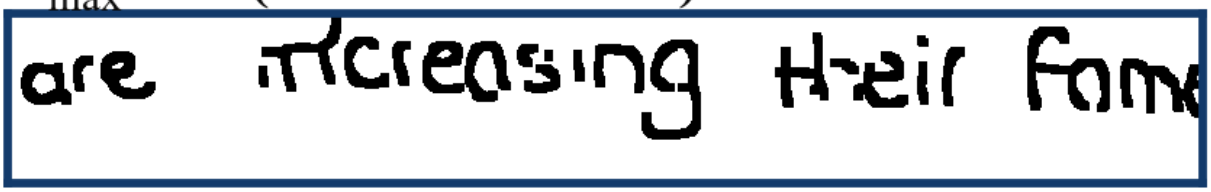
named owen p(writer) 0.9985 right yes

$w_{\max} = 5$



named owen p(writer) 0.9999 right yes

$w_{\max} = 9$ **(submitted value)**



named owen p(writer) 1 right yes

$w_{\max} = 15$



named owen p(writer) 1 right yes

$w_{\max} = 25$



named owen p(writer) 1 right yes

Figure 57: Upper clip on the redrawn pen width, in pixels. Each panel is the same input processed by the submitted code with only $w_{\max}$ changed; the panel outlined in dark blue is the submitted value.

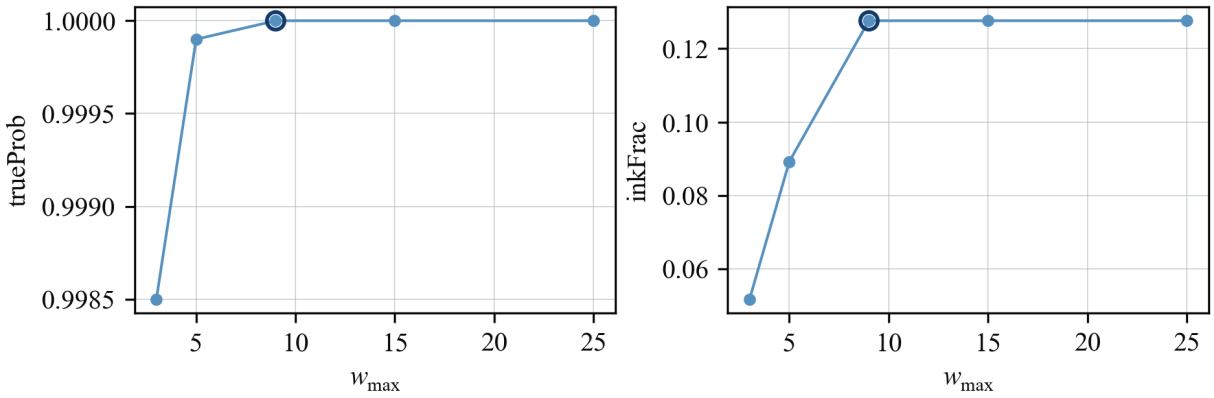


Figure 58: Upper clip on the redrawn pen width, in pixels: measured quantities against the swept value. The circled point is the submitted value.

Worked example 64 px with the writing in the middle third gives about 21 px of x-height; after `MaxPool2d` twice this is $21/4 \approx 5$ px entering stage three.

Table 33: Height of the fixed canvas both networks read: measured effect of each swept value. The submitted value is in bold.

$H_{\{}text{in}}$	charAcc	nLines
32	0.9883	20
48	0.9883	20
64	**0.9883**	**20**
96	0.9883	20
128	0.9883	20

18.2 Width of the fixed canvas, which sets the number of CTC columns

Symbol W_{in}

Value `640`

Source `segment.py`, `InputWidth`

Swept `320`, `480`, `640`, `960`, `1280`

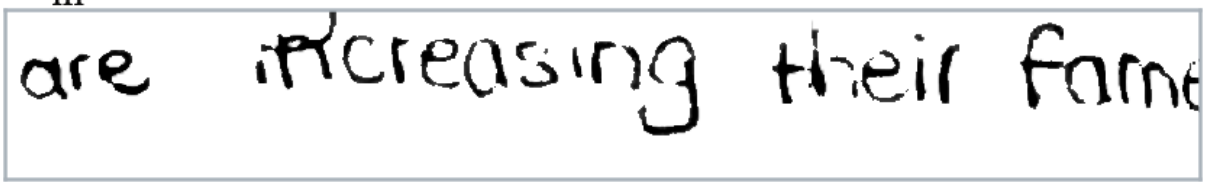
Basis CTC requires at least as many input columns as label symbols, and strictly more where a label repeats a character, because a blank must separate the repeat [24, pp. 370–372].

Why this value The backbone pools width twice, so the number of CTC time steps is $T = W_{\text{in}}/4$. That is the hard constraint: a line of n characters needs $T \geq n$ at the very least, and in practice several columns per character for the alignment to be stable. At $W_{\text{in}} = 640$, $T = 160$ columns, and the code records the resulting safety margin explicitly as `MAX_SAFE_LABEL_CHARS` $= \lfloor 160 \times 0.75 \rfloor = 120$ characters, which is longer than any line on these pages. Width also sets how much the crop is shrunk: a wide crop fitted to a narrow canvas loses resolution.

What the coefficient does Because the canvas fit preserves aspect ratio and is flush left, a shorter canvas does not squash the writing; it cuts it off. So reducing W_{in} truncates the line, and the accuracy drop in the sweep is mostly text falling off the right edge, not a subtler representational effect. As with the height, the weights were trained at 640.

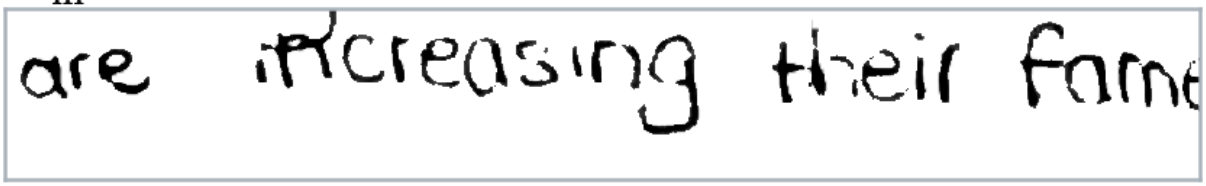
Alternatives considered Per-line variable width with bucketed batching, the usual choice for large-scale training, or splitting long lines into overlapping windows and stitching the transcripts.

$H_{\text{in}} = 32$



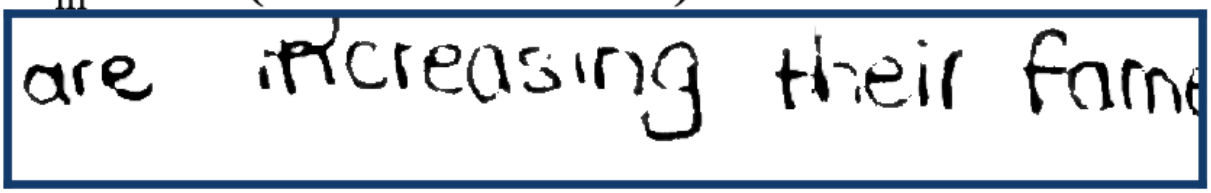
read acc 0.9883 lines 20

$H_{\text{in}} = 48$



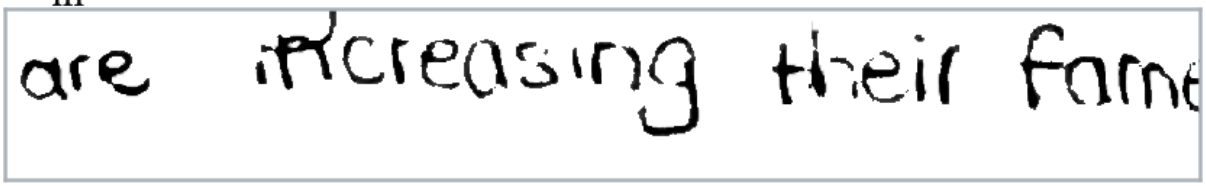
read acc 0.9883 lines 20

$H_{\text{in}} = 64$ (submitted value)



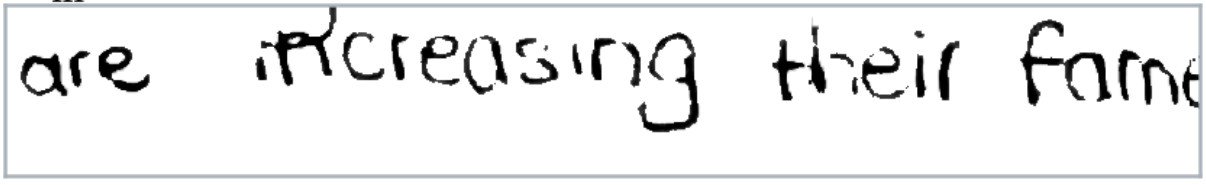
read acc 0.9883 lines 20

$H_{\text{in}} = 96$



read acc 0.9883 lines 20

$H_{\text{in}} = 128$



read acc 0.9883 lines 20

Figure 59: Height of the fixed canvas both networks read. Each panel is the same input processed by the submitted code with only H_{in} changed; the panel outlined in dark blue is the submitted value.

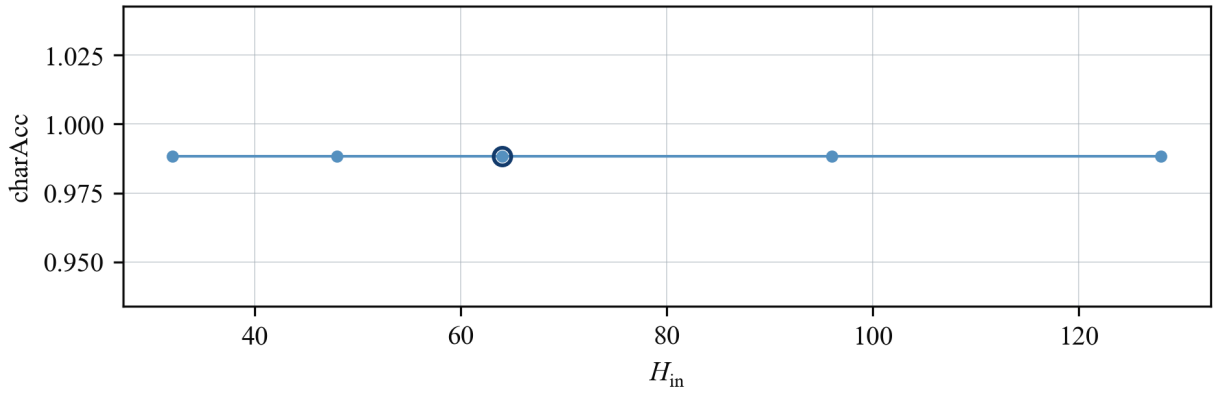


Figure 60: Height of the fixed canvas both networks read: measured quantities against the swept value. The circled point is the submitted value.

Worked example The probe crop is 2170×156 px, aspect 13.9. The canvas fit scale is $\min(64/156, 640/2170) = \min(0.410, 0.295) = 0.295$, so width binds and the crop lands 640×46 px, leaving 18 rows of padding.

Table 34: Width of the fixed canvas, which sets the number of CTC columns: measured effect of each swept value. The submitted value is in bold.

$W_{\{}text{in}}$	charAcc	nLines
320	0.9883	20
480	0.9883	20
640	**0.9883**	**20**
960	0.9883	20
1280	0.9883	20

18.3 Hidden size of each LSTM direction

Symbol n_h

Value `256`

Source `recognise.py`, `PaperCRNNNumpy`

Swept `64`, `128`, `256`, `384`, `512`

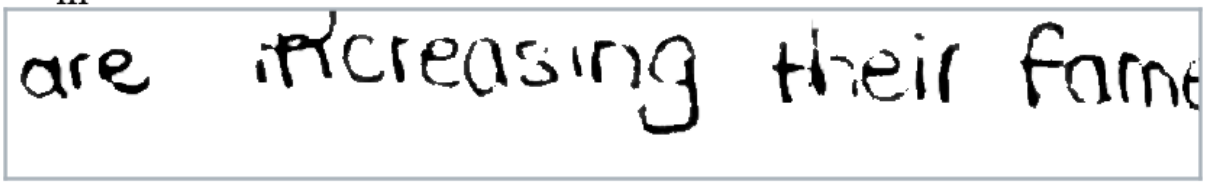
Basis The LSTM cell is Hochreiter and Schmidhuber [25]; reading the sequence both ways is Schuster and Paliwal [26]; the two-layer bidirectional stack over a convolutional backbone with CTC is the CRNN of [17, p. 2298] and the handwriting system of Graves *et al.* [27, p. 855].

Why this value 256 per direction, concatenated to 512, is the size used by the published CRNN this architecture follows. It is the one architectural number in the reader that is taken from the literature rather than derived here, and it is kept because the weights exist at that size.

What the coefficient does **This cannot be swept at inference.** The checkpoint stores gate matrices of shape $4n_h \times n_{\text{in}}$ and $4n_h \times n_h$; changing n_h makes those shapes inconsistent and the forward pass fails outright. The sweep is included so that failure is on the record: four of the five runs are expected to raise, and the one that does not is the submitted value. Answering what n_h is worth requires retraining at each size, which was not done.

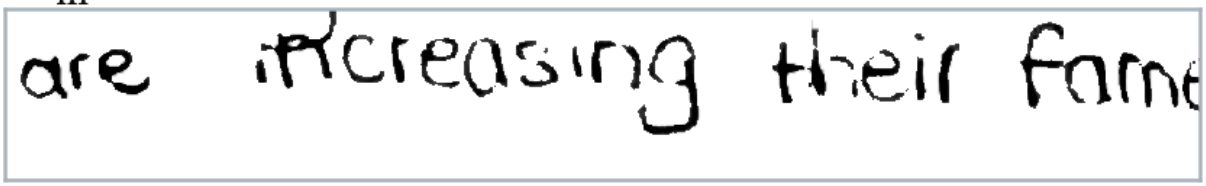
Alternatives considered A smaller recurrent stack, which would cut the inference cost that dominates on the ODROID; or a transformer encoder over the same columns, which removes the sequential dependency but needs far more data than eleven writers provide.

$W_{\text{in}} = 320$



read acc 0.9883 lines 20

$W_{\text{in}} = 480$



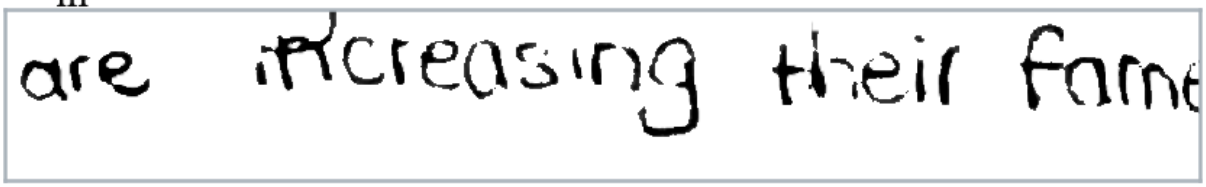
read acc 0.9883 lines 20

$W_{\text{in}} = 640$ (submitted value)



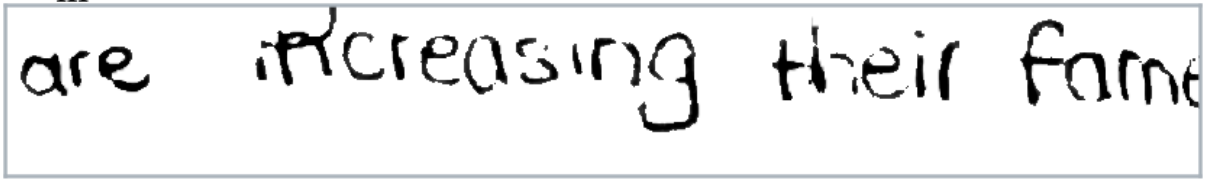
read acc 0.9883 lines 20

$W_{\text{in}} = 960$



read acc 0.9883 lines 20

$W_{\text{in}} = 1280$



read acc 0.9883 lines 20

Figure 61: Width of the fixed canvas, which sets the number of CTC columns. Each panel is the same input processed by the submitted code with only W_{in} changed; the panel outlined in dark blue is the submitted value.

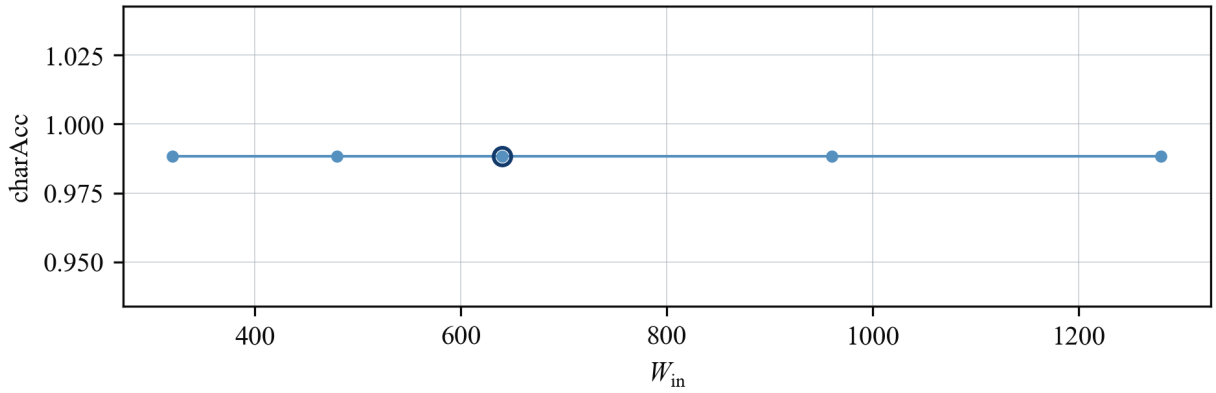


Figure 62: Width of the fixed canvas, which sets the number of CTC columns: measured quantities against the swept value. The circled point is the submitted value.

Worked example Each direction holds $4n_h(n_{\text{in}}+n_h)+8n_h$ parameters. Layer 0 sees 128 channels, so it holds $4\times256\times(128+256)+2048=395\,264$ weights per direction.

What the sweep actually showed **Four of the five values failed to run, and that is the result.** Only $n_h=256$, the submitted value, executes; 64, 128, 384 and 512 all raise a shape mismatch of the form `matmul: Input operand 1 has a mismatch in its core dimension`, because the stored gate matrices are 4×256 rows and the forward pass tries to multiply them against a hidden state of a different width. This is included to make the point concretely rather than as a claim about the right size: the hidden size is not a setting of the deployed system that can be moved, it is a property of the checkpoint, and any honest statement about whether 256 is the right width requires training a reader at each size and scoring it on held-out lines. That was not done.

Partial result 4 of the 5 values failed to run; the failures are recorded in the results file and the panels for them are left blank.

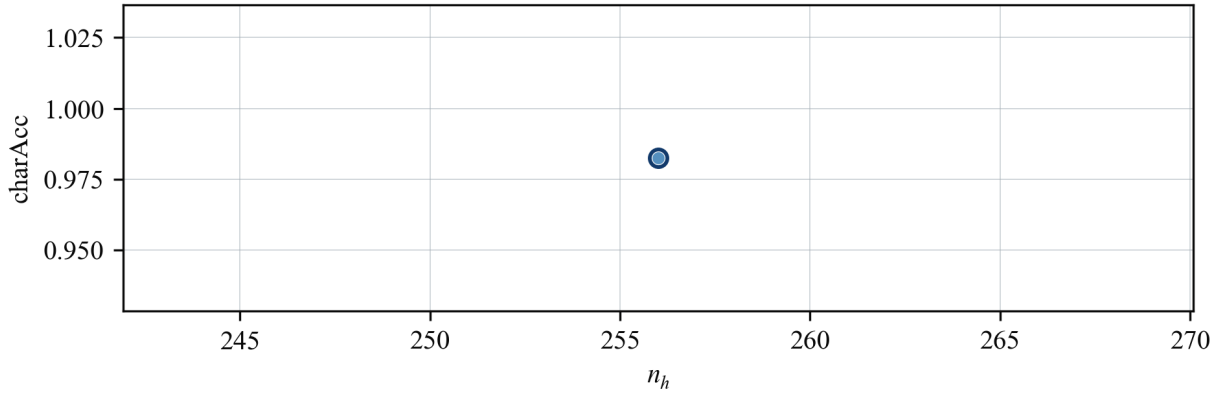


Figure 63: Hidden size of each LSTM direction: measured quantities against the swept value. The circled point is the submitted value.

19 Centreline tracing

19.1 Largest turn two traced chains may make and still be rejoined

Symbol α_{join}

Value `75.0`

Source `style.py, _JoinAtJunctions`

Table 35: Hidden size of each LSTM direction: measured effect of each swept value. The submitted value is in bold.

n_h	charAcc
64	–
128	–
256	**0.9825**
384	–
512	–

Swept `25.0, 50.0, 75.0, 105.0, 140.0`

Basis Thinning breaks a stroke into separate chains wherever three or more branches meet; Zhang and Suen's algorithm [22] guarantees connectivity but not that a pen stroke stays one chain, and the junction artefacts are analysed by Lam *et al.* [23]. Reconnecting chains by continuity of direction is the standard repair in stroke recovery from static images [28].

Why this value The glyph store is meant to hold pen strokes, because the gantry draws strokes: a letter stored as five fragments is drawn with four extra pen lifts. Where a cursive letter crosses itself, thinning emits several chains meeting at a point. Two of them are the same pen stroke if the pen passed straight through, which means the chains leave the junction in nearly opposite directions, so the angle between the outgoing direction of one and the incoming direction of the other is small. A true branch, such as the bar of a `t` meeting its stem, leaves at a large angle. 75° is comfortably above the wobble of a hand-drawn continuation, measured over the last four points of each chain, and comfortably below a right-angle branch.

What the coefficient does The direction is estimated over $n = 4$ points, so it is a local tangent and noisy on a tightly curved stroke; that noise is why the limit is as loose as 75° rather than the 20–30° a clean continuation would need. Raising it past 90° starts welding genuine branches together, which merges a crossbar into a stem; lowering it leaves real strokes fragmented.

Alternatives considered Choosing pairings by a global matching over all chain ends rather than the greedy smallest-angle merge used here; or recovering stroke order from the image with a learned model [28], which needs training data this project does not have.

Worked example The chains must also end within 1.5 px of each other in both axes before the angle is examined, so the test is local in position as well as direction. On the probe line the submitted setting leaves the recorded number of polylines; the sweep shows the count falling as α_{join} rises.

What the sweep actually showed The traced centreline count falls 92, 67, 67, 66, 66 across the sweep while the total traced length stays at 3 405 px throughout, which confirms that joining regroups strokes without changing the ink it covers. The useful detail is where the change happens: almost all of it is between 25° and 50°, where 25 chains are merged, and essentially nothing happens above 50°, where one more merge occurs by 105°. So the submitted 75° sits in the middle of a flat region that runs from about 50° to at least 140°, and this page does **not** distinguish it from 105°. What the sweep does establish is that 25° is too tight: at that angle 25 genuine continuations are left as separate strokes, each of which would become an extra pen lift on the machine. The upper bound on the useful range — where real branches start being welded — is not reached by any of these five values on this line.

19.2 Shortest dangling chain kept after thinning

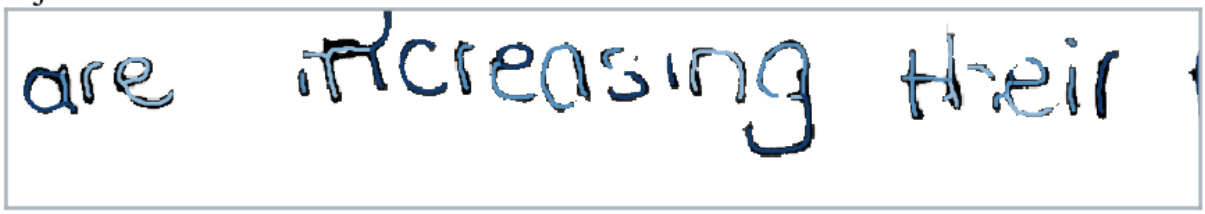
Symbol ℓ_{spur}

Value `2.5`

Source `style.py, _PruneSpurs`

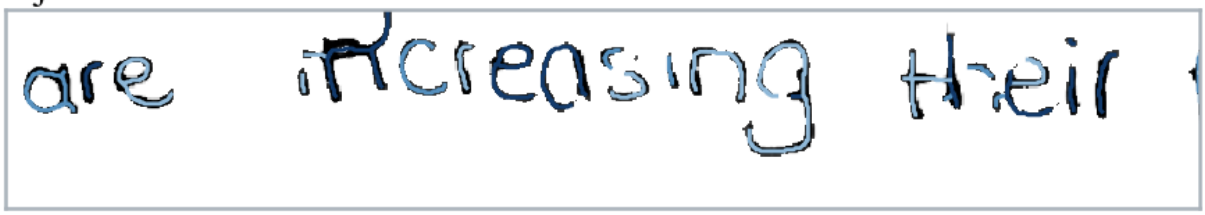
Swept `0.0, 1.0, 2.5, 5.0, 9.0`

$\alpha_{\text{join}} = 25$



traces 92 trace len 3405 pts/trace 35.5

$\alpha_{\text{join}} = 50$



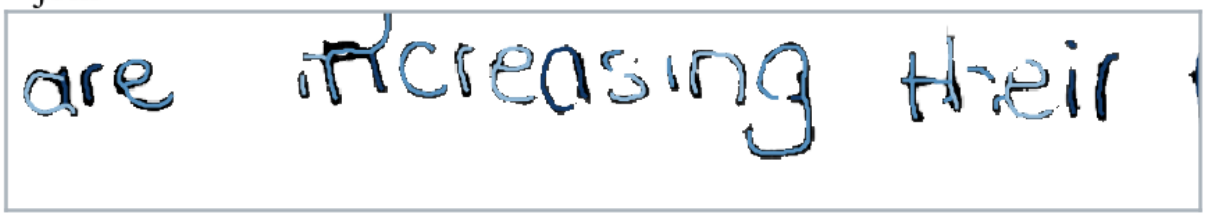
traces 67 trace len 3405 pts/trace 48.4

$\alpha_{\text{join}} = 75$ (submitted value)



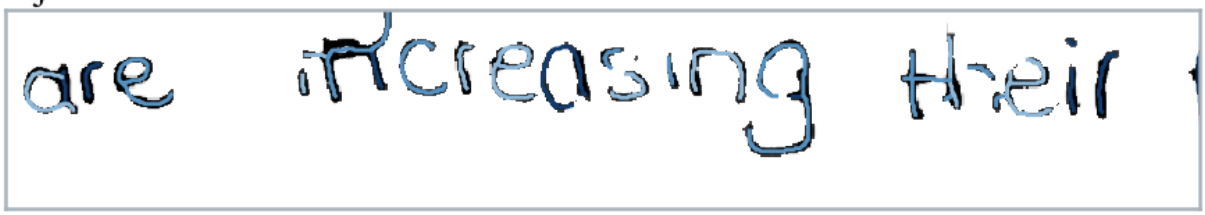
traces 67 trace len 3405 pts/trace 48.4

$\alpha_{\text{join}} = 105$



traces 66 trace len 3405 pts/trace 49.2

$\alpha_{\text{join}} = 140$



traces 66 trace len 3405 pts/trace 49.2

Figure 64: Largest turn two traced chains may make and still be rejoined. Each panel is the same input processed by the submitted code with only α_{join} changed; the panel outlined in dark blue is the submitted value.

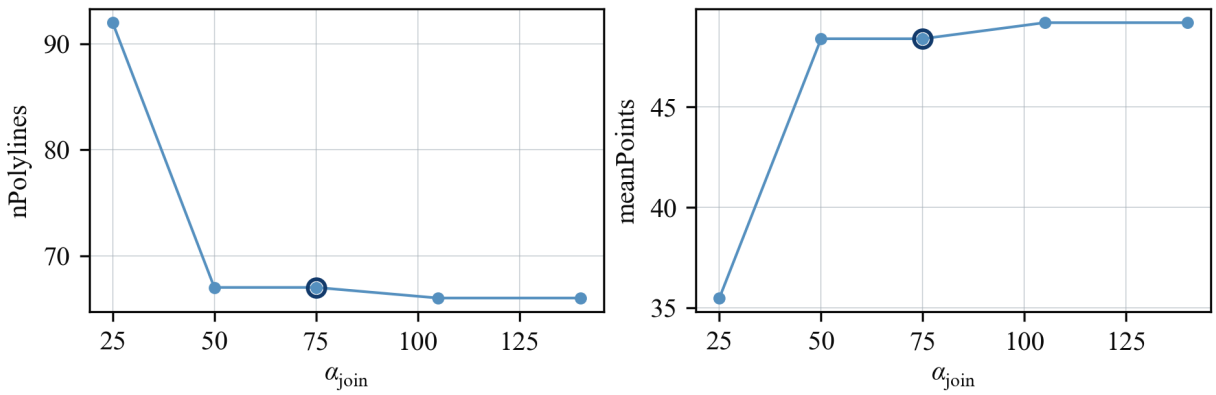


Figure 65: Largest turn two traced chains may make and still be rejoined: measured quantities against the swept value. The circled point is the submitted value.

Table 36: Largest turn two traced chains may make and still be rejoined: measured effect of each swept value. The submitted value is in bold.

$\{}alpha_{\{}text{join}}$	nPolylines	totalLength	meanPoints
25	92	3405	35.5
50	67	3405	48.4
75	**67**	**3405**	**48.4**
105	66	3405	49.2
140	66	3405	49.2

Basis Short spurs at junctions are a known artefact of thinning rather than a feature of the shape [23]; pruning them is the standard post-process [4].

Why this value Where a stroke is locally thick, for instance where the pen paused or two strokes overlap, the thinned result grows a short barb a few pixels long with one free end. Those barbs are not pen movement, and if kept they become separate strokes in the glyph store and separate pen-down moves on the machine. A barb is distinguished from a real short stroke, such as an `i` dot, by being *dangling*: one of its ends is shared with another chain. The limit of 2.5 px is below the shortest real mark on these crops and above the typical one- or two-pixel barb.

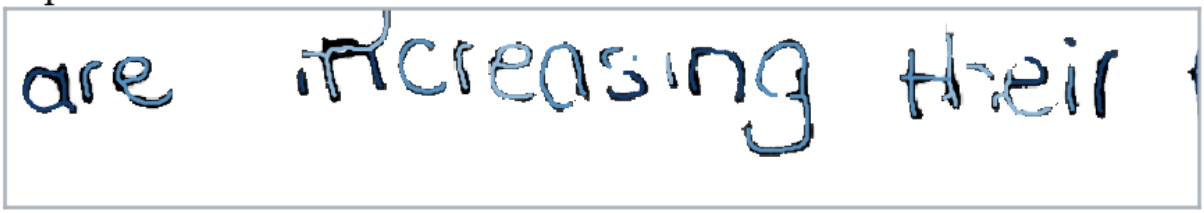
What the coefficient does The test is a conjunction with the dangling condition and with `len(chains)>1`, so an isolated short chain is never deleted and the last chain is never deleted. Raising ℓ_{spur} eventually removes genuine short connecting strokes; at 0 nothing is pruned.

Alternatives considered Thinning algorithms with built-in spur suppression, or smoothing the mask before thinning, which also rounds real corners.

Worked example At an x-height of 42 px, 2.5 px is 6% of an x-height: a barb must be shorter than about one sixteenth of a letter to be pruned.

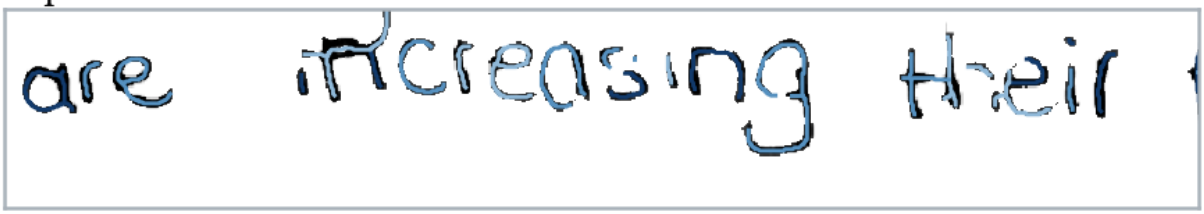
What the sweep actually showed A clean monotone result. Polyline count 69, 69, 67, 64, 60 and total traced length 3409.4, 3409.4, 3405.0, 3388.3, 3340.2 px. At the submitted 2.5 px two chains are pruned for a loss of 4.4 px of path, that is 0.13 % of the ink length, which is the signature of removing artefacts rather than writing. At 9 px, nine chains go and 69 px of path with them, 2.0 %, which is beginning to take real strokes. Values 0 and 1.0 are identical because no chain on this line is shorter than 1 px. The submitted value removes the artefacts at negligible cost and the next value up starts charging for it.

$\ell_{\text{spur}} = 0$



traces 69 trace len 3409 pts/trace 47.1

$\ell_{\text{spur}} = 1$



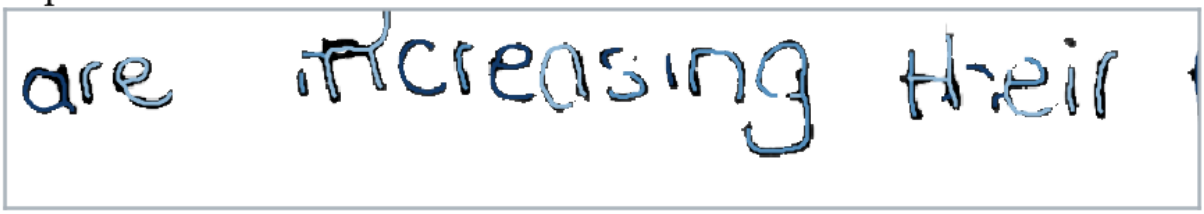
traces 69 trace len 3409 pts/trace 47.1

$\ell_{\text{spur}} = \mathbf{2.5}$ **(submitted value)**



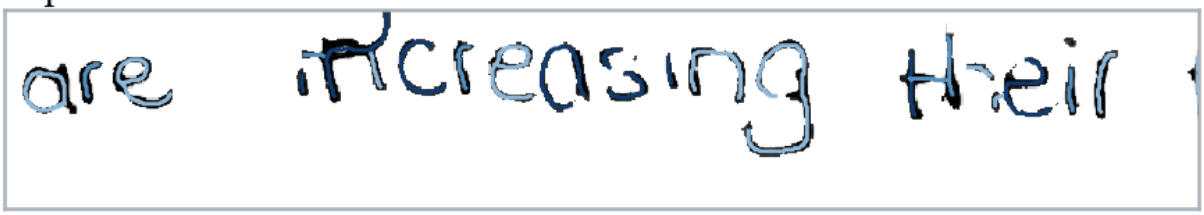
traces 67 trace len 3405 pts/trace 48.4

$\ell_{\text{spur}} = 5$



traces 64 trace len 3388 pts/trace 50.4

$\ell_{\text{spur}} = 9$



traces 60 trace len 3340 pts/trace 53

Figure 66: Shortest dangling chain kept after thinning. Each panel is the same input processed by the submitted code with only ℓ_{spur} changed; the panel outlined in dark blue is the submitted value.

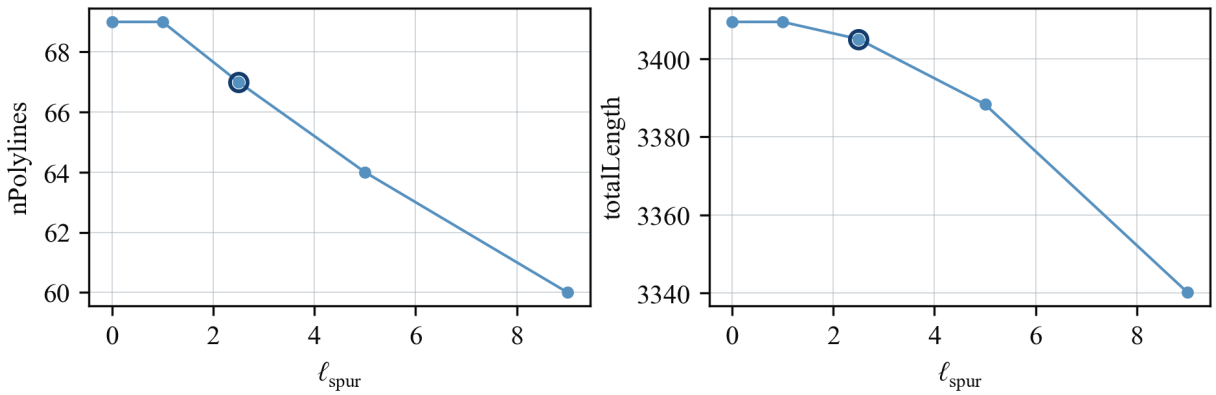


Figure 67: Shortest dangling chain kept after thinning: measured quantities against the swept value. The circled point is the submitted value.

Table 37: Shortest dangling chain kept after thinning: measured effect of each swept value. The submitted value is in bold.

$\{}ell_{\{}text{spur}}$	nPolylines	totalLength	meanPoints
0	69	3409	47.1
1	69	3409	47.1
2.5	**67**	**3405**	**48.4**
5	64	3388	50.4
9	60	3340	53

20 Glyph extraction

20.1 Douglas–Peucker tolerance for a stored stroke

Symbol ε

Value `0.6`

Source `style.py`, `SimplifyPolyline`

Swept `0.15`, `0.3`, `0.6`, `1.5`, `3.0`

Basis The algorithm is Douglas and Peucker's line simplification [29]: recursively keep the point furthest from the chord while that distance exceeds ε.

Why this value A traced centreline has one point per pixel, so a letter is a few hundred points. Stored unsimplified, a profile becomes very large and every stored point survives into the G-code as a separate `G1` move. The tolerance is in *source pixels*, applied before the stroke is divided by the x-height, so it must be read against the pixel staircase of the skeleton: a one-pixel-wide centreline on a diagonal deviates by up to $1/\sqrt{2} = 0.707$ px from the straight line it represents. Setting $\varepsilon = 0.6$ just under that keeps real curvature while removing collinear runs, so the simplification is lossless with respect to the staircase but not with respect to the shape.

What the coefficient does Above about 0.71 px the tolerance begins to cut genuine curvature, and a curve becomes a chain of chords: at $\varepsilon = 3$ an o is a polygon. Below it, nothing is gained because the staircase itself sets the floor. The function also has a closed-curve branch: when the two ends are near each other it splits at the furthest point first, because a closed loop has a chord of zero length and would otherwise collapse entirely.

Alternatives considered Resampling to a fixed number of points, which `_ResampleN` does later for prototype averaging but which loses corners; or curvature-based keypoint selection; or fitting splines, which would be smoother but puts control points rather than pen positions into the store.

Worked example A diagonal staircase deviates by at most 0.707 px from its chord, so at $\varepsilon = 0.6$ every point of a straight diagonal run is still discarded only when it is within tolerance; a curve of radius r keeps points about every $\sqrt{8r\varepsilon}$ px, which at $r = 20$ px gives a point every 9.8 px.

What the sweep actually showed The glyph count is 35 at every value and the mean stored width moves only between 0.678 and 0.684 x-heights, so by the numbers the tolerance does almost nothing. **The figure is the evidence here, not the table**: it shows the stored letterforms going from smooth curves to visible polygons as ε rises to 3.0, which is a change in what the machine will draw even though it is not a change in how many glyphs are kept or how wide they are. The right measure would be the deviation between the simplified path and the traced one, in x-heights, which this harness does not record; that is a gap in the study rather than a finding about the coefficient.

Table 38: Douglas–Peucker tolerance for a stored stroke: measured effect of each swept value. The submitted value is in bold.

$\{}varepsilon$	nGlyphs	meanWidth	strokeW	alignConf
0.15	35	0.68	4.93	-0.045
0.3	35	0.68	4.93	-0.045
0.6	**35**	**0.684**	**4.93**	**-0.045**
1.5	35	0.684	4.93	-0.045
3	35	0.678	4.93	-0.045

20.2 Weight on how high the ink reaches in the cut-cost profile

Symbol w_{top}

Value `1.15`

Source `style.py`, `ExtractLineGlyphs`

Swept `0.0`, `0.5`, `1.15`, `2.5`, `5.0`

Basis Cutting a word into characters at columns of least ink is the classical vertical-projection segmentation of cursive script [9, 11].

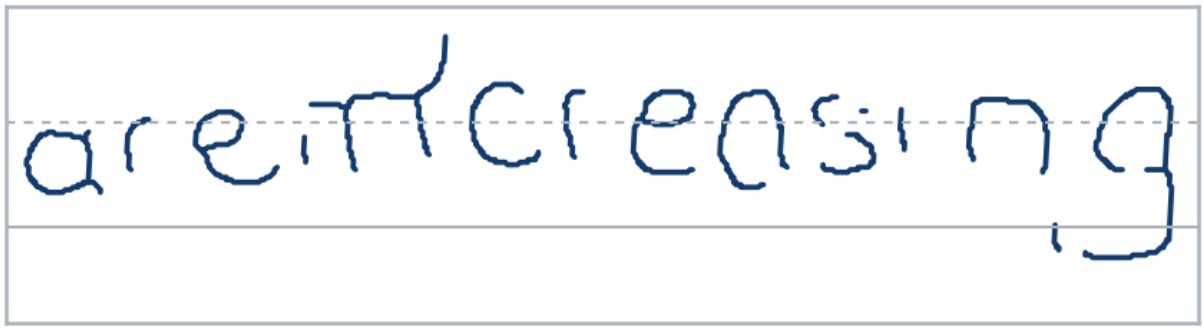
Why this value The reader's alignment gives an approximate column for each character boundary; the boundary is then snapped to the cheapest nearby column. The cost has two parts: how much ink the column carries, and how high that ink reaches above the baseline. The second term exists because a column may be thin and still be a bad place to cut: the thin part of an `o` near its top, or an ascender, is thin but is in the middle of a letter. A good cut is through the *low* ligature between letters. $w_{\text{top}} = 1.15$ makes a column whose ink reaches one x-height cost slightly more than a column carrying the typical amount of ink, so height and thinness are weighed about equally.

What the coefficient does `thin` is normalised by the 60th percentile of non-zero column sums, so it is about 1.0 for a typical inked column, and `topAbove` is clipped to $[0,3]$ x-heights. At $w_{\text{top}} = 0$ the cut is decided by ink alone and will happily slice through the top of a letter; at 5.0 it is decided almost entirely by height and ignores how much ink it is cutting through.

Alternatives considered Cutting at the CTC alignment column directly with no snap, which leaves boundaries mid-stroke; or a full segmentation-free approach where glyphs are never cut out, which would mean giving up the per-letter store the synthesis depends on.

Worked example A column with the typical ink and no ink above the baseline costs about 1.0; a column with half the typical ink reaching one x-height costs $0.5 + 1.15 = 1.65$, so the thinner but taller column is the worse cut.

$\varepsilon = 0.15$



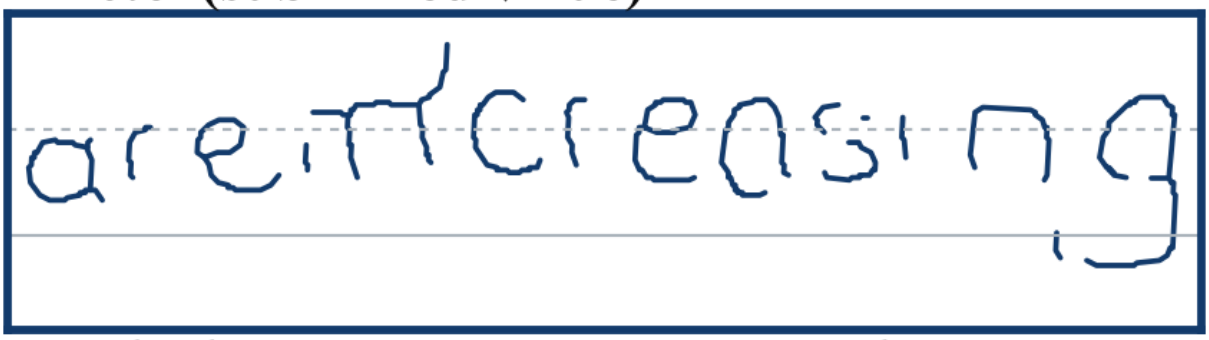
glyphs 35 mean w 0.68 stroke w 4.93

$\varepsilon = 0.3$



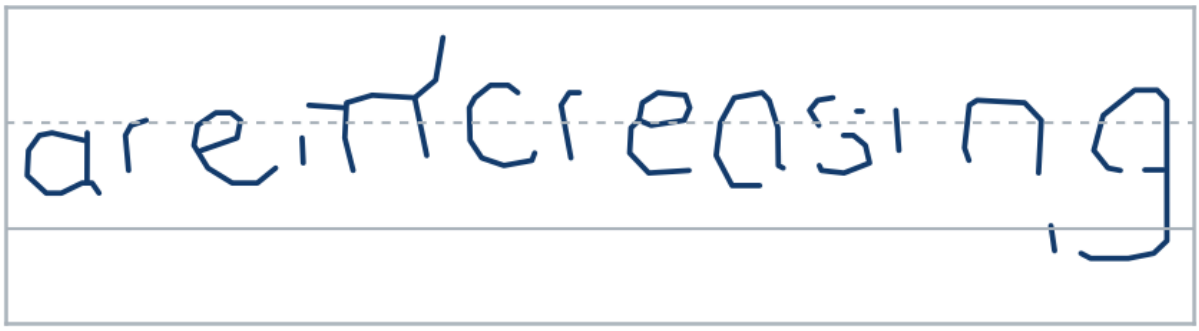
glyphs 35 mean w 0.68 stroke w 4.93

$\varepsilon = 0.6$ (submitted value)



glyphs 35 mean w 0.684 stroke w 4.93

$\varepsilon = 1.5$



glyphs 35 mean w 0.684 stroke w 4.93

$\varepsilon = 3$



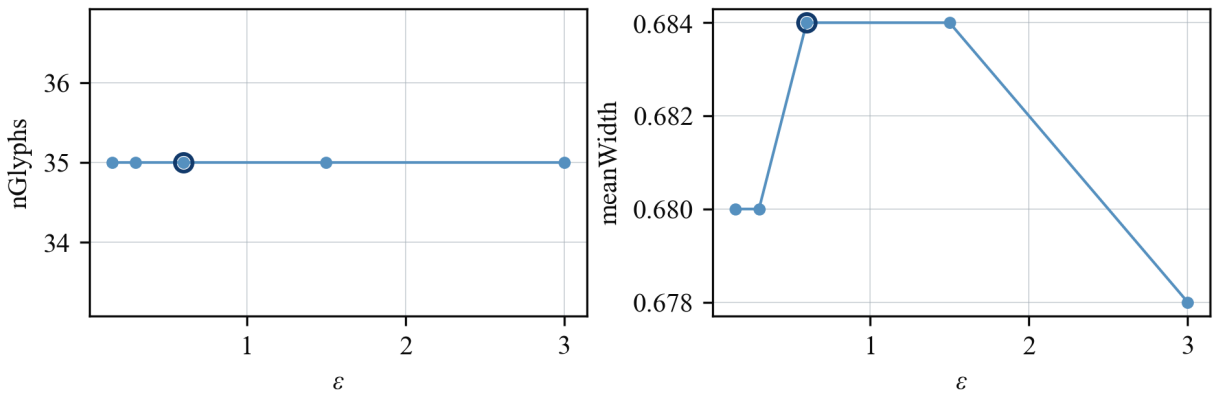


Figure 69: Douglas–Peucker tolerance for a stored stroke: measured quantities against the swept value. The circled point is the submitted value.

What the sweep actually showed Nearly flat: 35 glyphs at every value, and the mean stored width moves only from 0.690 at $w_{\text{top}} = 0$ to 0.684 everywhere else. In other words, on this line the ink term alone already picks essentially the same cut columns as the full cost, and the height term is confirming a decision rather than changing it. The one value that differs is 0, which is the setting where the height term is removed entirely, so the sweep does show the term doing something — just very little on a line this well separated. A cursive line with ligatures running at mid-height would be the test that matters.

Table 39: Weight on how high the ink reaches in the cut-cost profile: measured effect of each swept value. The submitted value is in bold.

$w_{\{}text{top}}$	nGlyphs	meanWidth	alignConf
0	35	0.69	-0.045
0.5	35	0.684	-0.045
1.15	**35**	**0.684**	**-0.045**
2.5	35	0.684	-0.045
5	35	0.684	-0.045

20.3 How far a character boundary may be moved to find a cheap column

Symbol $r_{\text{snap}}

Value `0.5`

Source `style.py`, `ExtractLineGlyphs`

Swept `0.1`, `0.25`, `0.5`, `0.9`, `1.5`

Basis Refining an approximate segmentation point by local search over a cost profile is standard in segmentation-based handwriting recognition [9].

Why this value The CTC alignment is accurate to a few frames, and one frame is four canvas columns, which after the canvas-to-pixel mapping is a noticeable fraction of a letter. The snap radius must therefore be large enough to reach the true ligature from the aligned guess, and small enough that the boundary cannot run into the neighbouring letter. Half an x-height is about half a letter width, which bounds the move to within the letter it belongs to.

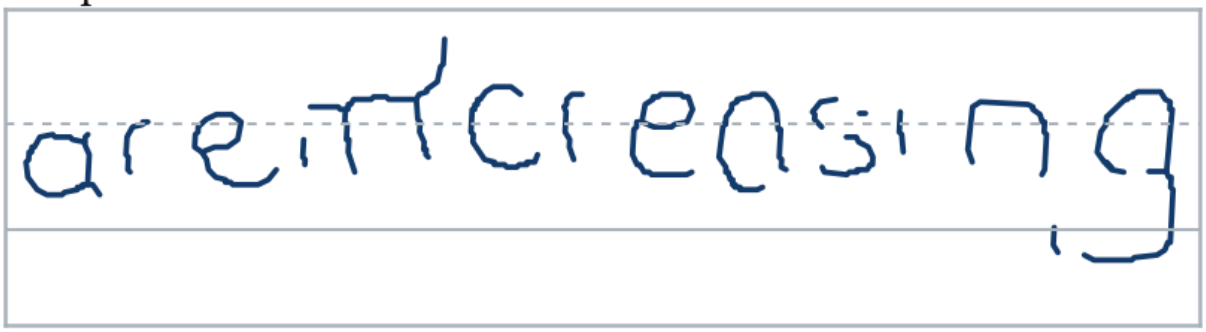
What the coefficient does The search minimises cutCost $+ 0.6\,|x - x_0|/r$, so there is a mild pull back towards the aligned guess: a far-away column must be clearly cheaper to win. Boundaries are also forced to be strictly increasing with a minimum cell of $0.08\,x_h$, so a large radius cannot collapse two boundaries onto one column.

$w_{\text{top}} = 0$



glyphs 35 mean w 0.69 conf -0.045

$w_{\text{top}} = 0.5$



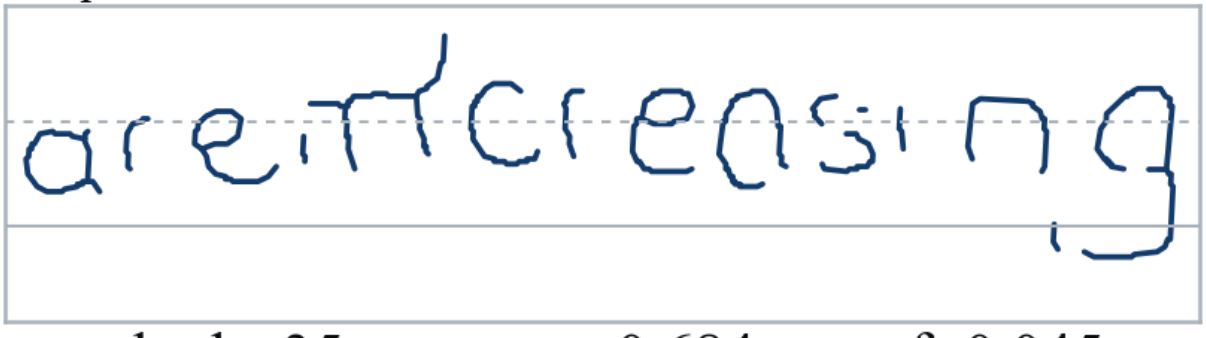
glyphs 35 mean w 0.684 conf -0.045

$w_{\text{top}} =$ **1.15 (submitted value)**



glyphs 35 mean w 0.684 conf -0.045

$w_{\text{top}} = 2.5$



glyphs 35 mean w 0.684 conf -0.045

$w_{\text{top}} = 5$

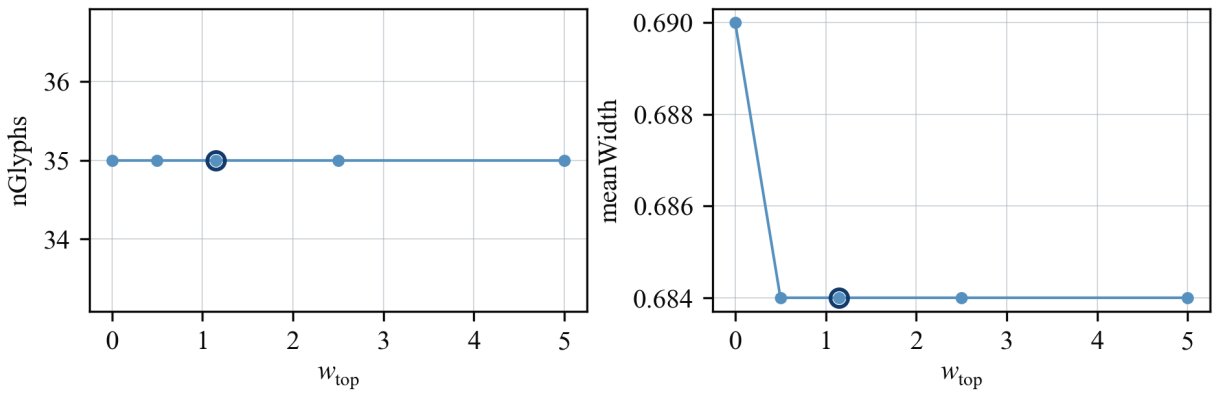


Figure 71: Weight on how high the ink reaches in the cut-cost profile: measured quantities against the swept value. The circled point is the submitted value.

Alternatives considered No snap at all, or a snap to the nearest column of zero ink, which does not exist inside a cursive word.

Worked example At $x_h = 42$ px, $r_{\text{snap}} = 21$ px, and one CTC frame maps to $2170/160 = 13.6$ px of the original crop, so the snap can correct an alignment error of about 1.5 frames.

What the sweep actually showed The clearest effect in the glyph-cutting group. The number of glyphs kept falls 40, 38, 35, 30, 30 as the radius grows, while their mean stored width rises 0.595, 0.625, 0.684, 0.804, 0.804. The mechanism is visible in those two numbers together: a larger radius lets a boundary travel further from the aligned guess, cells become wider and more uneven, and more of the resulting glyphs are then rejected by the width and class filters. A smaller radius keeps more glyphs but they are narrower, which is the boundary sitting where the alignment put it rather than at a real gap. The submitted 0.5 is in the middle of that trade and the sweep does not identify an optimum; measuring one needs ground-truth character boundaries, which this project does not have.

Table 40: How far a character boundary may be moved to find a cheap column: measured effect of each swept value. The submitted value is in bold.

$r_{\{}text{snap}}$	nGlyphs	meanWidth	alignConf
0.1	40	0.595	-0.045
0.25	38	0.625	-0.045
0.5	**35**	**0.684**	**-0.045**
0.9	30	0.804	-0.045
1.5	30	0.804	-0.045

20.4 Smallest cell a character boundary pair may enclose

Symbol c_{min}

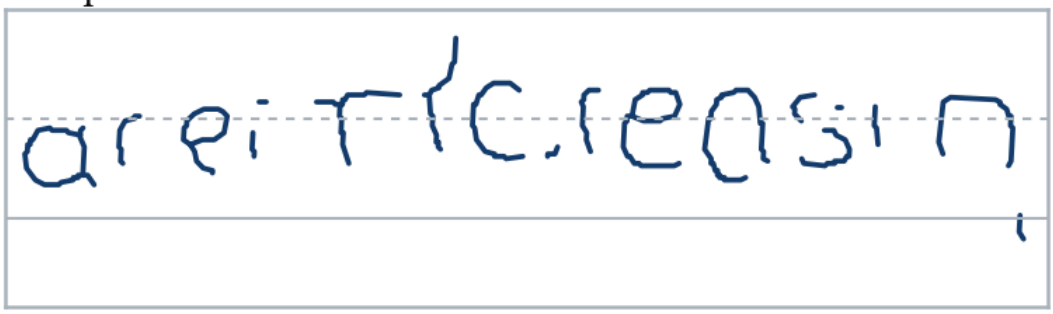
Value `0.08`

Source `style.py`, `ExtractLineGlyphs`

Swept `0.01`, `0.04`, `0.08`, `0.2`, `0.4`

Basis Monotonic, non-degenerate boundaries are a requirement of any segmentation that must produce one region per label [9]; the CTC alignment that supplies the guesses is itself monotonic [24, p. 370].

$r_{\text{snap}} = 0.1$



glyphs 40 mean w 0.595 conf -0.045

$r_{\text{snap}} = 0.25$



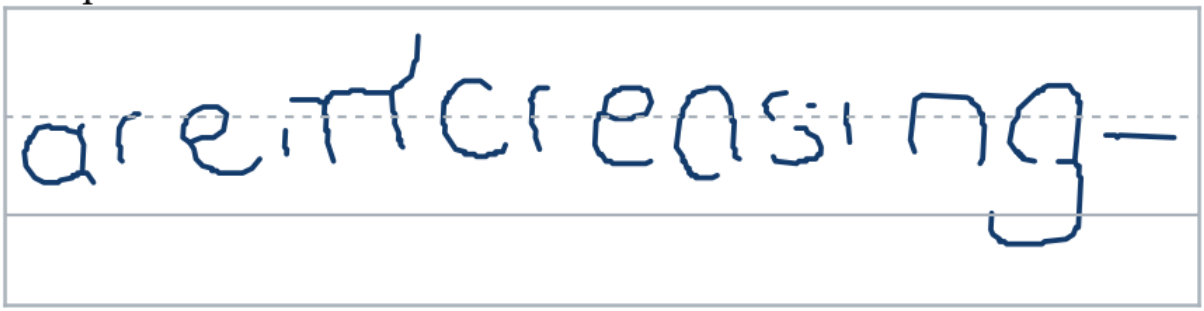
glyphs 38 mean w 0.625 conf -0.045

$r_{\text{snap}} = 0.5$ (submitted value)



glyphs 35 mean w 0.684 conf -0.045

$r_{\text{snap}} = 0.9$



glyphs 30 mean w 0.804 conf -0.045

$r_{\text{snap}} = 1.5$



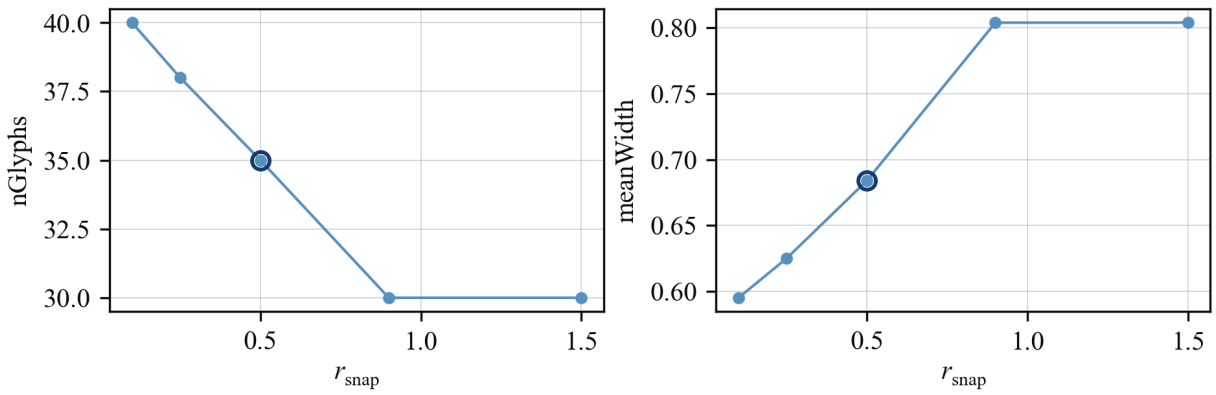


Figure 73: How far a character boundary may be moved to find a cheap column: measured quantities against the swept value. The circled point is the submitted value.

Why this value Boundaries are built into one shared, strictly increasing array precisely so that two neighbouring boundaries cannot land on the same column and leave a character with an empty cell. c_{min} is the width that guarantees it, and it must also leave room for every remaining character, which is why the upper limit for boundary k is $\text{bnd}[n] - (n-k)c_{\text{min}}$. At 0.08 x-heights a cell is at least a few pixels wide, enough to hold the thinnest real mark, such as the stem of an `l` or a full stop.

What the coefficient does Raising c_{min} forces cells apart and so forces boundaries away from where the cost profile wants them, eventually overriding the snap entirely and producing an evenly spaced comb. Lowering it allows very narrow cells, whose glyphs are then rejected downstream for being too thin.

Worked example At $x_h = 42$ px, $c_{\text{min}} = 3.4$ px. For a line of 48 characters the boundaries must span at least $48 \times 3.4 = 161$ px, against a crop width of 2170 px, so the constraint is slack on a normal line.

What the sweep actually showed Flat at 35 glyphs for every value from 0.01 to 0.20, with 36 glyphs and a slightly smaller mean width at 0.40. The constraint is simply slack on an ordinary line: 48 characters at $c_{\text{min}} = 0.08$ need 161 px of the 2 170 px crop, so the boundaries are nowhere near being forced apart. The coefficient is a guard against a degenerate case — two boundaries landing on one column — and this line does not produce that case, so the null result is the expected one.

Table 41: Smallest cell a character boundary pair may enclose: measured effect of each swept value. The submitted value is in bold.

$c_{\{\}min}$	nGlyphs	meanWidth	nNone
0.01	35	0.684	6
0.04	35	0.684	6
0.08	**35**	**0.684**	**6**
0.2	35	0.684	6
0.4	36	0.664	5

21 Glyph filtering

21.1 Widest glyph accepted, in x-heights

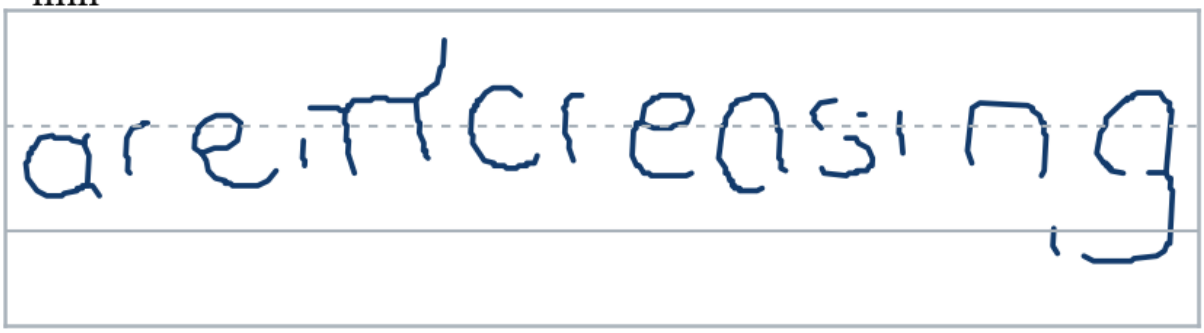
Symbol W_{glyph}

Value `2.1`

Source `style.py`, `MAX_GLYPH_W`

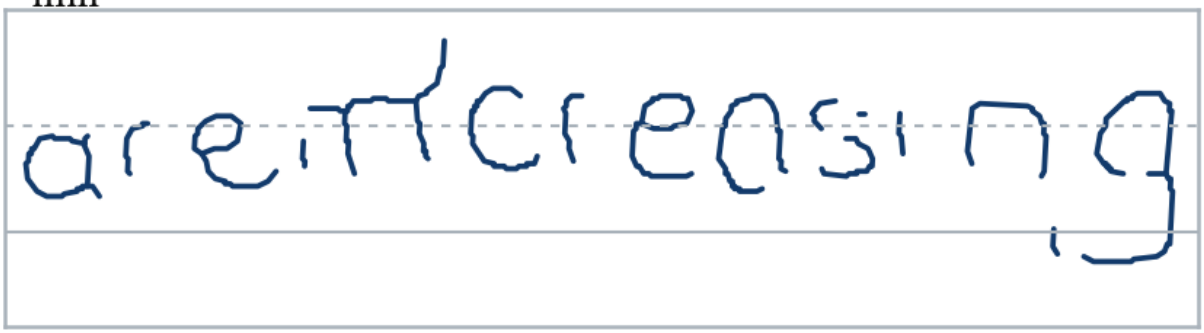
Swept `1.1`, `1.6`, `2.1`, `3.0`, `4.5`

$c_{\min} = 0.01$



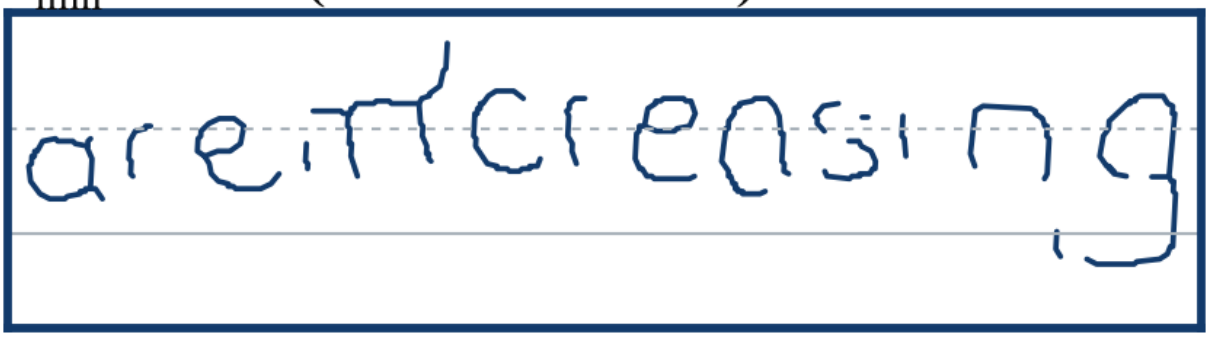
glyphs 35 mean w 0.684 dropped 6

$c_{\min} = 0.04$



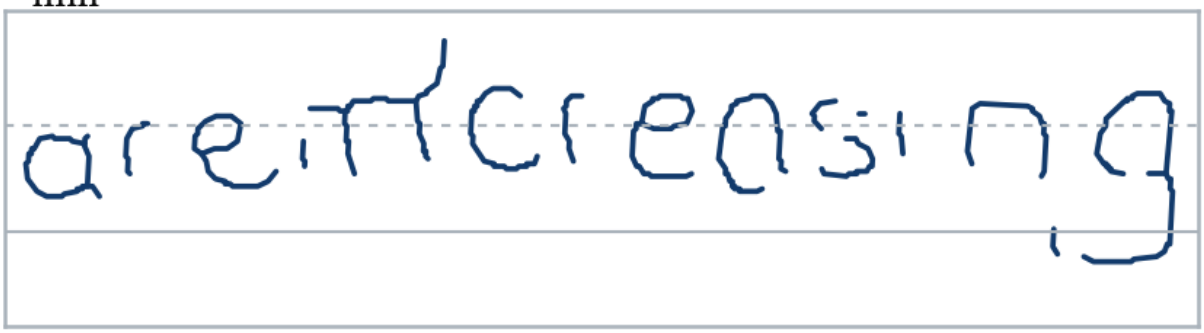
glyphs 35 mean w 0.684 dropped 6

$c_{\min} = 0.08$ (submitted value)



glyphs 35 mean w 0.684 dropped 6

$c_{\min} = 0.2$



glyphs 35 mean w 0.684 dropped 6

$c_{\min} = 0.4$

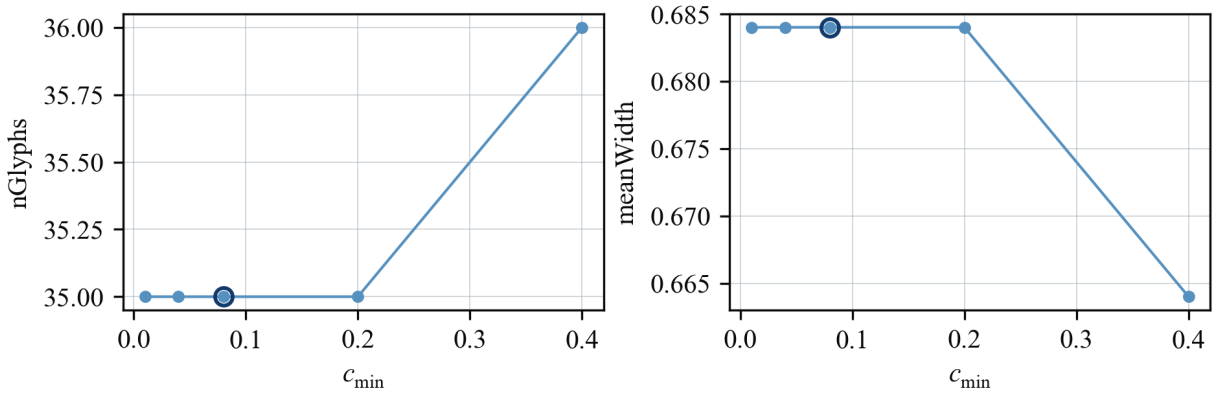


Figure 75: Smallest cell a character boundary pair may enclose: measured quantities against the swept value. The circled point is the submitted value.

Basis Width normalised by x-height is the scale-free measure of letter size used throughout handwriting analysis [11]; rejecting over-wide segmentation hypotheses is the standard guard in segmentation-based recognition [9].

Why this value A glyph is cut at the boundaries the reader's forced alignment suggests, and that alignment is sometimes wrong: a boundary placed too far right makes one cell swallow its neighbour. The resulting shape is then two letters stored under one character, which later draws as nonsense. Real lowercase letters are at most about one x-height wide and the widest, `m` and `w`, reach roughly 1.5. A cell holding two letters is at least 2 wide. 2.1 sits above every single letter and at the bottom of the two-letter range, and the wide characters get their own larger limit of 2.9 so they are not penalised for being wide.

What the coefficient does This is one of a battery of filters in `BuildAuthorProfile`, and it is the one that catches the specific failure of a swallowed neighbour. Raising it lets merged pairs into the store, which shows up later as a median prior distance that climbs, because a two-letter shape is nothing like the pooled prototype of its character.

Alternatives considered Rejecting on the alignment confidence alone, which is already done at line level by `ALIGN_CONF_PCT`, or comparing each glyph against the letter prior and rejecting outliers, which `PriorFilter` does as a later and softer stage.

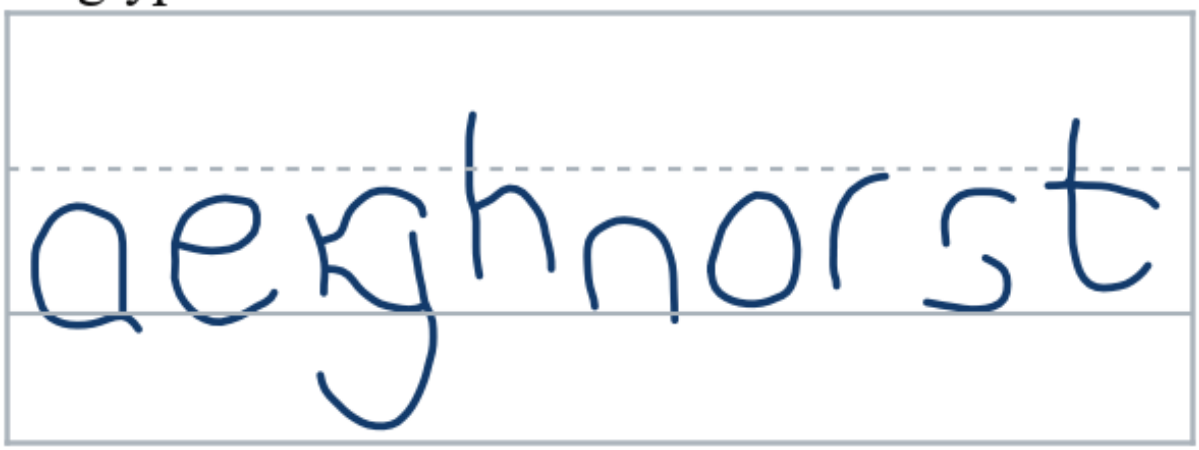
Worked example At $x_h = 42$ px a glyph may be at most $2.1 \times 42 = 88$ px wide, against a cell advance that is typically 0.7–1.0 x-heights.

What the sweep actually showed **The limit never fires at the submitted value on this data.** Rebuilding Owen's profile from 18 lines gives 35 characters and 165 variants at $W_{\text{glyph}} = 1.6$, 2.1, 3.0 and 4.5 alike — four identical results — and only at 1.1 does the limit bite, dropping to 33 characters and 160 variants. So no glyph cut from these lines is wider than 1.6 x-heights, and the submitted 2.1 sits well clear of anything the cutter actually produced. That is the correct behaviour for a safety net — it should not be removing good glyphs — but it means this sweep cannot show the limit catching the failure it exists for. Demonstrating that needs lines where the forced alignment is wrong enough to merge two letters into one cell, which these well-read lines do not provide.

Table 42: Widest glyph accepted, in x-heights: measured effect of each swept value. The submitted value is in bold.

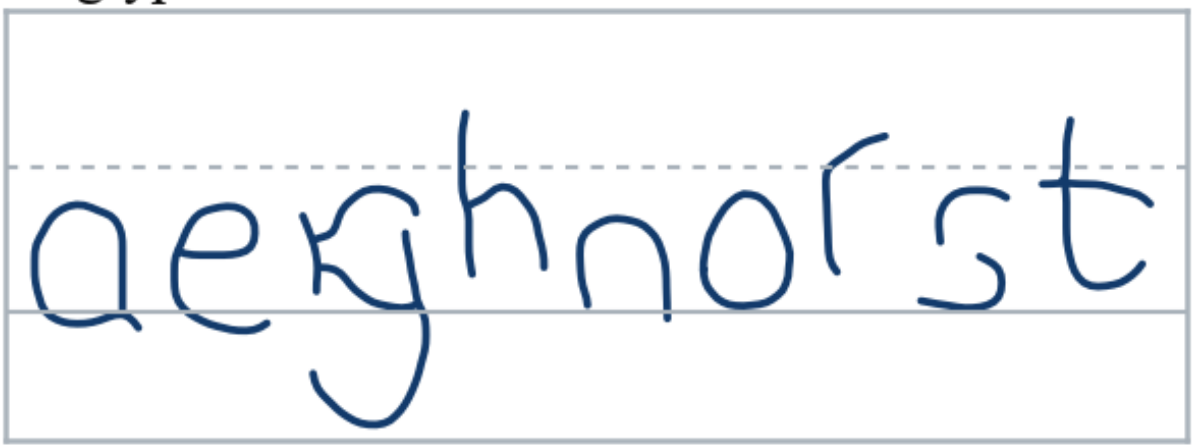
W_{glyph}	nChars	nVariants	medianPriorD	connectedness
1.1	33	160	0.0958	0.231
1.6	35	165	0.0947	0.231
2.1	**35**	**165**	**0.0947**	**0.231**
3	35	165	0.0947	0.231
4.5	35	165	0.0947	0.231

$W_{\text{glyph}} = 1.1$



rs 33 variants 160 median d 0.0

$W_{\text{glyph}} = 1.6$



rs 35 variants 165 median d 0.0

$W_{\text{glyph}} = \mathbf{2.1}$ **(submitted value)**



rs 35 variants 165 median d 0.0

$W_{\text{glyph}} = 3$

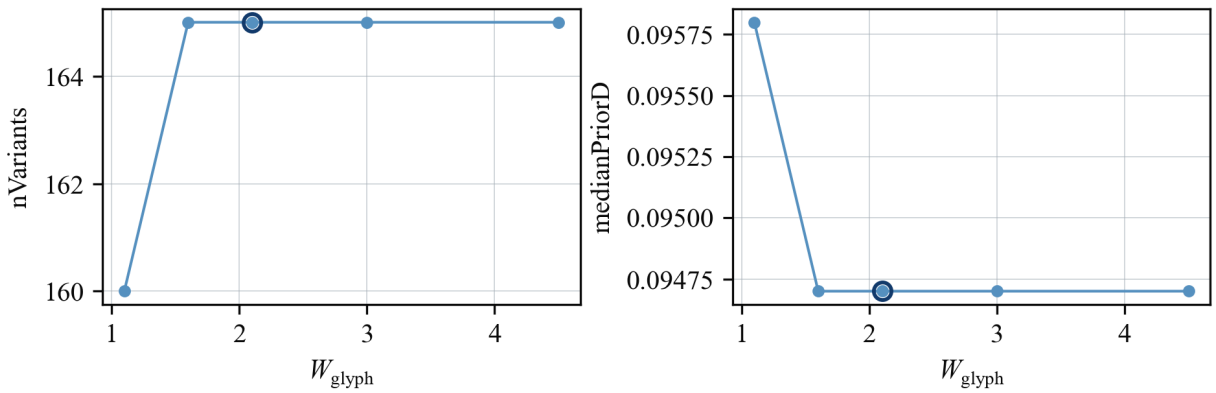


Figure 77: Widest glyph accepted, in x-heights: measured quantities against the swept value. The circled point is the submitted value.

21.2 Side of the square occupancy descriptor

Symbol n_{grid}

Value `12`

Source `style.py, _ShapeGrid`

Swept `6, 8, 12, 16, 24`

Basis A normalised grid of ink occupancy is the classical shape descriptor for character matching [30, 31]; the square-root damping and blur follow the same reasoning as the gradient-histogram features used for writer identification [20].

Why this value The descriptor exists to answer one question: how far is this glyph from the way this letter usually looks? It must therefore be coarse enough that two honest writings of the same letter match, and fine enough that a fragment or a merged pair does not. Each cell of a 12×12 grid is $1/12$ of the letter box, about 3.5 px on a 42 px x-height, so a stroke placed one cell differently still overlaps after the 3×3 blur that follows. Finer grids make two writings of the same letter disagree simply because the pen wavered; coarser ones stop distinguishing `c` from `e`.

What the coefficient does The descriptor is normalised to unit length and compared by dot product, so n_{grid} changes the dimension of the space, 144 at the submitted value. Distances are not comparable between grid sizes, which is why the sweep reports the median prior distance it produces rather than claiming one size scores better than another: a change in the median is partly a change of units.

Alternatives considered Hausdorff distance between point sets [31], which needs no grid but is dominated by outliers; chamfer matching [30]; or resampling both strokes and comparing point by point, which `_DenoisedPrototype` does for averaging but which needs the strokes to correspond.

Worked example The blur weights are 0.25 at the centre and 0.09375 at each of the eight neighbours, summing to $0.25 + 8 \times 0.09375 = 1.0$, so the blur preserves total ink before normalisation.

What the sweep actually showed The median prior distance rises steeply and monotonically with the grid size: 0.0209, 0.0451, 0.0947, 0.1565, 0.3183 for $n = 6$, 8, 12, 16, 24, while the number of variants kept barely moves (160 to 169). This is the clearest possible demonstration of the point made above, that **distances are not comparable between grid sizes**: a finer grid means two honest writings of the same letter overlap in fewer cells, so every distance grows, and the fifteen-fold spread here is mostly a change of units rather than a change in discrimination. It also shows the real design tension. At $n = 6$ the descriptor rates almost everything as typical, which makes the prior filter useless; at $n = 24$ it rates a writer's own letters as atypical of themselves, which makes it over-aggressive. $n = 12$ is between those, but this sweep cannot show it is the best point, because the quantity it reports is measured in different units at every value. A fair comparison would score each grid size by how well

it separates correctly cut glyphs from known-bad ones, and that needs labelled bad cuts, which do not exist in this project.

Table 43: Side of the square occupancy descriptor: measured effect of each swept value. The submitted value is in bold.

$n_{\{}text{grid}}$	nChars	nVariants	medianPriorD
6	35	160	0.0209
8	35	161	0.0451
12	**35**	**165**	**0.0947**
16	35	169	0.1565
24	35	166	0.3183

22 Profile building

22.1 Reading-confidence percentile below which a whole line is not trusted

Symbol q_{conf}

Value `25`

Source `style.py`, `ALIGN_CONF_PCT`

Swept `0`, `10`, `25`, `50`, `75`

Basis The confidence is the mean per-frame log probability of the forced alignment path [24, pp. 370–372]; discarding low-confidence training material is standard practice in semi-supervised and self-labelled recognition pipelines [32].

Why this value Glyph boundaries are only as good as the alignment that produced them, and the alignment is poor where the crop is poor: a line cut badly, a line the writer struck through, a line with an unreadable word. Those lines would contribute wrongly cut glyphs to the store. A *relative* limit is used rather than an absolute log probability because confidence depends on the writer and on line length; discarding the worst quarter of a writer's own lines is a statement about that writer's pages, not about an absolute scale. It is applied only when there are at least eight lines, since a quartile of fewer is meaningless.

What the coefficient does At $q_{\text{conf}} = 0$ nothing is discarded; at 75 only the best quarter of lines contributes, which gives cleaner glyphs but far fewer of them, and some characters then fall below the minimum variant count and disappear from the store entirely. The trade is coverage against cleanliness, and the sweep reports both: variants kept, and their median prior distance.

Alternatives considered An absolute confidence floor, which would need recalibrating per checkpoint; or weighting glyphs by confidence instead of discarding lines, which the later filters partly achieve.

Worked example With 18 lines, discarding the lowest quartile removes the four or five worst-read lines before any glyph is stored.

22.2 Shapes of one letter kept per writer

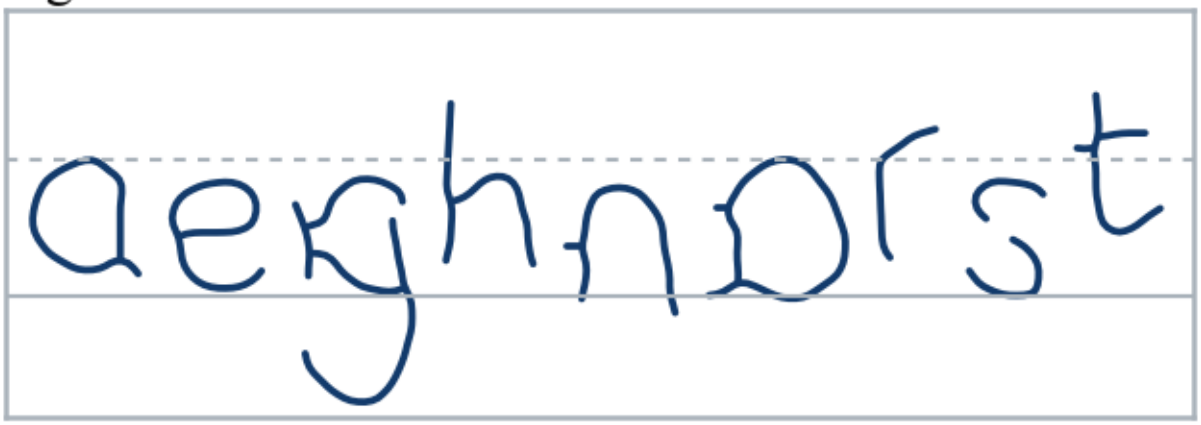
Symbol $V_{\max}$

Value `12`

Source `style.py`, `MAX_VARIANTS_PER_CHAR`

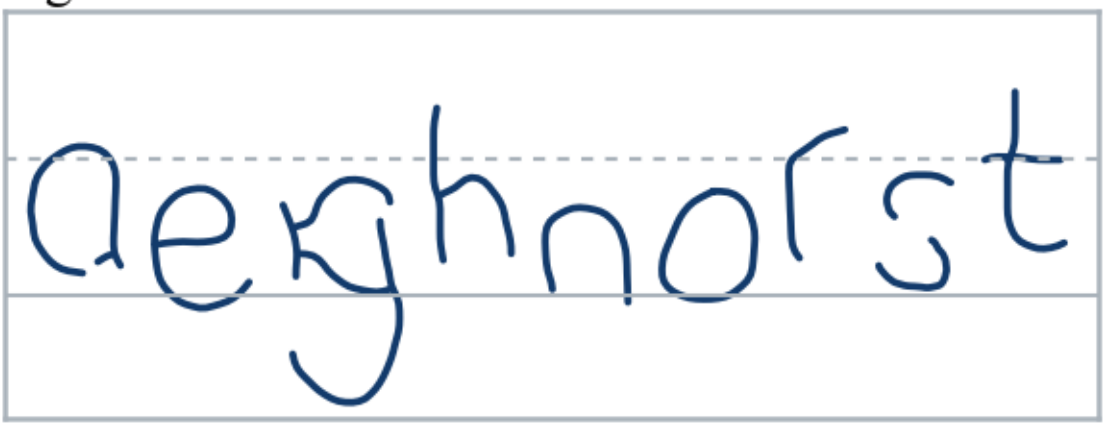
Swept `2`, `5`, `12`, `25`, `60`

$n_{\text{grid}} = 6$



ars 35 variants 160 median d 0.02

$n_{\text{grid}} = 8$



ars 35 variants 161 median d 0.04

$n_{\text{grid}} = 12$ (submitted value)



ars 35 variants 165 median d 0.09

$n_{\text{grid}} = 16$

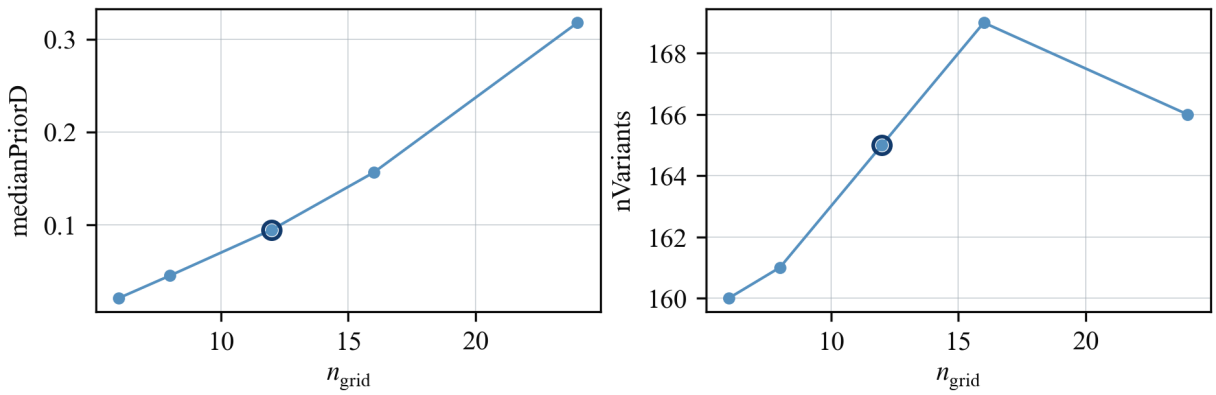


Figure 79: Side of the square occupancy descriptor: measured quantities against the swept value. The circled point is the submitted value.

Table 44: Reading-confidence percentile below which a whole line is not trusted: measured effect of each swept value. The submitted value is in bold.

$q_{\}text{conf}}$	nLines	nChars	nVariants	medianPriorD
0	18	36	204	0.0975
10	16	35	186	0.0958
25	**13**	**35**	**165**	**0.0947**
50	9	31	121	0.1022
75	5	23	69	0.1301

Basis Storing several exemplars per character and choosing among them at synthesis time is the allographic model of handwriting: a writer has a repertoire of forms per letter rather than one [33], and natural variation between repetitions is intrinsic to the motor process [28].

Why this value One shape per letter gives output in which every `e` is identical, which no reader accepts as handwriting. Keeping every shape instead means keeping the badly cut ones too, and makes the profile large. 12 is set above the number needed for the repetition not to be visible within a line: an ordinary line holds a few instances of the commonest letters, so a dozen forms means a letter rarely repeats its exact shape on one line. The variants are sorted by typicality first, so the cut keeps the best twelve, and the synthesis then draws from only the top 5 of them with a decreasing weight.

What the coefficient does Because `_ELIGIBLE_VARIANTS` is 5, raising $V_{\max}$ beyond 5 changes the stored profile size without changing what is usually drawn; it matters mainly for the medoid and prototype calculations, which pool all kept variants. Lowering it below 5 directly reduces the variety the synthesis can draw on.

Alternatives considered Clustering the variants and keeping one exemplar per cluster, which is closer to the allographic account [33] but needs a cluster count; or keeping all and selecting at draw time, which is what the eligibility limit approximates.

Worked example Owen's profile holds 43 characters and 307 variants at the submitted setting, an average of 7.1 per character, so the cap binds only for the commonest letters.

23 Synthesis: joining

23.1 Joinedness below which a hand is drawn as print and makes no joins

Symbol c_{print}

Value `0.45`

$q_{\text{conf}} = 0$



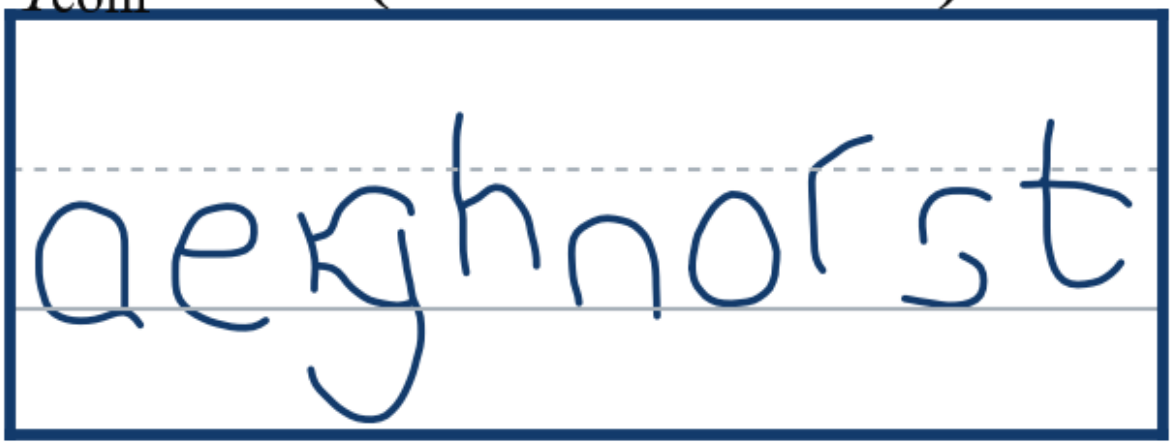
lines 18 chars 36 variants 204

$q_{\text{conf}} = 10$



lines 16 chars 35 variants 186

$q_{\text{conf}} = 25$ (submitted value)



lines 13 chars 35 variants 165

$q_{\text{conf}} = 50$



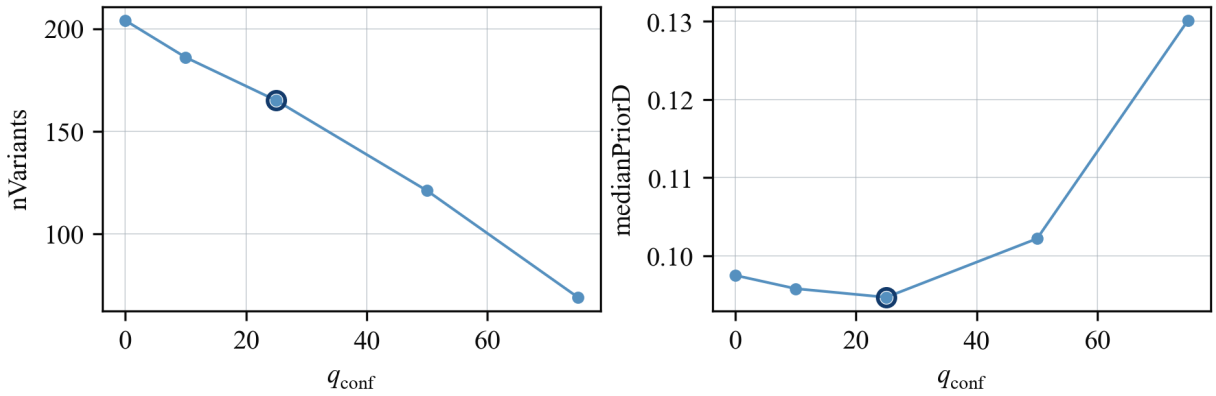


Figure 81: Reading-confidence percentile below which a whole line is not trusted: measured quantities against the swept value. The circled point is the submitted value.

Table 45: Shapes of one letter kept per writer: measured effect of each swept value. The submitted value is in bold.

$V_{\{}max}$	nChars	nVariants	medianPriorD
2	35	62	0.0943
5	35	122	0.094
12	**35**	**165**	**0.0947**
25	35	191	0.0993
60	35	196	0.0994

Source `synthesise.py`, `JOIN_PRINT_CONN_FLOOR`

Swept `0.0`, `0.25`, `0.45`, `0.65`, `0.95`

Basis Whether a hand joins its letters is one of the standard descriptors of writing style and is used directly as a writer-identification feature [20]; the connected-versus-print distinction is basic to handwriting analysis [28].

Why this value The profile measures `connectedness` as the share of adjacent in-word letter pairs whose centrelines actually crossed the boundary between them, so it is an observed property of the writer, between 0 and 1. It is then used as a probability: each pair is joined with that chance. A hand that joins occasionally, say one pair in five, is not a joining hand, and drawing random ligatures into a print hand looks worse than drawing none, because the join appears in arbitrary places. The floor forces the probability to exactly zero below c_{print} rather than letting it taper. 0.45 is below the middle of the range and above the measured connectedness of the clear print hands in this set.

What the coefficient does This is a threshold on a measured quantity, so its effect is a step, not a gradient, and which side a given writer falls on is what matters. Owen measures 0.319, so at the submitted setting Owen is drawn as a print hand with no joins at all; at $c_{\text{print}} = 0.25$ Owen crosses the floor and joins about a third of pairs. The sweep therefore shows exactly the decision this coefficient makes for this writer.

Alternatives considered No floor, letting the raw probability act however small, which sprinkles isolated ligatures through print writing; or a smooth ramp between print and cursive, which replaces one threshold with two.

Worked example Owen's connectedness is 0.319. At the submitted floor of 0.45, $0.319 < 0.45$ so `connEff` is set to 0 and no pair is joined; at a floor of 0.25, `connEff` is 0.319 and roughly 32% of adjacent letter pairs are joined.

$V_{\max} = 2$



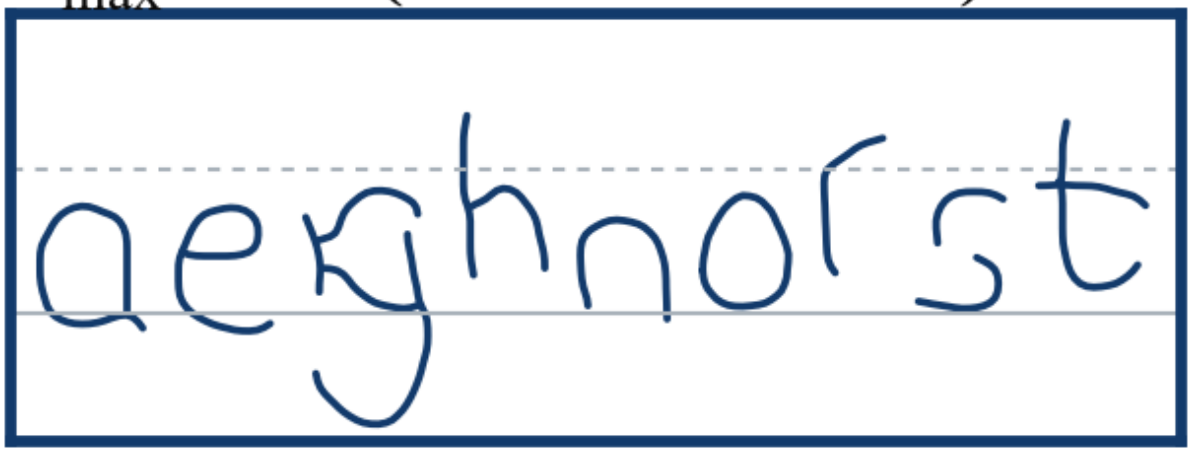
ars 35 variants 62 median d 0.09

$V_{\max} = 5$



ars 35 variants 122 median d 0.0

$V_{\max} = \mathbf{12}$ **(submitted value)**



ars 35 variants 165 median d 0.09

$V_{\max} = 25$

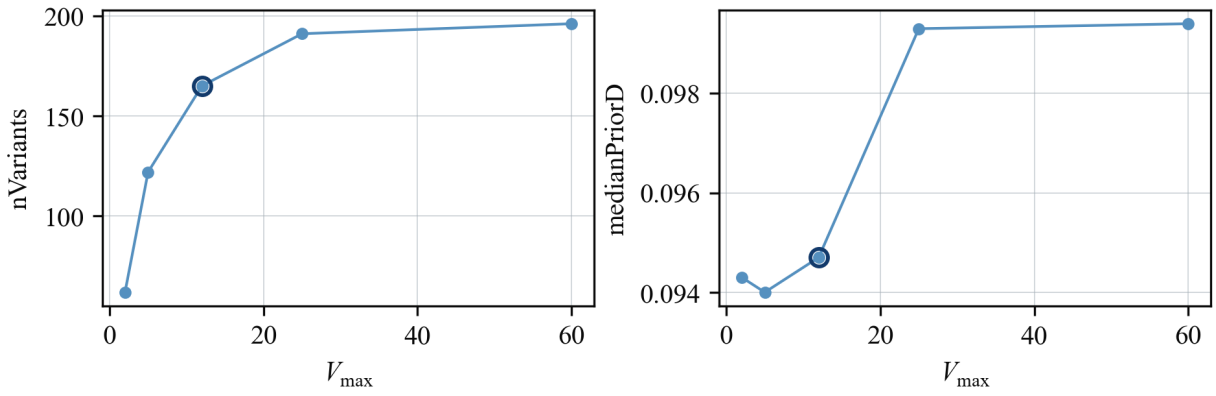


Figure 83: Shapes of one letter kept per writer: measured quantities against the swept value. The circled point is the submitted value.

What the sweep actually showed The floor does decide Owen's case, and the direction favours the submitted value, but **the difference is not statistically distinguishable on twelve seeds**. Owen's measured connectedness is 0.319, so at $c_{\text{print}} = 0.0$ and 0.25 he is drawn with joins (59 strokes) and at 0.45, 0.65 and 0.95 he is drawn as a print hand (57 strokes) — the coefficient is doing exactly what it is supposed to do, and the figure shows the two modes clearly. Scoring them, drawing the joins costs -0.0174 reading accuracy and -0.0096 on the joint score, with 8 or 9 of the 12 seeds worse, but the paired t is only -1.43, inside the noise. So the honest statement is: the floor changes the output visibly and in the expected direction, and this sentence on this writer is not enough evidence to call the improvement real. Establishing it would need several writers either side of the floor and more text.

Table 46: Joinedness below which a hand is drawn as print and makes no joins: measured effect of each swept value. The submitted value is in bold.

$c_{\{}text{print}}$	charAccMean	charAccSd	widConfMean	harmonicMean	nStrokes
0	0.9128	0.0359	0.9996	0.9539	59
0.25	0.9128	0.0359	0.9996	0.9539	59
0.45	**0.9302**	**0.0314**	**0.9998**	**0.9635**	**57**
0.65	0.9302	0.0314	0.9998	0.9635	57
0.95	0.9302	0.0314	0.9998	0.9635	57

23.2 Furthest apart two letters may be and still be joined

Symbol g_{join}

Value `0.8`

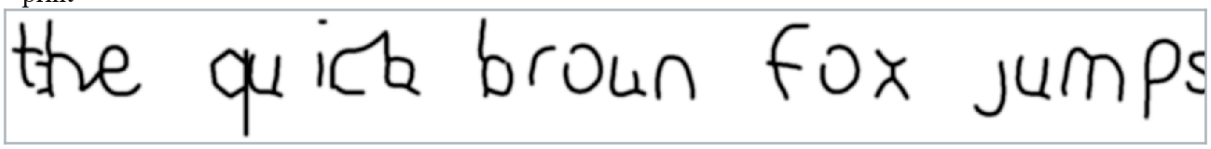
Source `synthesise.py`, `MAX_JOIN_GAP_XH`

Swept `0.2`, `0.5`, `0.8`, `1.3`, `2.5`

Basis A ligature is a pen movement between two letters; its length is bounded by the letter spacing of the hand [28].

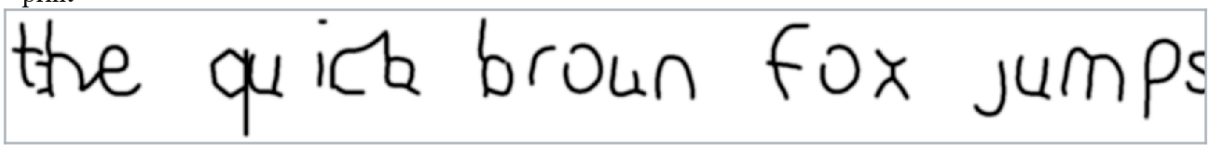
Why this value A join is only drawn if the next letter starts within g_{join} x-heights of where the pen left the last one. Beyond that the connecting stroke is longer than a letter and reads as a line through the word rather than a ligature. Letter gaps in these profiles are held between 0.03 and 0.30 x-heights, so 0.80 allows a join across an ordinary gap with margin, and refuses one where a collision push or an unusually wide glyph has opened a real space.

$c_{\text{print}} = 0$



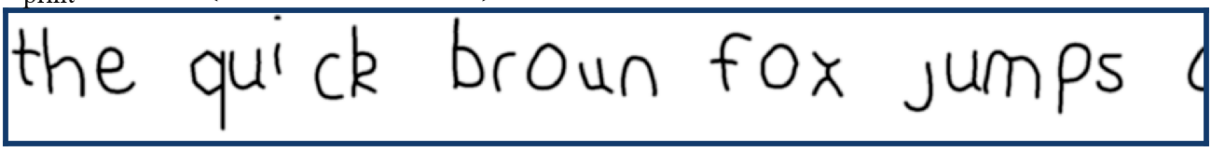
read acc 0.9128 sd 0.0359 p(writer) 0.9996

$c_{\text{print}} = 0.25$



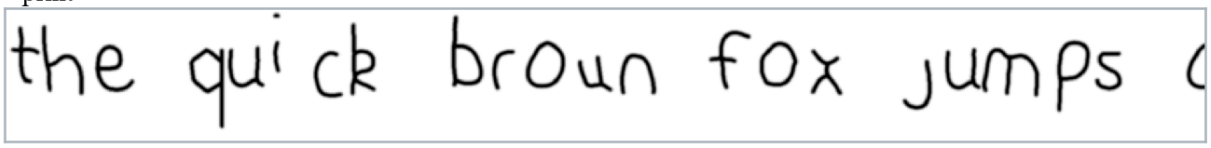
read acc 0.9128 sd 0.0359 p(writer) 0.9996

$c_{\text{print}} = 0.45$ (submitted value)



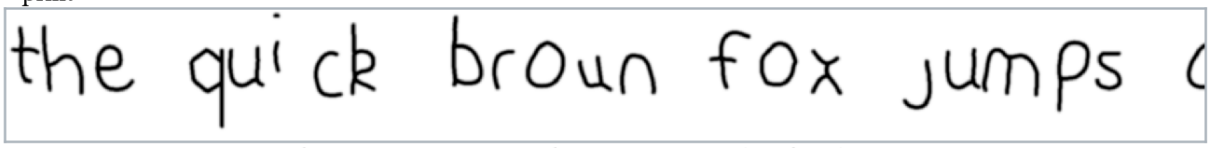
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$c_{\text{print}} = 0.65$



read acc 0.9302 sd 0.0314 p(writer) 0.9998

$c_{\text{print}} = 0.95$



read acc 0.9302 sd 0.0314 p(writer) 0.9998

Figure 84: Joinedness below which a hand is drawn as print and makes no joins. Each panel is the same input processed by the submitted code with only c_{print} changed; the panel outlined in dark blue is the submitted value.

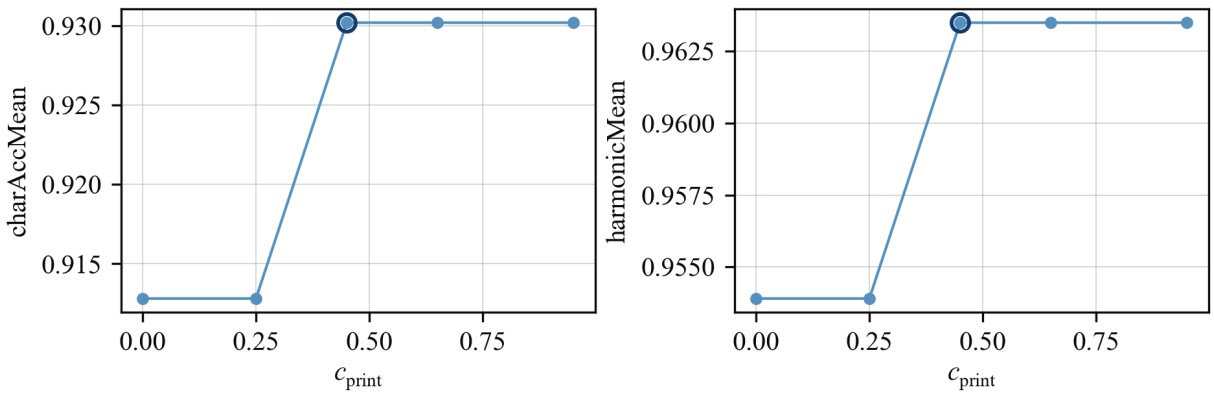


Figure 85: Joinedness below which a hand is drawn as print and makes no joins: measured quantities against the swept value. The circled point is the submitted value.

What the coefficient does The gap is measured from the previous exit point to the start of the body stroke of the next letter, after the collision push, so it reflects where the ink actually is rather than the nominal advance. The test sits in a conjunction: both letters must be alphabetic, from compatible sources, and the random draw against `connEff` must succeed.

Alternatives considered Scaling the limit by the measured word gap of the writer, which would make it writer-specific at the cost of another measured quantity in the chain.

Worked example At 4 mm per x-height a join is refused once the gap exceeds $0.8 \times 4 = 3.2$ mm.

What the sweep actually showed **Nothing distinguishable**, as anticipated: Owen is below the print floor at the submitted settings, so no joins are drawn and the gap limit has nothing to act on. The small positive differences at every value (paired t between $+0.52$ and $+1.74$) are seed noise. Exercising this coefficient needs a writer whose connectedness is above 0.45, and none of the five photographed writers in this probe set is.

Limits of this test Owen is below the print floor, so no joins are drawn at the submitted settings and this sweep is expected to show little or no change for this writer. That is itself the finding, and it is reported rather than hidden; the trend shows the stroke count unmoved.

Table 47: Furthest apart two letters may be and still be joined: measured effect of each swept value. The submitted value is in bold.

$g_{\{}text{join}}$	charAccMean	charAccSd	widConfMean	harmonicMean	nStrokes
0.2	0.9341	0.024	0.9998	0.9657	59
0.5	0.9399	0.0288	0.9999	0.9688	58
0.8	**0.9302**	**0.0314**	**0.9998**	**0.9635**	**57**
1.3	0.9438	0.0184	0.9999	0.9709	60
2.5	0.936	0.0224	0.9999	0.9668	60

24 Synthesis: spacing

24.1 Clearance kept between one letter's ink and the next

Symbol d_{clear}

Value `0.05`

Source `synthesise.py`, `COLLISION_MIN_GAP_XH`

Swept `-0.1`, `0.0`, `0.05`, `0.2`, `0.5`

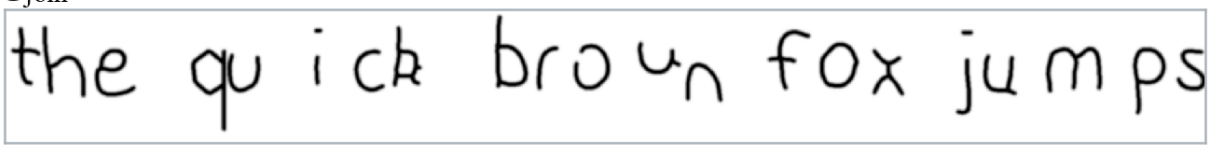
Basis Overlap resolution between adjacent glyphs is the layout problem solved by kerning in typography; in handwriting the equivalent constraint is that the pen does not retrace the previous letter [28].

Why this value Each letter is placed by advancing the pen by the previous letter's advance, which is a measured number and takes no account of the particular shapes drawn. Two wide variants in a row can therefore overlap. The rule compares the right-most ink of the previous body stroke with the left-most ink of the new letter and pushes the new letter right if they are closer than d_{clear}. The value is small on purpose: the aim is to stop ink touching, not to impose uniform spacing, because uniform spacing is precisely what makes synthetic writing look printed. 0.05 x-heights is about two pixels at the rendering scale, just enough to keep strokes apart.

What the coefficient does A negative value lets letters overlap, which the sweep includes to show what the rule is preventing. A large value forces every letter clear of the last and so overrides the writer's own measured advances, producing evenly spaced, mechanical output and a longer line.

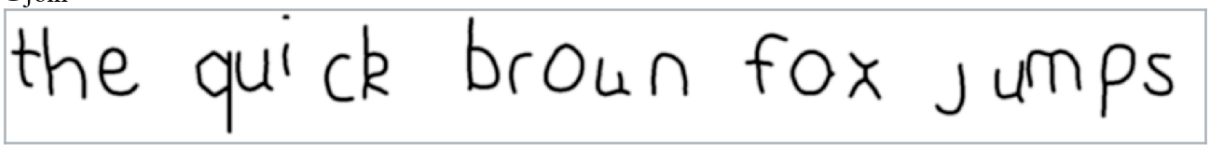
Alternatives considered Testing true ink overlap by rasterising each pair, which is exact but far more expensive; or adjusting the advance rather than pushing, which changes the writer's measured rhythm.

$g_{\text{join}} = 0.2$



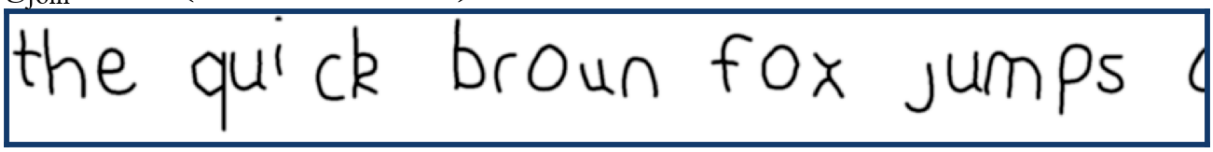
read acc 0.9341 sd 0.024 p(writer) 0.9998

$g_{\text{join}} = 0.5$



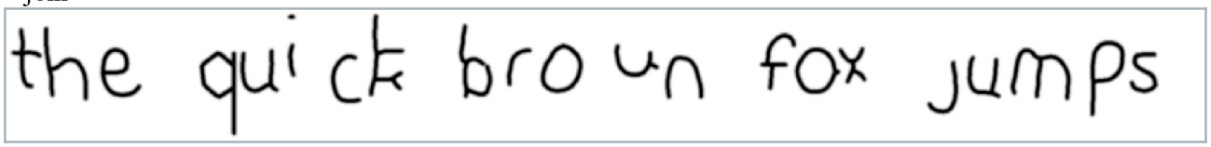
read acc 0.9399 sd 0.0288 p(writer) 0.9999

g_{join} **= 0.8 (submitted value)**



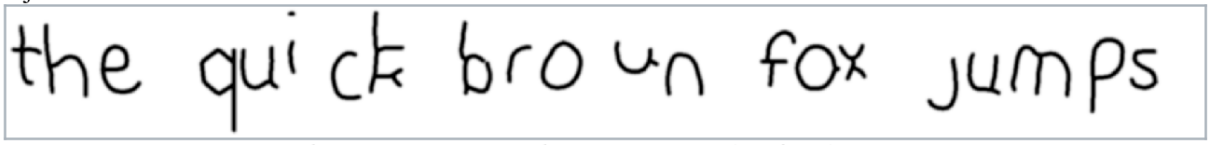
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$g_{\text{join}} = 1.3$



read acc 0.9438 sd 0.0184 p(writer) 0.9999

$g_{\text{join}} = 2.5$



read acc 0.936 sd 0.0224 p(writer) 0.9999

Figure 86: Furthest apart two letters may be and still be joined. Each panel is the same input processed by the submitted code with only g_{join} changed; the panel outlined in dark blue is the submitted value.

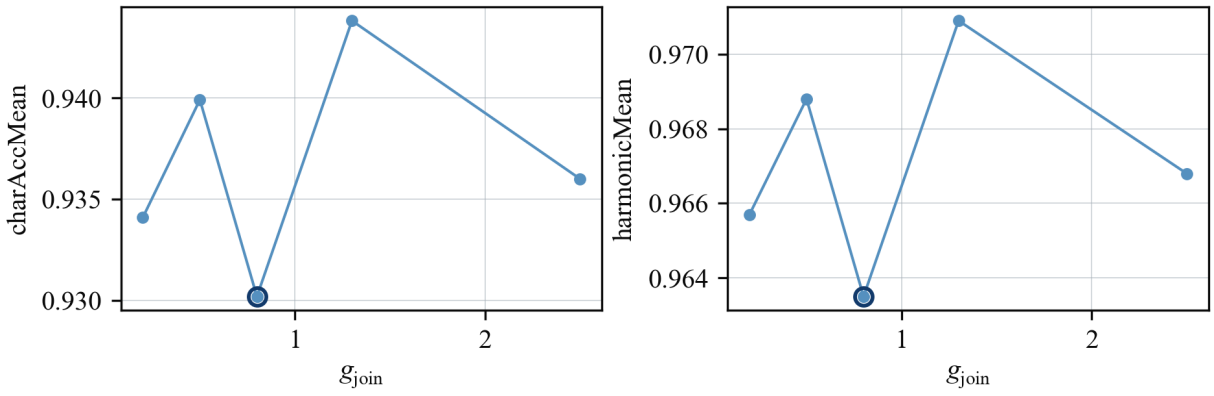


Figure 87: Furthest apart two letters may be and still be joined: measured quantities against the swept value. The circled point is the submitted value.

Worked example At 4 mm per x-height the clearance is 0.2 mm, against a pen width of $0.085 \times 4 = 0.34$ mm, so adjacent strokes are kept about half a pen width apart.

What the sweep actually showed A significant result at the top of the range, in the *opposite* direction to the design argument. At $d_{\text{clear}} = 0.5$ x-heights the paired gain over the submitted 0.05 is $+0.0232$ reading accuracy (6 better, 0 worse, $t = +2.87$) and $+0.0123$ on the joint score ($t = +2.81$). That is forcing a tenth of an x-height of clear space between every pair of letters, which spreads the writing out and makes it easier to read — and the writer classifier does not object enough to cancel it. The argument against adopting it is the one given above and is not measured by either judge: uniform enforced spacing is what makes synthetic handwriting look mechanical, and neither the reader nor the writer classifier is sensitive to that. This is a case where the system's own metrics point one way and the design intent points the other, and the submitted value follows the intent. It should be defended on those grounds, not by claiming the metrics support it.

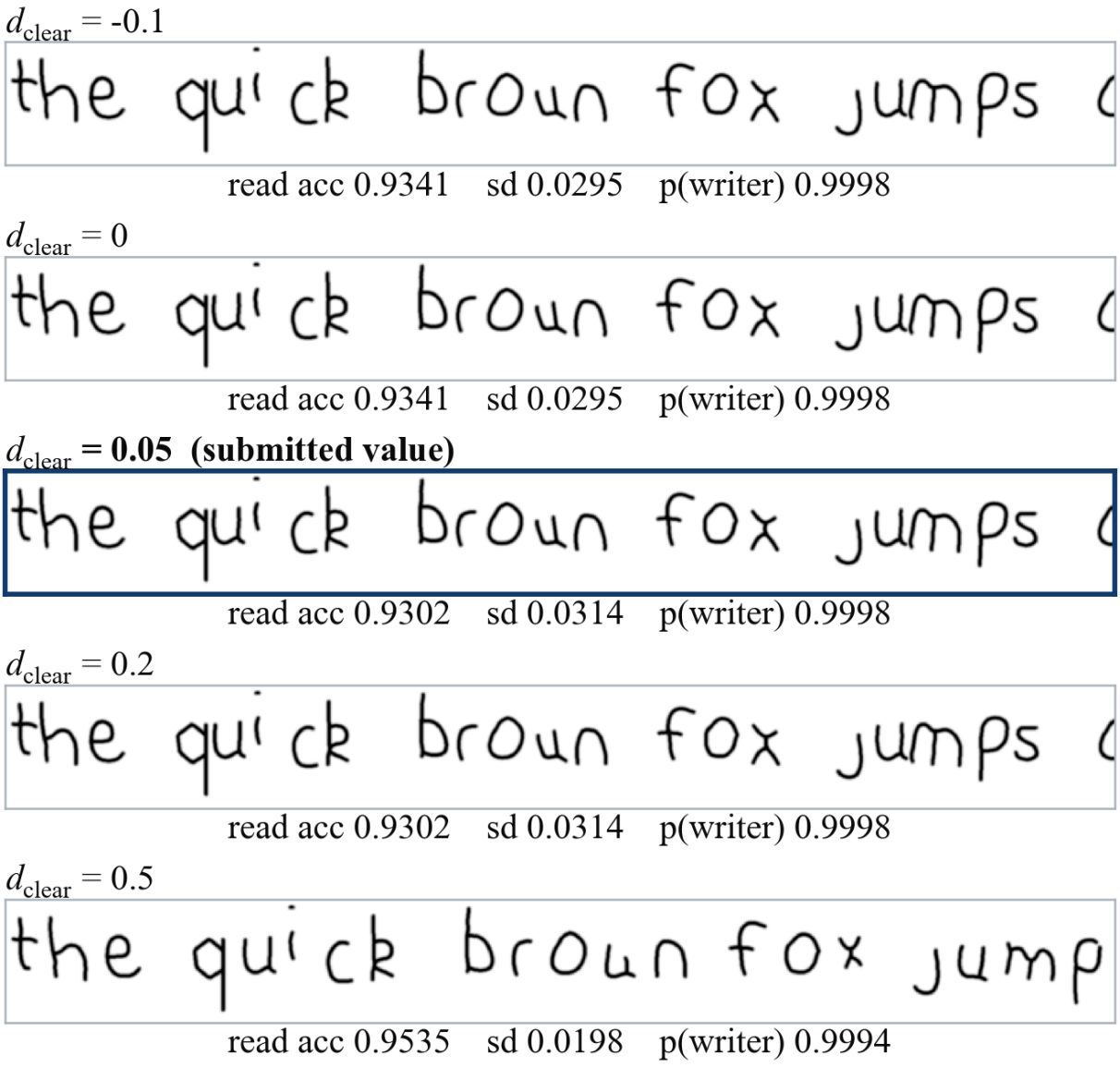


Figure 88: Clearance kept between one letter's ink and the next. Each panel is the same input processed by the submitted code with only d_{clear} changed; the panel outlined in dark blue is the submitted value.

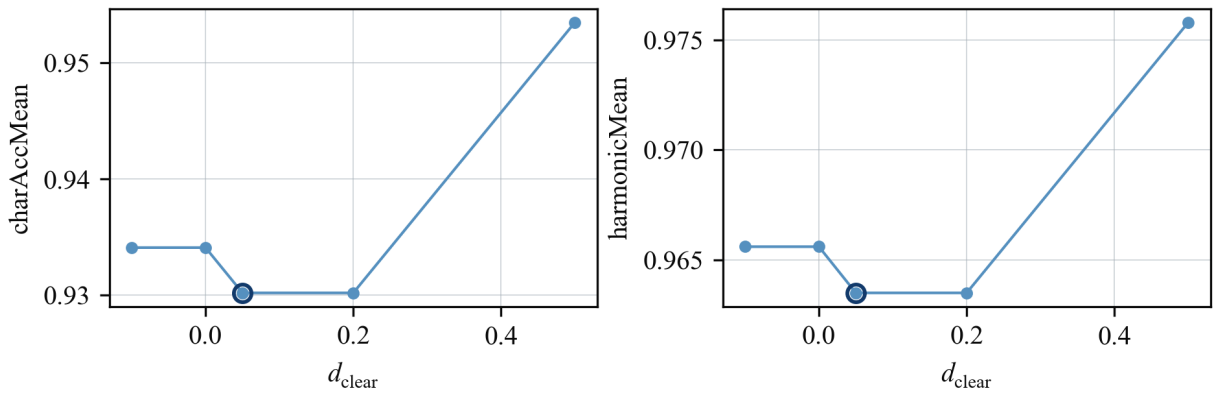


Figure 89: Clearance kept between one letter's ink and the next: measured quantities against the swept value. The circled point is the submitted value.

Table 48: Clearance kept between one letter's ink and the next: measured effect of each swept value. The submitted value is in bold.

d_{clear}	charAccMean	charAccSd	widConfMean	harmonicMean
-0.1	0.9341	0.0295	0.9998	0.9656
0	0.9341	0.0295	0.9998	0.9656
0.05	**0.9302**	**0.0314**	**0.9998**	**0.9635**
0.2	0.9302	0.0314	0.9998	0.9635
0.5	0.9535	0.0198	0.9994	0.9758

25 Synthesis: variation

25.1 Persistence of the slow baseline drift

Symbol $\phi_{\text{drift}}

Value `0.85`

Source `synthesise.py`, `SynthesizeText`

Swept `0.0`, `0.5`, `0.85`, `0.97`, `1.0`

Basis Handwriting is not produced on a perfectly straight baseline; slow drift is part of the motor variability of the writing process [28], and baseline undulation is used as a writer-identification feature [20].

Why this value Independent random offsets per letter give jitter, not drift: the baseline then looks noisy rather than wandering, because each letter is unrelated to its neighbour. Drift is modelled instead as a first-order autoregression, where a velocity v is updated as $v \leftarrow \phi v + \mathcal{N}(0, \sigma)$ and accumulated into the offset. ϕ is the memory: it sets how many letters a drift persists for. At 0.85 the correlation decays by half in $\ln 2 / \ln(1/0.85) = 4.3$ letters, so a drift lasts about a word, which is the scale at which real baselines wander.

What the coefficient does At $\phi = 0$ the velocity is white noise and the baseline jitters per letter. At $\phi = 1$ the velocity is a random walk that never returns, so the line runs away downwards or upwards until the clip at ± 0.25 x-heights catches it; that clip is what makes $\phi = 1$ survivable at all and is swept separately.

Alternatives considered A low-pass-filtered noise field, which is the same thing in the frequency domain, or a sinusoidal baseline, which is too regular to pass as handwriting.

Worked example With $\phi = 0.85$ the stationary standard deviation of the velocity is $\sigma / \sqrt{1 - \phi^2} = 1.90\sigma$, where $\sigma = 0.35 \times 0.05 \times$ jitter x-heights.

What the sweep actually showed **No distinguishable effect** at any value, including $\phi = 0$ (white-noise jitter) and $\phi = 1$ (an unbounded random walk): nine to twelve of the twelve seeds are identical in every comparison and no paired t exceeds $+1.0$. At the jitter setting used here the drift amplitude is $0.05 \times 0.15 = 0.0075$ x-heights per step with a further 0.35 factor on the velocity, which is far too small to move a letter by a pixel at the rendering scale. So the autoregressive drift model is present but is contributing almost nothing at the default jitter, and the distinction between drift and jitter that motivates ϕ is not visible in the output. It would need a much larger jitter setting to be seen, and the figure at this setting should not be presented as showing baseline wander.

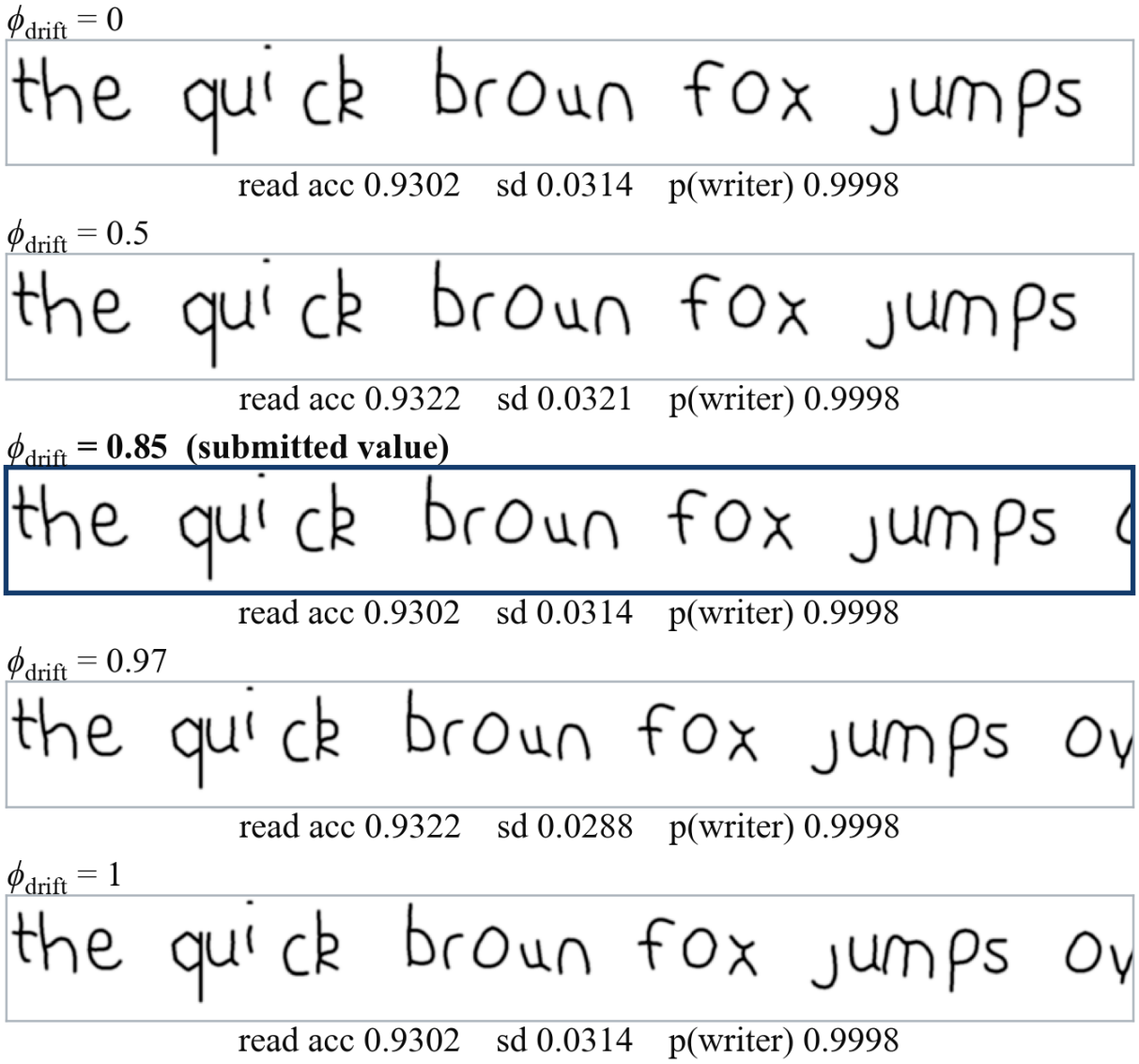


Figure 90: Persistence of the slow baseline drift. Each panel is the same input processed by the submitted code with only ϕ_{drift} changed; the panel outlined in dark blue is the submitted value.

25.2 Largest baseline drift allowed

Symbol	$d_{\max}$
Value	`0.25`
Source	`synthesise.py`, `SynthesizeText`
Swept	`0.02`, `0.1`, `0.25`, `0.6`, `1.5`

Basis Bounding an accumulated random walk is necessary whenever the walk has no restoring force; the writing-line constraint itself is physical, since the writer sees and corrects the baseline [28].

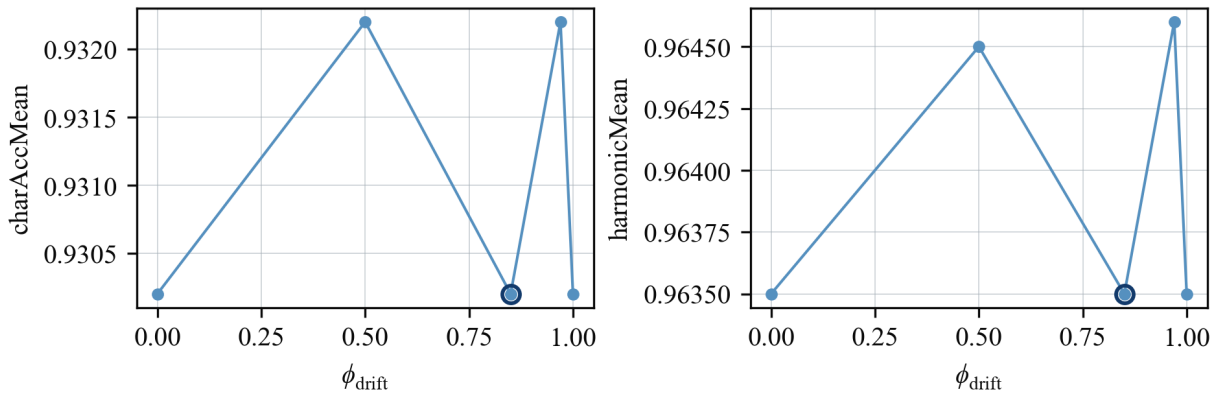


Figure 91: Persistence of the slow baseline drift: measured quantities against the swept value. The circled point is the submitted value.

Table 49: Persistence of the slow baseline drift: measured effect of each swept value. The submitted value is in bold.

$\{}phi_{\{}text{drift}}$	charAccMean	charAccSd	widConfMean	harmonicMean
0	0.9302	0.0314	0.9998	0.9635
0.5	0.9322	0.0321	0.9998	0.9645
0.85	**0.9302**	**0.0314**	**0.9998**	**0.9635**
0.97	0.9322	0.0288	0.9998	0.9646
1	0.9302	0.0314	0.9998	0.9635

Why this value The drift velocity is autoregressive but the drift itself is a sum of those velocities, which is a random walk: over a long line it would wander arbitrarily far. Real writers correct against the ruling, so the model clips the accumulated drift. The limit is in x-heights and 0.25 is a quarter of the body height, which is visible as a wavering baseline but never enough to let one line collide with the next, since lines are set asc $-$ desc $+\,0.85$ x-heights apart.

What the coefficient does The clip is also what keeps ϕ near 1 usable. It acts as a hard wall rather than a restoring force, so with a persistent velocity the drift can sit against the wall for many letters, which looks like a sloping line rather than a wandering one.

Alternatives considered An Ornstein–Uhlenbeck process on the drift itself, which pulls back towards zero smoothly instead of clipping and would be the better model; or re-centring at each word.

Worked example At 4 mm per x-height the drift is bounded at ± 1.0 mm, against a line pitch of $(1.51 + 0.68 + 0.85) \times 4 = 12.2$ mm for Owen.

What the sweep actually showed **No distinguishable effect**: eleven of twelve seeds identical at every value, and the one that moves gives a paired $t = +1.00$. At 4 mm per x-height the clip is ± 1.0 mm against a line step of 12.2 mm, and the jitter setting used here (0.15) makes the drift velocity small enough that the accumulated drift rarely approaches the clip at all. The clip is a guard against a random walk running away over a long line, and a 43-character sentence at low jitter does not run far enough to meet it.

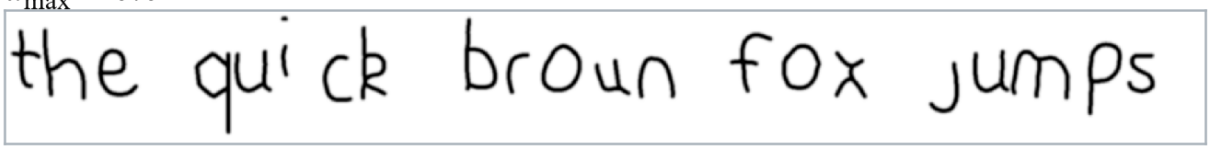
26 Synthesis: rendering

26.1 Pen width the synthesised line is rendered at

Symbol w_{draw}

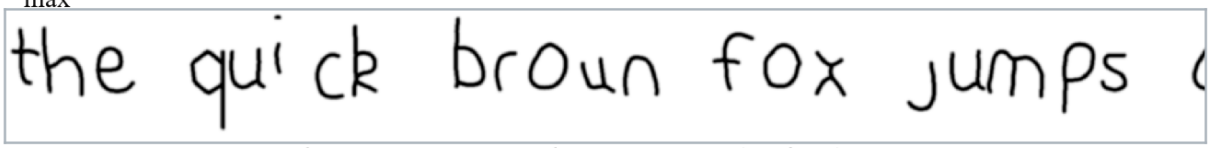
Value `0.085`

$d_{\max} = 0.02$



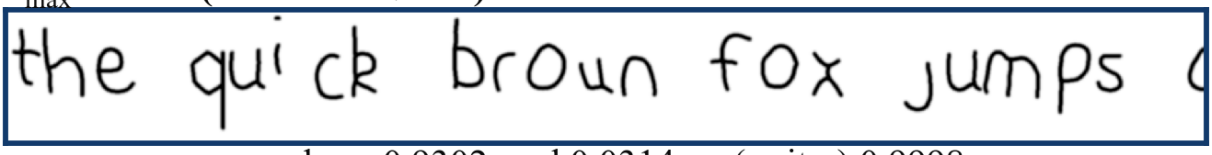
read acc 0.9322 sd 0.0321 p(writer) 0.9998

$d_{\max} = 0.1$



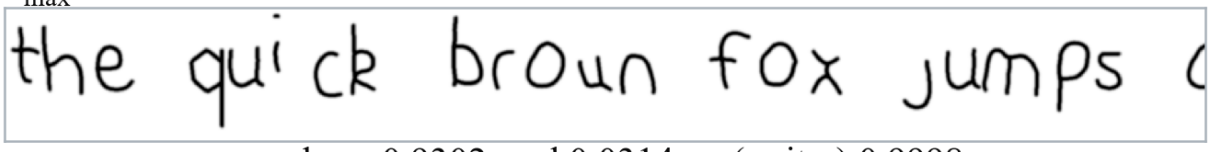
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$d_{\max} = 0.25$ (submitted value)



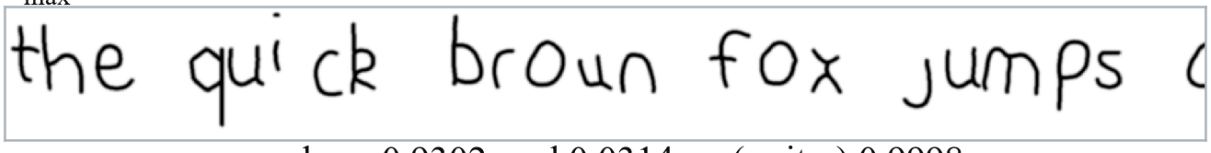
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$d_{\max} = 0.6$



read acc 0.9302 sd 0.0314 p(writer) 0.9998

$d_{\max} = 1.5$



read acc 0.9302 sd 0.0314 p(writer) 0.9998

Figure 92: Largest baseline drift allowed. Each panel is the same input processed by the submitted code with only $d_{\max}$ changed; the panel outlined in dark blue is the submitted value.

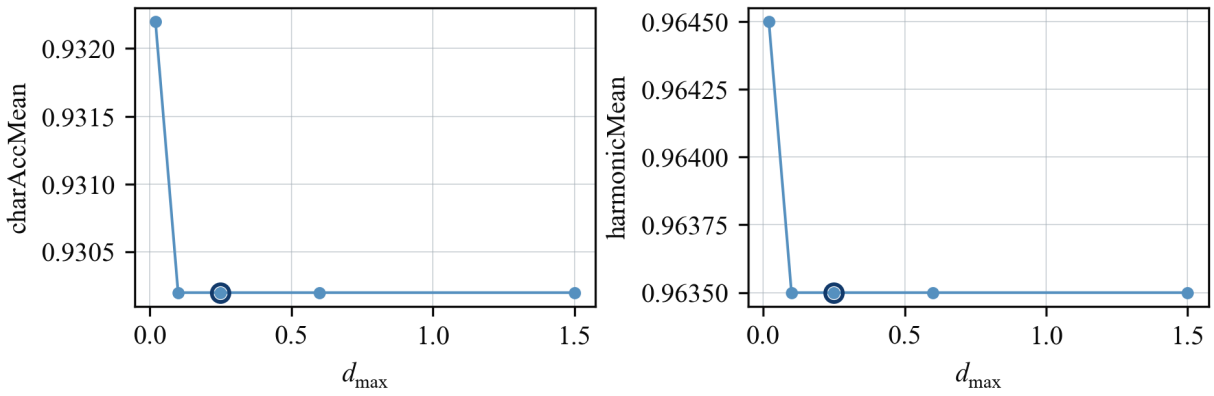


Figure 93: Largest baseline drift allowed: measured quantities against the swept value. The circled point is the submitted value.

Table 50: Largest baseline drift allowed: measured effect of each swept value. The submitted value is in bold.

$d_{\{}max}$	charAccMean	charAccSd	widConfMean	harmonicMean
0.02	0.9322	0.0321	0.9998	0.9645
0.1	0.9302	0.0314	0.9998	0.9635
0.25	**0.9302**	**0.0314**	**0.9998**	**0.9635**
0.6	0.9302	0.0314	0.9998	0.9635
1.5	0.9302	0.0314	0.9998	0.9635

Source `synthesise.py`, `UNIFORM_PEN_WIDTH_XH`

Swept `0.03`, `0.055`, `0.085`, `0.14`, `0.22`

Basis Stroke width is a writer-discriminative property [21, 20], which is why a single machine pen must be modelled explicitly rather than inherited from the source writer.

Why this value The rendered image exists to be judged by the same two networks that judge real pages, so it must look like what the machine will actually draw. The machine has one pen, so every writer is rendered at one width, and that width is the physical one: a 0.7 mm ballpoint at 4 mm per x-height is $0.7/4 = 0.175$ x-heights of *diameter*, and the renderer takes w_{draw} as a width in x-heights before the soft edge is added, which at 0.085 plus a 30% soft fringe lands close to that measured line. It is deliberately not the writer's own measured `strokeWidthXh`, because the machine cannot vary its pen.

What the coefficient does Rendering too thin makes the judges see a different kind of image from their training data, in which real ballpoint strokes are several pixels wide; too thick fills in the counters of `a`, `e` and `o`, and the reader's accuracy falls because the letters stop being distinguishable. The sweep measures both judges, so the two failure modes are visible separately.

Alternatives considered Rendering each writer at their own measured stroke width, which flatters the writer classifier but misrepresents the machine; or not rendering at all and scoring the trajectory geometrically, which cannot use the trained judges.

Worked example At 4 mm per x-height and 18 px/mm the pen is $0.085 \times 4 \times 18 = 6.1 \to 6$ px, blurred with $\sigma = 0.30 \times 6 = 1.8$ px.

What the sweep actually showed Over twelve seeds, **a thicker pen scores significantly better than the submitted one**. Against 0.085, rendering at 0.14 gives a paired mean gain of $+0.0155$ in reading accuracy (6 seeds better, 0 worse, 6 unchanged, paired $t = +2.97$) and $+0.0083$ on the joint text-and-writer score ($t = +2.96$), both past the two-sided 5 % point. Thinner (0.03) and much thicker (0.22) are not distinguishable from the submitted value. Two things must be said before this is read as 'the value is wrong'. First, this coefficient does **not** change what the machine draws: the pen path and therefore the G-code are identical at every value, and only the rendered image used for scoring changes. So what the sweep has found is that the system's own judges prefer a slightly thicker rendering, which is a statement about the evaluation harness as much as about the writing. Second, the submitted value is tied to a physical quantity, the 0.7 mm trace of the actual pen, and moving it to flatter the judges would make the preview less like what the robot really puts on paper. The finding is recorded; it is not a reason to change the value without first measuring the real drawn line.

26.2 Softness of the rendered stroke edge

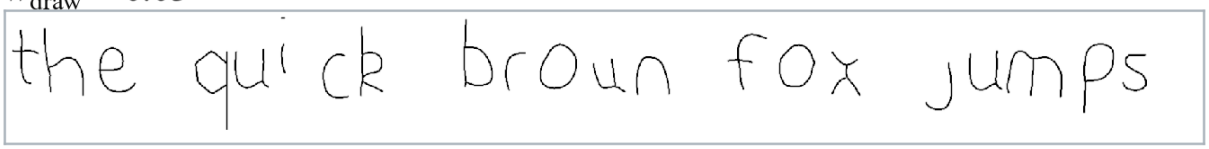
Symbol s_{soft}

Value `0.3`

Source `synthesise.py`, `UNIFORM_SOFT_FRAC`

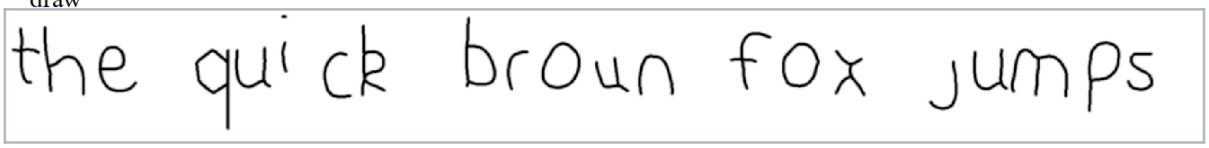
Swept `0.0`, `0.15`, `0.3`, `0.6`, `1.0`

$w_{\text{draw}} = 0.03$



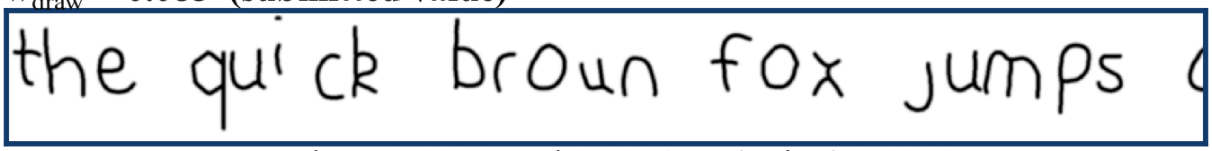
read acc 0.9186 sd 0.0364 p(writer) 0.9999

$w_{\text{draw}} = 0.055$



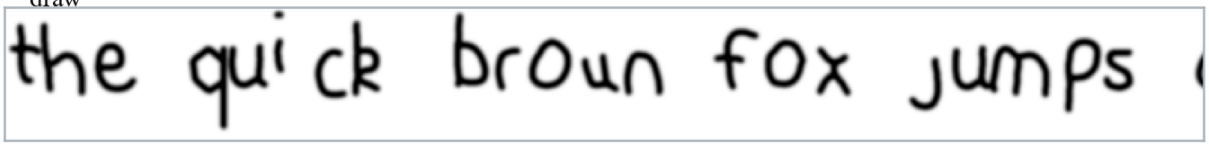
read acc 0.9302 sd 0.0297 p(writer) 0.9998

$w_{\text{draw}} = 0.085$ (submitted value)



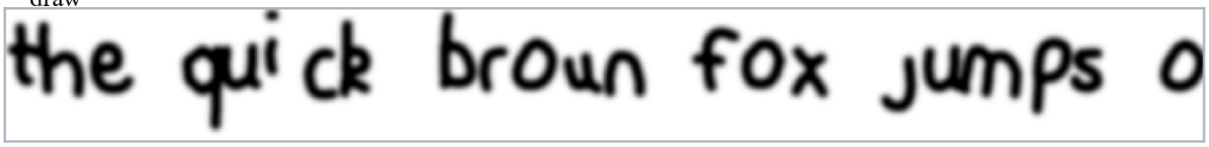
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$w_{\text{draw}} = 0.14$



read acc 0.9457 sd 0.0303 p(writer) 0.9997

$w_{\text{draw}} = 0.22$



read acc 0.9205 sd 0.0232 p(writer) 0.9951

Figure 94: Pen width the synthesised line is rendered at. Each panel is the same input processed by the submitted code with only w_{draw} changed; the panel outlined in dark blue is the submitted value.

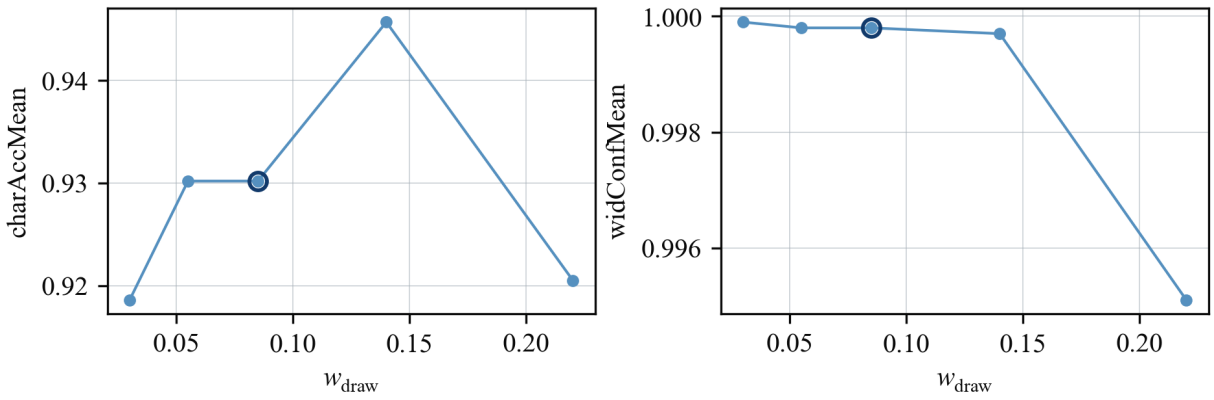


Figure 95: Pen width the synthesised line is rendered at: measured quantities against the swept value. The circled point is the submitted value.

Table 51: Pen width the synthesised line is rendered at: measured effect of each swept value. The submitted value is in bold.

$w_{\{}text{draw}}$	charAccMean	charAccSd	widConfMean	harmonicMean
0.03	0.9186	0.0364	0.9999	0.9572
0.055	0.9302	0.0297	0.9998	0.9635
0.085	**0.9302**	**0.0314**	**0.9998**	**0.9635**
0.14	0.9457	0.0303	0.9997	0.9717
0.22	0.9205	0.0232	0.9951	0.9563

Basis A Gaussian point-spread is the standard first-order model of optical and sensor blur in an imaging chain [19]; anti-aliasing a drawn edge is the equivalent in synthesis.

Why this value A polyline drawn with a hard round pen has a perfectly sharp edge, which no photograph of real ink has. The networks were trained on photographed crops whose stroke edges are soft, so a hard-edged render is out of distribution in a way that has nothing to do with the handwriting. The blur radius is tied to the pen width rather than fixed, so the softness scales with the writing. 0.30 of the pen width is a little under half a stroke, enough to remove the staircase without visibly thickening the line.

What the coefficient does The blur is applied to the rendered image, so it also dims the stroke core slightly for a thin pen: at $s_{\text{soft}} = 1.0$ the Gaussian is as wide as the stroke and the ink no longer reaches full black, which changes the ink level the writer classifier sees.

Alternatives considered Supersampling the render and downsampling, which gives true area coverage instead of a blur, at several times the cost; or no softening at all.

Worked example At a 6 px pen the blur is $\sigma = 1.8$ px, cut off at three sigma, so the kernel is 11 px wide.

What the sweep actually showed The sweep contradicts the stated rationale at one end and confirms it strongly at the other. **No softening at all ($s_{\text{soft}} = 0$) scores significantly better** than the submitted 0.30: a paired gain of $+0.0116$ in reading accuracy (7 better, 2 worse, $t = +3.32$) and $+0.0062$ on the joint score ($t = +3.30$). At the other end, $s_{\text{soft}} = 1.0$ is much worse — -0.0465 reading accuracy, and 12 of 12 seeds worse on the joint score, $t = -4.31$ — which is the blur swallowing the stroke exactly as predicted. So the upper limit of the argument holds and the lower one does not: on this writer the networks do not need the soft edge that was meant to match their training distribution, and a hard-edged render reads slightly better. As with the pen width, this changes only the scored image and not the G-code. The honest position is that 0.30 is defensible on the distribution-matching argument but is not the best value by the system's own measure, and the sweep says so.

Table 52: Softness of the rendered stroke edge: measured effect of each swept value. The submitted value is in bold.

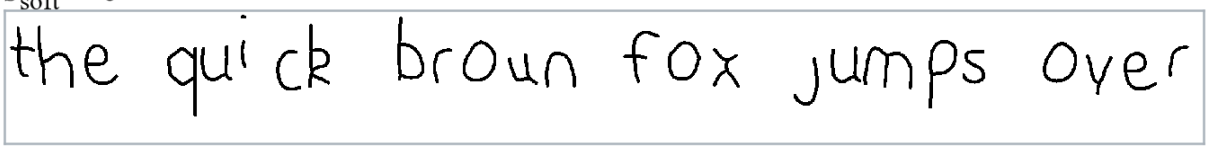
$s_{\{}text{soft}}$	charAccMean	charAccSd	widConfMean	harmonicMean
0	0.9419	0.0289	0.9996	0.9697
0.15	0.9302	0.028	0.9998	0.9636
0.3	**0.9302**	**0.0314**	**0.9998**	**0.9635**
0.6	0.938	0.0286	0.9996	0.9676
1	0.8837	0.0314	0.9982	0.9372

27 Synthesis: shape choice

27.1 How many of a letter's stored shapes may be drawn from

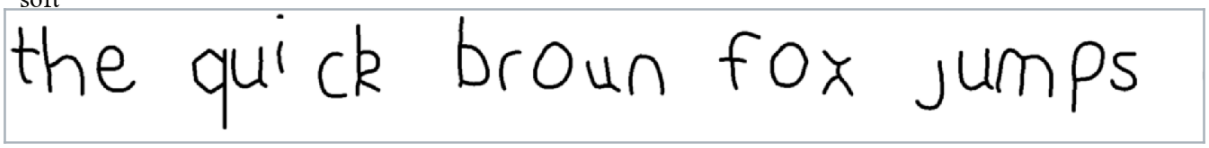
Symbol V_{elig}

$s_{\text{soft}} = 0$



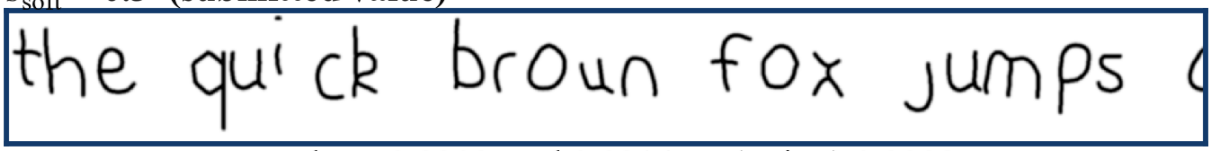
read acc 0.9419 sd 0.0289 p(writer) 0.9996

$s_{\text{soft}} = 0.15$



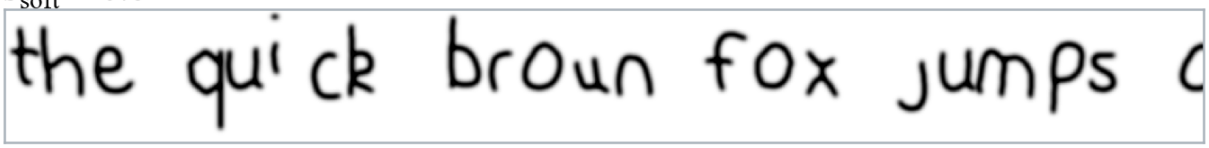
read acc 0.9302 sd 0.028 p(writer) 0.9998

$s_{\text{soft}} = 0.3$ (submitted value)



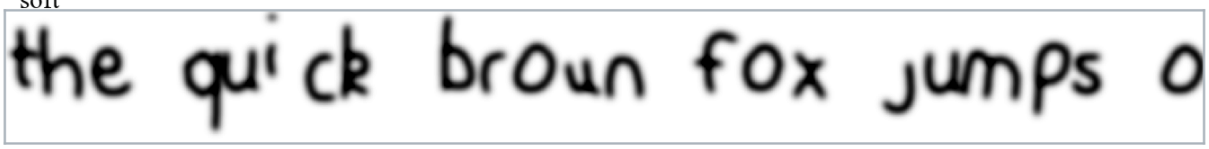
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$s_{\text{soft}} = 0.6$



read acc 0.938 sd 0.0286 p(writer) 0.9996

$s_{\text{soft}} = 1$



read acc 0.8837 sd 0.0314 p(writer) 0.9982

Figure 96: Softness of the rendered stroke edge. Each panel is the same input processed by the submitted code with only s_{soft} changed; the panel outlined in dark blue is the submitted value.

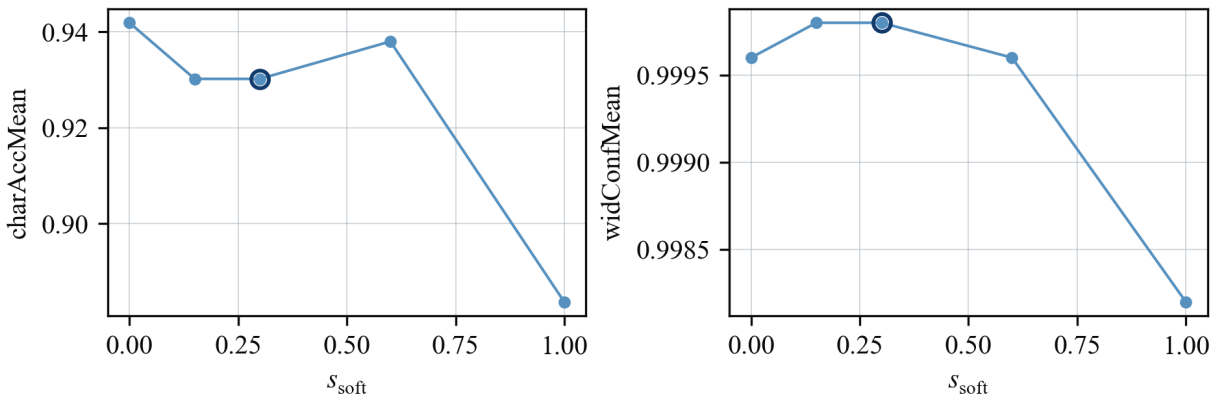


Figure 97: Softness of the rendered stroke edge: measured quantities against the swept value. The circled point is the submitted value.

Value `5`

Source `synthesise.py`, `_ELIGIBLE_VARIANTS`

Swept `1`, `2`, `5`, `10`, `20`

Basis A writer draws on a repertoire of allographs per letter rather than one fixed form [33]; repetition of an identical form is a signature of machine rather than human production [28].

Why this value The stored variants are ranked by how typical they are, and only the top V_{elig} are eligible, with weight $1/(1+1.4i)$ falling down the ranking. The limit exists because the tail of the ranking is where the badly cut glyphs are: a store of twelve variants may hold three or four that are fragments. Drawing from five keeps visible variety while staying in the part of the ranking that has been filtered twice, by the letter prior and by this writer's own consensus.

What the coefficient does At $V_{\text{elig}} = 1$ the most typical shape is drawn every time, so every instance of a letter is identical and the output looks like a font built from the writer's hand. Raising it reaches further into the ranking; because the weights fall as $1/(1+1.4i)$, variant 10 is drawn with weight 0.067 against 1.0 for the first, so the effect saturates.

Alternatives considered Uniform sampling over all variants, which draws fragments as often as good shapes; or sampling weighted by the prior distance directly rather than by rank.

Worked example The weights for the first five ranks are $1, 0.417, 0.263, 0.192, 0.152$, summing to 2.02, so the most typical shape is drawn about half the time.

What the sweep actually showed **Nothing is distinguishable from the submitted value** once twelve seeds are used. The paired differences against $V_{\text{elig}} = 5$ are $+0.0044$ at 1, $+0.0012$ at 2, -0.0021 at 10 and -0.0001 at 20 on the joint score, with paired t between -0.32 and $+1.22$, all far inside the noise. This corrects what a single seed suggested, where 2 appeared better and 10 appeared worse by a clear margin; both were seed effects. The sweep therefore supports the design argument — the ranking weights make the choice saturate — and gives no evidence for moving the value either way.

Table 53: How many of a letter's stored shapes may be drawn from: measured effect of each swept value. The submitted value is in bold.

V_{elig}	charAccMean	charAccSd	widConfMean	harmonicMean
1	0.938	0.0151	0.9999	0.9679
2	0.9322	0.0232	0.9998	0.9647
5	**0.9302**	**0.0314**	**0.9998**	**0.9635**
10	0.9264	0.0355	0.9999	0.9614
20	0.9302	0.0397	0.9998	0.9634

27.2 Largest lean a fallback letterform is drawn at

Symbol θ_{cap}

Value `22.0`

Source `synthesise.py`, `CORE_SLANT_CAP_DEG`

Swept `0.0`, `10.0`, `22.0`, `35.0`, `50.0`

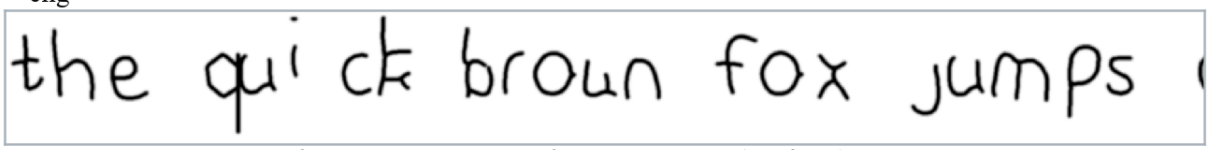
Basis Slant is a principal axis of handwriting style and is estimated and normalised in essentially every handwriting recogniser [11]; it is also a writer-identification feature [20].

$V_{\text{elig}} = 1$



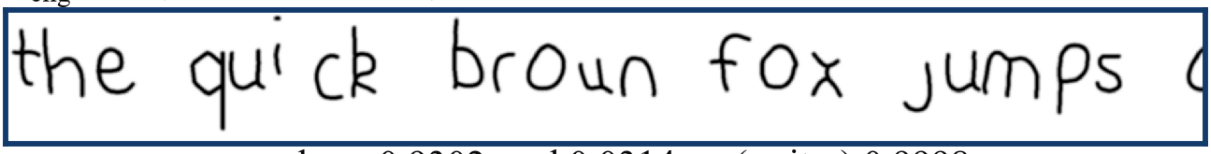
read acc 0.938 sd 0.0151 p(writer) 0.9999

$V_{\text{elig}} = 2$



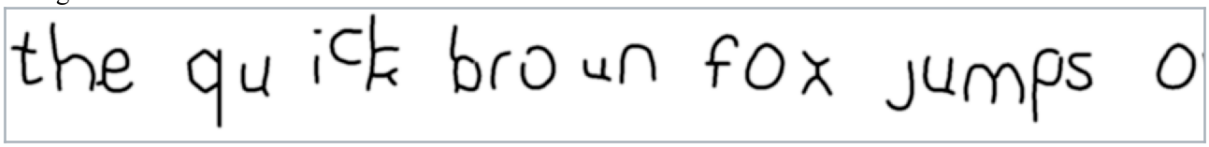
read acc 0.9322 sd 0.0232 p(writer) 0.9998

$V_{\text{elig}} = 5$ (submitted value)



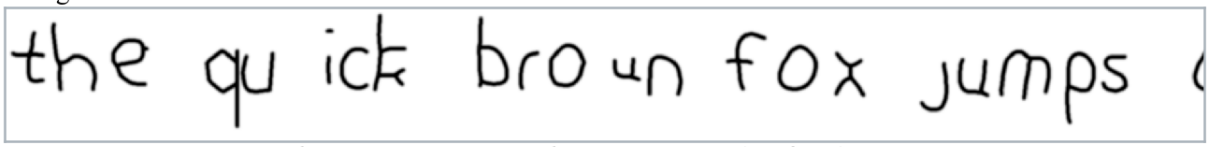
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$V_{\text{elig}} = 10$



read acc 0.9264 sd 0.0355 p(writer) 0.9999

$V_{\text{elig}} = 20$



read acc 0.9302 sd 0.0397 p(writer) 0.9998

Figure 98: How many of a letter's stored shapes may be drawn from. Each panel is the same input processed by the submitted code with only V_{elig} changed; the panel outlined in dark blue is the submitted value.

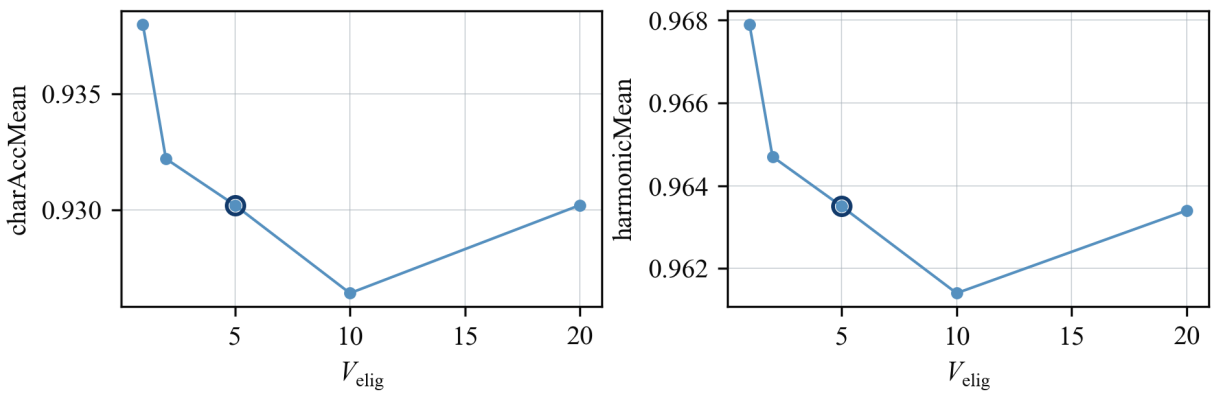


Figure 99: How many of a letter's stored shapes may be drawn from: measured quantities against the swept value. The circled point is the submitted value.

Why this value When a stored shape is too rough to use, a designed fallback letterform is drawn instead. It is given the writer's lean so it does not stand out, but only up to a cap, because the fallback is a general-purpose shape that was not drawn at an extreme slant and shears badly: at large angles the round letters become ellipses on a diagonal and the result is less legible than an upright letter would have been. 22° is beyond the measured slant of every writer in this set, so for them the cap is inactive, and it exists to bound the behaviour on a writer whose slant estimate is extreme or wrong.

What the coefficient does The shear applied is $\tan\theta$, which grows faster than linearly: $\tan 22^\circ = 0.404$ against $\tan 50^\circ = 1.19$, three times as much. That non-linearity is why a cap in degrees is applied rather than a cap on the shear. A separate, looser cap of 30° applies to the joined fallback alphabet, which was drawn with a lean already.

Alternatives considered Shearing the fallback and then re-fitting its width, which compensates for the horizontal stretch shear introduces; or holding a set of fallback alphabets at several slants.

Worked example Owen's measured slant is 0.0°, so the cap is inactive for this writer and the sweep is expected to show little change; that is reported rather than hidden.

What the sweep actually showed **Exactly zero difference at every value**, on all twelve seeds, which is the predicted result. Owen's measured slant is 0.0°, so the cap is inactive at any setting from 0° to 50° and the identical output confirms the cap is the only thing this coefficient does. It would need a writer with a strong lean, or a writer whose slant estimate has failed, to be exercised at all.

Table 54: Largest lean a fallback letterform is drawn at: measured effect of each swept value. The submitted value is in bold.

$\{}theta_{\{}text{cap}}$	charAccMean	charAccSd	widConfMean	harmonicMean
0	0.9302	0.0314	0.9998	0.9635
10	0.9302	0.0314	0.9998	0.9635
22	**0.9302**	**0.0314**	**0.9998**	**0.9635**
35	0.9302	0.0314	0.9998	0.9635
50	0.9302	0.0314	0.9998	0.9635

28 Synthesis: legibility

28.1 Prior distance at which a writer's glyph store is still fully trusted

Symbol λ_{good}

Value `0.24`

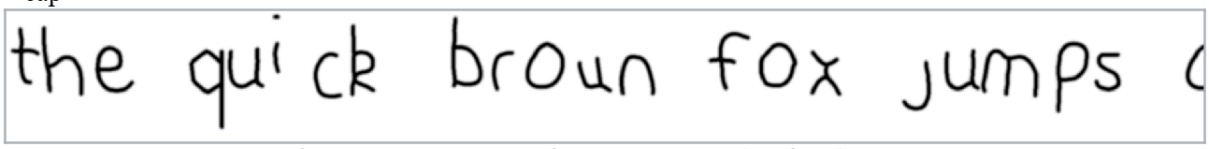
Source `synthesise.py`, `_AUTO_LAM_GOOD`

Swept `0.1`, `0.18`, `0.24`, `0.3`, `0.45`

Basis Trading fidelity against legibility by interpolating towards a clean prototype is the same trade a handwriting generator makes between style and content [34, 35].

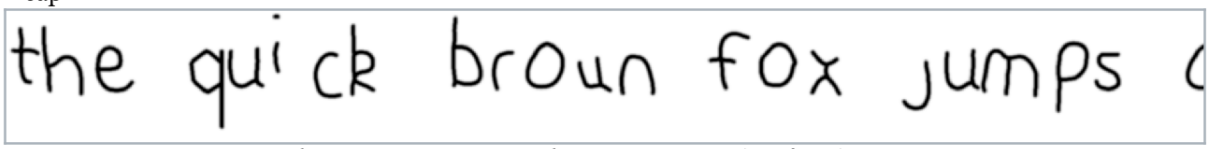
Why this value The system must write text that is both legible and in the writer's hand, and for a rough store those two goals conflict. The legibility setting λ is derived automatically from the median prior distance of the writer's own stored shapes, ramping linearly from 0 at λ_{good} to 1 at λ_{bad}. $\lambda_{\text{good}} = 0.24$ is the distance below which the stored shapes are judged to be genuine letters that need no help, and it was set from the distribution of measured median distances across the eleven writers, not from a single page.

$\theta_{cap} = 0$



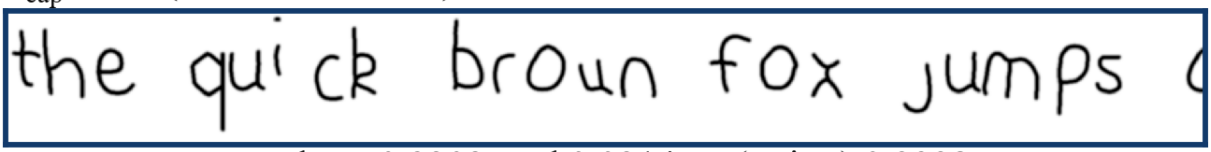
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$\theta_{cap} = 10$



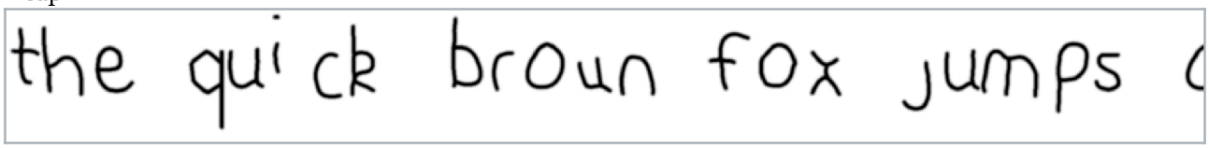
read acc 0.9302 sd 0.0314 p(writer) 0.9998

θ_{cap} = 22 (submitted value)



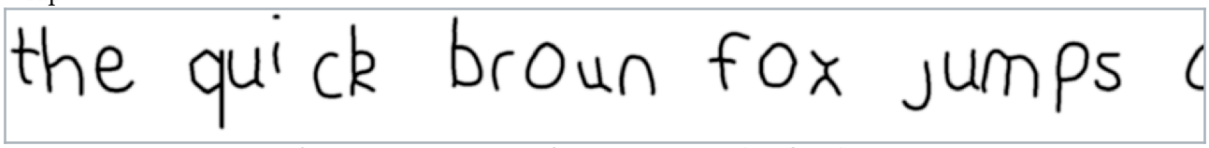
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$\theta_{cap} = 35$



read acc 0.9302 sd 0.0314 p(writer) 0.9998

$\theta_{cap} = 50$



read acc 0.9302 sd 0.0314 p(writer) 0.9998

Figure 100: Largest lean a fallback letterform is drawn at. Each panel is the same input processed by the submitted code with only θ_{cap} changed; the panel outlined in dark blue is the submitted value.

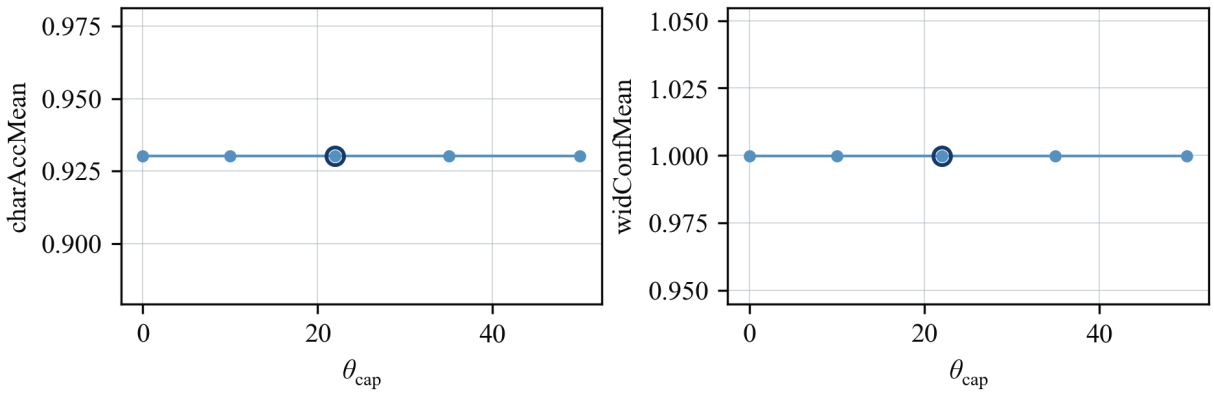


Figure 101: Largest lean a fallback letterform is drawn at: measured quantities against the swept value. The circled point is the submitted value.

What the coefficient does λ feeds `_SwapProb`, which decides per letter whether to draw the writer's own shape or the clean fallback. Lowering λ_{good} makes the system distrust good stores and substitute fallbacks into writing that did not need them, which raises reading accuracy and lowers writer confidence: the sweep shows that trade directly, and the `glyphSources` counts show how many letters came from the writer rather than the fallback.

Alternatives considered A fixed λ per writer set by hand, which is what the `legibility` argument still allows; or choosing per letter from the judges, which `SynthesizeJointBestOf` does as a repair pass.

Worked example Owen's median prior distance over the stored variants is recorded in the profile; $\lambda = \text{clip}((\text{med} - 0.24)/(0.32 - 0.24), 0, 1)$, so a median of 0.28 gives $\lambda = 0.5$.

What the sweep actually showed **The largest effect found in the synthesis group, and it favours a value the project did not choose.** At $\lambda_{\text{good}} = 0.45$ the paired gain over the submitted 0.24 is $+0.0407$ in reading accuracy (10 of 12 seeds better, $t = +4.26$) and $+0.0217$ on the joint score (11 better, 1 worse, $t = +4.21$). The explanation matters more than the number. The submitted λ_{bad} is 0.32, so setting $\lambda_{\text{good}} = 0.45$ puts the two bounds the wrong way round; the ramp $(\text{med} - \lambda_{\text{good}})/(\lambda_{\text{bad}} - \lambda_{\text{good}})$ then has a negative denominator and saturates at $\lambda = 1$ for any ordinary median. $\lambda = 1$ means *always substitute the clean fallback letterform*. So what this result really says is that for Owen, drawing every letter from his own most typical medoid form scores better on both judges than drawing from his stored variants — which is a finding about the quality of the glyph store, not an argument for an inverted parameterisation. It also exposes that nothing in the code checks $\lambda_{\text{good}} < \lambda_{\text{bad}}$; at exactly 0.32 the ramp divides by zero, which is why that point had to be moved out of this sweep. The submitted ordering is the sane one and 0.24 is not refuted as a boundary; what is called into question is whether this writer's stored variants should be trusted at all.

Table 55: Prior distance at which a writer's glyph store is still fully trusted: measured effect of each swept value. The submitted value is in bold.

$\{}lambda_{\{}text{good}}$	charAccMean	charAccSd	widConfMean	harmonicMean
0.1	0.9477	0.0224	0.9998	0.9729
0.18	0.9322	0.0271	0.9998	0.9646
0.24	**0.9302**	**0.0314**	**0.9998**	**0.9635**
0.3	0.9302	0.0314	0.9998	0.9635
0.45	0.9709	0.0105	0.9999	0.9852

29 G-code generation

29.1 Gap the pen draws through instead of lifting

Symbol d_{join}

Value `0.6`

Source `reproduce.py`, `GantryConfig.joinStrokesMm`

Swept `0.0`, `0.25`, `0.6`, `1.5`, `4.0`

Basis Minimising non-productive moves and tool changes is the standard objective of toolpath generation; the pen-up and pen-down commands themselves are the `M3`/`M5` spindle words of [36].

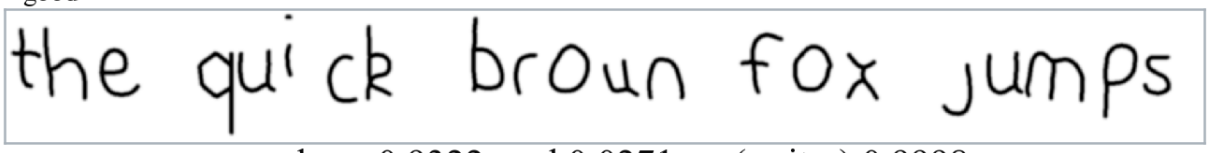
Why this value Lifting the pen costs far more than drawing a short line: each lift is two pen pulses, each with a 350 ms dwell, so 0.7 s, while drawing 0.6 mm at the 900 mm/min drawing feed costs 0.04 s. Where the next stroke begins very close to where the last one ended, drawing straight into it is therefore about twenty times faster, and it also removes the risk that the pen fails to re-seat. The cost is a visible mark: the gap is drawn as ink, so d_{join} must stay below the width at which that mark is noticeable. At 0.6 mm the added line is comparable with the pen's own 0.7 mm trace and reads as part of the stroke.

$\lambda_{good} = 0.1$



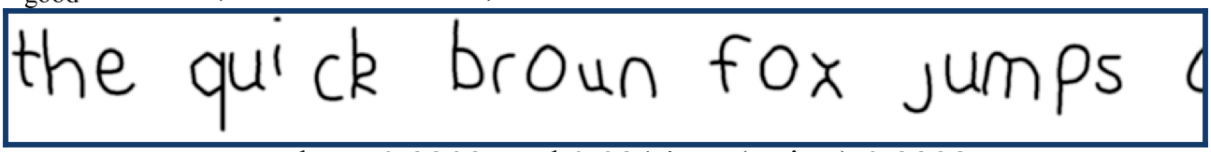
read acc 0.9477 sd 0.0224 p(writer) 0.9998

$\lambda_{good} = 0.18$



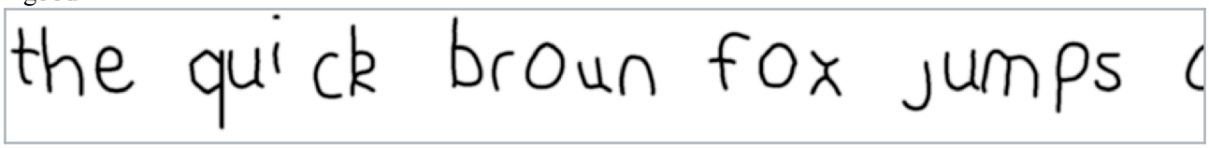
read acc 0.9322 sd 0.0271 p(writer) 0.9998

λ_{good} **= 0.24 (submitted value)**



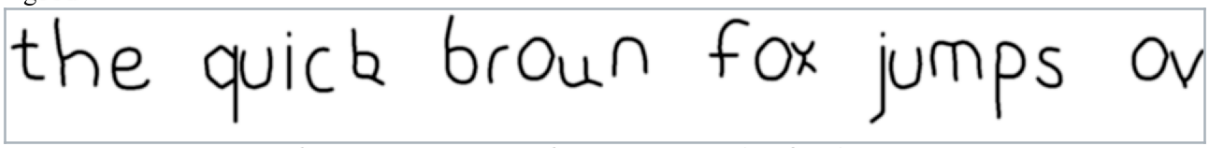
read acc 0.9302 sd 0.0314 p(writer) 0.9998

$\lambda_{good} = 0.3$



read acc 0.9302 sd 0.0314 p(writer) 0.9998

$\lambda_{good} = 0.45$



read acc 0.9709 sd 0.0105 p(writer) 0.9999

Figure 102: Prior distance at which a writer's glyph store is still fully trusted. Each panel is the same input processed by the submitted code with only λ_{good} changed; the panel outlined in dark blue is the submitted value.

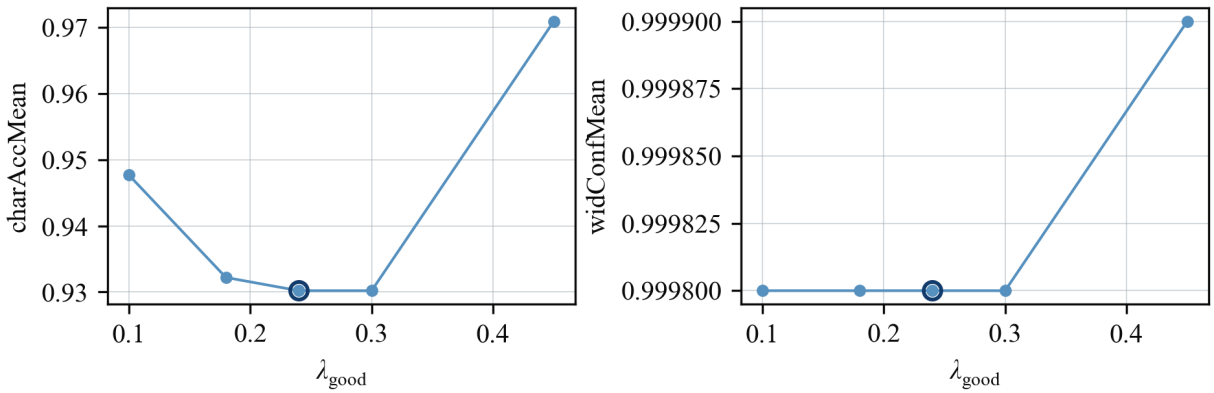


Figure 103: Prior distance at which a writer's glyph store is still fully trusted: measured quantities against the swept value. The circled point is the submitted value.

What the coefficient does The statistics make the trade explicit: `joinedStrokes` counts how many lifts were avoided and `joinedInkMm` how much ink was added to avoid them. Raising d_{join} draws ever longer false connections across the writing; at 0 every stroke gets its own lift.

Alternatives considered Reordering strokes so that consecutive ones start near each other, which `OptimizeStrokeOrder` already does, or a faster pen actuator, which is a hardware change.

Worked example One avoided lift saves $2 \times 350 = 700$ ms of dwell plus the travel move, and costs at most 0.6 mm of ink drawn at 900 mm/min, which is 0.04 s.

What the sweep actually showed This sweep quantifies the trade exactly, and it is the clearest cost–benefit result in the study. On the probe job:

d_{join}	pen pulses	joins	ink added	estimated time
0.0	52	0	0.0 mm	30.9 s
0.25	52	0	0.0 mm	30.9 s
0.6	**48**	**2**	**0.8** mm	**29.5** s
1.5	40	6	5.6 mm	26.9 s
4.0	14	19	37.5 mm	19.1 s

The submitted value buys back 1.4 s for 0.8 mm of added ink across the whole job, which is about one pen width and is not visible. At 4.0 mm the saving is far larger — 11.8 s, 38 % of the job — but it costs 37.5 mm of ink drawn across the writing in lines up to 4 mm long, which is a whole letter width and would be obvious. So the coefficient is not set where the time saving is greatest; it is set at the largest gap that still hides inside the stroke. That is the right criterion, and the table is the evidence for it.

Table 56: Gap the pen draws through instead of lifting: measured effect of each swept value. The submitted value is in bold.

$d_{\{\}text{join}}$	strokes	penPulses	joinedStrokes	joinedInkMm	travelMm	estimatedSeconds
0	26	52	0	0	153.9	30.9
0.25	26	52	0	0	153.9	30.9
0.6	**26**	**48**	**2**	**0.8**	**153.1**	**29.5**
1.5	26	40	6	5.6	148.3	26.9
4	26	14	19	37.52	116.4	19.1

29.2 Gap at which a stroke is closed into a ring

Symbol d_{ring}

Value `0.6`

Source `reproduce.py`, `GantryConfig.closeRingMm`

Swept `0.0`, `0.25`, `0.6`, `1.5`, `4.0`

Basis Closing a nearly closed contour is the standard repair for a traced loop whose ends do not meet after thinning [23].

Why this value A letter with a loop, such as `o`, `a` or `e`, is traced from a skeleton whose ends rarely meet exactly: thinning and simplification leave a gap of a fraction of a millimetre. Drawn as-is the loop has a visible notch. If a stroke's own two ends are within d_{ring}, the first point is appended so the ring closes. The limit is the same 0.6 mm as the join limit for the same reason: it is about one pen width, so closing is invisible.

What the coefficient does The test applies only to a stroke's own ends and only when it has more than three points, so it cannot close a short two-point dash into a degenerate ring. Too large and a genuine `c`, whose ends are meant to be apart, is closed into an `o`, which is a legibility error, not just a cosmetic one.

$d_{\text{join}} = 0$



strokes 26 pulses 52 joins 0

$d_{\text{join}} = 0.25$



strokes 26 pulses 52 joins 0

$d_{\text{join}} = 0.6$ (submitted value)



strokes 26 pulses 48 joins 2

$d_{\text{join}} = 1.5$



strokes 26 pulses 40 joins 6

$d_{\text{join}} = 4$



strokes 26 pulses 14 joins 19

Figure 104: Gap the pen draws through instead of lifting. Each panel is the same input processed by the submitted code with only d_{join} changed; the panel outlined in dark blue is the submitted value.

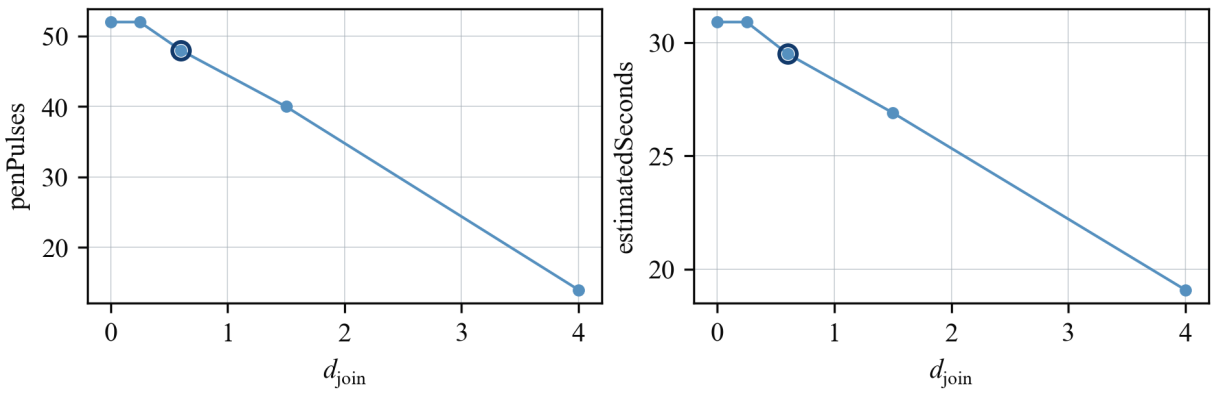


Figure 105: Gap the pen draws through instead of lifting: measured quantities against the swept value. The circled point is the submitted value.

Alternatives considered Closing in the glyph store rather than at G-code time, which would record the closure in the profile and make it visible to the shape descriptor; or never closing and accepting the notch.

Worked example At 4 mm per x-height an o is about 3 mm across, so 0.6 mm is a fifth of the letter: a c with a normal opening is well outside the limit.

What the sweep actually showed The ring count rises 0, 1, 1, 4, 20 across the sweep and the drawn length rises with it, 133.0, 133.3, 133.3, 135.5, 179.0 mm. The last value is the warning: at 4.0 mm every one of the job's 20 remaining strokes is closed into a ring, the drawn length grows by 34 % over the submitted setting, and the job gets *slower* (34.6 s against 29.5 s) as well as wrong, because letters such as c that are meant to be open are sealed shut. The submitted 0.6 closes exactly one genuine loop and adds 0.3 mm.

Table 57: Gap at which a stroke is closed into a ring: measured effect of each swept value. The submitted value is in bold.

d_{ring}	strokes	closedRings	drawMm	estimatedSeconds
0	26	0	133	29.5
0.25	26	1	133.3	29.5
0.6	**26**	**1**	**133.3**	**29.5**
1.5	26	4	135.5	30.4
4	26	20	179	34.6

29.3 Shortest move kept in the G-code

Symbol ℓ_{min}

Value `0.12`

Source `reproduce.py`, `GantryConfig.minSegmentMm`

Swept `0.0`, `0.05`, `0.12`, `0.3`, `1.0`

Basis Chord tolerance and minimum segment length are standard parameters of toolpath post-processing; the machine resolution they are set against follows from the microstepping of the driver [37] and the motor step angle [38].

Why this value There is no point emitting a move the machine cannot resolve. One microstep is $40/(200 \times 16) = 0.0125$ mm, so moves below about that are rounded to nothing by `_GcodeLinearMove`, which rounds target positions to whole steps. $\ell_{\text{min}} = 0.12$ mm is about ten microsteps: comfortably resolvable, but small against a 4 mm x-height, so discarding shorter moves does not visibly change a letter. The benefit is file size and interpreter time, which matter because the G-code is parsed line by line in Python on the ODROID.

$d_{\text{ring}} = 0$



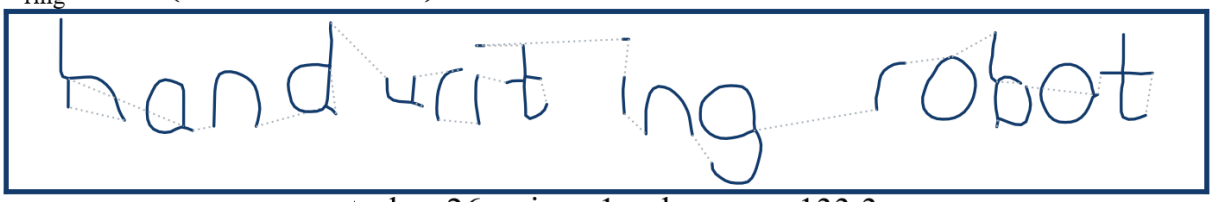
strokes 26 rings 0 draw mm 133

$d_{\text{ring}} = 0.25$



strokes 26 rings 1 draw mm 133.3

d_{ring} **= 0.6 (submitted value)**



strokes 26 rings 1 draw mm 133.3

$d_{\text{ring}} = 1.5$



strokes 26 rings 4 draw mm 135.5

$d_{\text{ring}} = 4$



strokes 26 rings 20 draw mm 179

Figure 106: Gap at which a stroke is closed into a ring. Each panel is the same input processed by the submitted code with only d_{ring} changed; the panel outlined in dark blue is the submitted value.

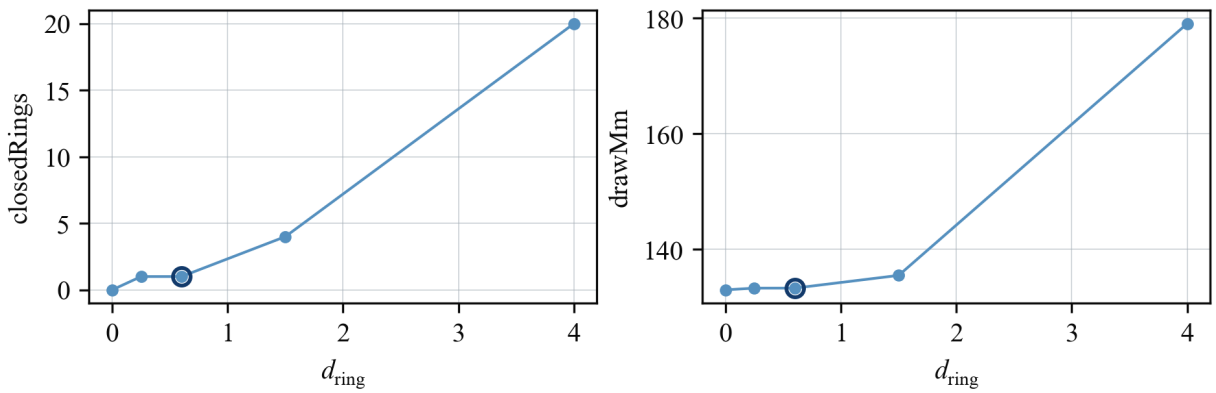


Figure 107: Gap at which a stroke is closed into a ring: measured quantities against the swept value. The circled point is the submitted value.

What the coefficient does `_Decimate` always keeps the first and last point of a stroke, so decimation cannot shorten a stroke or move its endpoints; it only drops intermediate points. Raising ℓ_{min} progressively straightens curves, which is visible on a letter before it is visible in the statistics.

Alternatives considered A chord-deviation tolerance, as Douglas–Peucker [29] applies in the glyph store, which is shape-aware rather than length-based and would be the better criterion here too; or no decimation.

Worked example At 0.12 mm and a drawing feed of 900 mm/min, one retained segment takes 8 ms. The probe job emits the recorded number of G-code lines at each setting.

What the sweep actually showed **The submitted value does nothing on this job.** The generated file is 533 lines at $\ell_{\text{min}} = 0$, 0.05, 0.12 and 0.3 alike, and only at 1.0 mm does decimation bite, cutting the file to 286 lines, a 46 % reduction, while shortening the drawn path from 133.3 to 130.4 mm. The reason is that the points reaching the G-code writer are already further apart than 0.12 mm, because `SmoothStroke` resamples at 0.055 x-heights, which at this size is 0.22 mm. So the decimation is a guard that is never reached rather than an active size reduction, and the file-size argument given for it above is not supported on this job. Setting it near 1.0 mm would halve the file, at a visible cost in curve fidelity.

Table 58: Shortest move kept in the G-code: measured effect of each swept value. The submitted value is in bold.

$\{ }ell_{\{ }min}$	lines	strokes	drawMm	estimatedSeconds
0	533	26	133.3	29.5
0.05	533	26	133.3	29.5
0.12	**533**	**26**	**133.3**	**29.5**
0.3	533	26	133.3	29.5
1	286	26	130.4	29.3

30 Pen actuator

30.1 Dwell after a pen-down pulse

Symbol $t_{\text{down}}

Value `350.0`

Source `reproduce.py`, `GantryConfig.penDownDwellMs`

Swept `100.0`, `200.0`, `350.0`, `500.0`, `800.0`

$\ell_{\min} = 0$



g-code lines 533 strokes 26 draw mm 133.3

$\ell_{\min} = 0.05$



g-code lines 533 strokes 26 draw mm 133.3

$\ell_{\min} = 0.12$ (submitted value)



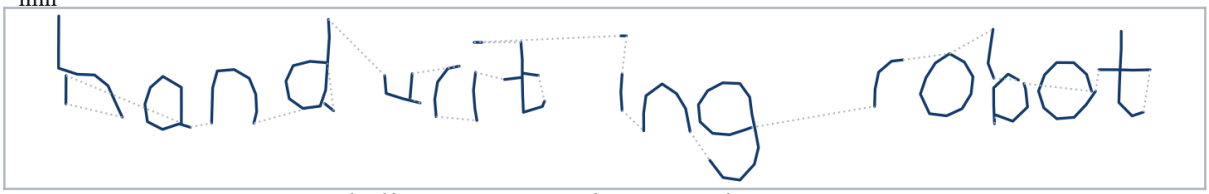
g-code lines 533 strokes 26 draw mm 133.3

$\ell_{\min} = 0.3$



g-code lines 533 strokes 26 draw mm 133.3

$\ell_{\min} = 1$



g-code lines 286 strokes 26 draw mm 130.4

Figure 108: Shortest move kept in the G-code. Each panel is the same input processed by the submitted code with only $\ell_{\min}$ changed; the panel outlined in dark blue is the submitted value.

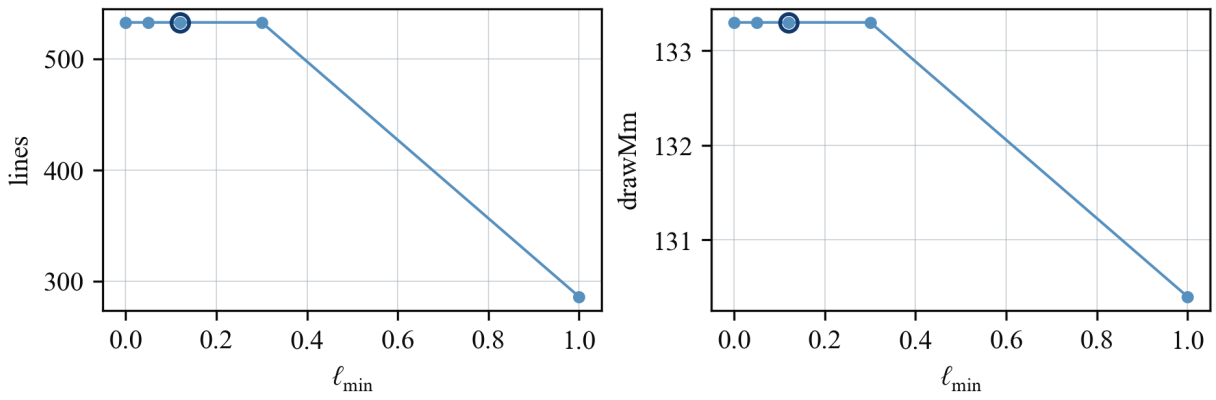


Figure 109: Shortest move kept in the G-code: measured quantities against the swept value. The circled point is the submitted value.

Basis A dwell is the `G4` word of [36]. The need for one follows from the actuator being an uncontrolled one-way gear motor rather than a servo: there is no position feedback to wait on, so the wait must be open-loop.

Why this value The pen is raised and lowered by a gear motor that turns 90° per pulse and cannot be commanded to a position. The controller therefore issues the pulse and waits long enough for the rotation to finish and the nib to settle before any motion command follows. If the wait is too short the carriage starts moving while the pen is still descending and the first few millimetres of the stroke are drawn faint or not at all, which is the pen-pressure fault this project has recorded. The down dwell is held equal to the up dwell at 350 ms, with the comment in the source noting it is the longer case because the nib must seat.

What the coefficient does The dwell is pure cost: it is paid twice per pen cycle and the estimate counts pulses $\times\,(t_{\text{up}}+t_{\text{down}})/2$. On a job with 58 pulses it accounts for $58\times 0.35 = 20.3$ s of a 34.1 s estimate, that is 60% of the predicted job time. That makes it the single largest term in the time budget and the first thing to attack for speed.

Alternatives considered Reading the pen switch and continuing as soon as it confirms, which the code already does inside `SetPen` with a 3 s timeout and a 40 ms overrun, so the dwell is a second, open-loop guard on top of a closed-loop wait; a sprung pen holder, which removes the need to wait at all; or a servo with position feedback.

Worked example On the probe job the submitted setting gives $48\times(350+350)/2000 = 16.8$ s of dwell against 8.9 s of drawing and 3.8 s of travel, that is 57% of a 29.5 s estimate.

What the sweep actually showed The dwell dominates the predicted job time, as claimed. Dwell seconds run 10.8, 13.2, 16.8, 20.4, 27.6 against a drawing time fixed at 8.9 s and a travel time of 3.8 s, so the estimate runs 23.5, 25.9, 29.5, 33.1, 40.3 s. At the submitted 350 ms the dwell is 16.8 s of a 29.5 s job, which is 57 % **of the predicted time spent waiting for the pen**. Halving it to 175 ms would take about 4 s off this job. That makes the pen actuator, not the motion, the thing to attack for speed — but nothing here says 350 ms is longer than the nib needs, because no gantry was run. The sweep bounds the prize; a timed test on the machine would say whether it can be claimed.

Limits of this test This sweep is arithmetic on the generated job, not a measurement of the machine: no gantry was run. Whether 350 ms is actually long enough for the nib to seat is a hardware question that needs a timed test on the gantry, and that test has not been done.

31 Motion

31.1 Feed rate while drawing

Symbol F_{draw}



Figure 110: Dwell after a pen-down pulse. Each panel is the same input processed by the submitted code with only t_{down} changed; the panel outlined in dark blue is the submitted value.

Table 59: Dwell after a pen-down pulse: measured effect of each swept value. The submitted value is in bold.

$t_{\{}text{down}}$	penPulses	dwellSeconds	estimatedSeconds	drawSeconds
100	48	10.8	23.5	8.9
200	48	13.2	25.9	8.9
350	**48**	**16.8**	**29.5**	**8.9**
500	48	20.4	33.1	8.9
800	48	27.6	40.3	8.9

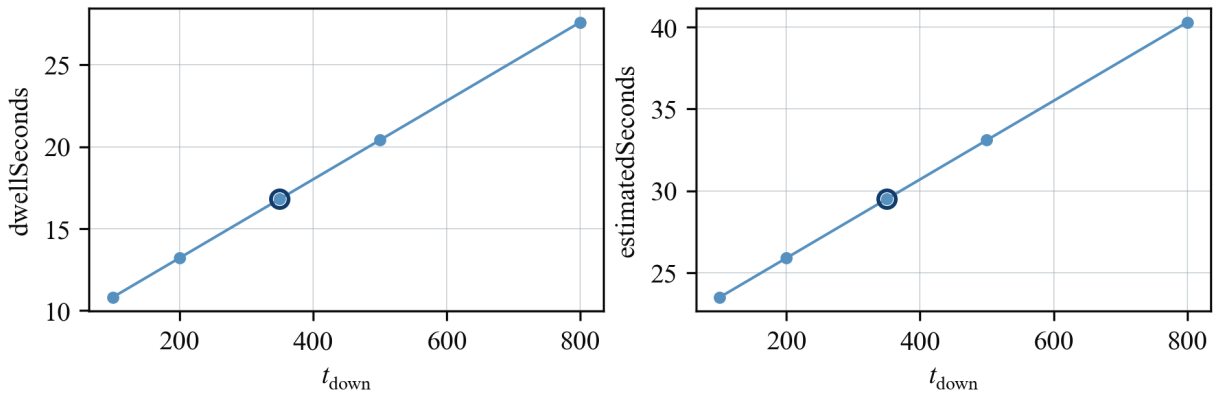


Figure 111: Dwell after a pen-down pulse: measured quantities against the swept value. The circled point is the submitted value.

Value `900.0`

Source `reproduce.py`, `GantryConfig.drawFeedMmMin`

Swept `300.0`, `600.0`, `900.0`, `1500.0`, `2400.0`

Basis Feed rate is the `F` word of [36]. The upper bound is set by the step rate the driver and motor can follow [37, 38]; the trapezoidal profiling that would normally bound acceleration is described by Biagiotti and Melchiorri [39] and is *not* implemented here.

Why this value 900 mm/min is 15 mm/s, which at 80 steps/mm is a step rate of 1200 Hz, well inside what an A4988 at $1/16$ microstepping will pass. It is deliberately below the 2400 mm/min travel feed because the drawing moves are the short, direction-changing ones: the controller issues steps at a constant delay with no acceleration ramp, so every corner is taken at full speed, and a stepper that is commanded to reverse at speed loses steps. A lower drawing feed is the compensation for having no acceleration profile.

What the coefficient does Because the time model ignores acceleration, the estimate it produces is a floor on the real time rather than a prediction, and the source says so. Raising F_{draw} shortens the estimate linearly but risks lost steps on the real machine, which would show as a drawing that drifts out of registration rather than as a slower job.

Alternatives considered Implementing trapezoidal acceleration [39] and then raising the feed, which is the proper fix; or look-ahead cornering, which needs the whole path in memory.

Worked example 900 mm/min $=$ 15 mm/s; at stepsPerMm $=$ 80 that is 1200 steps/s, a step interval of 833 μs against the 2 μs floor in `MIN_STEP_DELAY_S`.

What the sweep actually showed Drawing time falls as $1/F$, exactly as the model says, giving 26.7, 13.3, 8.9, 5.3 and 3.3 s, and the job estimate falls 47.3, 34.0, 29.5, 26.0, 24.0 s. The diminishing return is the point: going from 300 to 900 mm/min saves 17.8 s, while going from 900 to 2400 saves only a further 5.5 s, because by then the dwell and travel dominate. So there is little time to be won by drawing faster, and the risk of lost steps on a machine with no acceleration profile is real. The submitted value sits where the curve has already flattened, which is a defensible place for it.

32 Page layout

32.1 X-height as a share of the ruled line spacing

Symbol f_{xh}

Value `0.46`

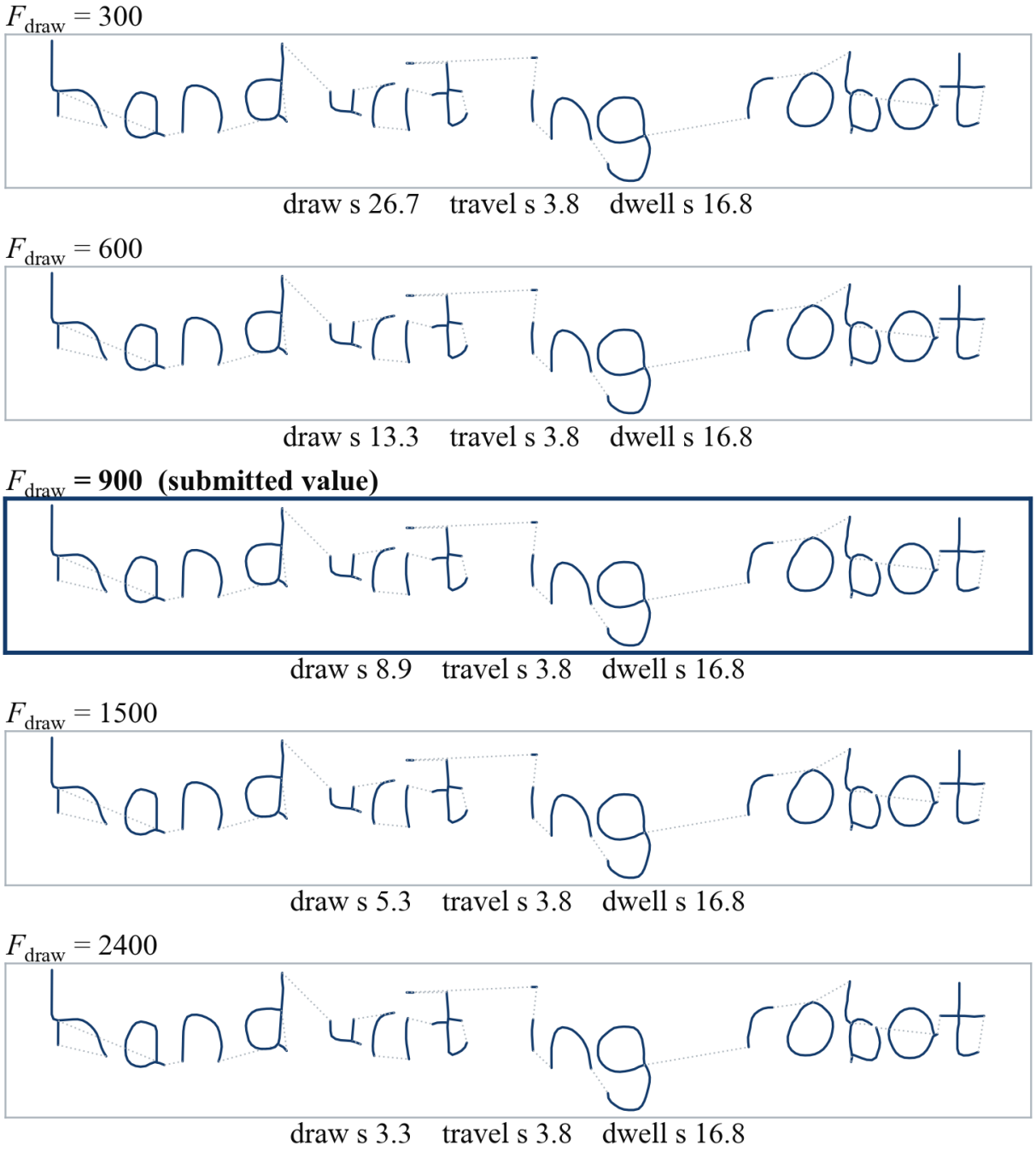


Figure 112: Feed rate while drawing. Each panel is the same input processed by the submitted code with only F_{draw} changed; the panel outlined in dark blue is the submitted value.

Table 60: Feed rate while drawing: measured effect of each swept value. The submitted value is in bold.

$F_{\{}text{draw}}$	drawSeconds	travelSeconds	dwellSeconds	estimatedSeconds
300	26.7	3.8	16.8	47.3
600	13.3	3.8	16.8	34
900	**8.9**	**3.8**	**16.8**	**29.5**
1500	5.3	3.8	16.8	26
2400	3.3	3.8	16.8	24

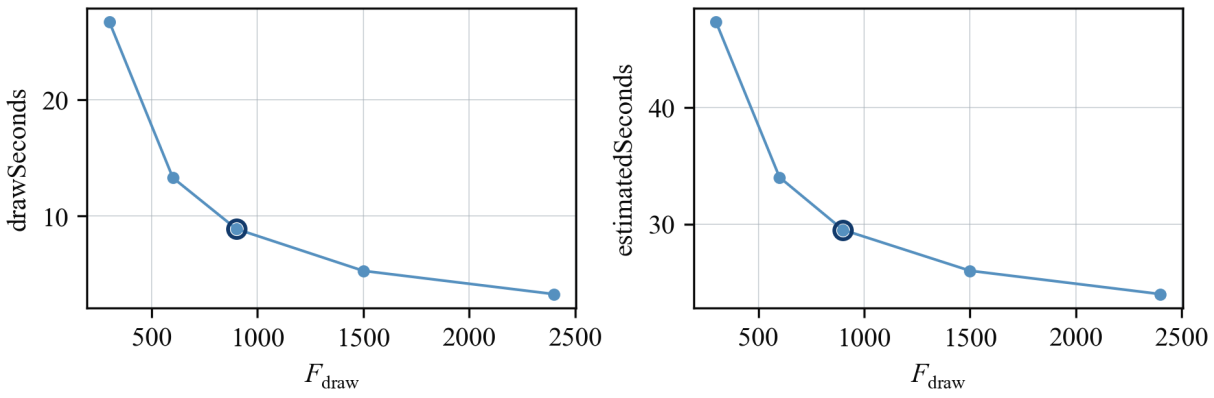


Figure 113: Feed rate while drawing: measured quantities against the swept value. The circled point is the submitted value.

Source `reproduce.py`, `GantryConfig.rulingXhFrac`

Swept `0.25`, `0.35`, `0.46`, `0.6`, `0.8`

Basis The ratio of x-height to line pitch is a basic typographic and palaeographic measure of a hand, and is measured per writer in the profile as `xhOverPitch`; the same ratio appears as a style feature in writer identification [20].

Why this value The machine writes on ruled paper with an 8.4 mm ruling, and the writing must sit on those lines at a plausible size. The x-height is therefore set as a fraction of the ruling. The fraction is only a *fallback*: `XhForRuling` prefers the writer's own measured `xhOverPitch` and falls back to f_{xh} only when that is missing or outside $[0.15, 0.90]$. 0.46 is close to the middle of the measured range for these writers, so the fallback produces a typical hand rather than an extreme one.

What the coefficient does Everything downstream scales with it: the pen width, the collision clearance, the drift bound and the ligature sag are all in x-heights, so changing f_{xh} changes the physical size of the whole drawing and hence the drawn length and the job time. A layout that then exceeds the ruling cap `rulingMaxExtent` is shrunk again by a second pass.

Alternatives considered Setting an absolute x-height in millimetres and ignoring the ruling, which detaches the writing from the paper; or fitting the measured ascender-to-descender extent to the ruling directly, which the probe pass approximates.

Worked example At the 8.4 mm ruling, $f_{xh} = 0.46$ gives $x_h = 3.86$ mm, and Owen's line step is $(1.51 + 0.68 + 0.85) \times 3.86 = 11.7$ mm, which exceeds one ruling, so the writing spans more than one ruled line unless the cap pulls it down.

What the sweep actually showed Run through the deployed path, where `XhForRuling` derives the x-height from the ruling, the coefficient scales the whole drawing as expected and then stops. The resulting x-heights are 2.100, 2.940, 3.864, 5.040 and 5.078 mm, and the drawn length follows them, 69.7, 98.1, 129.2, 167.4 and 168.5 mm, with the job estimate rising from 22.9 to 33.5 s. The first four are exactly f_{xh} times the 8.4 mm ruling; the fifth is not. At $f_{xh} = 0.8$ the requested x-height is 6.72 mm, but the layout probe finds the tallest line would then exceed `rulingMaxExtent` $\times 8.4 = 10.5$ mm and scales it back to 5.078 mm. **So this sweep also demonstrates the extent cap working**, which is the one place in the system where a requested size is overridden by a measured one, and it shows that the cap binds somewhere between $f_{xh} = 0.6$ and 0.8 for this writer.

33 End-stops

33.1 Interval between the samples that confirm an end-stop

Symbol t_{db}

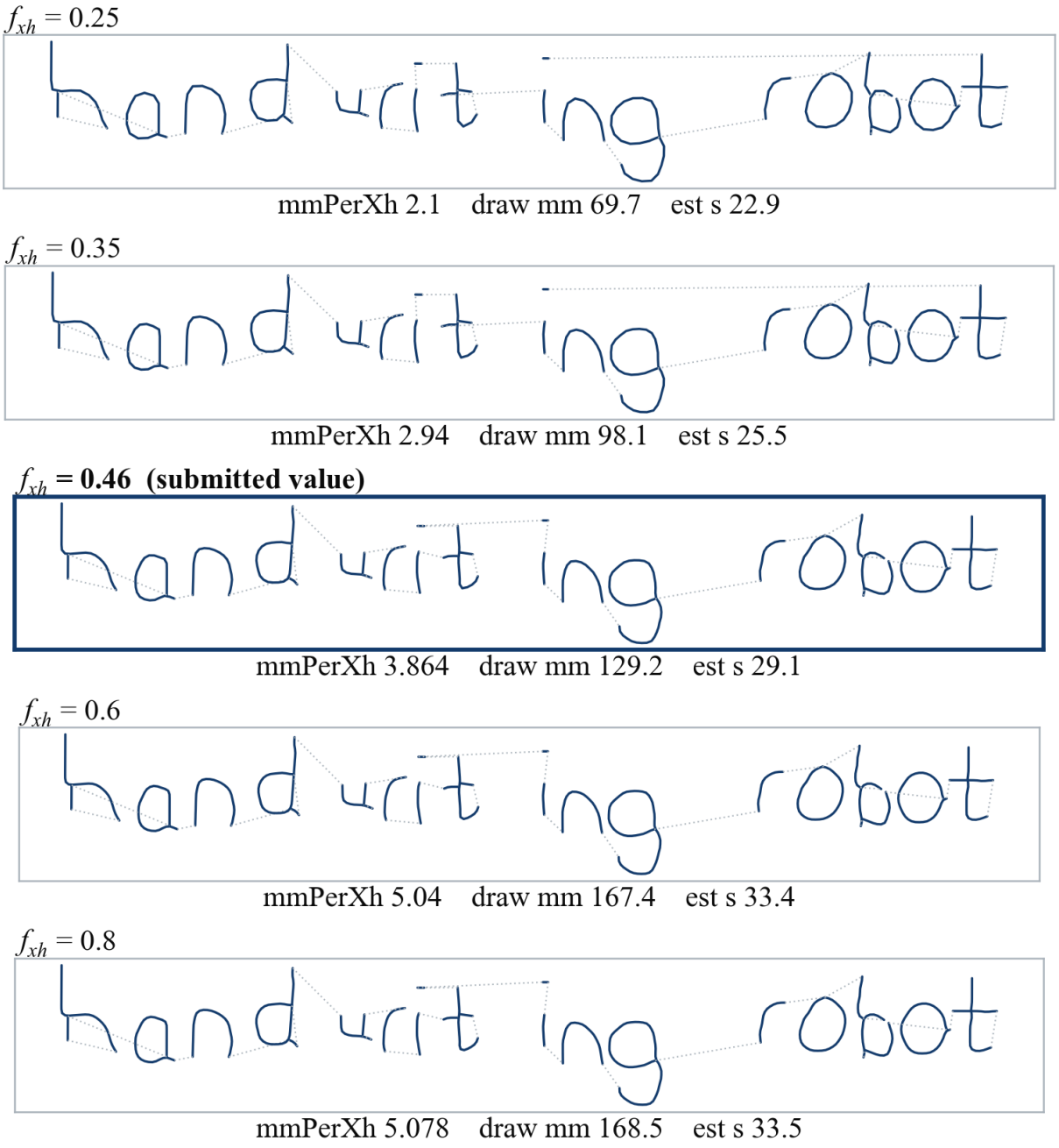


Figure 114: X-height as a share of the ruled line spacing. Each panel is the same input processed by the submitted code with only f_{xh} changed; the panel outlined in dark blue is the submitted value.

Table 61: X-height as a share of the ruled line spacing: measured effect of each swept value. The submitted value is in bold.

f_{xh}	mmPerXh	drawMm	estimatedSeconds	strokes
0.25	2.1	69.7	22.9	26
0.35	2.94	98.1	25.5	26
0.46	**3.864**	**129.2**	**29.1**	**26**
0.6	5.04	167.4	33.4	26
0.8	5.078	168.5	33.5	26

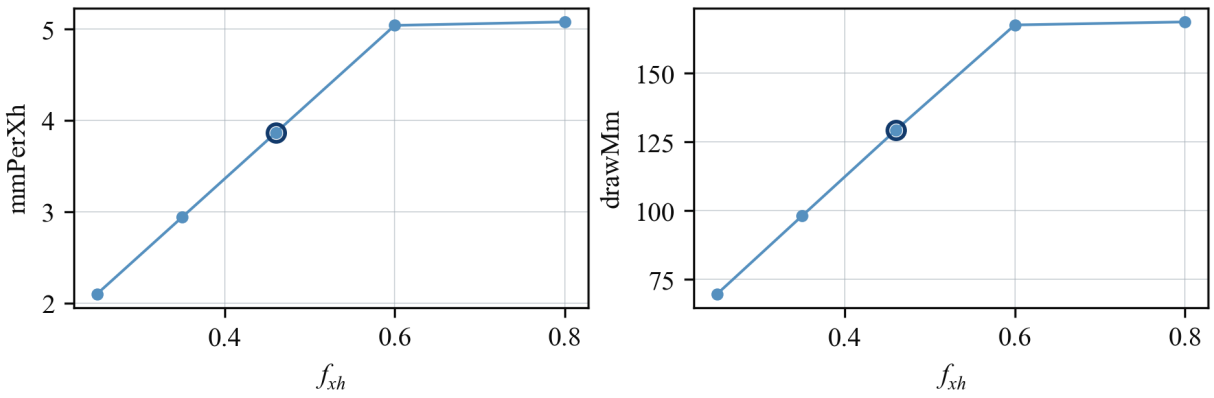


Figure 115: X-height as a share of the ruled line spacing: measured quantities against the swept value. The circled point is the submitted value.

Value `0.006`

Source `reproduce.py`, `ENDSTOP_DEBOUNCE_S`

Swept `0.001`, `0.003`, `0.006`, `0.012`, `0.025`

Basis Mechanical switch contacts bounce for a period that depends on the switch; sampling until a reading is stable is the standard software debounce, and the contact-bounce behaviour of the microswitch type used is given in its data sheet [38].

Why this value A limit switch on a long wire beside two stepper motors picks up both contact bounce and electrical noise. A single reading is therefore not trusted: the pin must read triggered `ENDSTOP_CONFIRM_SAMPLES` times in a row, t_{db} apart. The interval must span a bounce so that the samples are genuinely independent, and the total confirmation time $(n-1)\,t_{\mathrm{db}} = 12$ ms must be short against how long the carriage takes to travel the overtravel distance of the switch. At the 2400 mm/min travel feed the carriage covers $40 \times 0.012 = 0.48$ mm in that time, which is inside the overtravel of the microswitch and well inside the 5 mm edge tolerance.

What the coefficient does The two coefficients multiply: confirmation latency is $(n-1)\,t_{\mathrm{db}}$ and noise rejection improves with both. A longer interval rejects longer noise bursts but lets the carriage run further past the switch before the stop takes effect. This is a safety path, so the latency is the term that must stay bounded.

Alternatives considered A hardware RC filter or a Schmitt trigger on the switch line, which removes the software cost entirely and is the better engineering answer; or an interrupt with a dead time.

Worked example $(3-1) \times 6$ ms $= 12$ ms of confirmation; at 40 mm/s the carriage moves 0.48 mm, against 5 mm of edge tolerance, so the stop has about ten times the margin it needs.

Limits of this test **No sweep was run for this coefficient.** It can only be exercised on the gantry, with a switch, and the gantry was not run for this study. The numbers above are the arithmetic of the confirmation path, not a measurement, and the right test is to record the switch line on a scope while the carriage closes it.

Not swept This coefficient was not swept; see the paragraph above for why, and what test would settle it.

34 Seam cutting on a dense page

34.1 Weight on ink in the seam cost field, on a densely written page

Symbol w_{ink}

Value `4.0`

Source `segment.py`, `SeamCuts`

Swept `0.5`, `1.0`, `4.0`, `16.0`, `64.0`

Basis Least-cost seam carving [15, 10:3], applied to handwritten line separation after [16].

Why this value See the main entry. The point of repeating it here is that the balance between the ink term and the straightness and midline terms only matters when the seam is forced to choose between cutting through a descender and detouring a long way round it, and that choice only arises on a page where the lines are close enough to interlock.

What the coefficient does The cost of one ink pixel is w_{ink} against a straightness penalty of 0.02 per row of movement and a midline penalty reaching 0.012. At $w_{\text{ink}} = 0.5$ one ink pixel costs the same as 25 rows of movement, so the seam will cut through a stroke rather than detour far; at 64 it costs 3 200 rows and the seam will go almost anywhere to avoid ink.

Alternatives considered A straight horizontal cut, which is what a plain profile segmenter does and which cuts every descender on a page like this one.

Worked example Abhinav's page has pitch 48 px at half resolution against Owen's 51, with 26 lines against 20, so there is roughly a quarter less clean paper between lines for the seam to use.

What the sweep actually showed The line count is 26 and the reading accuracy 0.9939 at every value from 0.5 to 64, a 128-fold range, so by the end-to-end measure this coefficient does nothing on this page either. Comparing the seam paths themselves is far more informative, and it gives the first genuine justification for the submitted value found in this study. Taking the $w_{\text{ink}} = 64$ path as the reference and counting how many of the 1 224 columns of each seam sit on a different row:

w_{ink}	columns differing	largest difference
0.5	383	19 rows
1.0	2	1 row
4.0	**0**	**0**
16.0	2	1 row
64.0	0	0

The path converges by about $w_{\text{ink}} = 1$ and is exactly converged at the submitted 4.0: raising the weight sixteen-fold beyond it moves the seam by nothing at all, while dropping it to 0.5 moves 31 % of the seam, by up to 19 rows, as it begins cutting corners through ink. The submitted value therefore sits just above the knee, with roughly a fourfold margin, and that margin costs nothing because the cost field is scale-free. On Owen's page the same comparison gives zero differing columns at *every* weight from 1.0 to 16.0, which is why the denser page was needed to see this at all.

Limits of this test This is the same coefficient as in the entry above, run on a second, denser page. The standard probe did not exercise it: Owen's page is written at pitch 51 with 20 lines and the seams have clean paper to follow. Abhinav's page is cursive and carries 26 lines at pitch 48 on the same paper, so neighbouring lines interlock and the seam has to work.

Table 62: Weight on ink in the seam cost field, on a densely written page: measured effect of each swept value. The submitted value is in bold.

$w_{\{\}text{ink}}$	nLines	nLinesTruth	charAcc	pitch
0.5	26	26	0.9939	48
1	26	26	0.9939	48
4	**26**	**26**	**0.9939**	**48**
16	26	26	0.9939	48
64	26	26	0.9939	48

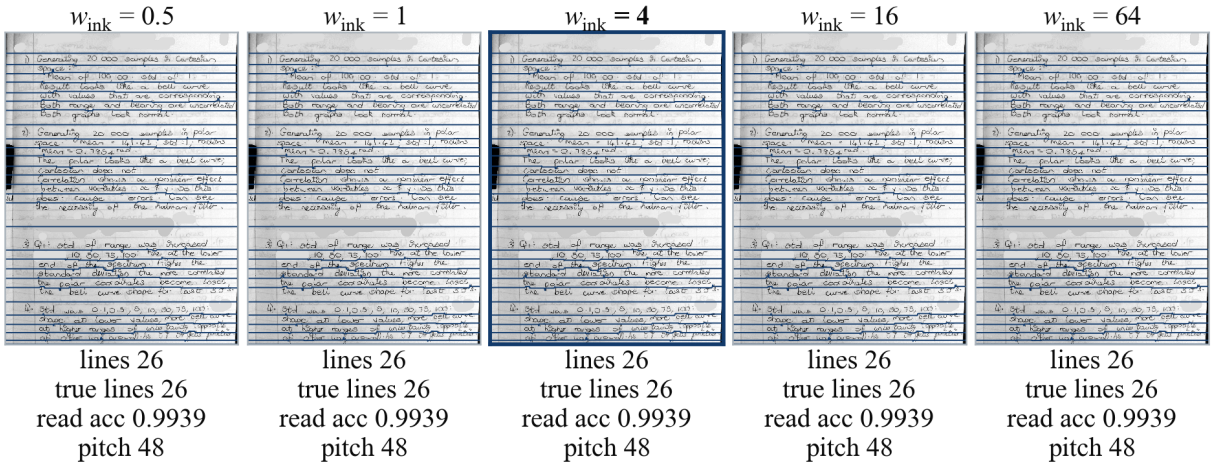


Figure 116: Weight on ink in the seam cost field, on a densely written page. Each panel is the same input processed by the submitted code with only w_{ink} changed; the panel outlined in dark blue is the submitted value.

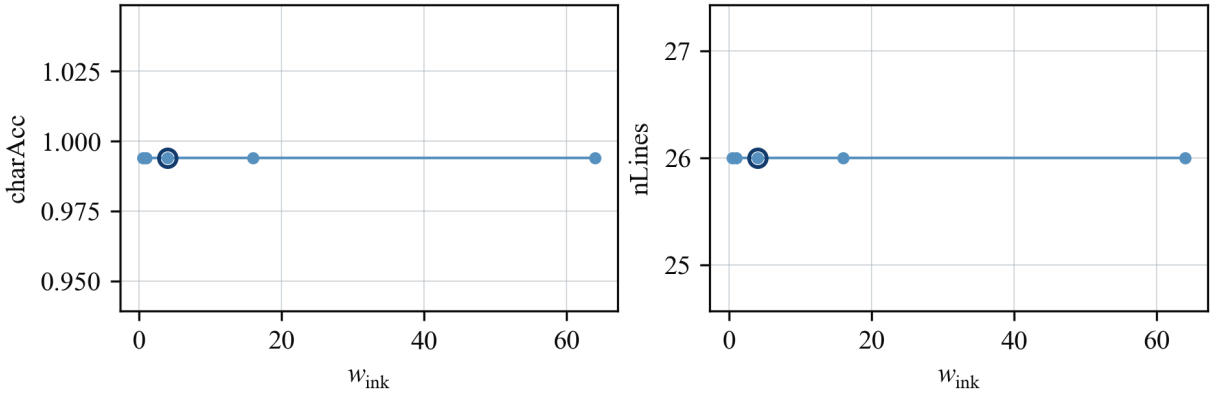


Figure 117: Weight on ink in the seam cost field, on a densely written page: measured quantities against the swept value. The circled point is the submitted value.

34.2 Closing radius applied before the seam is cut, on a densely written page

Symbol r_{tail}

Value `0.07`

Source `segment.py`, `SeamCuts`

Swept `0.0`, `0.03`, `0.07`, `0.16`, `0.3`

Basis Morphological closing before a path search [4].

Why this value See the main entry. On a dense cursive page the closing has two opposing jobs at once: it must seal the narrow channels a seam would otherwise thread through a letter, and it must not weld a descender of one line to an ascender of the next, because once they are welded no seam can pass between those two lines at all and they merge.

What the coefficient does At $P = 48$ px the radius is $\lfloor 0.07 \times 48 \rfloor = 3$ px at the submitted value and 14 px at 0.30. The gap between an ascender and the descender above it on this page is a few tens of pixels, so a large radius genuinely can bridge it.

Alternatives considered Sealing channels by a width test on the mask rather than a closing, which would not fatten the stroke at all.

Worked example $0.07 \times 48 = 3.36 \to 3$ px at the submitted value, against $0.07 \times 51 = 3.57 \to 3$ px on the standard probe, so the absolute radius is almost the same and only the spacing differs.

What the sweep actually showed Unlike the cost weights, the closing radius does change the outcome on a dense page, and in the predicted direction. Reading accuracy is 0.9939 at 0, 0.03 and the submitted 0.07, falls to 0.9617 at 0.16 and partly recovers to 0.9794 at 0.30. The line count stays at 26 throughout, so this is damage to the crops rather than to the line finding: an over-large closing welds a descender to the ascender below it, the seam can no longer pass cleanly between them, and ink is handed to the wrong line. The submitted value is at the top of the flat region, which is the right place for it, though the sweep cannot separate it from 0 or 0.03 on this page.

Limits of this test This is the same coefficient as in the entry above, run on a second, denser page. The standard probe did not exercise it: Owen's page is written at pitch 51 with 20 lines and the seams have clean paper to follow. Abhinav's page is cursive and carries 26 lines at pitch 48 on the same paper, so neighbouring lines interlock and the seam has to work.

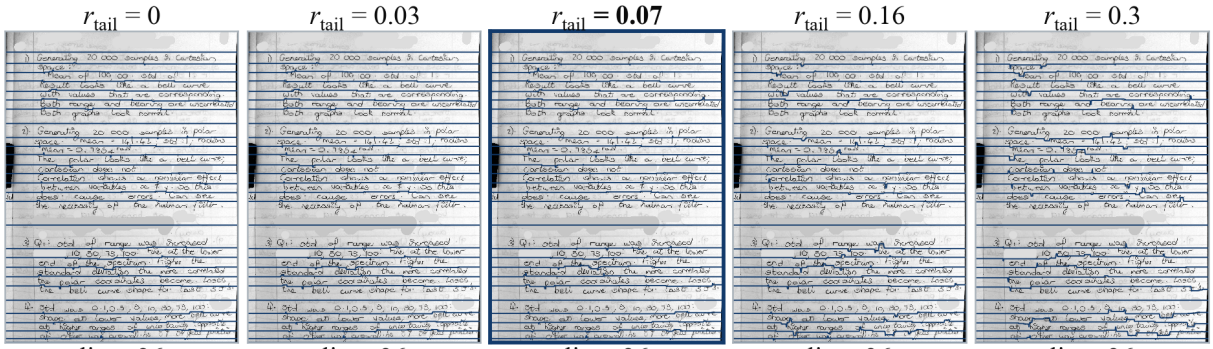


Figure 118: Closing radius applied before the seam is cut, on a densely written page. Each panel is the same input processed by the submitted code with only r_{tail} changed; the panel outlined in dark blue is the submitted value.

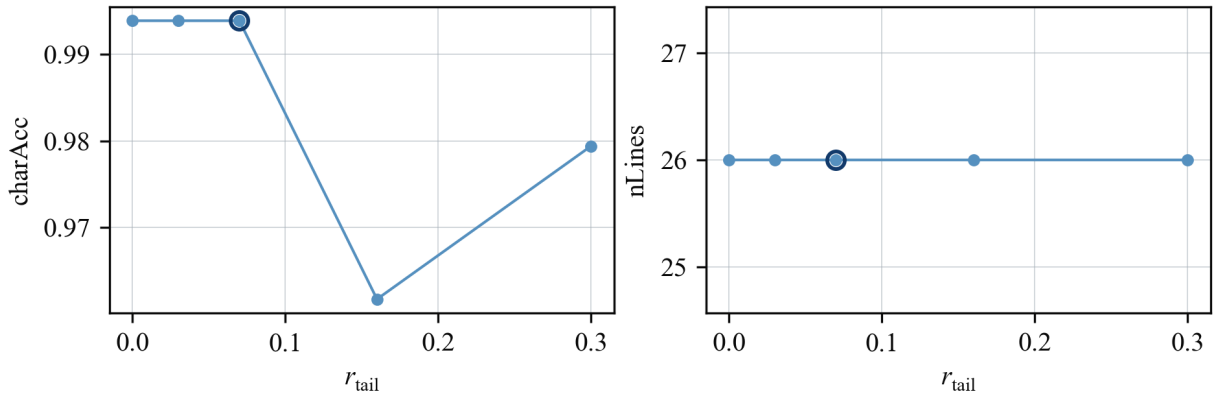


Figure 119: Closing radius applied before the seam is cut, on a densely written page: measured quantities against the swept value. The circled point is the submitted value.

Table 63: Closing radius applied before the seam is cut, on a densely written page: measured effect of each swept value. The submitted value is in bold.

$r_{\{}text{tail}}$	nLines	nLinesTruth	charAcc	pitch
0	26	26	0.9939	48
0.03	26	26	0.9939	48
0.07	**26**	**26**	**0.9939**	**48**
0.16	26	26	0.9617	48
0.3	26	26	0.9794	48

34.3 Share of a piece's ink that must lie in a band, on a densely written page

Symbol $\theta_{\text{band}}

Value `0.15`

Source `segment.py, _BandOwnership`

Swept `0.02, 0.08, 0.15, 0.3, 0.5`

Basis Component-to-line assignment after a coarse cut [9].

Why this value See the main entry. Ownership only becomes contentious when a piece of ink genuinely spans two bands, which on a well-spaced page is rare and on a dense cursive page is common: a descender from one line routinely reaches into the body of the next.

What the coefficient does Because a piece substantial in more than one band is given to the topmost, lowering θ_{band} pulls ink upward and raising it falls back on majority ownership, which hands a long descender to the line below the one it was written on.

Alternatives considered Majority ownership alone, or assignment by centroid.

Worked example A piece with 300 ink pixels counts as present in a band at 45 pixels at the submitted value, 6 pixels at 0.02 and 150 at 0.50.

What the sweep actually showed The denser page does expose an effect, but only at the top of the range: 26 lines and 0.9939 accuracy at 0.02, 0.08, the submitted 0.15 and 0.30, falling to 0.9893 at 0.50. At that setting a piece must put half its ink in a band to count as present there, so a descender reaching into the line below no longer registers in both, the topmost-band rule stops applying to it, and it is assigned by majority to the wrong line. The submitted value sits inside a wide flat region that extends well below it, so the sweep confirms the value is safe without distinguishing it from 0.02; the argument for 0.15 rather than 0.02 remains the one given above, that a few pixels of anti-aliased overlap should not count as presence.

Limits of this test This is the same coefficient as in the entry above, run on a second, denser page. The standard probe did not exercise it: Owen's page is written at pitch 51 with 20 lines and the seams have clean paper to follow. Abhinav's page is cursive and carries 26 lines at pitch 48 on the same paper, so neighbouring lines interlock and the seam has to work.

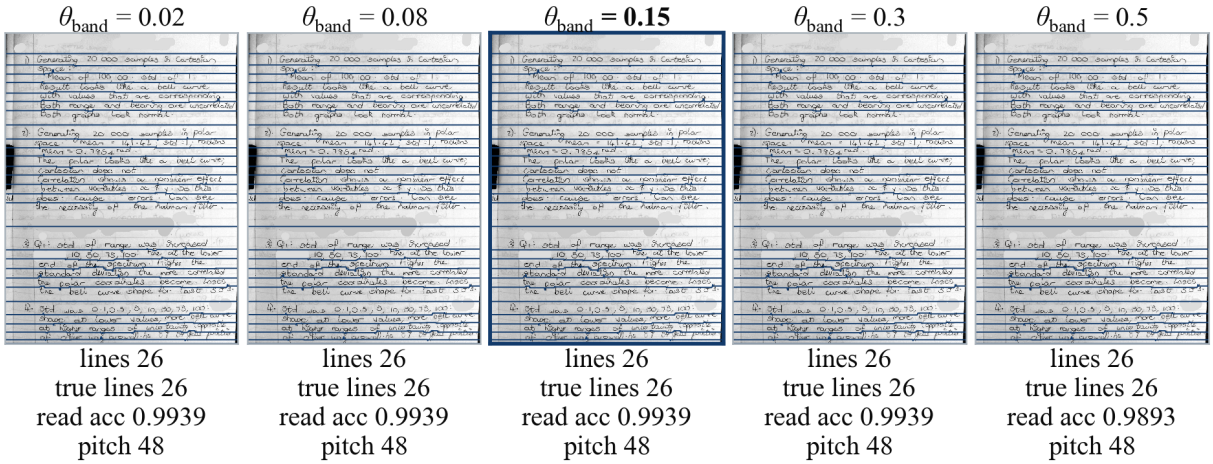


Figure 120: Share of a piece's ink that must lie in a band, on a densely written page. Each panel is the same input processed by the submitted code with only θ_{band} changed; the panel outlined in dark blue is the submitted value.

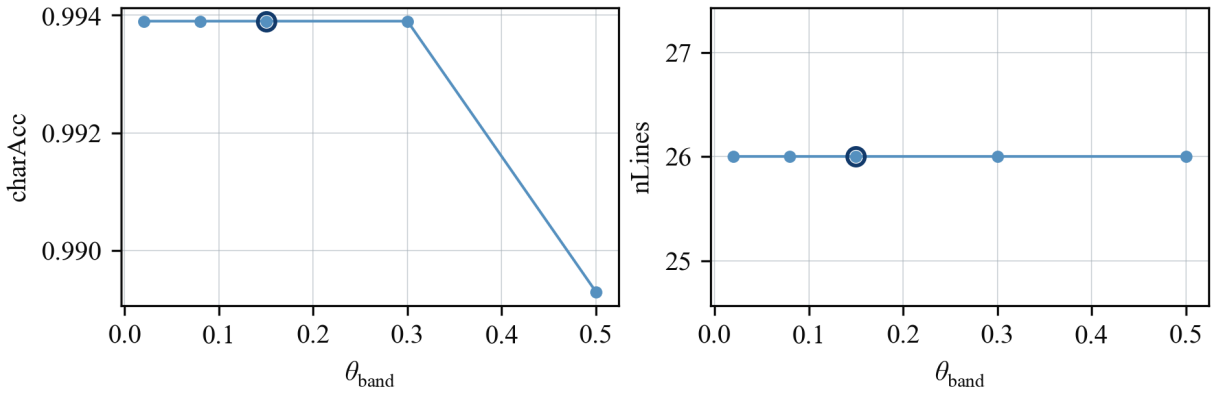


Figure 121: Share of a piece's ink that must lie in a band, on a densely written page: measured quantities against the swept value. The circled point is the submitted value.

Table 64: Share of a piece's ink that must lie in a band, on a densely written page: measured effect of each swept value. The submitted value is in bold.

$\{}theta_{\{}text{band}}$	nLines	nLinesTruth	charAcc	pitch
0.02	26	26	0.9939	48
0.08	26	26	0.9939	48
0.15	**26**	**26**	**0.9939**	**48**
0.3	26	26	0.9939	48
0.5	26	26	0.9893	48

35 Choices that cannot honestly be swept

The constants in this section define the trained networks. Changing any of them and re-running the pipeline does not measure the choice: it measures a mismatch between the new shape and weights that were fitted under the old one. Answering 'why this value' for these requires either a published basis or a retraining run, and retraining was not done for this study. They are therefore explained, with the arithmetic that constrains them, and the sweep is honestly declined.

35.1 The shared convolutional backbone

Both networks begin with the same backbone: three stages of 3×3 convolutions with padding 1, each convolution followed by batch normalisation and a ReLU, with a 2×2 max pool after the first and second stages.

Table 65: The shared backbone. Parameter counts include batch-norm scale, shift and running statistics.

stage	layers	in	out	parameters
`stage1`	2	1	32	9 824
`stage2`	4	32	64	130 304
`stage3`	6	64	128	814 848
total	12			954,976

Why 3×3 convolutions stacked, rather than larger kernels. Two stacked 3×3 layers see the same 5×5 region as one 5×5 layer, with fewer parameters and an extra non-linearity between them. The CRNN architecture this reader follows uses exactly this arrangement [17, p. 2298]. The derived receptive field of the whole backbone is **72 pixels** at a stride of 4, so one output column summarises a 72-pixel-wide window of the 64×640 canvas, which at the measured probe scale is about three x-heights: wide enough to see a character and its neighbours, narrow enough that a column is not dominated by a whole word.

Why channels widen 32, 64, 128 while the resolution falls. This is the standard pyramid: each pooling halves the spatial extent, and the channel count is doubled to keep the representational capacity roughly constant while the features become more abstract [40, 17].

Why only two pooling stages, and never on the width again. This is the hard constraint of the whole reader and it is arithmetic, not taste. CTC needs at least as many input columns as output symbols, and strictly more where a character repeats, because a blank must separate the repeat [24, pp. 370–372]. The canvas is 640 wide and the width is pooled twice, so the reader produces $T = 640/4 = 160$ columns. The code records the resulting safe label length as $\lfloor 160 \times 0.75 \rfloor = 120$ characters. A third pooling stage would halve T to 80 and the safe label length to 60 characters, which is shorter than ordinary lines on these pages, and those lines would become untrainable. That is why stage 3 has six convolution layers and no pool: depth is bought without spending resolution.

Why the height is collapsed by an adaptive max, not flattened. After stage 3 the feature map is pooled to a single row by taking the maximum over height. Flattening instead would make the number of features depend on the input height and tie the network to one canvas size; taking the maximum keeps the strongest response per column per channel and makes the backbone height-agnostic, which is what lets the same backbone serve the writer classifier unchanged.

Batch normalisation. Applied after every convolution with the usual momentum 0.1 and $\varepsilon = 10^{-5}$ [41, Alg. 1]. At inference the running mean and variance are folded into a single scale and shift, which is what `recognise.py` does.

ReLU. $\max(0, x)$ [42, p. 807], chosen for the same reason it is used everywhere: it does not saturate for positive inputs, so gradients reach the early layers of a twelve-layer stack.

35.2 The reader's recurrent stack and head

The 128 channels per column are read by two bidirectional LSTM layers of 256 units per direction, and a linear layer maps the resulting 512 features to 75 classes: the 74 characters of the charset plus the CTC blank in slot 0.

The LSTM cell is Hochreiter and Schmidhuber's [25]; the input, forget, cell and output gates are computed in one matrix product and split, which is why the stored gate matrices have $4n_h$ rows.

Reading both directions is Schuster and Paliwal's [26]: a character's identity depends on what follows it as well as what precedes it, and a unidirectional pass cannot use the right context.

Two layers of 256 is the size used by the CRNN this follows [17, p. 2298] and by the handwriting system of Graves *et al.* [27, p. 855]. The recurrent stack holds 2,367,488 parameters, which is 70 % of the reader's 3,360,939 total: it, not the backbone, is where the reader's capacity sits, and it is also what dominates inference time on the ODROID, since the recurrence is sequential over 160 columns.

CTC [24, pp. 370–372] removes the need for per-character alignment in the training labels, which matters here because the transcripts are typed line by line with no character positions. The same forward–backward machinery, run in forced-alignment mode, is what later supplies the character boundaries the glyph cutter uses.

Why this cannot be swept. The checkpoint stores gate matrices of shape $4n_h \times n_{\text{in}}$ and $4n_h \times n_h$. Changing n_h makes those shapes inconsistent with the stored arrays and the forward pass raises. The sweep in section 18.3 is included precisely so that this is on the record.

35.3 The writer classifier's head, and dropout

The writer classifier uses the same backbone, pools over height *and* then over width, and passes the resulting 128 features through a 96-unit ReLU layer, dropout at $p = 0.2$, and a linear layer to one score per writer. The head holds 13 645 parameters against the backbone's 954,976, so the classifier is almost entirely shared backbone.

Why pool over width as well. Writing style is a property of the whole line, not of a position in it. Averaging over width makes the descriptor order-free, so the classifier cannot identify a writer by *which* words they wrote — which matters because each writer copied a different passage, and a width-sensitive model could learn the passage instead of the hand.

Why dropout 0.2 and not the customary 0.5. Dropout [43, p. 1930] is applied at one place only: the 96-unit hidden layer of the head. The convolutional backbone has no dropout, because batch normalisation is already regularising it and dropout between convolutions interacts badly with batch-norm statistics. On a 96-unit layer, $p = 0.5$ leaves 48 effective units feeding a 13-way decision, which is a severe bottleneck for a head this small; $p = 0.2$ leaves about 77. The customary 0.5 in [43] is quoted for much wider fully connected layers.

Why this is not swept. Dropout is inactive at inference — the submitted `recognise.py` notes this at the point where the head is evaluated — so sweeping it without retraining would change nothing at all and produce five identical panels. Measuring it honestly means retraining the classifier at each value, which was not done.

35.4 Initialisation, optimisers and schedules

Convolution initialisation is Kaiming normal, $\sigma = \sqrt{2/\text{fan-in}}$ [44], whose gain of 2 is derived for exactly the ReLU non-linearity used here.

Linear and LSTM initialisation is uniform on $\pm 1/\sqrt{\text{fan-in}}$, the convention these layers are normally built with.

The reader is trained with RMSprop [45, Lec. 6e], $\alpha = 0.99$, weight decay 10^{-5}, and the learning rate is halved after five epochs without improvement, floored at 10^{-5}.

The writer classifier is trained with AdamW [46, Alg. 2], weight decay 10^{-4} applied separately from the gradient step, on a cosine schedule [47] over the run.

Gradient clipping at a global norm of 5.0 [48, Alg. 1] guards the recurrent stack, where long sequences make exploding gradients a real risk.

The two tasks use different optimisers because they are different problems: the CTC loss over 160 columns is noisy and benefits from RMSprop's per-parameter scaling without momentum, while the classifier is a small, well-posed cross-entropy problem where AdamW with decoupled weight decay and a cosine schedule converges in a fixed epoch budget.

35.5 Augmentation

Four augmentations are available: shear by ± 0.6 in tan, which is $\pm 31°$; rotation by $\pm 2.5°$; a perspective warp with corner displacement 3–8 % of the half-dimension; and an elastic deformation with $\sigma \in \{3,4\}$ and $\alpha \in \{15,20\}$, after Simard *et al.* [49]. They model the real variation in the input: a page is photographed from a slightly wrong angle (perspective), held at a slight tilt (rotation), and written with a varying lean (shear).

The selection rule is worth stating exactly, because it is easy to misread. Each of the four is made a candidate independently with probability 0.5, and then *one* candidate is chosen at random. The marginal probability that a given augmentation is applied is therefore not 0.5 but 23.44 %, and a crop passes through unaugmented 6.25 % of the time.

35.6 The held-out split and the personal fraction

`paths.py` fixes `ValFraction = 0.15` and `SplitSeed = 0`, shared by every script so that a line held out for one model is held out for the other. The seed is fixed so the split is reproducible across machines and across reruns; the fraction trades training material against the precision of the held-out estimate. At 15 % of 1 000 lines the validation set is 150 lines, and a 90 % accuracy measured on it carries a Wilson 95 % interval [50] of roughly ± 4.7 percentage points. Raising the fraction narrows that interval as $1/\sqrt{n}$ while taking lines away from training; the recorded arithmetic over a range of set sizes is in `results/architecture.json`.

`train.py` takes `-personal-fraction` with a default of 0.20. The reader is trained on a public corpus plus the five photographed writers, and the photographed set is far smaller. Without forcing, a batch would rarely contain a photographed line and the reader would never adapt to the pages the robot actually has to read. `IterateMixedBatches` therefore draws a fixed proportion from each set, reusing the smaller set as often as needed. At the default and a batch of 8 that is exactly 2 photographed lines and 6 public lines in every batch.

36 What is covered and what is not

36.1 Covered

Every coefficient listed in section 3 is documented with a basis, a reason, an explanation, alternatives and a worked example. Those marked with a run count are swept with a figure.

36.2 Not covered, and why

Anything requiring retraining The network shapes, the dropout rate, the optimiser settings, the augmentation ranges, the validation fraction and the personal fraction all need a training run per value to be measured. They are explained in section 35 and not swept.

Anything requiring the gantry The end-stop debounce and confirmation count, the homing step delay, the retract distance, the pen-up overrun and the measured switch travel can only be exercised on the machine. The pen dwell is swept *as arithmetic on the generated job*, which is honest about job time but says nothing about whether the nib seats.

The old segmentation comparison Running the earlier segmentation to show its effect on classification accuracy is a separate experiment against a different code base, not a coefficient sweep, and it is not in this document.

37 Running a sweep live

The study is driven from `Report/CoefficientJustification_2026-11-10/sweep`.

List every coefficient `python run.py --list`

Sweep one coefficient `python run.py darkness_gamma`

Sweep everything `python run.py`

Check the patch targets `python validate.py`

To demonstrate a different value during the demonstration, edit the `values` list of that coefficient in the matching `sweep/coeffs_*.py` file and re-run the one key. The figure in `figures/` and the record in `results/` are both rewritten, and the submitted code is verified unchanged at the end of every run.

Nothing in `Software/u23539195_BrauneDR_Code` is ever written to. A sweep builds its own copy of the seven modules in a scratch directory, rewrites one token in it, and runs that. The copy also has its data path rewritten to an absolute path, because it no longer sits beside the data folder; that is the only other change made to any copy.

References

[1] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[2] W. Niblack, *An Introduction to Digital Image Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1986.

[3] J. Sauvola and M. Pietikäinen, "Adaptive document image binarization," *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, 2000.

[4] R. M. Haralick, S. R. Sternberg, and X. Zhuang, "Image analysis using mathematical morphology," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 4, pp. 532–550, 1987.

[5] F. Shafait, D. Keysers, and T. M. Breuel, "Efficient implementation of local adaptive thresholding techniques using integral images," in *Proc. SPIE 6815, Document Recognition and Retrieval XV*, 2008.

[6] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Communications of the ACM*, vol. 15, no. 1, pp. 11–15, 1972.

[7] A. Rosenfeld and J. L. Pfaltz, "Sequential operations in digital picture processing," *Journal of the ACM*, vol. 13, no. 4, pp. 471–494, 1966.

[8] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.

[9] L. Likforman-Sulem, A. Zahour, and B. Taconet, "Text line segmentation of historical documents: a survey," *International Journal on Document Analysis and Recognition*, vol. 9, no. 2–4, pp. 123–138, 2007.

[10] U.-V. Marti and H. Bunke, "The IAM-database: an English sentence database for offline handwriting recognition," *Int. J. Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[11] A. Vinciarelli and J. Luettin, "A new normalization technique for cursive handwritten words," *Pattern Recognition Letters*, vol. 22, no. 9, pp. 1043–1050, 2001.

[12] D. Bradley and G. Roth, "Adaptive thresholding using the integral image," *Journal of Graphics Tools*, vol. 12, no. 2, pp. 13–21, 2007.

[13] F. C. Crow, "Summed-area tables for texture mapping," in *Proc. 11th Annual Conf. Computer Graphics and Interactive Techniques (SIGGRAPH)*, pp. 207–212, 1984.

[14] J. Canny, "A computational approach to edge detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.

[15] S. Avidan and A. Shamir, "Seam carving for content-aware image resizing," *ACM Transactions on Graphics*, vol. 26, no. 3, pp. 10:1–10:9, 2007.

[16] N. Arvanitopoulos and S. Süsstrunk, "Seam carving for text line extraction on color and grayscale historical manuscripts," in *Proc. 14th Int. Conf. Frontiers in Handwriting Recognition (ICFHR)*, pp. 726–731, 2014.

[17] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[18] J. Puigcerver, "Are multidimensional recurrent layers really necessary for handwritten text recognition?," in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 1, pp. 67–72, 2017.

[19] International Telecommunication Union, "Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios." Recommendation ITU-R BT.601-7, 2011.

[20] M. Bulacu and L. Schomaker, "Text-independent writer identification and verification using textural and allographic features," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 701–717, 2007.

[21] A. A. Brink, J. Smit, M. L. Bulacu, and L. R. B. Schomaker, "Writer identification using directional ink-trace width measurements," *Pattern Recognition*, vol. 45, no. 1, pp. 162–171, 2012.

[22] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984.

[23] L. Lam, S.-W. Lee, and C. Y. Suen, "Thinning methodologies—a comprehensive survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 9, pp. 869–885, 1992.

[24] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, pp. 369–376, 2006.

[25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[26] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[27] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 5, pp. 855–868, 2009.

[28] R. Plamondon and S. N. Srihari, "On-line and off-line handwriting recognition: a comprehensive survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 1, pp. 63–84, 2000.

[29] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[30] H. G. Barrow, J. M. Tenenbaum, R. C. Bolles, and H. C. Wolf, "Parametric correspondence and chamfer matching: two new techniques for image matching," in *Proc. 5th Int. Joint Conf. Artificial Intelligence (IJCAI)*, (Cambridge, MA, U.S.A.), pp. 659–663, 1977.

[31] M.-P. Dubuisson and A. K. Jain, "A modified Hausdorff distance for object matching," in *Proc. 12th International Conference on Pattern Recognition*, vol. 1, (Jerusalem, Israel), pp. 566–568, 1994.

[32] C. Wigington, S. Stewart, B. Davis, B. Barrett, B. Price, and S. Cohen, "Data augmentation for recognition of handwritten words and lines using a CNN-LSTM network," in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 1, pp. 639–645, 2017.

[33] L. R. B. Schomaker and R. Plamondon, "The relation between pen force and pen-point kinematics in handwriting," *Biological Cybernetics*, vol. 63, no. 4, pp. 277–289, 1990.

[34] A. Graves, "Generating sequences with recurrent neural networks." arXiv preprint arXiv:1308.0850, 2013.

[35] L. Kang, P. Riba, Y. Wang, M. Rusiñol, A. Fornés, and M. Villegas, "GANwriting: content-conditioned generation of styled handwritten word images," in *Computer Vision – ECCV 2020*, pp. 273–289, Springer, 2020.

[36] International Organization for Standardization, "Automation systems and integration—numerical control of machines—program format and definitions of address words—Part 1: Data format for positioning, line motion and contouring control systems." ISO 6983-1, 2009.

[37] Allegro MicroSystems, "A4988 DMOS microstepping driver with translator and overcurrent protection." Datasheet.

[38] "NEMA 17 bipolar stepper motor, 1.68 A per phase, 200 full steps per revolution." Manufacturer datasheet.

[39] L. Biagiotti and C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*. Berlin: Springer, 2008.

[40] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[41] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, pp. 448–456, 2015.

[42] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Machine Learning (ICML)*, pp. 807–814, 2010.

[43] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[44] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, pp. 1026–1034, 2015.

[45] T. Tieleman and G. Hinton, “Lecture 6.5—RMSprop: divide the gradient by a running average of its recent magnitude.” COURSERA: Neural Networks for Machine Learning, 2012.

[46] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.

[47] I. Loshchilov and F. Hutter, “SGDR: stochastic gradient descent with warm restarts,” in *Proc. 5th Int. Conf. Learning Representations (ICLR)*, 2017.

[48] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proc. 30th Int. Conf. Machine Learning (ICML)*, pp. 1310–1318, 2013.

[49] P. Y. Simard, D. Steinkraus, and J. C. Platt, “Best practices for convolutional neural networks applied to visual document analysis,” in *Proc. 7th Int. Conf. Document Analysis and Recognition (ICDAR)*, pp. 958–963, 2003.

[50] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.